

ZlibValidation

Generated on Mon Mar 31 2025 15:12:40 for ZlibValidation by Doxygen 1.9.5

Mon Mar 31 2025 15:12:40

1 ZlibValidation	1
1.1 Description	1
1.2 Usage	1
1.3 Development Diary	1
1.3.1 2025-01-27	1
1.3.2 2025-01-28	2
1.3.3 2025-02-01	2
1.3.4 2025-02-10	2
1.3.5 2025-02-11	2
1.3.6 2025-02-14	3
1.3.7 2025-02-15	3
1.3.8 2025-02-17	3
1.3.9 2025-02-18	3
1.3.10 2025-02-19	3
1.3.11 2025-02-20	4
1.3.12 2025-02-25	4
1.3.13 2025-02-26	4
1.3.14 2025-02-27	4
1.3.15 2025-02-28	4
1.3.16 2025-03-01	5
1.3.17 2025-03-07	5
1.3.18 2025-03-10	5
1.3.19 2025-03-14	5
1.3.20 2025-03-15	6
1.3.21 2025-03-16	6
1.3.22 2025-03-17	7
1.3.23 2025-03-18	7
1.3.24 2025-03-19	8
1.3.25 2025-03-21	8
1.3.26 2025-03-24	8
1.3.27 2025-03-25	8
1.3.28 2025-03-26	9
1.3.29 2025-03-27	9

1.3.30 2025-03-28	10
1.3.31 2025-03-29	11
1.3.32 2025-03-30	11
1.3.33 2025-03-31	12
2 Namespace Index	15
2.1 Namespace List	15
3 Hierarchical Index	17
3.1 Class Hierarchy	17
4 Class Index	19
4.1 Class List	19
5 File Index	21
5.1 File List	21
6 Namespace Documentation	23
6.1 slang Namespace Reference	23
6.2 slang::parsing Namespace Reference	23
6.3 slang::syntax Namespace Reference	23
6.3.1 Function Documentation	24
6.3.1.1 clone() [1/2]	25
6.3.1.2 clone() [2/2]	25
6.3.1.3 deepClone() [1/5]	26
6.3.1.4 deepClone() [2/5]	26
6.3.1.5 deepClone() [3/5]	27
6.3.1.6 deepClone() [4/5]	27
6.3.1.7 deepClone() [5/5]	28
6.4 slang::syntax::detail Namespace Reference	28
6.4.1 Typedef Documentation	28
6.4.1.1 InsertChangeMap	28
6.4.1.2 ListChangeMap	29
6.4.1.3 ModifyChangeMap	29
6.4.2 Function Documentation	29
6.4.2.1 transformTree()	29

7 Class Documentation	31
7.1 AttributesIterator Class Reference	31
7.1.1 Detailed Description	32
7.1.2 Constructor & Destructor Documentation	32
7.1.2.1 AttributesIterator()	32
7.1.2.2 ~AttributesIterator()	32
7.1.3 Member Function Documentation	32
7.1.3.1 end()	33
7.1.3.2 get()	33
7.1.3.3 next()	33
7.1.4 Member Data Documentation	34
7.1.4.1 attr_	34
7.1.4.2 attrs_	34
7.1.4.3 err_	34
7.2 CellExtractor Class Reference	35
7.2.1 Detailed Description	36
7.2.2 Constructor & Destructor Documentation	36
7.2.2.1 CellExtractor()	36
7.2.3 Member Function Documentation	36
7.2.3.1 foundTargetCell()	36
7.2.3.2 handle()	37
7.2.4 Member Data Documentation	37
7.2.4.1 foundTarget_	37
7.2.4.2 targetCell_	37
7.3 CellPrinter Class Reference	38
7.3.1 Detailed Description	39
7.3.2 Constructor & Destructor Documentation	39
7.3.2.1 CellPrinter()	39
7.3.3 Member Function Documentation	39
7.3.3.1 handle()	39
7.3.4 Member Data Documentation	39
7.3.4.1 foundTarget_	40
7.3.4.2 out_	40

7.3.4.3 targetCell_	40
7.4 slang::syntax::detail::ChangeCollection Struct Reference	40
7.4.1 Detailed Description	41
7.4.2 Member Function Documentation	41
7.4.2.1 clear()	41
7.4.2.2 empty()	41
7.4.3 Member Data Documentation	41
7.4.3.1 insertAfter	42
7.4.3.2 insertBefore	42
7.4.3.3 listInsertAtBack	42
7.4.3.4 listInsertAtFront	42
7.4.3.5 removeOrReplace	42
7.5 slang::syntax::ConstTokenOrSyntax Struct Reference	43
7.5.1 Detailed Description	44
7.5.2 Member Typedef Documentation	44
7.5.2.1 Base	44
7.5.3 Constructor & Destructor Documentation	44
7.5.3.1 ConstTokenOrSyntax() [1/3]	44
7.5.3.2 ConstTokenOrSyntax() [2/3]	45
7.5.3.3 ConstTokenOrSyntax() [3/3]	45
7.5.4 Member Function Documentation	45
7.5.4.1 isNode()	45
7.5.4.2 isToken()	45
7.5.4.3 node()	46
7.5.4.4 range()	46
7.5.4.5 token()	46
7.6 slang::syntax::DeferredSourceRange Class Reference	46
7.6.1 Detailed Description	47
7.6.2 Constructor & Destructor Documentation	47
7.6.2.1 DeferredSourceRange() [1/4]	47
7.6.2.2 DeferredSourceRange() [2/4]	47
7.6.2.3 DeferredSourceRange() [3/4]	47
7.6.2.4 DeferredSourceRange() [4/4]	48

7.6.3 Member Function Documentation	48
7.6.3.1 get()	48
7.6.3.2 operator*()	48
7.6.3.3 syntax()	49
7.6.4 Member Data Documentation	49
7.6.4.1 node	49
7.6.4.2 range	49
7.7 GateInfo Struct Reference	49
7.7.1 Detailed Description	50
7.7.2 Member Data Documentation	50
7.7.2.1 gateTypeName	50
7.7.2.2 inputSignals	50
7.7.2.3 kind	50
7.7.2.4 outputSignal	50
7.8 GroupsIterator Class Reference	51
7.8.1 Detailed Description	51
7.8.2 Constructor & Destructor Documentation	51
7.8.2.1 GroupsIterator()	52
7.8.2.2 ~GroupsIterator()	52
7.8.3 Member Function Documentation	52
7.8.3.1 end()	52
7.8.3.2 get()	53
7.8.3.3 next()	53
7.8.4 Member Data Documentation	53
7.8.4.1 err_	53
7.8.4.2 group_	54
7.8.4.3 groups_	54
7.9 slang::syntax::SeparatedSyntaxList< T >::iterator_base< U > Class Template Reference	54
7.9.1 Detailed Description	55
7.9.2 Member Typedef Documentation	56
7.9.2.1 ParentList	56
7.9.3 Constructor & Destructor Documentation	56
7.9.3.1 iterator_base() [1/2]	56

7.9.3.2 iterator_base() [2/2]	56
7.9.4 Member Function Documentation	56
7.9.4.1 advance()	57
7.9.4.2 dereference()	57
7.9.4.3 distance_to()	57
7.9.4.4 equals()	57
7.9.5 Member Data Documentation	58
7.9.5.1 index	58
7.9.5.2 list	58
7.10 LibAttribute Class Reference	58
7.10.1 Detailed Description	59
7.10.2 Constructor & Destructor Documentation	59
7.10.2.1 LibAttribute()	59
7.10.2.2 ~LibAttribute()	59
7.10.3 Member Function Documentation	60
7.10.3.1 getBoolean()	60
7.10.3.2 getFloat()	60
7.10.3.3 getInt()	61
7.10.3.4 getName()	61
7.10.3.5 getString()	62
7.10.3.6 getValues()	62
7.10.3.7 isComplex()	62
7.10.4 Member Data Documentation	63
7.10.4.1 attr_	63
7.10.4.2 err_	63
7.11 liberty_value_data Struct Reference	63
7.11.1 Detailed Description	63
7.11.2 Member Data Documentation	64
7.11.2.1 dim_sizes	64
7.11.2.2 dimensions	64
7.11.2.3 index_info	64
7.11.2.4 values	64
7.12 LibFile Class Reference	64

7.12.1 Detailed Description	66
7.12.2 Constructor & Destructor Documentation	66
7.12.2.1 LibFile()	66
7.12.2.2 ~LibFile()	66
7.12.3 Member Function Documentation	67
7.12.3.1 checkTimingArcMonotonicity()	67
7.12.3.2 logic()	68
7.12.3.3 modify()	70
7.12.3.4 mono()	70
7.12.3.5 parse()	72
7.12.3.6 read()	74
7.12.3.7 supercell()	75
7.12.3.8 verilog()	77
7.12.3.9 writeJsonToFile()	78
7.12.4 Member Data Documentation	79
7.12.4.1 basename_	79
7.12.4.2 err_	79
7.12.4.3 filename_	80
7.12.4.4 filepath_	80
7.12.4.5 jsonname_	80
7.12.4.6 lib_json_	80
7.12.4.7 libname_	80
7.12.4.8 logger_	81
7.12.4.9 loggername_	81
7.12.4.10 process_	81
7.12.4.11 temperature_	81
7.12.4.12 voltage_	81
7.13 LibGroup Class Reference	82
7.13.1 Detailed Description	82
7.13.2 Constructor & Destructor Documentation	82
7.13.2.1 LibGroup()	83
7.13.2.2 ~LibGroup()	83
7.13.3 Member Function Documentation	83

7.13.3.1	getAttrs()	83
7.13.3.2	getGroups()	84
7.13.3.3	getName()	84
7.13.3.4	getType()	85
7.13.4	Member Data Documentation	85
7.13.4.1	err_	85
7.13.4.2	group_	85
7.14	LibraryComparator Class Reference	85
7.14.1	Detailed Description	86
7.14.2	Constructor & Destructor Documentation	87
7.14.2.1	LibraryComparator()	87
7.14.3	Member Function Documentation	88
7.14.3.1	compareCell()	88
7.14.3.2	compareLut()	89
7.14.3.3	comparePin()	91
7.14.3.4	compareTimingArc()	92
7.14.3.5	generateReport()	93
7.14.4	Member Data Documentation	94
7.14.4.1	abstol_	94
7.14.4.2	comp_json_	94
7.14.4.3	comp_lib_path_	94
7.14.4.4	ref_json_	94
7.14.4.5	ref_lib_path_	95
7.14.4.6	reltol_	95
7.15	LogicExtractor Class Reference	95
7.15.1	Detailed Description	97
7.15.2	Constructor & Destructor Documentation	97
7.15.2.1	LogicExtractor()	97
7.15.3	Member Function Documentation	97
7.15.3.1	deriveLogicRecursive()	97
7.15.3.2	formatExpression()	98
7.15.3.3	getExtractedGates()	98
7.15.3.4	getInternalWires()	98

7.15.3.5	getLogicExpressions()	99
7.15.3.6	getPrimaryInputs()	99
7.15.3.7	getPrimaryOutputs()	100
7.15.3.8	handle() [1/5]	100
7.15.3.9	handle() [2/5]	101
7.15.3.10	handle() [3/5]	101
7.15.3.11	handle() [4/5]	101
7.15.3.12	handle() [5/5]	101
7.15.4	Member Data Documentation	101
7.15.4.1	gateOutputDrivers_	102
7.15.4.2	inTargetModule_	102
7.15.4.3	internalWires_	102
7.15.4.4	logicCache_	102
7.15.4.5	parsingComplete_	102
7.15.4.6	portDirections_	103
7.15.4.7	primaryInputs_	103
7.15.4.8	primaryOutputs_	103
7.15.4.9	targetCell_	103
7.16	ModuleRewriter Class Reference	104
7.16.1	Detailed Description	105
7.16.2	Constructor & Destructor Documentation	105
7.16.2.1	ModuleRewriter()	105
7.16.3	Member Function Documentation	106
7.16.3.1	handle() [1/2]	106
7.16.3.2	handle() [2/2]	106
7.16.4	Member Data Documentation	107
7.16.4.1	cellName_	107
7.16.4.2	depth_	107
7.16.4.3	inputPins_	107
7.16.4.4	instance_count_	107
7.16.4.5	logger_	108
7.16.4.6	moduleName_	108
7.16.4.7	outputPins_	108

7.16.4.8 portInfoMap_	108
7.17 slang::syntax::PtrTokenOrSyntax Struct Reference	109
7.17.1 Detailed Description	110
7.17.2 Member Typedef Documentation	110
7.17.2.1 Base	110
7.17.3 Constructor & Destructor Documentation	110
7.17.3.1 PtrTokenOrSyntax() [1/3]	110
7.17.3.2 PtrTokenOrSyntax() [2/3]	111
7.17.3.3 PtrTokenOrSyntax() [3/3]	111
7.17.4 Member Function Documentation	111
7.17.4.1 isNode()	111
7.17.4.2 isToken()	111
7.17.4.3 node()	112
7.17.4.4 range()	112
7.17.4.5 token()	112
7.18 slang::syntax::detail::RemoveChange Struct Reference	113
7.18.1 Detailed Description	113
7.19 slang::syntax::detail::ReplaceChange Struct Reference	114
7.19.1 Detailed Description	115
7.20 slang::syntax::SeparatedSyntaxList< T > Class Template Reference	115
7.20.1 Detailed Description	117
7.20.2 Member Typedef Documentation	117
7.20.2.1 const_iterator	117
7.20.2.2 iterator	118
7.20.3 Constructor & Destructor Documentation	118
7.20.3.1 SeparatedSyntaxList() [1/2]	118
7.20.3.2 SeparatedSyntaxList() [2/2]	118
7.20.4 Member Function Documentation	118
7.20.4.1 begin() [1/2]	118
7.20.4.2 begin() [2/2]	119
7.20.4.3 cbegin()	119
7.20.4.4 cend()	119
7.20.4.5 clone()	119

7.20.4.6 elems() [1/2]	120
7.20.4.7 elems() [2/2]	120
7.20.4.8 empty()	120
7.20.4.9 end() [1/2]	121
7.20.4.10 end() [2/2]	121
7.20.4.11 getChild() [1/2]	121
7.20.4.12 getChild() [2/2]	121
7.20.4.13 getChildPtr()	122
7.20.4.14 operator[]() [1/2]	122
7.20.4.15 operator[]() [2/2]	122
7.20.4.16 resetAll()	123
7.20.4.17 setChild()	123
7.20.4.18 size()	123
7.20.5 Member Data Documentation	124
7.20.5.1 elements	124
7.21 si2drExprT Struct Reference	124
7.21.1 Detailed Description	125
7.21.2 Member Data Documentation	125
7.21.2.1 b	125
7.21.2.2 d	125
7.21.2.3 i	125
7.21.2.4 left	125
7.21.2.5 right	126
7.21.2.6 s	126
7.21.2.7 type	126
7.21.2.8	126
7.21.2.9 valuetype	126
7.22 si2OID Struct Reference	127
7.22.1 Detailed Description	127
7.22.2 Member Data Documentation	127
7.22.2.1 v1	127
7.22.2.2 v2	127
7.23 slang::syntax::detail::SyntaxChange Struct Reference	128

7.23.1 Detailed Description	128
7.23.2 Member Data Documentation	129
7.23.2.1 first	129
7.23.2.2 second	129
7.23.2.3 separator	129
7.24 slang::syntax::SyntaxList< T > Class Template Reference	130
7.24.1 Detailed Description	131
7.24.2 Constructor & Destructor Documentation	131
7.24.2.1 SyntaxList() [1/2]	131
7.24.2.2 SyntaxList() [2/2]	132
7.24.3 Member Function Documentation	132
7.24.3.1 clone()	132
7.24.3.2 getChild() [1/2]	132
7.24.3.3 getChild() [2/2]	133
7.24.3.4 getChildPtr()	133
7.24.3.5 resetAll()	133
7.24.3.6 setChild()	134
7.25 slang::syntax::SyntaxListBase Class Reference	134
7.25.1 Detailed Description	135
7.25.2 Constructor & Destructor Documentation	135
7.25.2.1 SyntaxListBase()	136
7.25.3 Member Function Documentation	136
7.25.3.1 clone()	136
7.25.3.2 getChild() [1/2]	136
7.25.3.3 getChild() [2/2]	136
7.25.3.4 getChildCount()	137
7.25.3.5 getChildPtr()	137
7.25.3.6 isChildOptional()	137
7.25.3.7 isKind()	137
7.25.3.8 resetAll()	137
7.25.3.9 setChild()	138
7.25.4 Member Data Documentation	138
7.25.4.1 childCount	138

7.26 slang::syntax::SyntaxNode Class Reference	138
7.26.1 Detailed Description	140
7.26.2 Member Typedef Documentation	140
7.26.2.1 Token	141
7.26.3 Constructor & Destructor Documentation	141
7.26.3.1 SyntaxNode()	141
7.26.4 Member Function Documentation	141
7.26.4.1 as() [1/2]	141
7.26.4.2 as() [2/2]	141
7.26.4.3 as_if() [1/2]	142
7.26.4.4 as_if() [2/2]	142
7.26.4.5 childNode() [1/2]	142
7.26.4.6 childNode() [2/2]	142
7.26.4.7 childToken()	142
7.26.4.8 childTokenPtr()	143
7.26.4.9 getChild() [1/2]	143
7.26.4.10 getChild() [2/2]	143
7.26.4.11 getChildCount()	143
7.26.4.12 getChildPtr()	143
7.26.4.13 getFirstToken()	143
7.26.4.14 getFirstTokenPtr()	144
7.26.4.15 getLastToken()	144
7.26.4.16 getLastTokenPtr()	144
7.26.4.17 isChildOptional()	144
7.26.4.18 isEquivalentTo()	144
7.26.4.19 isKind()	145
7.26.4.20 sourceRange()	145
7.26.4.21 toString()	145
7.26.4.22 visit() [1/2]	145
7.26.4.23 visit() [2/2]	146
7.26.5 Member Data Documentation	146
7.26.5.1 kind	146
7.26.5.2 parent	146

7.26.5.3 previewNode	146
7.27 slang::syntax::SyntaxPrinter Class Reference	147
7.27.1 Detailed Description	148
7.27.2 Constructor & Destructor Documentation	148
7.27.2.1 SyntaxPrinter() [1/2]	148
7.27.2.2 SyntaxPrinter() [2/2]	148
7.27.3 Member Function Documentation	148
7.27.3.1 append()	148
7.27.3.2 print() [1/4]	149
7.27.3.3 print() [2/4]	149
7.27.3.4 print() [3/4]	149
7.27.3.5 print() [4/4]	150
7.27.3.6 printFile()	150
7.27.3.7 setIncludeComments()	150
7.27.3.8 setIncludeDirectives()	151
7.27.3.9 setIncludeMissing()	151
7.27.3.10 setIncludePreprocessed()	151
7.27.3.11 setIncludeSkipped()	152
7.27.3.12 setIncludeTrivia()	152
7.27.3.13 setSquashNewlines()	152
7.27.3.14 str()	153
7.27.4 Member Data Documentation	153
7.27.4.1 buffer	153
7.27.4.2 includeComments	153
7.27.4.3 includeDirectives	153
7.27.4.4 includeMissing	154
7.27.4.5 includePreprocessed	154
7.27.4.6 includeSkipped	154
7.27.4.7 includeTrivia	154
7.27.4.8 sourceManager	154
7.27.4.9 squashNewlines	155
7.28 slang::syntax::SyntaxRewriter< TDerived > Class Template Reference	155
7.28.1 Detailed Description	157

7.28.2 Member Typedef Documentation	157
7.28.2.1 Token	157
7.28.3 Constructor & Destructor Documentation	157
7.28.3.1 SyntaxRewriter()	157
7.28.4 Member Function Documentation	157
7.28.4.1 insertAfter()	158
7.28.4.2 insertAtBack()	158
7.28.4.3 insertAtFront()	158
7.28.4.4 insertBefore()	159
7.28.4.5 makeComma()	159
7.28.4.6 makeld() [1/2]	159
7.28.4.7 makeld() [2/2]	160
7.28.4.8 makeToken()	160
7.28.4.9 parse()	161
7.28.4.10 remove()	161
7.28.4.11 replace()	162
7.28.4.12 transform()	162
7.28.5 Member Data Documentation	163
7.28.5.1 alloc	163
7.28.5.2 commits	163
7.28.5.3 factory	163
7.28.5.4 SingleSpace	164
7.28.5.5 sourceManager	164
7.28.5.6 tempTrees	164
7.29 slang::syntax::SyntaxTree Class Reference	164
7.29.1 Detailed Description	167
7.29.2 Member Typedef Documentation	167
7.29.2.1 MacroList	167
7.29.2.2 TreeOrError	167
7.29.3 Constructor & Destructor Documentation	168
7.29.3.1 SyntaxTree() [1/3]	168
7.29.3.2 SyntaxTree() [2/3]	168
7.29.3.3 ~SyntaxTree()	168

7.29.3.4 SyntaxTree() [3/3]	168
7.29.4 Member Function Documentation	169
7.29.4.1 allocator()	169
7.29.4.2 create()	169
7.29.4.3 diagnostics()	169
7.29.4.4 fromBuffer()	170
7.29.4.5 fromBuffers()	170
7.29.4.6 fromFile() [1/2]	171
7.29.4.7 fromFile() [2/2]	171
7.29.4.8 fromFileInMemory()	172
7.29.4.9 fromFiles() [1/2]	172
7.29.4.10 fromFiles() [2/2]	172
7.29.4.11 fromLibraryMapBuffer()	173
7.29.4.12 fromLibraryMapFile()	173
7.29.4.13 fromLibraryMapText()	173
7.29.4.14 fromText() [1/3]	174
7.29.4.15 fromText() [2/3]	174
7.29.4.16 fromText() [3/3]	175
7.29.4.17 getDefaultSourceManager()	175
7.29.4.18 getDefinedMacros()	175
7.29.4.19 getMetadata()	176
7.29.4.20 getSourceLibrary()	176
7.29.4.21 options()	176
7.29.4.22 root() [1/2]	176
7.29.4.23 root() [2/2]	177
7.29.4.24 sourceManager() [1/2]	177
7.29.4.25 sourceManager() [2/2]	177
7.29.5 Member Data Documentation	177
7.29.5.1 alloc	177
7.29.5.2 diagnosticsBuffer	178
7.29.5.3 isLibraryUnit	178
7.29.5.4 library	178
7.29.5.5 macros	178

7.29.5.6 metadata	178
7.29.5.7 options_	179
7.29.5.8 rootNode	179
7.29.5.9 sourceMan	179
7.30 slang::syntax::SyntaxVisitor< TDerived > Class Template Reference	179
7.30.1 Detailed Description	180
7.30.2 Member Function Documentation	180
7.30.2.1 visit()	180
7.30.2.2 visitDefault()	181
7.30.2.3 visitToken()	181
7.31 slang::syntax::TokenList Class Reference	182
7.31.1 Detailed Description	183
7.31.2 Constructor & Destructor Documentation	183
7.31.2.1 TokenList() [1/2]	183
7.31.2.2 TokenList() [2/2]	183
7.31.3 Member Function Documentation	184
7.31.3.1 clone()	184
7.31.3.2 getChild() [1/2]	184
7.31.3.3 getChild() [2/2]	184
7.31.3.4 getChildPtr()	185
7.31.3.5 resetAll()	185
7.31.3.6 setChild()	185
7.32 slang::syntax::TokenOrSyntax Struct Reference	186
7.32.1 Detailed Description	187
7.32.2 Constructor & Destructor Documentation	187
7.32.2.1 TokenOrSyntax() [1/3]	187
7.32.2.2 TokenOrSyntax() [2/3]	187
7.32.2.3 TokenOrSyntax() [3/3]	187
7.32.3 Member Function Documentation	188
7.32.3.1 node()	188
7.33 ValuesIterator Class Reference	188
7.33.1 Detailed Description	189
7.33.2 Constructor & Destructor Documentation	189

7.33.2.1 ValuesIterator()	189
7.33.2.2 ~ValuesIterator()	189
7.33.3 Member Function Documentation	189
7.33.3.1 end()	190
7.33.3.2 next()	190
7.33.4 Member Data Documentation	190
7.33.4.1 bool_	190
7.33.4.2 err_	191
7.33.4.3 exprp_	191
7.33.4.4 float_	191
7.33.4.5 int_	191
7.33.4.6 str_	191
7.33.4.7 values_	192
7.33.4.8 vtype_	192
7.34 VerilogVisitor Class Reference	192
7.34.1 Detailed Description	193
7.34.2 Constructor & Destructor Documentation	193
7.34.2.1 VerilogVisitor()	194
7.34.3 Member Function Documentation	194
7.34.3.1 handle() [1/5]	194
7.34.3.2 handle() [2/5]	194
7.34.3.3 handle() [3/5]	195
7.34.3.4 handle() [4/5]	195
7.34.3.5 handle() [5/5]	195
7.34.4 Member Data Documentation	196
7.34.4.1 depth_	196
7.34.4.2 inTargetModule_	196
7.34.4.3 targetCell_	196
8 File Documentation	197
8.1 include/compare.hpp File Reference	197
8.1.1 Typedef Documentation	198
8.1.1.1 json	198
8.2 compare.hpp	198

8.3 include/iterators.hpp File Reference	199
8.4 iterators.hpp	200
8.5 include/json_utils.hpp File Reference	200
8.5.1 Typedef Documentation	202
8.5.1.1 json	202
8.5.2 Function Documentation	202
8.5.2.1 generateCellJson()	202
8.5.2.2 generateLutJson()	204
8.5.2.3 generatePinJson()	205
8.5.2.4 generatePowerJson()	208
8.6 json_utils.hpp	209
8.7 include/lib_attribute.hpp File Reference	210
8.8 lib_attribute.hpp	211
8.9 include/lib_file.hpp File Reference	211
8.9.1 Typedef Documentation	212
8.9.1.1 json	212
8.10 lib_file.hpp	213
8.11 include/lib_group.hpp File Reference	213
8.12 lib_group.hpp	214
8.13 include/verilog_utils.hpp File Reference	215
8.13.1 Function Documentation	216
8.13.1.1 extractAndPrintNetlistInfo()	216
8.13.1.2 extractLogicFromVerilog()	217
8.13.1.3 getAST()	217
8.14 verilog_utils.hpp	218
8.15 include/version.h File Reference	221
8.15.1 Macro Definition Documentation	221
8.15.1.1 APP_AUTHOR	221
8.15.1.2 APP_CONTACT	222
8.15.1.3 APP_NAME	222
8.15.1.4 APP_VERSION	222
8.15.1.5 APP_VERSION_MAJOR	222
8.15.1.6 APP_VERSION_MINOR	222

8.15.1.7 APP_VERSION_PATCH	223
8.15.1.8 BUILD_TIMESTAMP	223
8.16 version.h	223
8.17 include_3rd_party/si2dr_liberty.h File Reference	223
8.17.1 Macro Definition Documentation	228
8.17.1.1 LONG_DOUBLE	228
8.17.1.2 SI2_ARGS	228
8.17.1.3 SI2_LONG_MAX	228
8.17.1.4 SI2_LONG_MIN	229
8.17.1.5 SI2_ULONG_MAX	229
8.17.1.6 SI2DR_FALSE	229
8.17.1.7 SI2DR_MAX_FLOAT32	229
8.17.1.8 SI2DR_MAX_FLOAT64	229
8.17.1.9 SI2DR_MAX_INT32	230
8.17.1.10 SI2DR_MAX_STRING_LEN	230
8.17.1.11 SI2DR_MIN_FLOAT32	230
8.17.1.12 SI2DR_MIN_FLOAT64	230
8.17.1.13 SI2DR_MIN_INT32	230
8.17.1.14 SI2DR_TRUE	231
8.17.2 Typedef Documentation	231
8.17.2.1 si2BooleanT	231
8.17.2.2 si2DoubleT	231
8.17.2.3 si2drAttrComplexValldT	231
8.17.2.4 si2drAttrldT	231
8.17.2.5 si2drAttrslT	232
8.17.2.6 si2drAttrTypeT	232
8.17.2.7 si2drBooleanT	232
8.17.2.8 si2drDefineldT	232
8.17.2.9 si2drDefinesldT	232
8.17.2.10 si2drErrorT	232
8.17.2.11 si2drExprT	233
8.17.2.12 si2drExprTypeT	233
8.17.2.13 si2drFloat32T	233

8.17.2.14 si2drFloat64T	233
8.17.2.15 si2drGroupIdT	233
8.17.2.16 si2drGroupsIdT	233
8.17.2.17 si2drInt32T	234
8.17.2.18 si2drIterIdT	234
8.17.2.19 si2drMessageHandlerT	234
8.17.2.20 si2drNamesIdT	234
8.17.2.21 si2drObjectIdT	234
8.17.2.22 si2drObjectsIdT	235
8.17.2.23 si2drObjectTypeT	235
8.17.2.24 si2drSeverityT	235
8.17.2.25 si2drStringT	235
8.17.2.26 si2drValuesIdT	235
8.17.2.27 si2drValueTypeT	235
8.17.2.28 si2drVoidT	236
8.17.2.29 si2FloatT	236
8.17.2.30 si2IteratorIdT	236
8.17.2.31 si2LongT	236
8.17.2.32 si2ObjectIdT	236
8.17.2.33 si2StringT	236
8.17.2.34 si2TypeT	237
8.17.2.35 si2VoidT	237
8.17.3 Enumeration Type Documentation	237
8.17.3.1 si2BooleanEnum	237
8.17.3.2 si2drAttrTypeT	237
8.17.3.3 si2drErrorT	238
8.17.3.4 si2drExprTypeT	238
8.17.3.5 si2drObjectTypeT	239
8.17.3.6 si2drSeverityT	239
8.17.3.7 si2drValueTypeT	239
8.17.3.8 si2TypeEnum	240
8.17.4 Function Documentation	240
8.17.4.1 get_function()	240

8.17.4.2 liberty_destroy_value_data()	241
8.17.4.3 liberty_get_element()	241
8.17.4.4 liberty_get_values_data()	241
8.17.4.5 main_liberty_parser()	241
8.17.4.6 print_version()	241
8.17.4.7 SI2_ARGS() [1/41]	241
8.17.4.8 SI2_ARGS() [2/41]	242
8.17.4.9 SI2_ARGS() [3/41]	242
8.17.4.10 SI2_ARGS() [4/41]	242
8.17.4.11 SI2_ARGS() [5/41]	242
8.17.4.12 SI2_ARGS() [6/41]	242
8.17.4.13 SI2_ARGS() [7/41]	242
8.17.4.14 SI2_ARGS() [8/41]	243
8.17.4.15 SI2_ARGS() [9/41]	243
8.17.4.16 SI2_ARGS() [10/41]	243
8.17.4.17 SI2_ARGS() [11/41]	243
8.17.4.18 SI2_ARGS() [12/41]	243
8.17.4.19 SI2_ARGS() [13/41]	243
8.17.4.20 SI2_ARGS() [14/41]	244
8.17.4.21 SI2_ARGS() [15/41]	244
8.17.4.22 SI2_ARGS() [16/41]	244
8.17.4.23 SI2_ARGS() [17/41]	244
8.17.4.24 SI2_ARGS() [18/41]	244
8.17.4.25 SI2_ARGS() [19/41]	244
8.17.4.26 SI2_ARGS() [20/41]	245
8.17.4.27 SI2_ARGS() [21/41]	245
8.17.4.28 SI2_ARGS() [22/41]	245
8.17.4.29 SI2_ARGS() [23/41]	245
8.17.4.30 SI2_ARGS() [24/41]	245
8.17.4.31 SI2_ARGS() [25/41]	245
8.17.4.32 SI2_ARGS() [26/41]	246
8.17.4.33 SI2_ARGS() [27/41]	246
8.17.4.34 SI2_ARGS() [28/41]	246

8.17.4.35 SI2_ARGS() [29/41]	246
8.17.4.36 SI2_ARGS() [30/41]	246
8.17.4.37 SI2_ARGS() [31/41]	246
8.17.4.38 SI2_ARGS() [32/41]	247
8.17.4.39 SI2_ARGS() [33/41]	247
8.17.4.40 SI2_ARGS() [34/41]	247
8.17.4.41 SI2_ARGS() [35/41]	247
8.17.4.42 SI2_ARGS() [36/41]	247
8.17.4.43 SI2_ARGS() [37/41]	247
8.17.4.44 SI2_ARGS() [38/41]	248
8.17.4.45 SI2_ARGS() [39/41]	248
8.17.4.46 SI2_ARGS() [40/41]	248
8.17.4.47 SI2_ARGS() [41/41]	248
8.17.4.48 si2drDefaultMessageHandler()	248
8.17.4.49 si2drPIGetMessageHandler()	249
8.17.4.50 si2drPISetMessageHandler()	249
8.17.4.51 usage()	249
8.18 si2dr_liberty.h	249
8.19 include_3rd_party/SyntaxNode.h File Reference	259
8.19.1 Detailed Description	260
8.20 SyntaxNode.h	260
8.21 include_3rd_party/SyntaxPrinter.h File Reference	266
8.21.1 Detailed Description	267
8.22 SyntaxPrinter.h	267
8.23 include_3rd_party/SyntaxTree.h File Reference	268
8.23.1 Detailed Description	269
8.24 SyntaxTree.h	269
8.25 include_3rd_party/SyntaxVisitor.h File Reference	271
8.25.1 Detailed Description	273
8.25.2 Macro Definition Documentation	273
8.25.2.1 DERIVED	273
8.26 SyntaxVisitor.h	273
8.27 README.md File Reference	276

8.28 src/compare.cpp File Reference	276
8.29 compare.cpp	276
8.30 src/iterators.cpp File Reference	282
8.31 iterators.cpp	282
8.32 src/json_utils.cpp File Reference	283
8.32.1 Function Documentation	284
8.32.1.1 generateCellJson()	284
8.32.1.2 generateLutJson()	286
8.32.1.3 generatePinJson()	287
8.32.1.4 generatePowerJson()	290
8.32.1.5 generateTimingJson()	291
8.32.1.6 parseStringToVector()	293
8.33 json_utils.cpp	294
8.34 src/lib_attribute.cpp File Reference	297
8.35 lib_attribute.cpp	298
8.36 src/lib_file.cpp File Reference	298
8.37 lib_file.cpp	299
8.38 src/lib_group.cpp File Reference	311
8.39 lib_group.cpp	311
8.40 src/main.cpp File Reference	311
8.40.1 Function Documentation	312
8.40.1.1 compareLibFiles()	313
8.40.1.2 funcLibFile()	314
8.40.1.3 main()	315
8.40.1.4 monoCheckLibFile()	316
8.40.1.5 parseLibFile()	318
8.40.1.6 printInfo()	320
8.40.1.7 supercellLibFile()	321
8.40.1.8 verilogLibFile()	323
8.41 main.cpp	324
8.42 src/verilog_utils.cpp File Reference	332
8.42.1 Function Documentation	333
8.42.1.1 extractAndPrintNetlistInfo()	333

8.42.1.2 <code>extractLogicFromVerilog()</code>	334
8.42.1.3 <code>getAST()</code>	334
8.43 <code>verilog_utils.cpp</code>	335
索引	355

Chapter 1

ZlibValidation

1.1 Description

Command line tool to validate standard cell libraries in .lib format.

1.2 Usage

```
ZlibValidation
Usage: ./zlibvalidation [OPTIONS] [SUBCOMMAND]

Options:
  -h,--help          Print this help message and exit
  -v,--version        Display program version information and exit

Subcommands:
  parse              Parse the Liberty file and write JSON to a file
  mono               Check the monotonicity of timing arc values
  compare            Compare the comparison library against the reference library and report differences
  supercell          Generate supercells for the given Liberty file
  zlibboost          ZlibBoost - Multi-threaded Library Processing Tool
  clear              Clear the log, JSON, map, markdown files in this directory
  verilog            Generate Verilog file for given Liberty file
  spice              Generate SPICE file for given Liberty file
  func               Check functional equivalence of two Liberty files or Verilog files
```

1.3 Development Diary

1.3.1 2025-01-27

- 创建了一个C++空项目，使用CMake管理编译链，并手动添加了外部库 libsi2dr_liberty.a。
- 添加了 CLI11 库以处理命令行参数解析。
- 添加了 spdlog 日志库，实现高效的日志格式化输出。

1.3.2 2025-01-28

- 添加 `nlohmann/json` 库用于 JSON 数据存储和检索。
- 在 `version.h` 中存储项目作者、版本、构建日期等信息，由 CMake 自动维护。
- 简化了 `--version` 参数解析的代码实现，并添加了 `--mode` 选项以选择工作模式。
- 使用 `spdlog` 库的日志记录器将 debug 级别及以上的日志输出到文件，将 info 级别及以上的日志输出到控制台。
- 将库文件顶层封装为 `LibFile` 对象，提供了读取、解析和修改库文件的方法，包含名称和 PVT 信息作为公共属性。
 - 但仍然使用过程化方法循环迭代读取库文件名称和 PVT 信息。
- 按照 C++ 标准库、第三方库和本地库的顺序重新排列了 `#include` 语句。

1.3.3 2025-02-01

- 使用面向对象编程（OOP）重构了原先用 C 语言循环遍历的代码，将简单属性的读取封装为 `LibAttribute` 类，
 - 并将函数返回的 `char *` 类型的属性值转换为 `std::string` 类型，便于外部使用。
- 将属性迭代过程封装为 `AttributesIterator` 类，在析构函数中处理迭代退出，实现资源获取即初始化（RAII）。

1.3.4 2025-02-10

- 将组的迭代过程封装为 `GroupsIterator` 类，同样在析构函数中处理迭代退出。
- 封装了 `LibGroup` 类，提供了读取组名、类别、属性、子组的方法。
- 将类的定义和实现分离，将类的实现放在 `src` 目录下，将类的定义放在 `include` 目录下，修改了 `CMakeLists.txt` 自动收集文件，便于模块化编译。

1.3.5 2025-02-11

- 类的头文件改为 `hpp` 后缀，修改了 `CMakeLists.txt` 中的文件收集规则，CMake 编译时间戳更新。
- 为 `LibFile` 类添加了输出同名 json 格式文件的方法 `LibFile::writeJsonToFile`，目前能输出 PVT、`cell_name`、`cell_footprint`、`cell_area` 信息。
- [x] voltage 浮点数误差未解决。

1.3.6 2025-02-14

- 在 `json_utils.cpp` 中，实现了以 JSON 数据结构循环迭代存储库 (lib) 信息的功能，具体包括 `generateCellJson`, `generatePinJson`, `generatePowerJson`, `generateLutJson` 等函数，并添加了同名的 `hpp` 头文件。
- [x] 时序 (timing) 信息解析功能待完成。
- 新增辅助函数 `parseStringToVector`，该函数可将以逗号分隔的字符串解析为浮点数向量，以便于读取 Look Up Table (LUT)。
- 为 `LibFile` 对象添加了私有属性 `lib_json_`，用于存储库 (lib) 的 JSON 对象。
- 封装了用于迭代复杂属性值的 `ValuesIterator` 类。
- 修改了 `LibAttribute::isComplex()` 函数的返回值类型，由原类型变更为 `bool` 类型。
- 重写了 `LibGroup::getName()` 函数，使其直接返回 `std::string` 类型的组名字。
- 重构了迭代器使用代码。移除了中间变量类似 `si2drGroupsIdT sub_groups` 的声明步骤。直接使用 `lib_group.getGroups()` 的返回值进行初始化 `GroupsIterator`，提升了代码的可读性和简洁性。

1.3.7 2025-02-15

- 实现 `generateTimingJson` 函数，用于全面解析时序信息。

1.3.8 2025-02-17

- 简化了迭代器在外部的调用流程，删除所有迭代器的 `begin()` 方法，改为在构造函数内部初始化。

1.3.9 2025-02-18

- 新建 `LibFile::mono()` 方法，准备实现库文件输出引脚的 values 单调递增性检查。
- 修改了 `setupLogger()` 函数，将日志文件名可作为参数传入，便于根据不同工作模式切换日志文件。

1.3.10 2025-02-19

- 修改了 `mode == "mono"` 情况下的逻辑，使用 C++17 引入的 `std::filesystem::exists()` 函数检查同名 JSON 文件是否存在，如果 JSON 文件不存在，则先调用库文件解析功能，生成 JSON 文件，然后再执行单调性检查。
- 实现了 `LibFile::mono()` 方法，检查 `cell_rise`、`cell_fall`、`rise_transition`、`fall_transition` 这四种 timing 信息，针对其 LUT 数据结构，检查其 value 值在表格的每一行是否保持单调递增。

1.3.11 2025-02-20

- `LibFile::mono()` 方法增加结果统计功能，在程序运行的最后输出单调性检查中 passed (通过) 和 failed (失败) 的 cell 数量，输出格式参考 `liberate_lv` 工具的风格。
- 以 `sl018_ff_3.96_-40.lib` 为例，与 `liberate_lv` 工具对比测试，结果均为 205 out of 559 cells failed。
- 对于非单调信息警告，增加输出 when 信息，方便用户查看具体位置。

1.3.12 2025-02-25

- 重构命令行参数解析，使用 `CLI11` 库的子命令，在每个回调函数中实现 `parse`、`mono` 功能。

1.3.13 2025-02-26

- 子命令可自定义输出文件名、日志文件名，并设置了相应的默认值。
- 改为在 `Logger` 初始化时打印版本信息，避免在 `-h`、`-v` 时多余打印。
- `LibFile::mono()` 方法增加对 `input_slew` 的单调性检查，如果在输入时有指定，就检查 value 矩阵的每一列是否单调。经过测试，`tcbn65lpbc.lib` 输出与 `liberate_lv` 工具一致。

1.3.14 2025-02-27

- 多线程并行尝试，采用 GDB 调试：
 - `si2dr_liberty` 库在解析 `Liberty` 文件时，可能使用了共享的数据结构（例如哈希表、字符串表）来存储解析结果或中间数据。
 - 在多线程并行解析多个文件时，不同的线程同时调用 `si2dr_liberty` 库的函数，并发地访问和操作这些共享数据结构。
 - `si2dr_liberty` 库可能没有采取足够的线程同步措施来保护这些共享数据结构，导致了数据竞争。
 - 数据竞争最终导致了内存损坏，使得 `strcmp` 函数在后续操作中访问了无效内存，触发了段错误。

1.3.15 2025-02-28

- 修改 `LibFile` 的构造函数与析构函数，`read` 方法改为私有，移入 `parse` 方法。将 `si2dr` 初始化与销毁放在 `parse` 方法中，使得 `mono` 方法与 `si2dr` 库解耦，从而实现多线程并行验证单调性。
- [x] 顶层 `si2drPIGetGroups` 未退出的 bug: `WARNING: si2drPIQuit: GetGroups called 1 more times than IterQuit`，内存泄漏？
- 学到了可以在 `CMakeLists.txt` 中使用 `set(CMAKE_BUILD_TYPE Release)` 减少编译器优化程度，配合 GDB 等调试工具检查堆栈信息。

1.3.16 2025-03-01

- `sub_groups_iter` 是在每次 `for` 循环的迭代中声明的局部变量，在每次迭代结束后其作用域结束，析构函数被调用，释放相关资源。所以 `si2drPIQuit` 未发现内存泄漏。
- 顶层 `group_iter` 的析构函数发生在 `parse` 方法结束时，所以 `parse` 方法末尾的 `si2drPIQuit` 检测到了顶层 `groups` 未退出的情况。
- 综上，`si2drPIQuit` 的警告是由于当时顶层 `groups` 未退出导致的，在 `parse` 方法结束后能正确释放，不会造成实际上的内存泄漏。
- [] 顺序执行 `parse`，`log` 输出仍存在问题：解析第二第三个库的时候会多余打印之前库的PVT信息，时间上也比单独解析一个库的总是要慢 (`parse` 6s 左右，`mono` 1s)。并行执行 `mono`，`log` 输出会集中到最后一个库的文件。
- 给 `LibFile` 类的私有属性添加了初始值，避免了未初始化的问题，吗？

1.3.17 2025-03-07

- 实现了 `compare` 子命令的解析，能够对两个库文件依次进行解析，并生成对应的 JSON 文件。
- [] 仍然存在日志重复输出的问题，多文件处理时数据可能会相互干扰。

1.3.18 2025-03-10

- 新增 `supercell` 子命令和 `LibFile::supercell` 方法，用于生成超级单元以供验证。
- 集成了 `zlibboost` 子命令，支持调用开源库特征化工具 `zlibboost`。已与开发者取得联系。
- 调整了子命令输出文件的位置为当前目录，避免文件混淆。
- 添加了 `clear` 子命令，方便清理生成的 JSON 文件和日志文件。

1.3.19 2025-03-14

- 添加了 `tabulate/table` 库，通过 CMake FetchContent 进行管理，用于生成格式化的表格输出。
- 新建了 `compare.cpp` 和 `compare.hpp` 文件，创建了 `LibraryComparator` 类，用于比较两个库文件的差异。目前实现了读取参考库 JSON 文件的功能。

1.3.20 2025-03-15

- 使用 `std::filesystem::path` 重构了 `LibFile` 类的文件路径管理，方便文件路径的拼接和处理。`LibFile` 类添加了成员变量 `basename_`、`filename_`、`libname_`、`jsonname_` 和 `loggername_`，用于存储文件名、库名、JSON 文件名和日志文件名。
- 通过作用域 (scope) 管理 `LibFile::parse` 方法中的顶层迭代器的生命周期，提前结束并调用 `si2drPIQuit` 保证资源释放，解决了分析多个库时日志重复输出和数据保存错误的问题。
- `clear` 子命令增加了删除 `.map` 和 `.md` 文件的功能。
- 实现了 `mono` 子命令的多线程并行。首先检查是否是多文件输入，如果是，就顺序检查是否准备好了每个文件的 JSON 文件，否则顺序进行多文件库解析。在所有文件的 JSON 准备就绪后，根据文件数量创建线程池，每个线程负责一个库的单调性检查。
- 重构了 `spdlog` 日志初始化，返回 `logger` 对象而不是设置全局默认 `logger`。保留名为 `APP_NAME` 的全局 `logger`，用于输出总体提示信息。
- 为 `LibFile` 类添加了独立的成员变量 `logger`，用于记录每个库文件的日志信息。
- 完成了 `supercell` 子命令的多线程并行，处理逻辑与 `mono` 子命令类似。
- `LibraryComparator::generateReport()` 方法测试了 `tabulate` 库的 markdown 表格输出功能，完善了报告的头部信息输出，包括参考库、比较库、相对容差、软件信息、报告生成时间和图例说明。

1.3.21 2025-03-16

- 针对 `compare` 子命令，增加了对报告文件名的检查，确保其以 `.md` 或 `.txt` 结尾。若不符合，则给出警告并自动添加 `.md` 后缀。
- 进一步完善了 `LibraryComparator::generateReport()` 方法。现在，该方法能够遍历比较库中的所有 `cell`，并在参考库中查找对应的 `cell`。如果找到，则调用 `LibraryComparator::compareCell()` 方法进行比较；否则，会记录 `cell` 未找到的警告信息。
- 新增了 `LibraryComparator::compareCell()` 方法，用于比较两个库中同名 `cell` 的输出引脚。该方法会遍历比较库 `cell` 的每个输出引脚，在参考库 `cell` 中寻找同名引脚。如果找到，则调用 `comparePin` 函数比较引脚；如果未找到，则记录警告信息。如果比较库 `cell` 没有输出引脚，则会记录一条信息级别的日志。
- 新增了 `LibraryComparator::comparePin()` 方法，用于比较两个库中同名 `cell` 的同名引脚。它会遍历比较库 `pin` 的每个 `timing arc`，并在参考库 `pin` 中查找相同类型的 `timing arc`。如果找到，则调用 `compareTimingArc` 函数进行比较；如果未找到，则记录警告信息。如果比较库 `pin` 没有 `timing arc`，则记录一条信息级别的日志。

- 修改了 `LibraryComparator::compareTimingArc()` 方法，用于比较两个 timing arc JSON 对象。该方法首先记录正在比较的 timing arc 的类型，然后提取 "related_pin" 属性。接着，它遍历预定义的 timing arc 名称列表 ("cell_rise", "cell_fall", "rise_transition", "fall_transition")，并在比较库中查找这些 timing arc。如果找到，则在参考库中查找对应的 timing arc，并调用 `compareLut` 方法进行比较。如果参考库中未找到对应的 timing arc，则记录警告信息。
- 修改了 `LibraryComparator::compareLut()` 方法（原 `compareValue` 方法，已更正命名），用于比较两个 JSON 对象中具有相同名称的 value。它首先提取两个 JSON 对象中的 "index_1" 和 "index_2" 数组，如果这两个数组不相等，则记录错误信息并返回。如果索引匹配，则比较 LUT 中的实际 value 值，并根据相对容差 (reltol) 和绝对容差 (abstol) 标记差异。
- 重新设计了层级比较方法，现在可以逐层级地传入 `cell_name`、`pin_name`、`timing_type`、`related_pin` 和 `arc_name`，以便在最内层制表或提示时能够准确输出层级信息。
- 修复了 `LibraryComparator::compareValue()` 方法的命名错误，已更正为 `LibraryComparator::compareLut`
- 已经可以输出表格，数据数量和商用工具结果一致，但格式还需要进一步调整。
- 新增了 `abstol` 参数，默认值为 0.002ns，用于设置绝对容差，与库验证工具保持一致。

1.3.22 2025-03-17

- 新增 `verilog` 和 `spice` 子命令，用于生成 Verilog 和 SPICE 代码。
- 完善 `LibraryComparator` 报告生成：
 - 为每个 cell 增加表头，仅在存在表格数据时输出表格，避免冗余输出。
 - 增加 cell 结果统计功能，输出结果统计表格，目前仅支持 Timing 中 Delay 的比较。

1.3.23 2025-03-18

- 优化统计表格，包含 | Cell Name | Data Type | Failed Count | Avg Diff | Avg Diff% | Max Diff | Max Diff% | Outliers |。
- 优化报告对比表格，包含 | Pin Name | Reference | Comparison | Diff | Diff % | Type | Arc Name | Row # | Index_1 | Column # | Index_2 | Note |
- 报告表格的 stdout 输出增加表头加粗、黄色的格式化。
- 增加 `si2drSimpleAttrGetBooleanValue` 封装，用于获取布尔类型的属性值，以判断引脚是否是时钟引脚。
- `LibFile::mono()` 增加对 `min_pulse_width` 的单调性检查：
 - 针对包含 "input_pins" 的 cell，遍历每个输入引脚。
 - 如果引脚是时钟引脚，则检查其 "timing_arcs"。

- 针对 "timing_type" 为 "min_pulse_width" 的 timing arc, 对 "rise_constraint" 和 "fall_constraint" 调用 checkTimingArcMonotonicity 进行单调性检查。
- checkTimingArcMonotonicity 函数日志区分有无 when 信息, 输出不同日志。
- checkTimingArcMonotonicity 核心比较功能取消了等于的情况, 相等情况默认为通过。
- checkTimingArcMonotonicity 核心比较功能增加了判断 related_pin 是否等于当前 pin, 不等于则跳过。最终改为: 如果矩阵中当前的值小于前一个值, 并且当前引脚不是相关引脚, 或者当前值和前一个值都为零, 那么就认为这些值不是单调递增的。

1.3.24 2025-03-19

- 完善 supercell 方法, 如果有有时钟信号引脚, 链式长度被设为 1, 再生成超级单元。
- 解决 voltage 浮点数误差问题, `std::round(voltage_ * 100) / 100.0` 保留两位小数。

1.3.25 2025-03-21

- 新增 func 子命令, 用于检查库或者 Verilog 文件的逻辑等价性。

1.3.26 2025-03-24

- 调研了 C++ 操作 Verilog 相关的库, 找到了一个实用的 Verilog 库: `slang`, 它可以解析出抽象语法树 (AST)。已修改 CMakeLists 文件, 将 slang 集成到项目中, 目前正在研究其 API。

1.3.27 2025-03-25

- 新增 `verilog_utils.cpp` 和 `verilog_utils.hpp` 文件, 并创建了 `VerilogVisitor` 类, 用于自定义遍历 Verilog AST。
- 验证了 slang 对不同类型逻辑单元的解析能力:
 - 简单组合逻辑 (如 AND2D0)、复杂时序逻辑 (如 CMPE42D1) 和基本时序逻辑 (如 DFQD1) 均能被正确解析。
 - 用户自定义原语 (UDP), 例如 `tsmc_dff`, 会被错误地识别为模块实例化, 并产生 "Invalid instance declaration" 警告。
 - 成功提取了模块名、端口方向 (input、output) 及名称。
 - 能够解析命名端口连接的层次化实例化, 并获取模块名、实例名称和端口映射关系。
 - 能够解析门级原语实例化, 并获取门类型、实例名称、输出端口和输入端口。

1.3.28 2025-03-26

- 进一步完善了对 UDP (如 `tsmc_dff`) 端口映射关系的解析, 但其内部 `Entry` 尚未处理。
- 尝试了通过继承 `slang::syntax::SyntaxRewriter` 类创建自定义类 `CellExtractor`, 实现了只保留目标模块、删除其他模块, 并将修改后的语法树另存为新文件。使用 `slang::syntax::SyntaxPrinter::printFile(*tree)` 方法可以将 Verilog 代码输出到文件。
- 创建了 `CellPrinter` 类, 继承自 `slang::syntax::SyntaxVisitor` 类。当访问到目标模块时, 使用 `module.toString()` 方法也能将 Verilog 代码输出到文件, 但发现输出结果中空行会被删除。
- 修改了 `LibFile::supercell()` 方法, 该方法可以根据输入的 `cell_names` 生成指定的 `supercell`, 并在遇到未找到的单元时提示警告信息。
- 优化了时钟引脚的处理方式, 现在时钟引脚不再被视为 `supercell` 的输入引脚, 而是仅记录为时序单元, 并跳过存入 `input_pins` 集合的步骤。
- 引入了 Doxygen 文档生成工具, 用于可视化分析项目, 并生成了 Doxygen HTML 文档和 LaTeX 参考手册。
- [x] 尚未解决手册中文显示不正确的问题, 可能需要自定义 LaTeX 头文件。

1.3.29 2025-03-27

- **重大失误!** 误操作 Git 分支, 导致部分代码修改丢失。务必牢记: **** 及时提交代码! ****
- 实现了 `LibFile::verilog` 方法的核心流程:
 - 首先调用 `this->supercell()` 生成超级单元。
 - 读取 `.map` 文件, 提取单元映射关系到 `std::pair<std::string, std::string>` 中 (原始单元名 -> 超级单元名/模块名)。
 - 从 `lib_json_` 中提取单元的输入/输出端口信息。
 - 根据是否存在时钟引脚, 确定 Verilog 模块中 `instance` 的生成数量 (时序逻辑为 1, 组合逻辑等于 `chain length`)。
 - 通过字符串操作生成 ANSI 风格的端口列表, 并与模块名组合成完整的 `fullModuleText`。
 - 使用 `slang::syntax::SyntaxTree::fromText` 从 `fullModuleText` 生成语法树。
 - 将语法树传递给 `ModuleRewriter` 类, 使用其 `transform` 方法遍历模块成员, 并通过 `handle()` 方法进行处理。
 - 最后, 使用 `slang::syntax::SyntaxPrinter::printFile(*tree)` 将处理后的语法树输出到文件。
- `LibFile::verilog` 目前能够正确输出模块名和 ANSI 风格的端口声明。

1.3.30 2025-03-28

- 实现了在模块成员列表中插入新节点的功能。具体而言，在 `ModuleRewriter::handle(const slang::syntax::ModuleDeclarationSyntax &module)` 方法中，通过 `insertAtBack(module.members, newDataNode)` 函数，成功地在模块成员的末尾插入了链式单元所需的中间变量声明，例如 `wire OP_i;`。
- 成功地在模块内部添加了关键连接网络变量。通过解析 `CELLNAME__X#__CRITICALPORT__OUTPUTPORT` 格式的字符串，提取出关键输入端口 (critical input port) 和输出端口 (OUTPUTPORT)，用于后续的实例化端口连接。
- 增加了用于处理多输出接口的中间网络变量 `P__UNUSEOUTPUTPORT`，用于连接不需要考察的输出端口。
- 成功地在模块内添加了模块实例化所需的模块名和实例名称，完成了端口映射关系的连接处理。
- 验证了代码对简单组合逻辑单元（如 `INVD0` 反相器）和复杂组合逻辑单元（如 `FA1D0` 全加器）的输出能力，结果均符合预期。
- 为 `ModuleRewriter` 类添加了私有属性 `std::map<std::string, std::string> portInfoMap_`，用于存储端口映射关系，将端口名映射到其方向 (input/output)。
- 完成了实例化端口连接的细节处理：
 - 如果是关键输入端口且为第一个实例，则直接连接到输入端口；否则，连接到 `OP_(i-1)` 这样的中间网络变量。
 - 如果是关键输出端口，且为最后一个实例，则连接到输出端口；否则，连接到 `OP_i` 中间网络变量。
 - 如果是其他输出端口且不是最后一个实例，则连接到 `P_i__portName` 这样的中间网络变量；否则，连接名等于端口名。
 - 其余输入端口，连接名等于端口名。
- 文档方面，GitHub 远程仓库已开放为公开访问，并配置了 GitHub Pages。可以通过 <https://cedar17.github.io/ZlibValidation/> 访问项目文档，该页面包含 Doxygen 生成的 HTML 文档和仓库地址的链接。
- 配置了 GitHub Actions，实现了 Doxygen 文档和 Graphviz 图的自动化构建，以及 LaTeX 参考手册的自动编译（暂不支持中文）。每次在 `dev` 和 `main` 分支的提交都会触发文档和参考手册的生成，并发布到 `gh-pages` 分支。
- 将第三方库放置到 `include_3rd_party` 目录下，方便管理。修改了 `CMakeLists` 文件，将第三方库的头文件路径添加到 `include_directories` 中。修改了 `Doxyfile` 文件，将第三方库的头文件路径添加到 `INPUT` 中，便于文档生成工具理解第三方库函数、类的关系及使用方法。

1.3.31 2025-03-29

- 修复了 Doxygen 生成 LaTeX 参考手册时中文显示不正确的问题。通过修改 Doxyfile 文件，添加 `EXTRA_PACKAGES = ctex` 选项，并指定 `lualatex` 作为编译器，成功解决了中文显示问题。同时，从 `include_3rd_party` 目录中移除了 `Allsyntax.h` 文件，避免了由于 LaTeX 文档过大导致的 "TeX capacity exceeded, sorry [main memory size=5000000]" 内存不足问题。
- 实现了 Verilog 文件的顶层模块生成功能，采用非 ANSI 风格的端口声明方式，格式与 `liberate_lv` 工具生成的完全一致。实现过程如下：
 - 首先将各模块的 Verilog 代码存放到临时文件中。
 - 在处理模块的过程中，使用 `std::vector<std::string>` 收集每个模块的 `input_pins` 和 `output_pins`。
 - 遍历所有模块名称列表，拼接模块名和引脚名，生成顶层模块的 `all_input_ports` 和 `all_output_ports`。
 - 使用字符串拼接方式构建 `validate_top` 模块的完整代码。
 - 最后将临时文件内容与顶层模块代码合并，输出到最终的 Verilog 文件中。
- 更改 `deploy-docs.yml`，修复了 GitHub Action 生成的 LaTeX 参考手册中的图像显示问题。具体步骤如下：
 - 设置系统时区为 Asia/Shanghai (东八区)。
 - 安装 miniconda 并更新 conda。
 - 通过 conda 安装 1.9.5 版本的 Doxygen，并安装 graphviz 以解决依赖关系，确保每个 dot 图都能正确编译出 pdf 文件。
 - 使用 doxygen 命令生成 HTML 文档和 tex 文件。
 - 采用 action marketplace 提供的 `xu-cheng/texlive-action@v2` 安装全量的 texlive2024。
 - 进入 `./doc_doxygen/latex` 目录，运行 `make` 命令使用 `lualatex` 编译 tex 文件，生成 pdf 文件。
 - 使用 `JamesIves/github-pages-deploy-action@v4` 将生成的 pdf 文件上传到 GitHub Pages 发布，并保留 `README.md` 和 `index.html` 文件。

1.3.32 2025-03-30

- 完成了 `LibFile::logic(const std::string &cell_name)` 方法，该方法返回指定 `cell_name` 所有输出引脚对应的逻辑表达式，以 `std::map<std::string, std::string>` 的形式返回，存储输出引脚名称到其逻辑表达式字符串的映射关系。
- 实现了 `func` 子命令的基本框架，用于检查库或 Verilog 文件的逻辑等价性。
 - 完善了 `funcLibFile` 函数，能够处理 Liberty 文件和 Verilog 文件作为参考和比较对象。
 - 增加了文件类型判断，根据文件扩展名选择不同的处理方式。

- 实现了对 .lib 文件的解析，并提取指定 cell 的逻辑表达式。
- 目前仅支持 Liberty 文件的逻辑表达式提取，Verilog 文件的逻辑表达式提取功能尚未完成 (TODO)。
- 增加了对 report 文件名的检查，确保其以 .md 或 .txt 结尾。若不符合，则给出警告并自动添加 .md 后缀。
- 实现了从 Liberty 文件中提取逻辑表达式的功能，使用了 `LibFile::logic` 方法。
- 增加了日志输出，记录每个 cell 的逻辑表达式提取和比较过程。
- 逻辑比较部分目前为空，待实现 (TODO)。

1.3.33 2025-03-31

- 实现了从 Verilog 结构网表中提取逻辑表达式的功能，使用了 `LogicExtractor` 类。
 - 设计了如何使用 Slang 库从 Verilog 结构网表中提取逻辑表达式的方案：
 - * 使用 Slang 解析 Verilog 文件获取语法树 (AST)。
 - * 遍历 AST 定位目标模块。
 - * 在目标模块内构建网表的内部表示，识别输入、输出、线网及门级连接关系。
 - * 对每个输出端口，递归反向追踪门和线网直至到达主输入端口。
 - * 在回溯过程中根据遇到的门类型构建逻辑表达式。
 - * 应用记忆化技术避免对相同线网/信号的重复计算。
 - 创建了 `LogicExtractor` 类，用于从 Verilog 代码中提取逻辑表达式。
 - * `LogicExtractor` 类使用 Slang 库解析 Verilog 文件，并构建目标模块的网表表示。
 - * 实现了 `handle(const slang::syntax::ModuleDeclarationSyntax &module)` 方法，用于定位目标模块，并初始化内部数据结构。
 - * 实现了 `handle(const slang::syntax::PortDeclarationSyntax &portDecl)` 方法，用于提取端口信息，包括端口方向和名称。
 - * 实现了 `handle(const slang::syntax::NetDeclarationSyntax &netDecl)` 方法，用于提取线网信息。
 - * 实现了 `handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst)` 方法，用于提取门级单元信息，包括门类型、输入和输出信号。
 - * 实现了 `getLogicExpressions()` 方法，用于根据提取的网表信息，递归地推导每个输出端口的逻辑表达式。
 - * 实现了 `deriveLogicRecursive(const std::string &signalName)` 方法，用于递归地推导指定信号的逻辑表达式。
 - * 实现了 `formatExpression(const GateInfo &gateInfo, const std::vector<std::string> &inputExprs)` 方法，用于根据门类型和输入表达式，生成逻辑表达式字符串。
 - 实现了 `extractAndPrintNetlistInfo(const std::string &verilog_file, const std::string &cell)` 函数，用于提取并打印网表信息，包括输入、输出、线网和门级单元。

- 实现了 `extractLogicFromVerilog(const std::string &verilog_file, const std::string &cell)` 函数，用于从 Verilog 文件中提取指定 cell 的逻辑表达式，并以 `std::map<std::string, std::string>` 的形式返回。

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

slang	23
slang::parsing	23
slang::syntax	23
slang::syntax::detail	28

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AttributesIterator	31
slang::syntax::detail::ChangeCollection	40
slang::syntax::DeferredSourceRange	46
GateInfo	49
GroupsIterator	51
iterator_facade	
slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >	54
LibAttribute	58
liberty_value_data	63
LibFile	64
LibGroup	82
LibraryComparator	85
si2drExprT	124
si2OID	127
std::span	
slang::syntax::SyntaxList< T >	130
slang::syntax::TokenList	182
slang::syntax::detail::SyntaxChange	128
slang::syntax::detail::RemoveChange	113
slang::syntax::detail::ReplaceChange	114
slang::syntax::SyntaxNode	138
slang::syntax::SyntaxListBase	134
slang::syntax::SeparatedSyntaxList< T >	115

slang::syntax::SyntaxList< T >	130
slang::syntax::TokenList	182
slang::syntax::SyntaxPrinter	147
slang::syntax::SyntaxTree	164
slang::syntax::SyntaxVisitor< TDerived >	179
slang::syntax::SyntaxRewriter< CellExtractor >	155
CellExtractor	35
slang::syntax::SyntaxRewriter< ModuleRewriter >	155
ModuleRewriter	104
slang::syntax::SyntaxRewriter< TDerived >	155
slang::syntax::SyntaxVisitor< CellExtractor >	179
slang::syntax::SyntaxVisitor< CellPrinter >	179
CellPrinter	38
slang::syntax::SyntaxVisitor< LogicExtractor >	179
LogicExtractor	95
slang::syntax::SyntaxVisitor< ModuleRewriter >	179
slang::syntax::SyntaxVisitor< VerilogVisitor >	179
VerilogVisitor	192
ValuesIterator	188
std::variant	
slang::syntax::ConstTokenOrSyntax	43
slang::syntax::TokenOrSyntax	186
slang::syntax::PtrTokenOrSyntax	109

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AttributesIterator	31
CellExtractor	35
CellPrinter	38
slang::syntax::detail::ChangeCollection	40
slang::syntax::ConstTokenOrSyntax	
A token or a constant syntax node	43
slang::syntax::DeferredSourceRange	46
GateInfo	49
GroupsIterator	51
slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >	54
LibAttribute	58
liberty_value_data	63
LibFile	64
LibGroup	82
LibraryComparator	85
LogicExtractor	95
ModuleRewriter	104
slang::syntax::PtrTokenOrSyntax	
A token pointer or a syntax node	109
slang::syntax::detail::RemoveChange	113
slang::syntax::detail::ReplaceChange	114
slang::syntax::SeparatedSyntaxList< T >	115
si2drExprT	124
si2OID	127

slang::syntax::detail::SyntaxChange	128
slang::syntax::SyntaxList< T >	
A syntax node that represents a list of child syntax nodes	130
slang::syntax::SyntaxListBase	
A base class for syntax nodes that represent a list of items	134
slang::syntax::SyntaxNode	
Base class for all syntax nodes	138
slang::syntax::SyntaxPrinter	147
slang::syntax::SyntaxRewriter< TDerived >	155
slang::syntax::SyntaxTree	164
slang::syntax::SyntaxVisitor< TDerived >	179
slang::syntax::TokenList	
A syntax node that represents a list of child tokens	182
slang::syntax::TokenOrSyntax	
A token or a syntax node	186
ValuesIterator	188
VerilogVisitor	192

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

include/ compare.hpp	197
include/ iterators.hpp	199
include/ json_utils.hpp	200
include/ lib_attribute.hpp	210
include/ lib_file.hpp	211
include/ lib_group.hpp	213
include/ verilog_utils.hpp	215
include/ version.h	221
include_3rd_party/ si2dr_liberty.h	223
include_3rd_party/ SyntaxNode.h	
Base class and utilities for syntax nodes	259
include_3rd_party/ SyntaxPrinter.h	
Support for printing syntax nodes and tokens	266
include_3rd_party/ SyntaxTree.h	
Top-level parser interface	268
include_3rd_party/ SyntaxVisitor.h	
Syntax tree visitor support	271
src/ compare.cpp	276
src/ iterators.cpp	282
src/ json_utils.cpp	283
src/ lib_attribute.cpp	297
src/ lib_file.cpp	298
src/ lib_group.cpp	311
src/ main.cpp	311
src/ verilog_utils.cpp	332

Chapter 6

Namespace Documentation

6.1 slang Namespace Reference

Namespaces

- namespace [parsing](#)
- namespace [syntax](#)

6.2 slang::parsing Namespace Reference

6.3 slang::syntax Namespace Reference

Namespaces

- namespace [detail](#)

Classes

- struct [ConstTokenOrSyntax](#)
A token or a constant syntax node.
- class [DeferredSourceRange](#)
- struct [PtrTokenOrSyntax](#)
A token pointer or a syntax node.
- class [SeparatedSyntaxList](#)
- class [SyntaxList](#)
A syntax node that represents a list of child syntax nodes.

- class [SyntaxListBase](#)
A base class for syntax nodes that represent a list of items.
- class [SyntaxNode](#)
Base class for all syntax nodes.
- class [SyntaxPrinter](#)
- class [SyntaxRewriter](#)
- class [SyntaxTree](#)
- class [SyntaxVisitor](#)
- class [TokenList](#)
A syntax node that represents a list of child tokens.
- struct [TokenOrSyntax](#)
A token or a syntax node.

Functions

- SLANG_EXPORT [SyntaxNode](#) * [clone](#) (const [SyntaxNode](#) &node, BumpAllocator &alloc)
Performs a shallow clone of the given syntax node.
- SLANG_EXPORT [SyntaxNode](#) * [deepClone](#) (const [SyntaxNode](#) &node, BumpAllocator &alloc)
Performs a deep clone of the given syntax node.
- template<std::derived_from< [SyntaxNode](#) > T>
T * [clone](#) (const T &node, BumpAllocator &alloc)
Performs a shallow clone of the given syntax node.
- template<std::derived_from< [SyntaxNode](#) > T>
T * [deepClone](#) (const T &node, BumpAllocator &alloc)
Performs a deep clone of the given syntax node.
- template<typename T >
[SyntaxList](#)< T > * [deepClone](#) (const [SyntaxList](#)< T > &node, BumpAllocator &alloc)
- template<typename T >
[SeparatedSyntaxList](#)< T > * [deepClone](#) (const [SeparatedSyntaxList](#)< T > &node, BumpAllocator &alloc)
- [TokenList](#) * [deepClone](#) (const [TokenList](#) &node, BumpAllocator &alloc)

6.3.1 Function Documentation

6.3.1.1 clone() [1/2]

```
SLANG_EXPORT SyntaxNode * slang::syntax::clone (
    const SyntaxNode & node,
    BumpAllocator & alloc )
```

Performs a shallow clone of the given syntax node.

All members will be simply copied to the new instance. The instance will be allocated with the provided allocator. Here is the caller graph for this function:



6.3.1.2 clone() [2/2]

```
template<std::derived_from< SyntaxNode > T>
T * slang::syntax::clone (
    const T & node,
    BumpAllocator & alloc )
```

Performs a shallow clone of the given syntax node.

All members will be simply copied to the new instance. The instance will be allocated with the provided allocator.

Definition at line 233 of file [SyntaxNode.h](#).

Here is the call graph for this function:

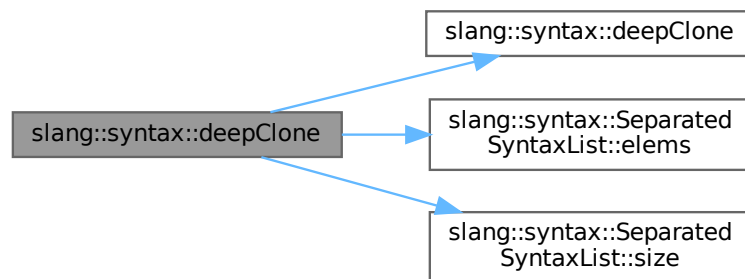


6.3.1.3 deepClone() [1/5]

```
template<typename T >
SeparatedSyntaxList< T > * slang::syntax::deepClone (
    const SeparatedSyntaxList< T > & node,
    BumpAllocator & alloc )
```

Definition at line 477 of file [SyntaxNode.h](#).

Here is the call graph for this function:



6.3.1.4 deepClone() [2/5]

```
template<typename T >
SyntaxList< T > * slang::syntax::deepClone (
    const SyntaxList< T > & node,
    BumpAllocator & alloc )
```

Definition at line 343 of file [SyntaxNode.h](#).

Here is the call graph for this function:



6.3.1.5 deepClone() [3/5]

```
SLANG_EXPORT SyntaxNode * slang::syntax::deepClone (
    const SyntaxNode & node,
    BumpAllocator & alloc )
```

Performs a deep clone of the given syntax node.

All members will be cloned recursively to create a complete new copy of the syntax tree. All cloned instances will be allocated with the provided allocator. Here is the caller graph for this function:



6.3.1.6 deepClone() [4/5]

```
template<std::derived_from< SyntaxNode > T>
T * slang::syntax::deepClone (
    const T & node,
    BumpAllocator & alloc )
```

Performs a deep clone of the given syntax node.

All members will be cloned recursively to create a complete new copy of the syntax tree. All cloned instances will be allocated with the provided allocator.

Definition at line 243 of file [SyntaxNode.h](#).

Here is the call graph for this function:



6.3.1.7 deepClone() [5/5]

```
TokenList * slang::syntax::deepClone (
    const TokenList & node,
    BumpAllocator & alloc ) [inline]
```

Definition at line 488 of file [SyntaxNode.h](#).

6.4 slang::syntax::detail Namespace Reference

Classes

- struct [ChangeCollection](#)
- struct [RemoveChange](#)
- struct [ReplaceChange](#)
- struct [SyntaxChange](#)

Typedefs

- using [InsertChangeMap](#) = flat_hash_map< const [SyntaxNode](#) *, std::vector< [SyntaxChange](#) > >
- using [ModifyChangeMap](#) = flat_hash_map< const [SyntaxNode](#) *, std::variant< [RemoveChange](#), [ReplaceChange](#) > >
- using [ListChangeMap](#) = flat_hash_map< const [SyntaxNode](#) *, std::vector< [SyntaxChange](#) > >

Functions

- SLANG_EXPORT std::shared_ptr< [SyntaxTree](#) > [transformTree](#) (BumpAllocator &&alloc, const std::shared_ptr< [SyntaxTree](#) > &tree, const [ChangeCollection](#) &commits, const std::vector< std::shared_ptr< [SyntaxTree](#) > > &tempTrees, const SourceLibrary *library)

6.4.1 Typedef Documentation

6.4.1.1 InsertChangeMap

```
using slang::syntax::detail::InsertChangeMap = typedef flat_hash_map<const SyntaxNode*, std::vector<SyntaxChange> >
```

Definition at line 68 of file [SyntaxVisitor.h](#).

6.4.1.2 ListChangeMap

```
using slang::syntax::detail::ListChangeMap = typedef flat_hash_map<const SyntaxNode*, std::vector<SyntaxChange>  
>
```

Definition at line 70 of file [SyntaxVisitor.h](#).

6.4.1.3 ModifyChangeMap

```
using slang::syntax::detail::ModifyChangeMap = typedef flat_hash_map<const SyntaxNode*, std::variant<RemoveChange,  
ReplaceChange> >
```

Definition at line 69 of file [SyntaxVisitor.h](#).

6.4.2 Function Documentation

6.4.2.1 transformTree()

```
SLANG_EXPORT std::shared_ptr< SyntaxTree > slang::syntax::detail::transformTree (  
    BumpAllocator && alloc,  
    const std::shared_ptr< SyntaxTree > & tree,  
    const ChangeCollection & commits,  
    const std::vector< std::shared_ptr< SyntaxTree > > & tempTrees,  
    const SourceLibrary * library )
```

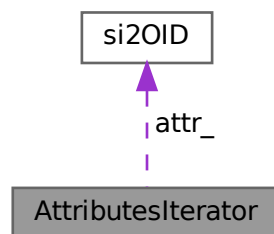

Chapter 7

Class Documentation

7.1 AttributesIterator Class Reference

```
#include <iterators.hpp>
```

Collaboration diagram for AttributesIterator:



Public Member Functions

- [AttributesIterator](#) ([si2drAttrslidT](#) attrs, [si2drErrorT](#) &err)
- [~AttributesIterator](#) ()
- void [next](#) ()
- bool [end](#) ()
- [LibAttribute](#) [get](#) ()

Private Attributes

- [si2drAttrsIdT](#) *attrs_*
- [si2drAttrIdT](#) *attr_*
- [si2drErrorT](#) & *err_*

7.1.1 Detailed Description

Definition at line 23 of file [iterators.hpp](#).

7.1.2 Constructor & Destructor Documentation

7.1.2.1 AttributesIterator()

```
AttributesIterator::AttributesIterator (
    si2drAttrsIdT attrs,
    si2drErrorT & err )
```

Definition at line 13 of file [iterators.cpp](#).

7.1.2.2 ~AttributesIterator()

```
AttributesIterator::~AttributesIterator ( )
```

Definition at line 17 of file [iterators.cpp](#).

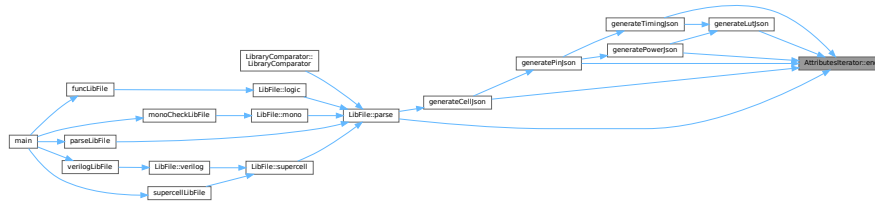
7.1.3 Member Function Documentation

7.1.3.1 end()

```
bool AttributesIterator::end ( )
```

Definition at line 20 of file [iterators.cpp](#).

Here is the caller graph for this function:

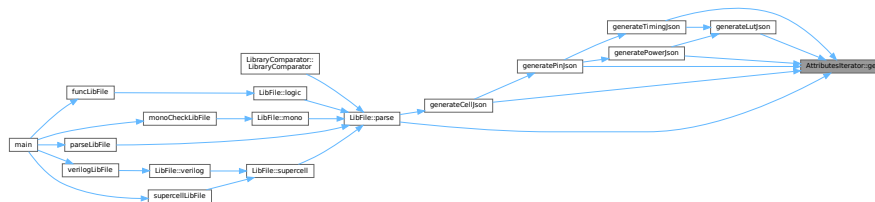


7.1.3.2 get()

```
LibAttribute AttributesIterator::get ( )
```

Definition at line 22 of file [iterators.cpp](#).

Here is the caller graph for this function:

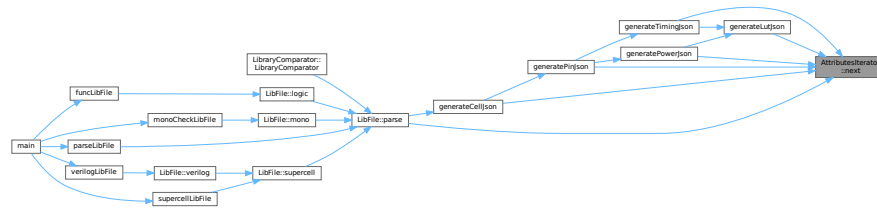


7.1.3.3 next()

```
void AttributesIterator::next ( )
```

Definition at line 19 of file [iterators.cpp](#).

Here is the caller graph for this function:



7.1.4 Member Data Documentation

7.1.4.1 attr_

`si2drAttrIdT` `AttributesIterator::attr_` [private]

Definition at line 33 of file [iterators.hpp](#).

7.1.4.2 attrs_

`si2drAttrsIdT` `AttributesIterator::attrs_` [private]

Definition at line 32 of file [iterators.hpp](#).

7.1.4.3 err_

`si2drErrorT&` `AttributesIterator::err_` [private]

Definition at line 34 of file [iterators.hpp](#).

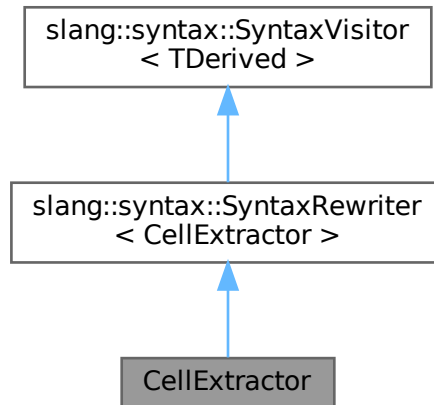
The documentation for this class was generated from the following files:

- [include/iterators.hpp](#)
- [src/iterators.cpp](#)

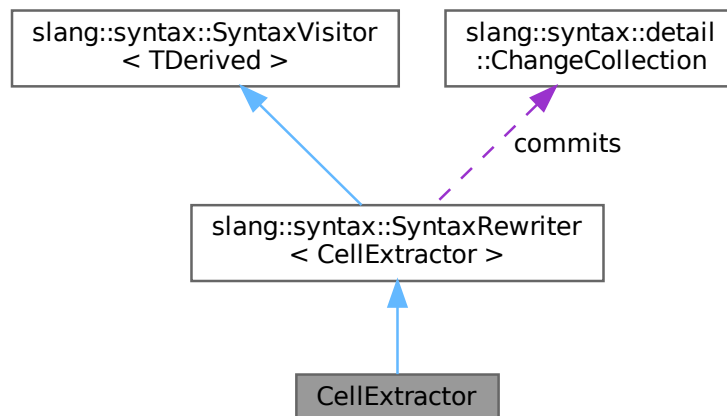
7.2 CellExtractor Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for CellExtractor:



Collaboration diagram for CellExtractor:



Public Member Functions

- `CellExtractor` (const std::string &targetCell)
- void `handle` (const slang::syntax::ModuleDeclarationSyntax &module)
- bool `foundTargetCell` () const

Private Attributes

- `const std::string & targetCell_`
- `bool foundTarget_`

Additional Inherited Members

7.2.1 Detailed Description

Definition at line 43 of file [verilog_utils.hpp](#).

7.2.2 Constructor & Destructor Documentation

7.2.2.1 CellExtractor()

```
CellExtractor::CellExtractor (  
    const std::string & targetCell ) [inline], [explicit]
```

Definition at line 45 of file [verilog_utils.hpp](#).

7.2.3 Member Function Documentation

7.2.3.1 foundTargetCell()

```
bool CellExtractor::foundTargetCell ( ) const
```

Definition at line 366 of file [verilog_utils.cpp](#).

Here is the caller graph for this function:

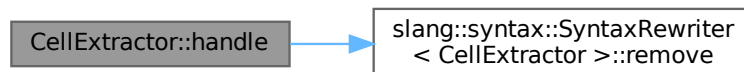


7.2.3.2 handle()

```
void CellExtractor::handle (  
    const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 350 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



7.2.4 Member Data Documentation

7.2.4.1 foundTarget_

```
bool CellExtractor::foundTarget_ [private]
```

Definition at line 52 of file [verilog_utils.hpp](#).

7.2.4.2 targetCell_

```
const std::string& CellExtractor::targetCell_ [private]
```

Definition at line 51 of file [verilog_utils.hpp](#).

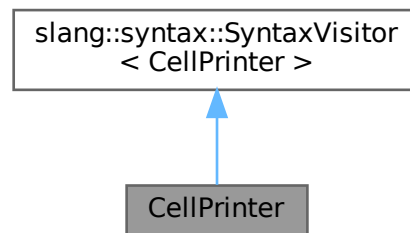
The documentation for this class was generated from the following files:

- [include/verilog_utils.hpp](#)
- [src/verilog_utils.cpp](#)

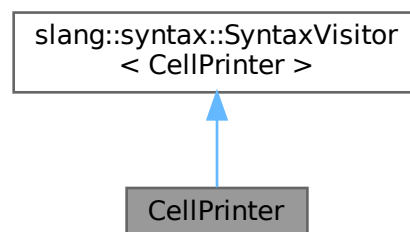
7.3 CellPrinter Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for CellPrinter:



Collaboration diagram for CellPrinter:



Public Member Functions

- [CellPrinter](#) (const std::string &targetCell, std::ostream &out)
- void [handle](#) (const slang::syntax::ModuleDeclarationSyntax &module)

Private Attributes

- const std::string & [targetCell_](#)
- std::ostream & [out_](#)
- bool [foundTarget_](#)

7.3.1 Detailed Description

Definition at line 56 of file [verilog_utils.hpp](#).

7.3.2 Constructor & Destructor Documentation

7.3.2.1 CellPrinter()

```
CellPrinter::CellPrinter (
    const std::string & targetCell,
    std::ostream & out ) [inline], [explicit]
```

Definition at line 58 of file [verilog_utils.hpp](#).

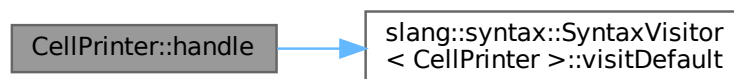
7.3.3 Member Function Documentation

7.3.3.1 handle()

```
void CellPrinter::handle (
    const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 369 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



7.3.4 Member Data Documentation

7.3.4.1 foundTarget_

```
bool CellPrinter::foundTarget_ [private]
```

Definition at line 65 of file [verilog_utils.hpp](#).

7.3.4.2 out_

```
std::ostream& CellPrinter::out_ [private]
```

Definition at line 64 of file [verilog_utils.hpp](#).

7.3.4.3 targetCell_

```
const std::string& CellPrinter::targetCell_ [private]
```

Definition at line 63 of file [verilog_utils.hpp](#).

The documentation for this class was generated from the following files:

- include/[verilog_utils.hpp](#)
- src/[verilog_utils.cpp](#)

7.4 slang::syntax::detail::ChangeCollection Struct Reference

```
#include <SyntaxVisitor.h>
```

Public Member Functions

- void [clear](#) ()
- bool [empty](#) () const

Public Attributes

- [InsertChangeMap](#) [insertBefore](#)
- [InsertChangeMap](#) [insertAfter](#)
- [ModifyChangeMap](#) [removeOrReplace](#)
- [ListChangeMap](#) [listInsertAtFront](#)
- [ListChangeMap](#) [listInsertAtBack](#)

7.4.1 Detailed Description

Definition at line 72 of file [SyntaxVisitor.h](#).

7.4.2 Member Function Documentation

7.4.2.1 clear()

```
void slang::syntax::detail::ChangeCollection::clear ( ) [inline]
```

Definition at line 79 of file [SyntaxVisitor.h](#).

Here is the caller graph for this function:



7.4.2.2 empty()

```
bool slang::syntax::detail::ChangeCollection::empty ( ) const [inline]
```

Definition at line 87 of file [SyntaxVisitor.h](#).

Here is the caller graph for this function:



7.4.3 Member Data Documentation

7.4.3.1 insertAfter

[InsertChangeMap](#) slang::syntax::detail::ChangeCollection::insertAfter

Definition at line 74 of file [SyntaxVisitor.h](#).

7.4.3.2 insertBefore

[InsertChangeMap](#) slang::syntax::detail::ChangeCollection::insertBefore

Definition at line 73 of file [SyntaxVisitor.h](#).

7.4.3.3 listInsertAtBack

[ListChangeMap](#) slang::syntax::detail::ChangeCollection::listInsertAtBack

Definition at line 77 of file [SyntaxVisitor.h](#).

7.4.3.4 listInsertAtFront

[ListChangeMap](#) slang::syntax::detail::ChangeCollection::listInsertAtFront

Definition at line 76 of file [SyntaxVisitor.h](#).

7.4.3.5 removeOrReplace

[ModifyChangeMap](#) slang::syntax::detail::ChangeCollection::removeOrReplace

Definition at line 75 of file [SyntaxVisitor.h](#).

The documentation for this struct was generated from the following file:

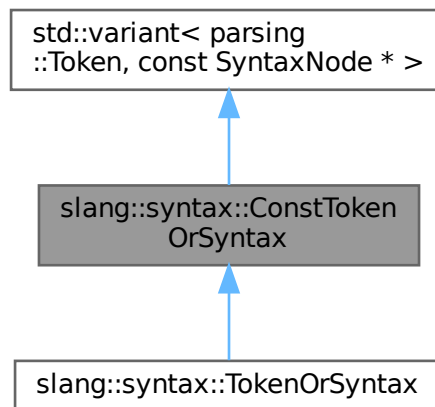
- `include_3rd_party/SyntaxVisitor.h`

7.5 slang::syntax::ConstTokenOrSyntax Struct Reference

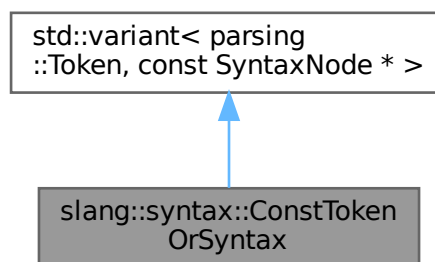
A token or a constant syntax node.

```
#include <SyntaxNode.h>
```

Inheritance diagram for slang::syntax::ConstTokenOrSyntax:



Collaboration diagram for slang::syntax::ConstTokenOrSyntax:



Public Types

- using [Base](#) = std::variant< parsing::Token, const [SyntaxNode](#) * >

Public Member Functions

- [ConstTokenOrSyntax](#) (parsing::Token [token](#))
- [ConstTokenOrSyntax](#) (const [SyntaxNode](#) *[node](#))
- [ConstTokenOrSyntax](#) (nullptr_t)
- bool [isToken](#) () const
- bool [isNode](#) () const
- parsing::Token [token](#) () const
- const [SyntaxNode](#) * [node](#) () const
- SourceRange [range](#) () const

Gets the source range for the token or syntax node.

7.5.1 Detailed Description

A token or a constant syntax node.

Definition at line 50 of file [SyntaxNode.h](#).

7.5.2 Member Typedef Documentation

7.5.2.1 Base

```
using slang::syntax::ConstTokenOrSyntax::Base = std::variant<parsing::Token, const SyntaxNode*>
```

Definition at line 51 of file [SyntaxNode.h](#).

7.5.3 Constructor & Destructor Documentation

7.5.3.1 ConstTokenOrSyntax() [1/3]

```
slang::syntax::ConstTokenOrSyntax::ConstTokenOrSyntax (
    parsing::Token token ) [inline]
```

Definition at line 52 of file [SyntaxNode.h](#).

7.5.3.2 ConstTokenOrSyntax() [2/3]

```
slang::syntax::ConstTokenOrSyntax::ConstTokenOrSyntax (
    const SyntaxNode * node ) [inline]
```

Definition at line 53 of file [SyntaxNode.h](#).

7.5.3.3 ConstTokenOrSyntax() [3/3]

```
slang::syntax::ConstTokenOrSyntax::ConstTokenOrSyntax (
    nullptr_t ) [inline]
```

Definition at line 54 of file [SyntaxNode.h](#).

7.5.4 Member Function Documentation

7.5.4.1 isNode()

```
bool slang::syntax::ConstTokenOrSyntax::isNode ( ) const [inline]
```

Returns

true if the object is a syntax node.

Definition at line 60 of file [SyntaxNode.h](#).

7.5.4.2 isToken()

```
bool slang::syntax::ConstTokenOrSyntax::isToken ( ) const [inline]
```

Returns

true if the object is a token.

Definition at line 57 of file [SyntaxNode.h](#).

7.5.4.3 node()

```
const SyntaxNode * slang::syntax::ConstTokenOrSyntax::node ( ) const [inline]
```

Gets access to the object as a syntax node (throwing an exception if it's not actually a syntax node).

Definition at line 68 of file [SyntaxNode.h](#).

7.5.4.4 range()

```
SourceRange slang::syntax::ConstTokenOrSyntax::range ( ) const
```

Gets the source range for the token or syntax node.

7.5.4.5 token()

```
parsing::Token slang::syntax::ConstTokenOrSyntax::token ( ) const [inline]
```

Gets access to the object as a token (throwing an exception if it's not actually a token).

Definition at line 64 of file [SyntaxNode.h](#).

The documentation for this struct was generated from the following file:

- include_3rd_party/[SyntaxNode.h](#)

7.6 slang::syntax::DeferredSourceRange Class Reference

```
#include <SyntaxNode.h>
```

Public Member Functions

- [DeferredSourceRange](#) ()=default
- [DeferredSourceRange](#) (SourceRange range)
- [DeferredSourceRange](#) (const [SyntaxNode](#) &node)
- [DeferredSourceRange](#) (const [SyntaxNode](#) &node, SourceRange range)
- SourceRange [get](#) () const
- SourceRange [operator*](#) () const
- const [SyntaxNode](#) * [syntax](#) () const

Private Attributes

- SourceRange [range](#)
- PointerIntPair< const [SyntaxNode](#) *, 1, 1, bool > [node](#)

7.6.1 Detailed Description

Represents a source range or a way to get one by materializing it from a syntax node. This exists to avoid computing the source range of a node unless it's actually needed.

Definition at line [250](#) of file [SyntaxNode.h](#).

7.6.2 Constructor & Destructor Documentation

7.6.2.1 DeferredSourceRange() [1/4]

```
slang::syntax::DeferredSourceRange::DeferredSourceRange ( ) [default]
```

7.6.2.2 DeferredSourceRange() [2/4]

```
slang::syntax::DeferredSourceRange::DeferredSourceRange (
    SourceRange range ) [inline]
```

Definition at line [253](#) of file [SyntaxNode.h](#).

7.6.2.3 DeferredSourceRange() [3/4]

```
slang::syntax::DeferredSourceRange::DeferredSourceRange (
    const SyntaxNode & node ) [inline]
```

Definition at line [254](#) of file [SyntaxNode.h](#).

7.6.2.4 DeferredSourceRange() [4/4]

```
slang::syntax::DeferredSourceRange::DeferredSourceRange (
    const SyntaxNode & node,
    SourceRange range ) [inline]
```

Definition at line [255](#) of file [SyntaxNode.h](#).

7.6.3 Member Function Documentation

7.6.3.1 get()

```
SourceRange slang::syntax::DeferredSourceRange::get ( ) const [inline]
```

Definition at line [258](#) of file [SyntaxNode.h](#).

Here is the caller graph for this function:



7.6.3.2 operator*()

```
SourceRange slang::syntax::DeferredSourceRange::operator* ( ) const [inline]
```

Definition at line [267](#) of file [SyntaxNode.h](#).

Here is the call graph for this function:



7.6.3.3 syntax()

```
const SyntaxNode * slang::syntax::DeferredSourceRange::syntax ( ) const [inline]
```

Definition at line 269 of file [SyntaxNode.h](#).

7.6.4 Member Data Documentation

7.6.4.1 node

```
PointerIntPair<const SyntaxNode*, 1, 1, bool> slang::syntax::DeferredSourceRange::node [mutable], [private]
```

Definition at line 273 of file [SyntaxNode.h](#).

7.6.4.2 range

```
SourceRange slang::syntax::DeferredSourceRange::range [mutable], [private]
```

Definition at line 272 of file [SyntaxNode.h](#).

The documentation for this class was generated from the following file:

- include_3rd_party/[SyntaxNode.h](#)

7.7 GateInfo Struct Reference

```
#include <verilog_utils.hpp>
```

Public Attributes

- slang::parsing::TokenKind [kind](#)
- std::string [gateTypeName](#)
- std::vector< std::string > [inputSignals](#)
- std::string [outputSignal](#)

7.7.1 Detailed Description

Definition at line 97 of file [verilog_utils.hpp](#).

7.7.2 Member Data Documentation

7.7.2.1 gateTypeName

```
std::string GateInfo::gateTypeName
```

Definition at line 99 of file [verilog_utils.hpp](#).

7.7.2.2 inputSignals

```
std::vector<std::string> GateInfo::inputSignals
```

Definition at line 100 of file [verilog_utils.hpp](#).

7.7.2.3 kind

```
slang::parsing::TokenKind GateInfo::kind
```

Definition at line 98 of file [verilog_utils.hpp](#).

7.7.2.4 outputSignal

```
std::string GateInfo::outputSignal
```

Definition at line 101 of file [verilog_utils.hpp](#).

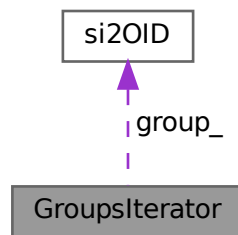
The documentation for this struct was generated from the following file:

- include/[verilog_utils.hpp](#)

7.8 GroupsIterator Class Reference

```
#include <iterators.hpp>
```

Collaboration diagram for GroupsIterator:



Public Member Functions

- `GroupsIterator` (`si2drGroupsIdT` groups, `si2drErrorT` &err)
- `~GroupsIterator` ()
- void `next` ()
- bool `end` ()
- `LibGroup` `get` ()

Private Attributes

- `si2drGroupsIdT` groups_
- `si2drGroupIdT` group_
- `si2drErrorT` & err_

7.8.1 Detailed Description

Definition at line 9 of file `iterators.hpp`.

7.8.2 Constructor & Destructor Documentation

7.8.2.1 GroupsIterator()

```
GroupsIterator::GroupsIterator (
    si2drGroupsIdT groups,
    si2drErrorT & err )
```

Definition at line 3 of file [iterators.cpp](#).

7.8.2.2 ~GroupsIterator()

```
GroupsIterator::~GroupsIterator ( )
```

Definition at line 6 of file [iterators.cpp](#).

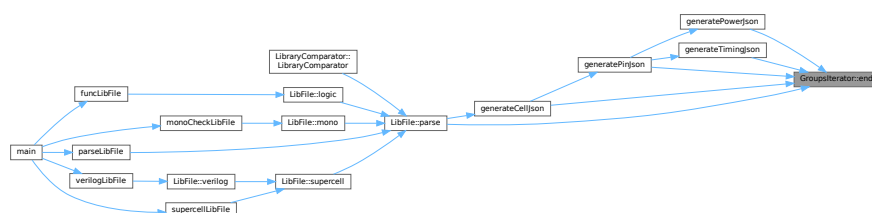
7.8.3 Member Function Documentation

7.8.3.1 end()

```
bool GroupsIterator::end ( )
```

Definition at line 9 of file [iterators.cpp](#).

Here is the caller graph for this function:

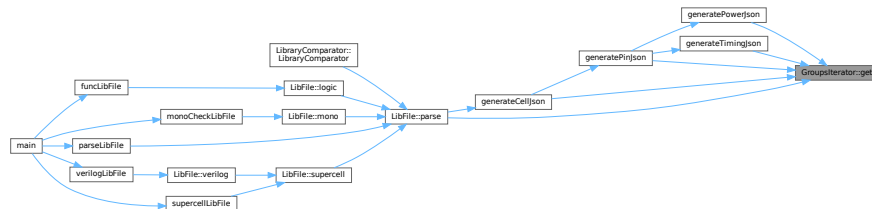


7.8.3.2 get()

`LibGroup GroupsIterator::get ( )`

Definition at line 11 of file [iterators.cpp](#).

Here is the caller graph for this function:

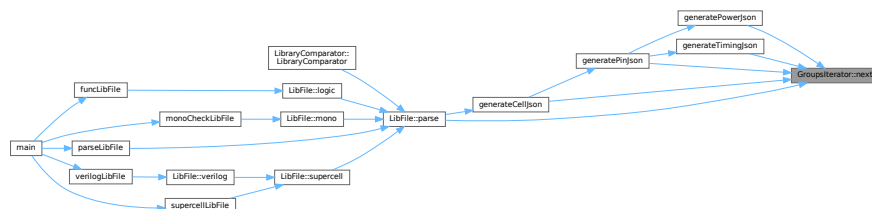


7.8.3.3 next()

`void GroupsIterator::next ( )`

Definition at line 8 of file [iterators.cpp](#).

Here is the caller graph for this function:



7.8.4 Member Data Documentation

7.8.4.1 err_

`si2drErrorT& GroupsIterator::err_ [private]`

Definition at line 20 of file [iterators.hpp](#).

7.8.4.2 group_

`si2drGroupIdT GroupsIterator::group_ [private]`

Definition at line 19 of file [iterators.hpp](#).

7.8.4.3 groups_

`si2drGroupsIdT GroupsIterator::groups_ [private]`

Definition at line 18 of file [iterators.hpp](#).

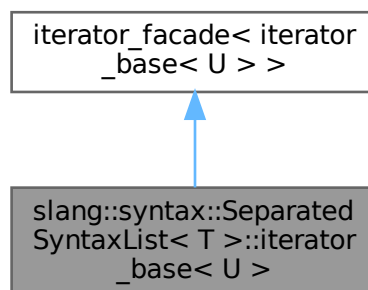
The documentation for this class was generated from the following files:

- [include/iterators.hpp](#)
- [src/iterators.cpp](#)

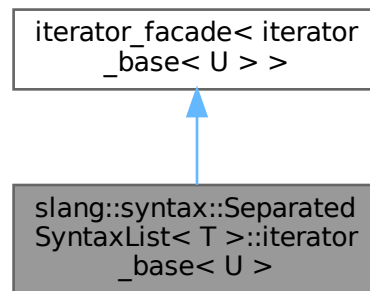
7.9 slang::syntax::SeparatedSyntaxList< T >::iterator_base< U > > Class Template Reference

```
#include <SyntaxNode.h>
```

Inheritance diagram for slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >:



Collaboration diagram for slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >:



Public Types

- using `ParentList` = `std::conditional_t< std::is_const_v< std::remove_pointer_t< U > >, const SeparatedSyntaxList, SeparatedSyntaxList >`

Public Member Functions

- `iterator_base` ()
- `iterator_base` (`ParentList` &`list`, `size_t` `index`)
- `U` `dereference` () const
- `bool` `equals` (const `iterator_base` &`other`) const
- `ptrdiff_t` `distance_to` (const `iterator_base` &`other`) const
- `void` `advance` (`ptrdiff_t` `n`)

Private Attributes

- `ParentList` * `list`
- `size_t` `index`

7.9.1 Detailed Description

```

template<typename T>
template<typename U>
class slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >

```

An iterator that will iterate over just the nodes (and skip the delimiters) in the parent `SeparatedSyntaxList`.

Definition at line 386 of file `SyntaxNode.h`.

7.9.2 Member Typedef Documentation

7.9.2.1 ParentList

```
template<typename T >
template<typename U >
using slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >::ParentList = std::conditional_t<std::is_↵
const_v<std::remove_pointer_t<U> >, const SeparatedSyntaxList, SeparatedSyntaxList>
```

Definition at line 388 of file [SyntaxNode.h](#).

7.9.3 Constructor & Destructor Documentation

7.9.3.1 iterator_base() [1/2]

```
template<typename T >
template<typename U >
slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >::iterator_base ( ) [inline]
```

Definition at line 391 of file [SyntaxNode.h](#).

7.9.3.2 iterator_base() [2/2]

```
template<typename T >
template<typename U >
slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >::iterator_base (
    ParentList & list,
    size_t index ) [inline]
```

Definition at line 392 of file [SyntaxNode.h](#).

7.9.4 Member Function Documentation

7.9.4.1 advance()

```
template<typename T >
template<typename U >
void slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >::advance (
    ptrdiff_t n ) [inline]
```

Definition at line 404 of file [SyntaxNode.h](#).

7.9.4.2 dereference()

```
template<typename T >
template<typename U >
U slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >::dereference ( ) const [inline]
```

Definition at line 394 of file [SyntaxNode.h](#).

7.9.4.3 distance_to()

```
template<typename T >
template<typename U >
ptrdiff_t slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >::distance_to (
    const iterator_base< U > & other ) const [inline]
```

Definition at line 400 of file [SyntaxNode.h](#).

7.9.4.4 equals()

```
template<typename T >
template<typename U >
bool slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >::equals (
    const iterator_base< U > & other ) const [inline]
```

Definition at line 396 of file [SyntaxNode.h](#).

7.9.5 Member Data Documentation

7.9.5.1 index

```
template<typename T >
template<typename U >
size_t slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >::index [private]
```

Definition at line 408 of file [SyntaxNode.h](#).

7.9.5.2 list

```
template<typename T >
template<typename U >
ParentList* slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >::list [private]
```

Definition at line 407 of file [SyntaxNode.h](#).

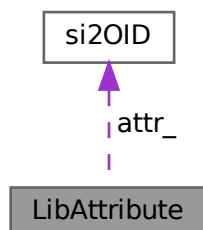
The documentation for this class was generated from the following file:

- include_3rd_party/[SyntaxNode.h](#)

7.10 LibAttribute Class Reference

```
#include <lib_attribute.hpp>
```

Collaboration diagram for LibAttribute:



Public Member Functions

- [LibAttribute](#) ([si2drAttrIdT](#) attr, [si2drErrorT](#) &err)
- [~LibAttribute](#) ()
- [std::string](#) [getName](#) ()
- [bool](#) [isComplex](#) ()
- [si2drValuesIdT](#) [getValues](#) ()
- [long int](#) [getInt](#) ()
- [double](#) [getFloat](#) ()
- [std::string](#) [getString](#) ()
- [bool](#) [getBoolean](#) ()

Private Attributes

- [si2drAttrIdT](#) attr_
- [si2drErrorT](#) & err_

7.10.1 Detailed Description

Definition at line 8 of file [lib_attribute.hpp](#).

7.10.2 Constructor & Destructor Documentation

7.10.2.1 LibAttribute()

```
LibAttribute::LibAttribute (  
    si2drAttrIdT attr,  
    si2drErrorT & err )
```

Definition at line 3 of file [lib_attribute.cpp](#).

7.10.2.2 ~LibAttribute()

```
LibAttribute::~LibAttribute ( )
```

Definition at line 5 of file [lib_attribute.cpp](#).

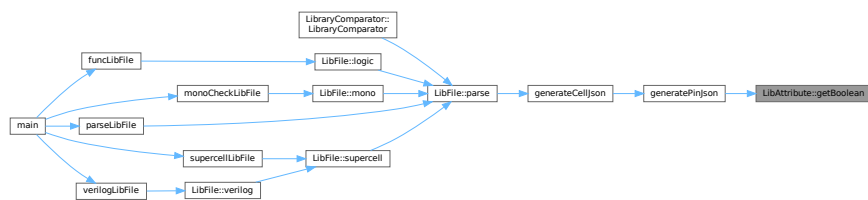
7.10.3 Member Function Documentation

7.10.3.1 getBoolean()

```
bool LibAttribute::getBoolean ( )
```

Definition at line 25 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:

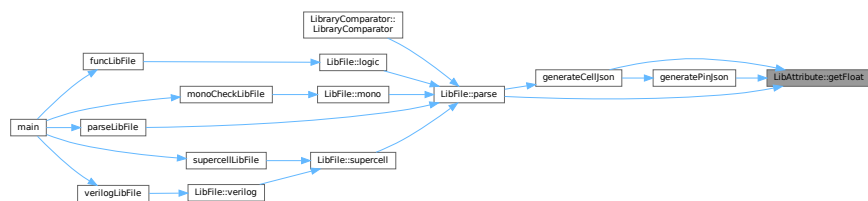


7.10.3.2 getFloat()

```
double LibAttribute::getFloat ( )
```

Definition at line 18 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:

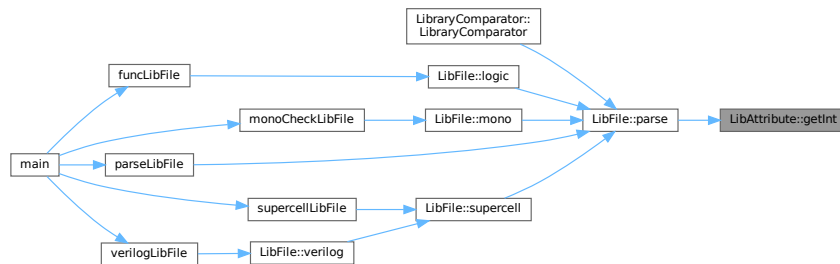


7.10.3.3 getInt()

```
long int LibAttribute::getInt ( )
```

Definition at line 16 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:

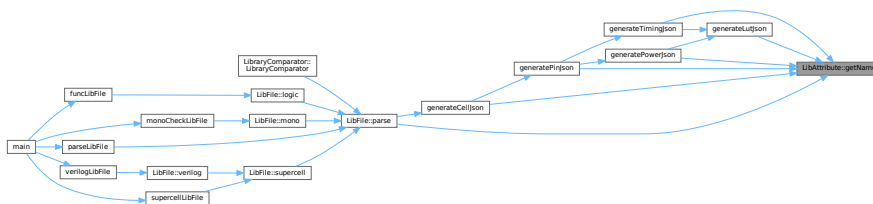


7.10.3.4 getName()

```
std::string LibAttribute::getName ( )
```

Definition at line 7 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:

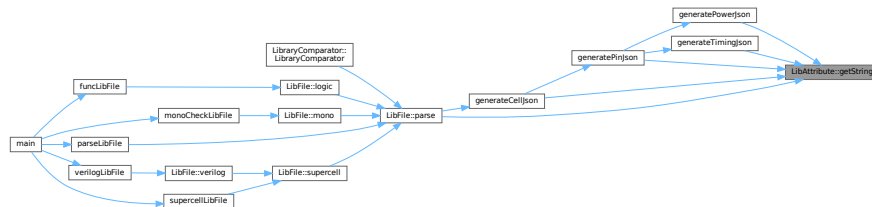


7.10.3.5 getString()

```
std::string LibAttribute::getString ( )
```

Definition at line 20 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:

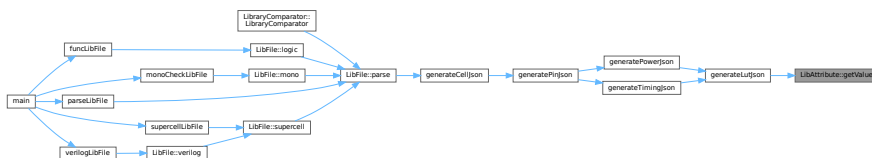


7.10.3.6 getValues()

```
si2drValuesIdT LibAttribute::getValues ( )
```

Definition at line 14 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:



7.10.3.7 isComplex()

```
bool LibAttribute::isComplex ( )
```

Definition at line 12 of file [lib_attribute.cpp](#).

7.10.4 Member Data Documentation

7.10.4.1 attr_

`si2drAttrIdT` `LibAttribute::attr_` [private]

Definition at line 21 of file `lib_attribute.hpp`.

7.10.4.2 err_

`si2drErrorT&` `LibAttribute::err_` [private]

Definition at line 22 of file `lib_attribute.hpp`.

The documentation for this class was generated from the following files:

- `include/lib_attribute.hpp`
- `src/lib_attribute.cpp`

7.11 liberty_value_data Struct Reference

```
#include <si2dr_liberty.h>
```

Public Attributes

- `int` `dimensions`
- `int *` `dim_sizes`
- `LONG_DOUBLE **` `index_info`
- `LONG_DOUBLE *` `values`

7.11.1 Detailed Description

Definition at line 612 of file `si2dr_liberty.h`.

7.11.2 Member Data Documentation

7.11.2.1 dim_sizes

```
int* liberty_value_data::dim_sizes
```

Definition at line 615 of file [si2dr_liberty.h](#).

7.11.2.2 dimensions

```
int liberty_value_data::dimensions
```

Definition at line 614 of file [si2dr_liberty.h](#).

7.11.2.3 index_info

```
LONG_DOUBLE** liberty_value_data::index_info
```

Definition at line 617 of file [si2dr_liberty.h](#).

7.11.2.4 values

```
LONG_DOUBLE* liberty_value_data::values
```

Definition at line 621 of file [si2dr_liberty.h](#).

The documentation for this struct was generated from the following file:

- [include_3rd_party/si2dr_liberty.h](#)

7.12 LibFile Class Reference

```
#include <lib_file.hpp>
```

Public Member Functions

- **LibFile** (const std::string &filepath, const std::string &loggername)
Constructs a [LibFile](#) object with specified file path and logger name.
- **~LibFile** ()
- void **writeJsonToFile** ()
Writes the JSON data stored in the object to a file.
- void **parse** ()
Parses the Liberty file and extracts library information into a JSON structure.
- void **modify** ()
- void **mono** (const bool is_slew)
Validates that lookup tables are monotonically increasing with respect to the output load.
- void **supercell** (const int chain_length, const std::vector< std::string > &cell_names)
Creates supercells based on standard cells in the library file.
- void **verilog** (const int chain_length, const std::vector< std::string > &cell_names)
Generates Verilog files for cell validation based on the library file.
- std::map< std::string, std::string > **logic** (const std::string &cell_name)

Public Attributes

- std::shared_ptr< spdlog::logger > **logger_**
- std::filesystem::path **filepath_**
- std::string **basename_**
- std::string **filename_**
- std::string **libname_** = ""
- std::string **jsonname_** = ""
- std::string **loggername_** = ""
- **json lib_json_** = json::object()

Private Member Functions

- void **read** ()
Reads the Liberty file specified by the filename_ member variable.
- bool **checkTimingArcMonotonicity** (const json &cell, const json &pin, const json &arc, const std::string &type, bool is_slew)
Checks if the values in a timing arc are monotonically increasing.

Private Attributes

- **si2drErrorT err_**
- int **process_** = 0
- float **voltage_** = 0.0
- int **temperature_** = 0

7.12.1 Detailed Description

Definition at line 13 of file [lib_file.hpp](#).

7.12.2 Constructor & Destructor Documentation

7.12.2.1 LibFile()

```
LibFile::LibFile (
    const std::string & filepath,
    const std::string & loggername )
```

Constructs a [LibFile](#) object with specified file path and logger name.

This constructor initializes a [LibFile](#) object with the given file path and creates a logger with the specified name. It sets up:

- Two logging sinks: a colored console sink and a file sink
- File path information including filename, basename, and a corresponding JSON filename
- Log levels (info for console, debug for file)

Parameters

<i>filepath</i>	The path to the file to be processed
<i>loggername</i>	The name for the logger and the log file

Definition at line 28 of file [lib_file.cpp](#).

7.12.2.2 ~LibFile()

```
LibFile::~LibFile ( )
```

Definition at line 48 of file [lib_file.cpp](#).

7.12.3 Member Function Documentation

7.12.3.1 checkTimingArcMonotonicity()

```
bool LibFile::checkTimingArcMonotonicity (
    const json & cell,
    const json & pin,
    const json & arc,
    const std::string & timing_arc_name,
    bool is_slew ) [private]
```

Checks if the values in a timing arc are monotonically increasing.

This function examines the values in the specified timing arc to ensure they are monotonically increasing. It checks monotonicity in two dimensions:

1. Across rows (by output load capacitance)
2. Across columns (by input slew, only if `is_slew` is true)

The function logs detailed information about any non-monotonic values found, including cell name, pin names, and conditional statements (when clause) if present.

Parameters

<i>cell</i>	The JSON object representing the cell being checked
<i>pin</i>	The JSON object representing the pin being checked
<i>arc</i>	The JSON object representing the timing arc being checked
<i>timing_arc_name</i>	The name of the timing arc to check (e.g., "cell_rise", "cell_fall")
<i>is_slew</i>	Boolean flag indicating whether to check monotonicity across input slew values

Returns

true if all values in the timing arc are monotonic, false otherwise

Exceptions

<i>None, but</i>	logs errors or warnings for invalid data formats or non-monotonic values
------------------	--

Definition at line 218 of file [lib_file.cpp](#).

Here is the caller graph for this function:

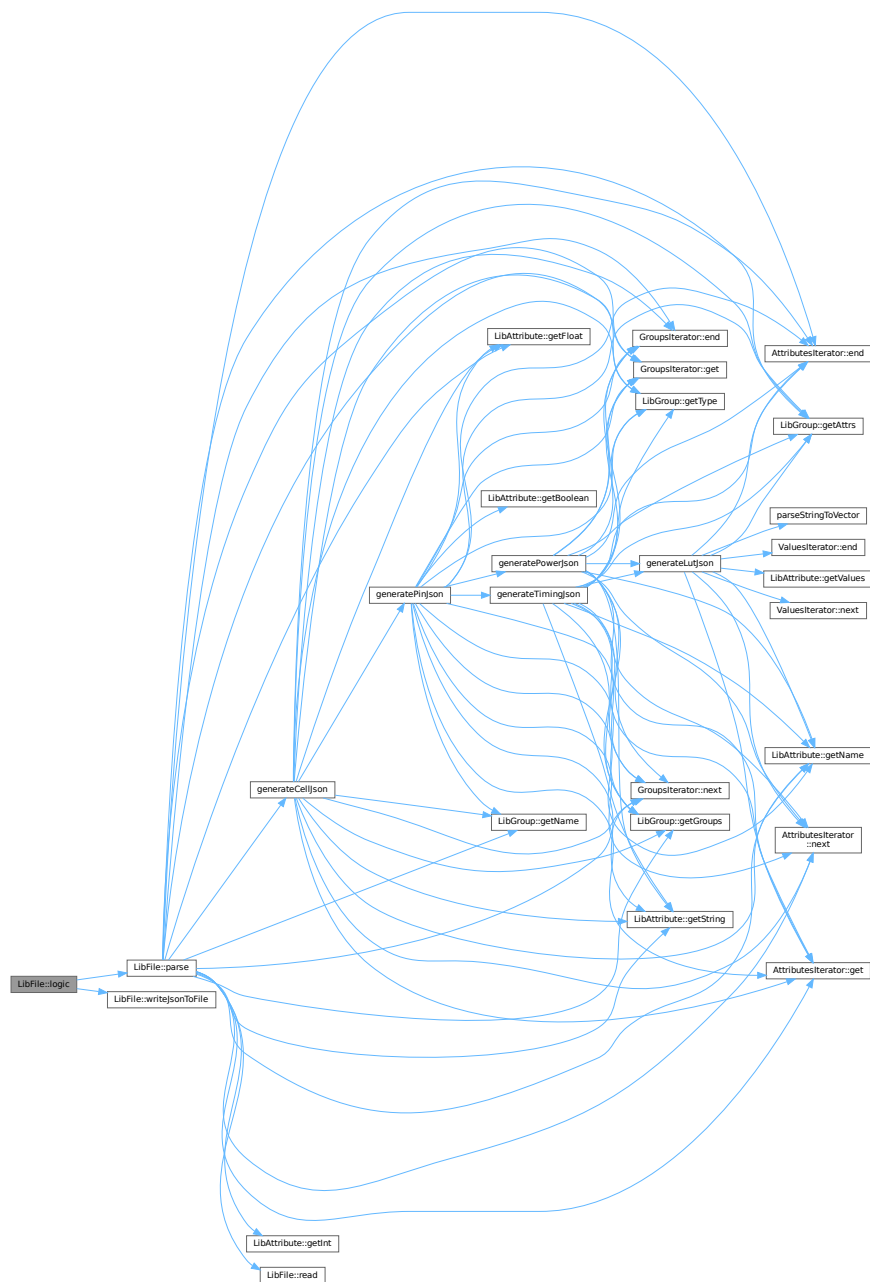


7.12.3.2 `logic()`

```
std::map< std::string, std::string > LibFile::logic (  
    const std::string & cell_name )
```

Definition at line [891](#) of file [lib_file.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.12.3.3 `modify()`

```
void LibFile::modify ( )
```

Definition at line 195 of file [lib_file.cpp](#).

7.12.3.4 `mono()`

```
void LibFile::mono (
    const bool is_slew )
```

Validates that lookup tables are monotonically increasing with respect to the output load.

This function checks the following timing data in the current library to ensure monotonicity:

- `cell_rise` and `retaining_rise`
- `cell_fall` and `retaining_fall`
- `rise_transition` and `retain_rise_slew`
- `fall_transition` and `retain_fall_slew`
- `min_pulse_width` constraints (`rise_constraint`, `fall_constraint`)

The function first checks if a parsed JSON representation exists. If not, it parses the Liberty file and creates the JSON file. It then evaluates each timing arc in all cells and reports the pass/fail status at the end.

Parameters

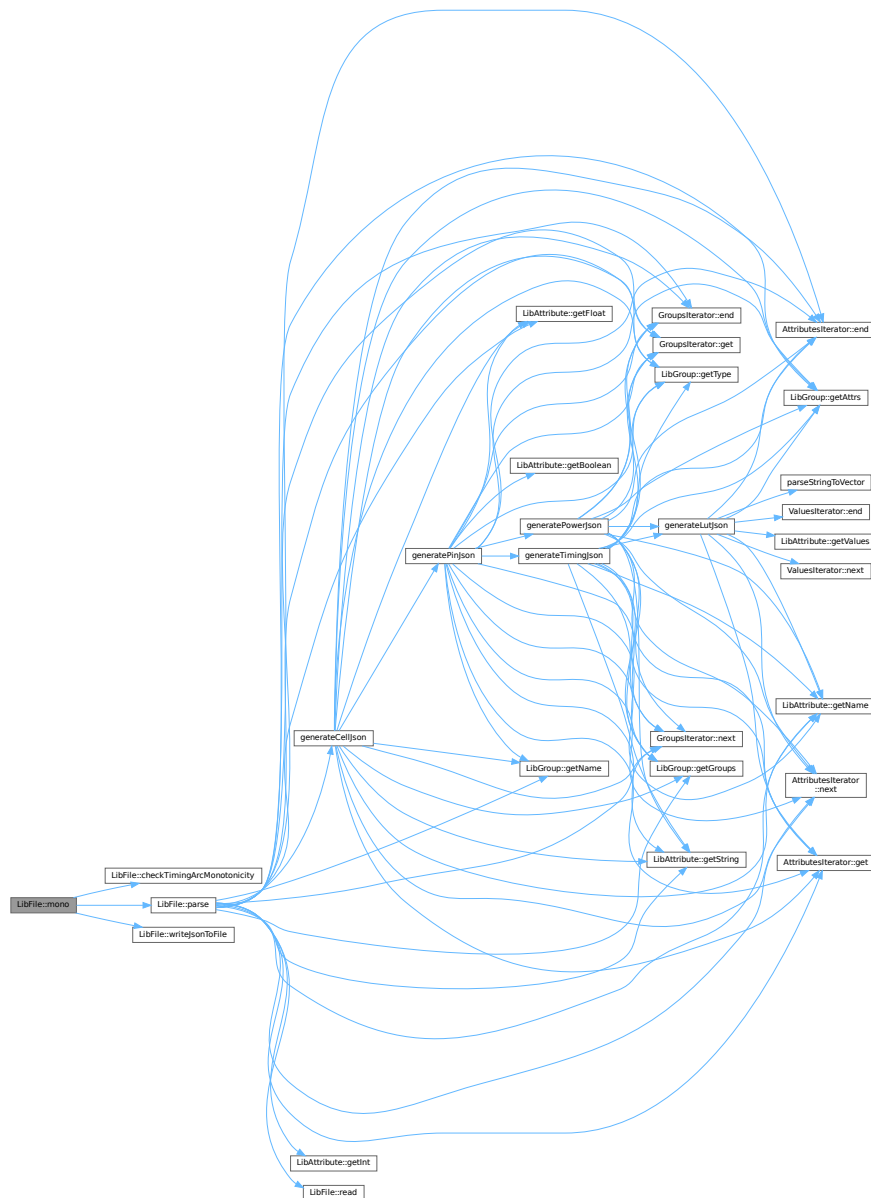
<code><i>is_slew</i></code>	Boolean flag indicating whether to check slew values rather than delay values
-----------------------------	---

The function outputs:

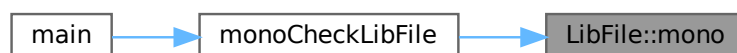
- Number of cells that passed/failed the monotonicity check
- List of cells that failed the check (if any)

Definition at line 344 of file [lib_file.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.12.3.5 parse()

```
void LibFile::parse ( )
```

Parses the Liberty file and extracts library information into a JSON structure.

This method handles the parsing of a Liberty file by:

1. Initializing the SI2 parser interface error handler
2. Reading the Liberty file contents
3. Extracting the library name
4. Processing second-level groups including:
 - Cell definitions, which are added to the JSON structure
 - Operating conditions, extracting PVT (Process, Voltage, Temperature) values

The parsed data is stored in the `lib_json_` member variable, with the following structure:

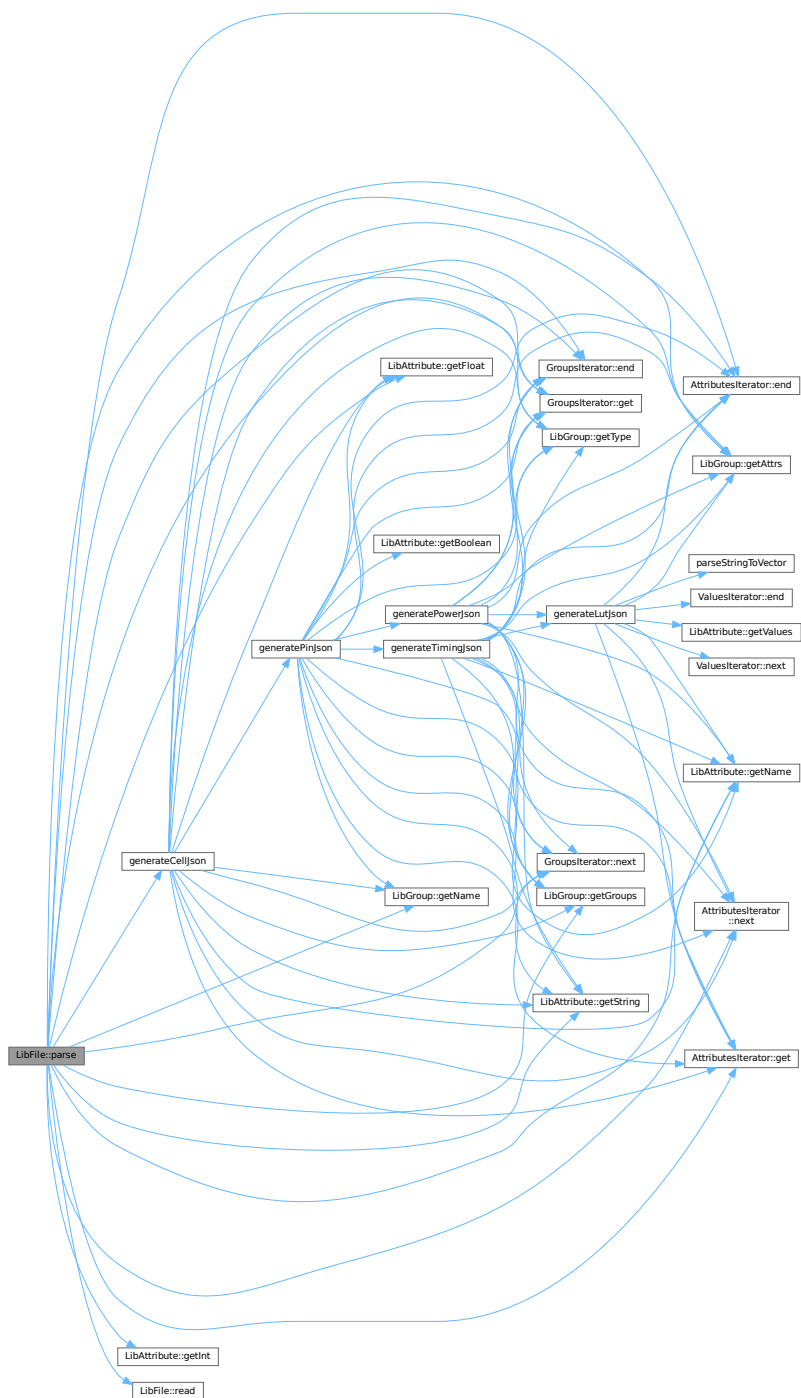
- "library_name": Name of the library
- "cells": Array of cell definitions
- "process": Process value
- "voltage": Voltage value (rounded to 2 decimal places)
- "temperature": Temperature value

Note

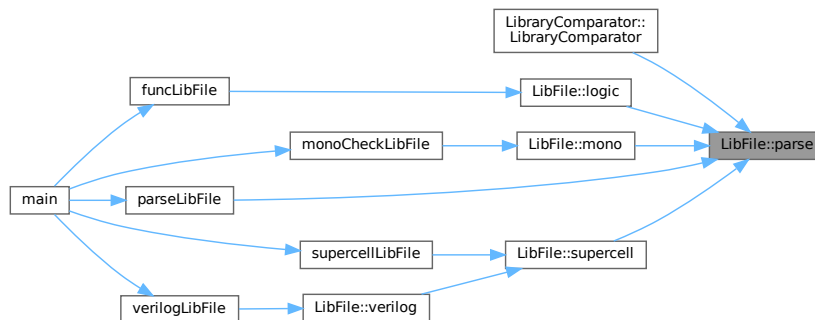
This method creates local scopes to ensure proper cleanup of SI2 iterators.

Definition at line [129](#) of file [lib_file.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.12.3.6 read()

```
void LibFile::read ( ) [private]
```

Reads the Liberty file specified by the `filename_` member variable.

This function attempts to read a Liberty file and logs the process. It measures the time taken to read the file and logs any errors encountered during the read operation. If an error occurs, the function logs the error and terminates the program.

- Logs the start of the read operation.
- Measures the duration of the read operation.
- Checks for specific errors (invalid name or syntax error) and logs them.
- Terminates the program with specific exit codes if errors are detected.
- Logs the completion of the read operation and the time taken.

Note

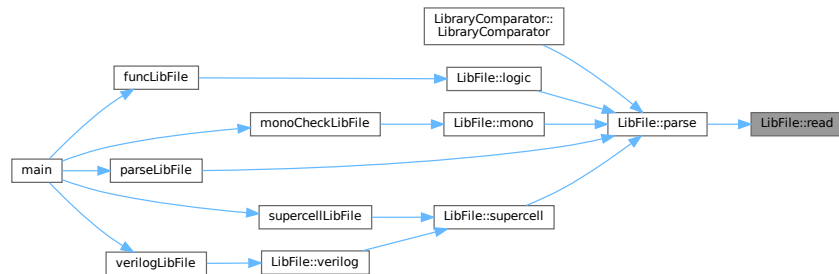
This function uses the `spdlog` library for logging and the `si2drReadLibertyFile` function for reading the Liberty file.

Exceptions

<i>This</i>	function will terminate the program with exit codes 301 or 401 if specific errors are encountered.
-------------	--

Definition at line 90 of file [lib_file.cpp](#).

Here is the caller graph for this function:



7.12.3.7 supercell()

```

void LibFile::supercell (
    const int chain_length,
    const std::vector< std::string > & cell_names )

```

Creates supercells based on standard cells in the library file.

This method creates supercells by combining standard cells in chains. For each input-output pin pair of a cell, it creates a supercell entry with naming format: `<cellname>X<chain_length><input_pin>↔____<output_pin>`. The results are written to a .map file.

For sequential cells (with clock pins), the chain length is always set to 1 regardless of the requested chain length.

Parameters

<i>chain_length</i>	The length of the chain for combinational cells
<i>cell_names</i>	Vector of specific cell names to process. If empty, all cells will be processed.

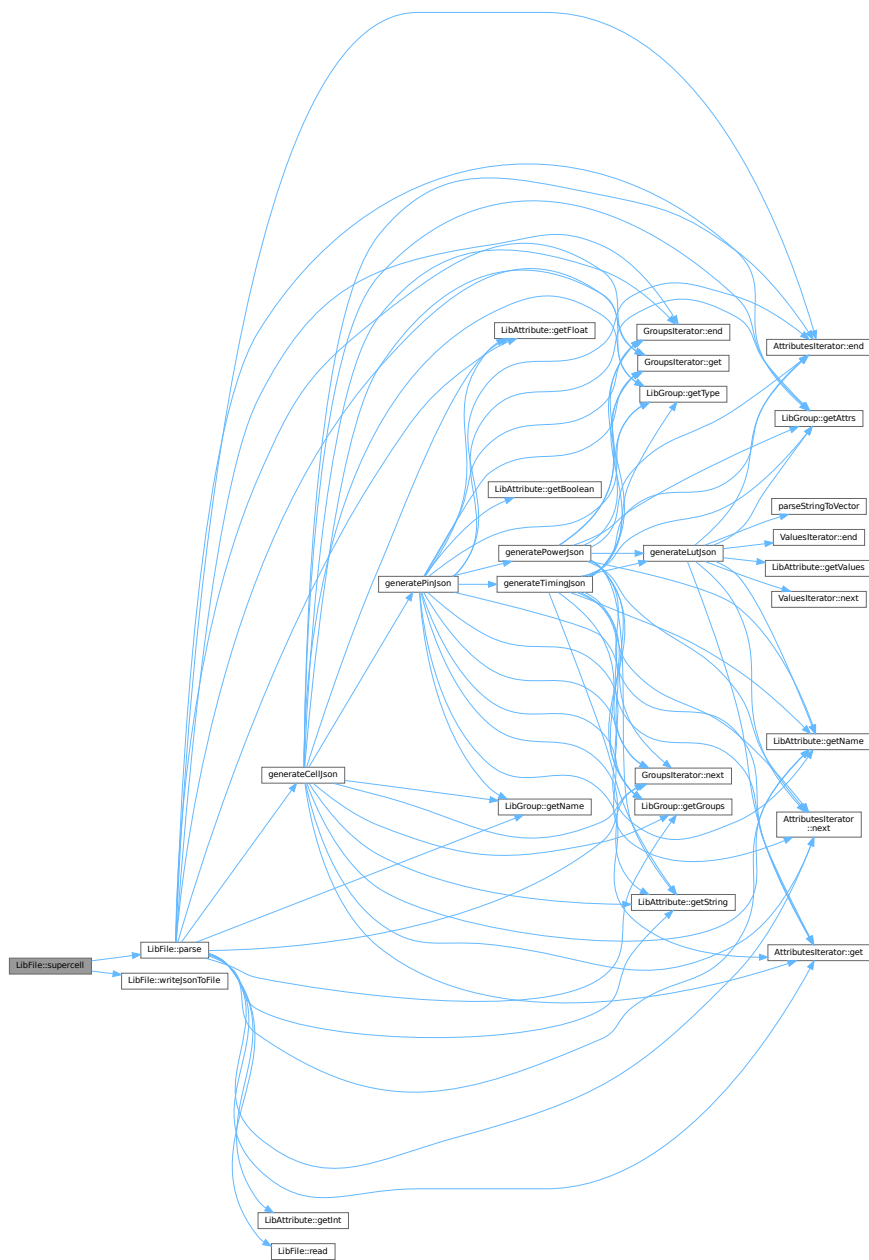
Note

Before creating supercells, this method checks for the existence of a JSON representation of the Liberty file and parses it if not found.

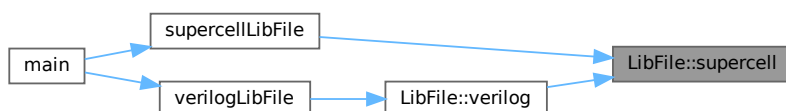
The method will report any requested cells that were not found in the library.

Definition at line 473 of file [lib_file.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.12.3.8 verilog()

```
void LibFile::verilog (
    const int chain_length,
    const std::vector< std::string > & cell_names )
```

Generates Verilog files for cell validation based on the library file.

This function creates Verilog modules for each cell in the specified chain and generates a top-level module that connects them all together. The process involves:

1. Creating supercells based on the specified chain length and cell names
2. Reading the mapping file to identify supercell name relationships
3. Generating individual Verilog modules for each supercell, handling both sequential and combinational cells differently
4. Creating ANSI-style port definitions for each module
5. Using the slang library to build proper syntax trees for Verilog modules
6. Creating a top-level module that instantiates all the individual modules
7. Writing the complete Verilog output to the output file

Sequential cells are instantiated once, while combinational cells are instantiated multiple times based on the specified chain length.

Parameters

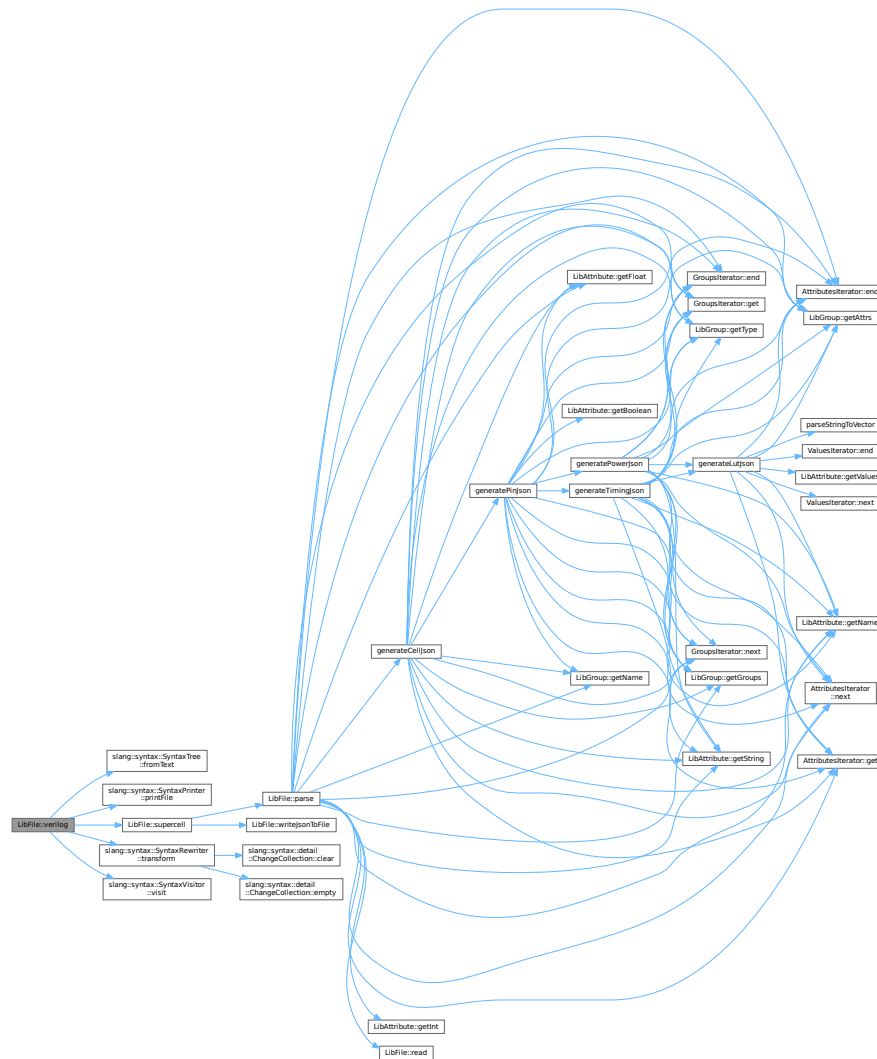
<i>chain_length</i>	The number of times to chain combinational cells together
<i>cell_names</i>	Vector of cell names to include in the Verilog generation

Note

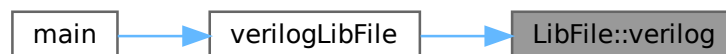
The function writes a temporary file during processing that might not be removed when the function completes (commented out cleanup code).

Definition at line 627 of file [lib_file.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.12.3.9 writeJsonToFile()

```
void LibFile::writeJsonToFile ( )
```

Writes the JSON data stored in the object to a file.

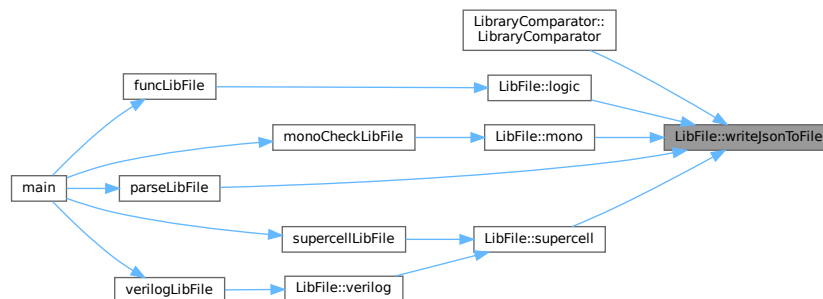
This method opens the file specified by `jsonname_` and writes the content of `lib_json_` with an indentation of 2 spaces. If the file cannot be opened, an error message is logged. Upon successful writing, an informational message is logged.

Exceptions

<i>None</i>	directly, but <code>std::ofstream</code> operations may throw exceptions
-------------	--

Definition at line 59 of file [lib_file.cpp](#).

Here is the caller graph for this function:



7.12.4 Member Data Documentation

7.12.4.1 `basename_`

`std::string LibFile::basename_`

Definition at line 19 of file [lib_file.hpp](#).

7.12.4.2 `err_`

`si2drErrorT LibFile::err_` [private]

Definition at line 34 of file [lib_file.hpp](#).

7.12.4.3 filename_

```
std::string LibFile::filename_
```

Definition at line 20 of file [lib_file.hpp](#).

7.12.4.4 filepath_

```
std::filesystem::path LibFile::filepath_
```

Definition at line 18 of file [lib_file.hpp](#).

7.12.4.5 jsonname_

```
std::string LibFile::jsonname_ = ""
```

Definition at line 22 of file [lib_file.hpp](#).

7.12.4.6 lib_json_

```
json LibFile::lib_json_ = json::object()
```

Definition at line 24 of file [lib_file.hpp](#).

7.12.4.7 libname_

```
std::string LibFile::libname_ = ""
```

Definition at line 21 of file [lib_file.hpp](#).

7.12.4.8 logger_

```
std::shared_ptr<spdlog::logger> LibFile::logger_
```

Definition at line 17 of file [lib_file.hpp](#).

7.12.4.9 loggername_

```
std::string LibFile::loggername_ = ""
```

Definition at line 23 of file [lib_file.hpp](#).

7.12.4.10 process_

```
int LibFile::process_ = 0 [private]
```

Definition at line 35 of file [lib_file.hpp](#).

7.12.4.11 temperature_

```
int LibFile::temperature_ = 0 [private]
```

Definition at line 37 of file [lib_file.hpp](#).

7.12.4.12 voltage_

```
float LibFile::voltage_ = 0.0 [private]
```

Definition at line 36 of file [lib_file.hpp](#).

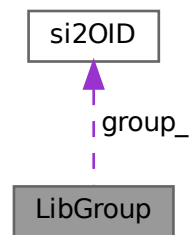
The documentation for this class was generated from the following files:

- [include/lib_file.hpp](#)
- [src/lib_file.cpp](#)

7.13 LibGroup Class Reference

```
#include <lib_group.hpp>
```

Collaboration diagram for LibGroup:



Public Member Functions

- `LibGroup` (`si2drGroupldT` group, `si2drErrorT` &err)
- `~LibGroup` ()
- `std::string` `getName` ()
- `std::string` `getType` ()
- `si2drAttrslldT` `getAttrs` ()
- `si2drGroupslldT` `getGroups` ()

Private Attributes

- `si2drGroupldT` `group_`
- `si2drErrorT` & `err_`

7.13.1 Detailed Description

Definition at line 8 of file `lib_group.hpp`.

7.13.2 Constructor & Destructor Documentation

7.13.2.1 LibGroup()

```
LibGroup::LibGroup (
    si2drGroupIdT group,
    si2drErrorT & err )
```

Definition at line 3 of file [lib_group.cpp](#).

7.13.2.2 ~LibGroup()

```
LibGroup::~~LibGroup ( )
```

Definition at line 5 of file [lib_group.cpp](#).

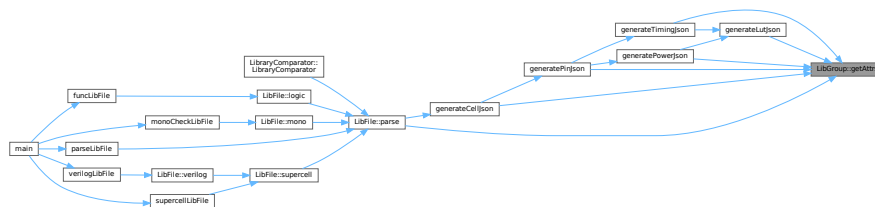
7.13.3 Member Function Documentation

7.13.3.1 getAttrs()

```
si2drAttrsIdT LibGroup::getAttrs ( )
```

Definition at line 19 of file [lib_group.cpp](#).

Here is the caller graph for this function:

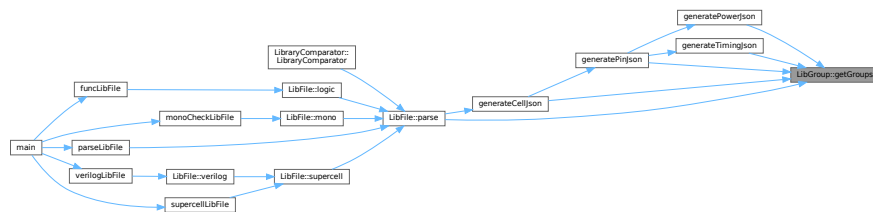


7.13.3.2 getGroups()

```
si2drGroupsIdT LibGroup::getGroups ( )
```

Definition at line 21 of file [lib_group.cpp](#).

Here is the caller graph for this function:

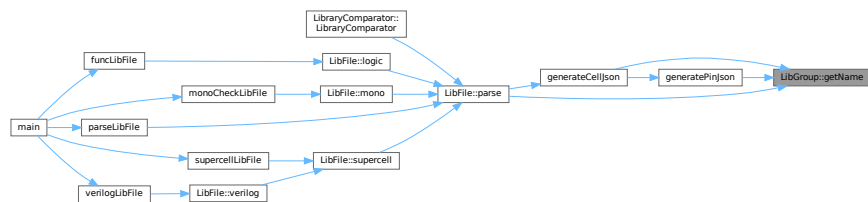


7.13.3.3 getName()

```
std::string LibGroup::getName ( )
```

Definition at line 7 of file [lib_group.cpp](#).

Here is the caller graph for this function:

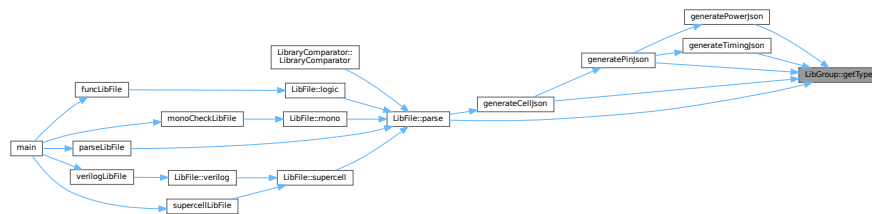


7.13.3.4 getType()

```
std::string LibGroup::getType ( )
```

Definition at line 14 of file [lib_group.cpp](#).

Here is the caller graph for this function:



7.13.4 Member Data Documentation

7.13.4.1 err_

```
si2drErrorT& LibGroup::err_ [private]
```

Definition at line 19 of file [lib_group.hpp](#).

7.13.4.2 group_

```
si2drGroupIdT LibGroup::group_ [private]
```

Definition at line 18 of file [lib_group.hpp](#).

The documentation for this class was generated from the following files:

- [include/lib_group.hpp](#)
- [src/lib_group.cpp](#)

7.14 LibraryComparator Class Reference

```
#include <compare.hpp>
```

Public Member Functions

- [LibraryComparator](#) ([LibFile](#) &ref_libfile, [LibFile](#) &comp_libfile, double reltol, double abstol)
Constructor for the [LibraryComparator](#) class.
- void [generateReport](#) (const std::string &output_file)
Generates a comparison report between the reference and comparison libraries.

Public Attributes

- std::filesystem::path [ref_lib_path_](#)
- std::filesystem::path [comp_lib_path_](#)

Private Member Functions

- void [compareCell](#) (const std::string &cell_name, const [json](#) &ref_cell, const [json](#) &comp_cell, Table &table)
Compares the output pins of a cell between a reference JSON and a comparison JSON.
- void [comparePin](#) (const std::string &cell_name, const std::string &pin_name, const [json](#) &ref_pin, const [json](#) &comp_pin, Table &table)
Compares the timing arcs of a given pin between a reference JSON and a comparison JSON.
- void [compareTimingArc](#) (const std::string &cell_name, const std::string &pin_name, const std::string &timing_type, const [json](#) &ref_timing_arc, const [json](#) &comp_timing_arc, Table &table)
Compares a timing arc between two JSON objects.
- void [compareLut](#) (const std::string &cell_name, const std::string &pin_name, const std::string &timing_type, const std::string &related_pin, const std::string &arc_name, const [json](#) &ref_lut, const [json](#) &comp_lut, Table &table)
Compares lookup tables (LUTs) between reference and comparison libraries for a specific timing arc.

Private Attributes

- [json](#) [ref_json_](#)
- [json](#) [comp_json_](#)
- double [reltol_](#)
- double [abstol_](#)

7.14.1 Detailed Description

Definition at line 12 of file [compare.hpp](#).

7.14.2 Constructor & Destructor Documentation

7.14.2.1 LibraryComparator()

```
LibraryComparator::LibraryComparator (
    LibFile & ref_libfile,
    LibFile & comp_libfile,
    double reltol,
    double abstol )
```

Constructor for the [LibraryComparator](#) class.

Initializes a [LibraryComparator](#) object by loading JSON data from reference and comparison library files. If JSON files don't exist, parses the library files first and creates the JSON files.

The constructor performs the following steps:

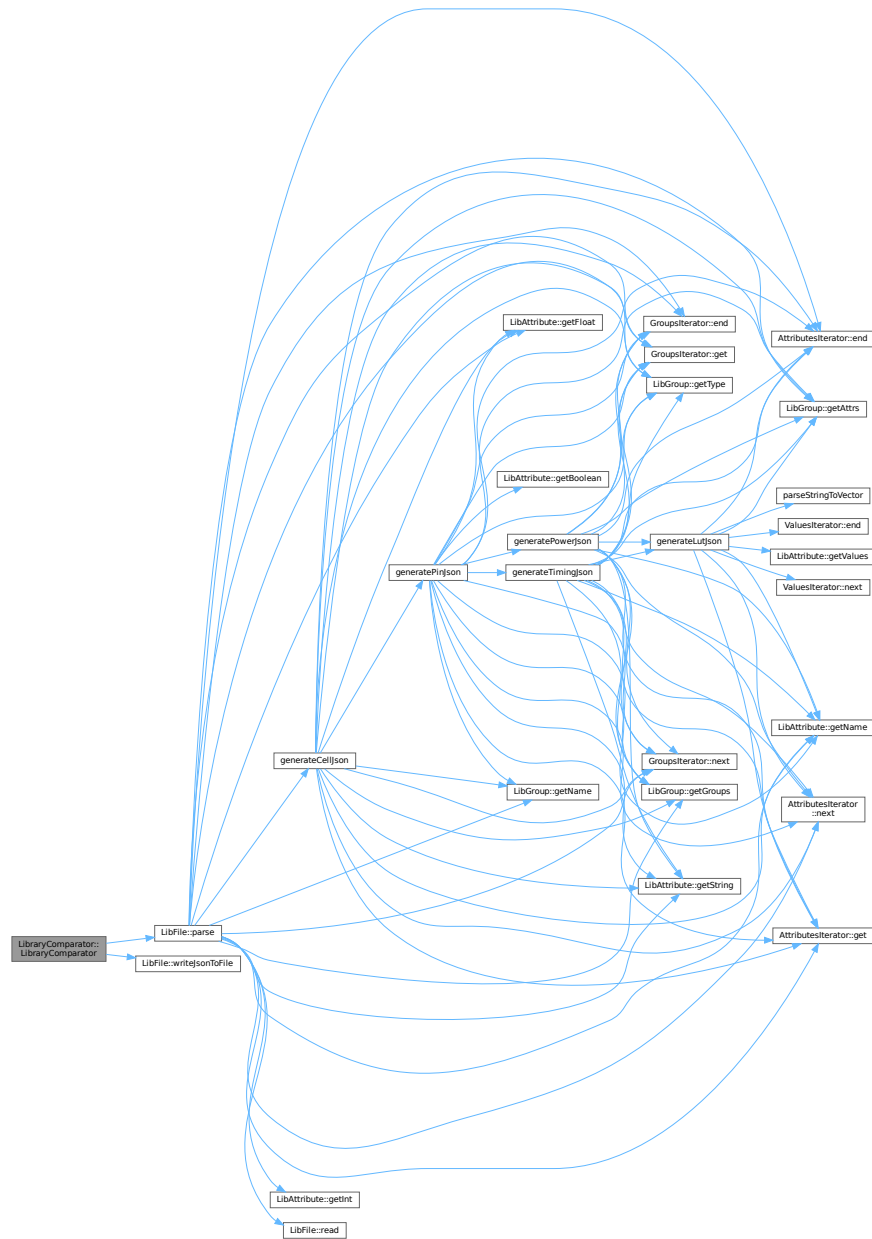
1. Looks for reference JSON file, parses library file if not found
2. Loads reference JSON data into `ref_json_` member
3. Looks for comparison JSON file, parses library file if not found
4. Loads comparison JSON data into `comp_json_` member
5. Stores file paths and initializes tolerance values for comparison

Parameters

<i>ref_libfile</i>	Reference library file object
<i>comp_libfile</i>	Comparison library file object
<i>reltol</i>	Relative tolerance for numerical comparisons (default defined elsewhere)
<i>abstol</i>	Absolute tolerance for numerical comparisons (default defined elsewhere)

Definition at line 31 of file [compare.cpp](#).

Here is the call graph for this function:



7.14.3 Member Function Documentation

7.14.3.1 compareCell()

```
void LibraryComparator::compareCell (
    const std::string & cell_name,
```

```
const json & ref_cell,
const json & comp_cell,
Table & table ) [private]
```

Compares the output pins of a cell between a reference JSON and a comparison JSON.

This function iterates through the output pins of the comparison cell and checks if each pin exists in the reference cell. If a pin is found in both cells, it calls the `comparePin` function to compare the details of the pin. If a pin is not found in the reference cell, a warning is logged. If the comparison cell does not contain any output pins, an informational message is logged.

Parameters

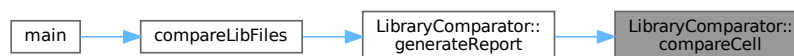
<i>cell_name</i>	The name of the cell being compared.
<i>ref_cell</i>	The reference JSON object containing the cell data.
<i>comp_cell</i>	The comparison JSON object containing the cell data.
<i>table</i>	The table object where the comparison results are stored.

Definition at line 308 of file [compare.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.14.3.2 compareLut()

```
void LibraryComparator::compareLut (
    const std::string & cell_name,
    const std::string & pin_name,
```

```

const std::string & timing_type,
const std::string & related_pin,
const std::string & arc_name,
const json & ref_lut,
const json & comp_lut,
Table & table ) [private]

```

Compares lookup tables (LUTs) between reference and comparison libraries for a specific timing arc.

This function compares the lookup table values between reference and comparison libraries for a given timing arc, checking the consistency of indices and values. It verifies that the indices match between libraries and then compares each value in the LUT against relative and absolute tolerance thresholds. Any discrepancies that exceed the tolerance limits are recorded in the provided table.

Parameters

<i>cell_name</i>	The name of the cell containing the LUT.
<i>pin_name</i>	The name of the pin associated with the LUT.
<i>timing_type</i>	The timing type of the arc (e.g., "rise_transition", "fall_transition").
<i>related_pin</i>	The related pin for the timing arc.
<i>arc_name</i>	The name of the specific timing arc being compared.
<i>ref_lut</i>	The lookup table from the reference library (in JSON format).
<i>comp_lut</i>	The lookup table from the comparison library (in JSON format).
<i>table</i>	Table object to store comparison results for any mismatches found.

Note

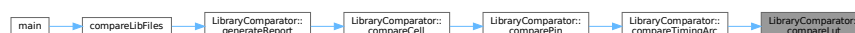
The function logs various information/warning/error messages:

- Logs index mismatches as warnings
- Logs value mismatches as debug messages
- Logs format errors in LUT structure as errors
- Records values exceeding tolerance in the provided table

The comparison uses both relative tolerance (`reltol_`) and absolute tolerance (`abstol_`) to determine if values differ significantly.

Definition at line 107 of file [compare.cpp](#).

Here is the caller graph for this function:



7.14.3.3 comparePin()

```
void LibraryComparator::comparePin (
    const std::string & cell_name,
    const std::string & pin_name,
    const json & ref_pin,
    const json & comp_pin,
    Table & table ) [private]
```

Compares the timing arcs of a given pin between a reference JSON and a comparison JSON.

This function iterates through the timing arcs of the comparison pin and looks for matching timing types in the reference pin. If a matching timing type is found, it calls the compareTimingArc function to compare the timing arcs. If a timing type is not found in the reference JSON, a warning is logged.

Parameters

<i>cell_name</i>	The name of the cell containing the pin.
<i>pin_name</i>	The name of the pin to compare.
<i>ref_pin</i>	The reference JSON object containing the pin's data.
<i>comp_pin</i>	The comparison JSON object containing the pin's data.
<i>table</i>	A reference to a Table object where comparison results are stored.

Definition at line 269 of file [compare.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.14.3.4 compareTimingArc()

```
void LibraryComparator::compareTimingArc (
    const std::string & cell_name,
    const std::string & pin_name,
    const std::string & timing_type,
    const json & ref_timing_arc,
    const json & comp_timing_arc,
    Table & table ) [private]
```

Compares a timing arc between two JSON objects.

This function compares a specific timing arc (e.g., cell_rise, cell_fall, rise_transition, fall_transition) between a reference JSON object and a comparison JSON object. It extracts the related pin from the reference timing arc and iterates through a predefined list of arc names. If an arc name is found in the comparison timing arc, it checks if the same arc name exists in the reference timing arc. If both exist, it calls the compareLut function to compare the LUT data associated with the arc. If the arc name is found in the comparison JSON but not in the reference JSON, a warning message is logged.

Parameters

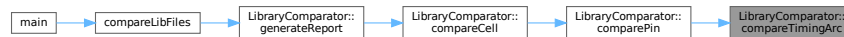
<i>cell_name</i>	The name of the cell.
<i>pin_name</i>	The name of the pin.
<i>timing_type</i>	The type of timing (e.g., "combinational", "sequential").
<i>ref_timing_arc</i>	The reference JSON object containing the timing arc information.
<i>comp_timing_arc</i>	The comparison JSON object containing the timing arc information.
<i>table</i>	The table to store comparison results.

Definition at line 229 of file [compare.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.14.3.5 generateReport()

```
void LibraryComparator::generateReport (
    const std::string & output_file )
```

Generates a comparison report between the reference and comparison libraries.

This function generates a detailed comparison report between the reference library and the comparison library. The report includes metadata such as the reference and comparison library paths, absolute and relative tolerances, and the application details. It also includes a legend explaining various symbols used in the report.

The function iterates through each cell in the comparison library, compares it with the corresponding cell in the reference library, and generates a table of differences. If there are any differences, it calculates summary statistics such as the average and maximum differences, and the number of outliers. The results are written to the specified output file in either Markdown or plain text format.

Parameters

<i>output_file</i>	The path to the output file where the report will be written.
--------------------	---

Definition at line 350 of file [compare.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.14.4 Member Data Documentation

7.14.4.1 abstol_

```
double LibraryComparator::abstol_ [private]
```

Definition at line 23 of file [compare.hpp](#).

7.14.4.2 comp_json_

```
json LibraryComparator::comp_json_ [private]
```

Definition at line 21 of file [compare.hpp](#).

7.14.4.3 comp_lib_path_

```
std::filesystem::path LibraryComparator::comp_lib_path_
```

Definition at line 16 of file [compare.hpp](#).

7.14.4.4 ref_json_

```
json LibraryComparator::ref_json_ [private]
```

Definition at line 20 of file [compare.hpp](#).

7.14.4.5 ref_lib_path_

```
std::filesystem::path LibraryComparator::ref_lib_path_
```

Definition at line 15 of file [compare.hpp](#).

7.14.4.6 reltol_

```
double LibraryComparator::reltol_ [private]
```

Definition at line 22 of file [compare.hpp](#).

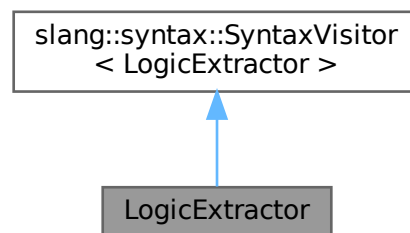
The documentation for this class was generated from the following files:

- include/[compare.hpp](#)
- src/[compare.cpp](#)

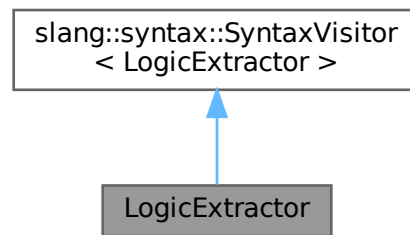
7.15 LogicExtractor Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for LogicExtractor:



Collaboration diagram for LogicExtractor:



Public Member Functions

- `LogicExtractor` (const std::string &targetCell)
- void `handle` (const slang::syntax::ModuleDeclarationSyntax &module)
- void `handle` (const slang::syntax::PortDeclarationSyntax &portDecl)
- void `handle` (const slang::syntax::NonAnsiPortListSyntax &portList)
- void `handle` (const slang::syntax::NetDeclarationSyntax &netDecl)
- void `handle` (const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst)
- const std::unordered_map< std::string, `GateInfo` > & `getExtractedGates` () const
- const std::unordered_set< std::string > & `getPrimaryInputs` () const
- const std::unordered_set< std::string > & `getPrimaryOutputs` () const
- const std::unordered_set< std::string > & `getInternalWires` () const
- std::map< std::string, std::string > `getLogicExpressions` ()

Private Member Functions

- std::string `deriveLogicRecursive` (const std::string &signalName)
- std::string `formatExpression` (const `GateInfo` &gateInfo, const std::vector< std::string > &inputExprs)

Private Attributes

- const std::string & `targetCell_`
- bool `inTargetModule_`
- bool `parsingComplete_`
- std::unordered_set< std::string > `primaryInputs_`
- std::unordered_set< std::string > `primaryOutputs_`
- std::unordered_set< std::string > `internalWires_`
- std::unordered_map< std::string, std::string > `portDirections_`
- std::unordered_map< std::string, `GateInfo` > `gateOutputDrivers_`
- std::unordered_map< std::string, std::string > `logicCache_`

7.15.1 Detailed Description

Definition at line 105 of file [verilog_utils.hpp](#).

7.15.2 Constructor & Destructor Documentation

7.15.2.1 LogicExtractor()

```
LogicExtractor::LogicExtractor (
    const std::string & targetCell ) [inline], [explicit]
```

Definition at line 108 of file [verilog_utils.hpp](#).

7.15.3 Member Function Documentation

7.15.3.1 deriveLogicRecursive()

```
std::string LogicExtractor::deriveLogicRecursive (
    const std::string & signalName ) [private]
```

Definition at line 989 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.15.3.2 formatExpression()

```
std::string LogicExtractor::formatExpression (
    const GateInfo & gateInfo,
    const std::vector< std::string > & inputExprs ) [private]
```

Definition at line 1046 of file [verilog_utils.cpp](#).

Here is the caller graph for this function:



7.15.3.3 getExtractedGates()

```
const std::unordered_map< std::string, GateInfo > & LogicExtractor::getExtractedGates ( ) const [inline]
```

Definition at line 125 of file [verilog_utils.hpp](#).

Here is the caller graph for this function:

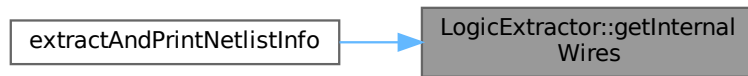


7.15.3.4 getInternalWires()

```
const std::unordered_set< std::string > & LogicExtractor::getInternalWires ( ) const [inline]
```

Definition at line 130 of file [verilog_utils.hpp](#).

Here is the caller graph for this function:



7.15.3.5 getLogicExpressions()

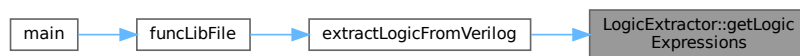
```
std::map< std::string, std::string > LogicExtractor::getLogicExpressions ( )
```

Definition at line 959 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:

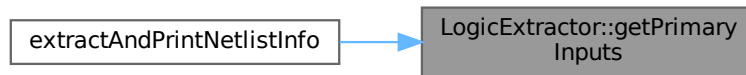


7.15.3.6 getPrimaryInputs()

```
const std::unordered_set< std::string > & LogicExtractor::getPrimaryInputs ( ) const [inline]
```

Definition at line 128 of file [verilog_utils.hpp](#).

Here is the caller graph for this function:

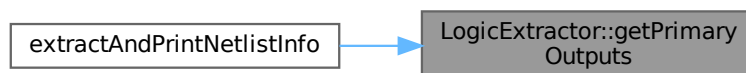


7.15.3.7 getPrimaryOutputs()

```
const std::unordered_set< std::string > & LogicExtractor::getPrimaryOutputs ( ) const [inline]
```

Definition at line 129 of file [verilog_utils.hpp](#).

Here is the caller graph for this function:

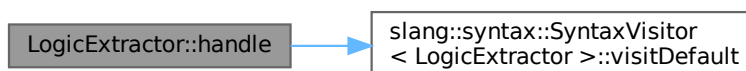


7.15.3.8 handle() [1/5]

```
void LogicExtractor::handle (
    const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 614 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



7.15.3.9 handle() [2/5]

```
void LogicExtractor::handle (
    const slang::syntax::NetDeclarationSyntax & netDecl )
```

Definition at line 810 of file [verilog_utils.cpp](#).

7.15.3.10 handle() [3/5]

```
void LogicExtractor::handle (
    const slang::syntax::NonAnsiPortListSyntax & portList )
```

Definition at line 721 of file [verilog_utils.cpp](#).

7.15.3.11 handle() [4/5]

```
void LogicExtractor::handle (
    const slang::syntax::PortDeclarationSyntax & portDecl )
```

Definition at line 658 of file [verilog_utils.cpp](#).

7.15.3.12 handle() [5/5]

```
void LogicExtractor::handle (
    const slang::syntax::PrimitiveInstantiationSyntax & primitiveInst )
```

Definition at line 830 of file [verilog_utils.cpp](#).

7.15.4 Member Data Documentation

7.15.4.1 gateOutputDrivers_

```
std::unordered_map<std::string, GateInfo> LogicExtractor::gateOutputDrivers_ [private]
```

Definition at line 153 of file [verilog_utils.hpp](#).

7.15.4.2 inTargetModule_

```
bool LogicExtractor::inTargetModule_ [private]
```

Definition at line 140 of file [verilog_utils.hpp](#).

7.15.4.3 internalWires_

```
std::unordered_set<std::string> LogicExtractor::internalWires_ [private]
```

Definition at line 146 of file [verilog_utils.hpp](#).

7.15.4.4 logicCache_

```
std::unordered_map<std::string, std::string> LogicExtractor::logicCache_ [private]
```

Definition at line 156 of file [verilog_utils.hpp](#).

7.15.4.5 parsingComplete_

```
bool LogicExtractor::parsingComplete_ [private]
```

Definition at line 141 of file [verilog_utils.hpp](#).

7.15.4.6 portDirections_

```
std::unordered_map<std::string, std::string> LogicExtractor::portDirections_ [private]
```

Definition at line 150 of file [verilog_utils.hpp](#).

7.15.4.7 primaryInputs_

```
std::unordered_set<std::string> LogicExtractor::primaryInputs_ [private]
```

Definition at line 144 of file [verilog_utils.hpp](#).

7.15.4.8 primaryOutputs_

```
std::unordered_set<std::string> LogicExtractor::primaryOutputs_ [private]
```

Definition at line 145 of file [verilog_utils.hpp](#).

7.15.4.9 targetCell_

```
const std::string& LogicExtractor::targetCell_ [private]
```

Definition at line 139 of file [verilog_utils.hpp](#).

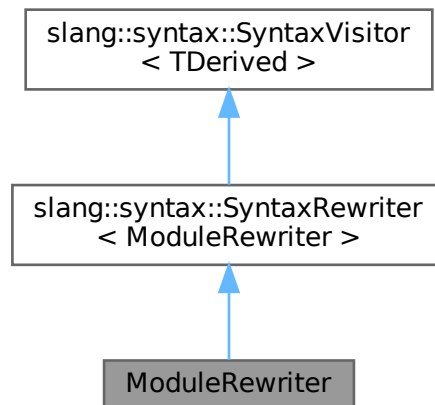
The documentation for this class was generated from the following files:

- [include/verilog_utils.hpp](#)
- [src/verilog_utils.cpp](#)

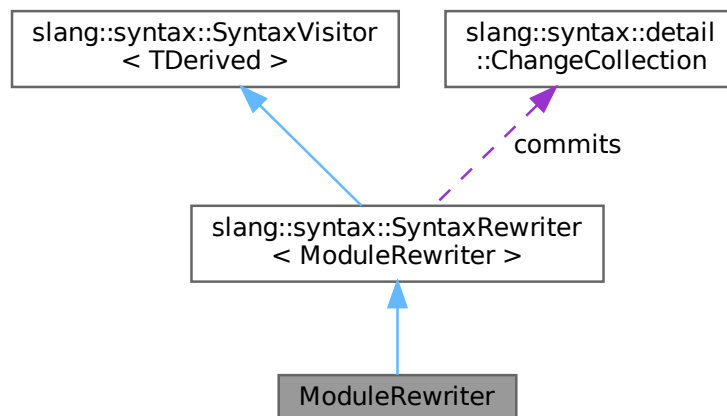
7.16 ModuleRewriter Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for ModuleRewriter:



Collaboration diagram for ModuleRewriter:



Public Member Functions

- `ModuleRewriter` (const std::vector< std::string > &inputPins, const std::vector< std::string > &outputPins, const std::pair< std::string, std::string > &supercell_entry, int instance_count, std::shared_ptr< spdlog::logger > logger)

- void `handle` (const `slang::syntax::SyntaxNode` &node)
Processes a syntax node in the module's AST.
- void `handle` (const `slang::syntax::ModuleDeclarationSyntax` &module)

Public Attributes

- `std::shared_ptr< spdlog::logger >` `logger_`

Private Attributes

- `const std::vector< std::string > &` `inputPins_`
- `const std::vector< std::string > &` `outputPins_`
- `const std::string` `cellName_`
- `const std::string` `moduleName_`
- `std::map< std::string, std::string >` `portInfoMap_`
- `int` `depth_`
- `int` `instance_count_`

Additional Inherited Members

7.16.1 Detailed Description

Definition at line 69 of file `verilog_utils.hpp`.

7.16.2 Constructor & Destructor Documentation

7.16.2.1 ModuleRewriter()

```
ModuleRewriter::ModuleRewriter (
    const std::vector< std::string > & inputPins,
    const std::vector< std::string > & outputPins,
    const std::pair< std::string, std::string > & supercell_entry,
    int instance_count,
    std::shared_ptr< spdlog::logger > logger ) [inline], [explicit]
```

Definition at line 71 of file `verilog_utils.hpp`.

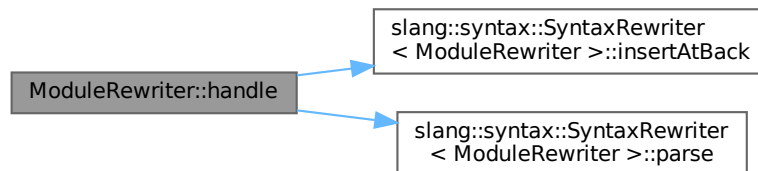
7.16.3 Member Function Documentation

7.16.3.1 `handle()` [1/2]

```
void ModuleRewriter::handle (
    const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 408 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



7.16.3.2 `handle()` [2/2]

```
void ModuleRewriter::handle (
    const slang::syntax::SyntaxNode & node )
```

Processes a syntax node in the module's AST.

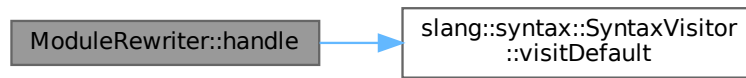
This method is called for each syntax node during the traversal of the AST. It logs debug information about the current node and continues processing its child nodes by recursively calling the appropriate visit methods.

Parameters

<i>node</i>	The syntax node to process
-------------	----------------------------

Definition at line 398 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



7.16.4 Member Data Documentation

7.16.4.1 cellName_

```
const std::string ModuleRewriter::cellName_ [private]
```

Definition at line 85 of file [verilog_utils.hpp](#).

7.16.4.2 depth_

```
int ModuleRewriter::depth_ [private]
```

Definition at line 88 of file [verilog_utils.hpp](#).

7.16.4.3 inputPins_

```
const std::vector<std::string>& ModuleRewriter::inputPins_ [private]
```

Definition at line 83 of file [verilog_utils.hpp](#).

7.16.4.4 instance_count_

```
int ModuleRewriter::instance_count_ [private]
```

Definition at line 89 of file [verilog_utils.hpp](#).

7.16.4.5 logger_

```
std::shared_ptr<spdlog::logger> ModuleRewriter::logger_
```

Definition at line 78 of file [verilog_utils.hpp](#).

7.16.4.6 moduleName_

```
const std::string ModuleRewriter::moduleName_ [private]
```

Definition at line 86 of file [verilog_utils.hpp](#).

7.16.4.7 outputPins_

```
const std::vector<std::string>& ModuleRewriter::outputPins_ [private]
```

Definition at line 84 of file [verilog_utils.hpp](#).

7.16.4.8 portInfoMap_

```
std::map<std::string, std::string> ModuleRewriter::portInfoMap_ [private]
```

Definition at line 87 of file [verilog_utils.hpp](#).

The documentation for this class was generated from the following files:

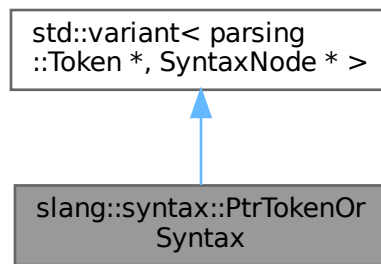
- include/[verilog_utils.hpp](#)
- src/[verilog_utils.cpp](#)

7.17 slang::syntax::PtrTokenOrSyntax Struct Reference

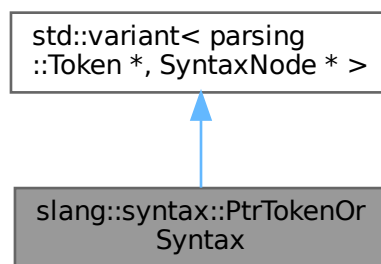
A token pointer or a syntax node.

```
#include <SyntaxNode.h>
```

Inheritance diagram for slang::syntax::PtrTokenOrSyntax:



Collaboration diagram for slang::syntax::PtrTokenOrSyntax:



Public Types

- using [Base](#) = std::variant< parsing::Token *, [SyntaxNode](#) * >

Public Member Functions

- [PtrTokenOrSyntax](#) (parsing::Token **token*)
- [PtrTokenOrSyntax](#) (SyntaxNode **node*)
- [PtrTokenOrSyntax](#) (nullptr_t)
- bool [isToken](#) () const
- bool [isNode](#) () const
- parsing::Token * [token](#) () const
- SyntaxNode * [node](#) () const
- SourceRange [range](#) () const

7.17.1 Detailed Description

A token pointer or a syntax node.

Definition at line 24 of file [SyntaxNode.h](#).

7.17.2 Member Typedef Documentation

7.17.2.1 Base

```
using slang::syntax::PtrTokenOrSyntax::Base = std::variant<parsing::Token*, SyntaxNode*>
```

Definition at line 25 of file [SyntaxNode.h](#).

7.17.3 Constructor & Destructor Documentation

7.17.3.1 PtrTokenOrSyntax() [1/3]

```
slang::syntax::PtrTokenOrSyntax::PtrTokenOrSyntax (
    parsing::Token * token ) [inline]
```

Definition at line 26 of file [SyntaxNode.h](#).

7.17.3.2 PtrTokenOrSyntax() [2/3]

```
slang::syntax::PtrTokenOrSyntax::PtrTokenOrSyntax (  
    SyntaxNode * node ) [inline]
```

Definition at line 27 of file [SyntaxNode.h](#).

7.17.3.3 PtrTokenOrSyntax() [3/3]

```
slang::syntax::PtrTokenOrSyntax::PtrTokenOrSyntax (  
    nullptr_t ) [inline]
```

Definition at line 28 of file [SyntaxNode.h](#).

7.17.4 Member Function Documentation

7.17.4.1 isNode()

```
bool slang::syntax::PtrTokenOrSyntax::isNode ( ) const [inline]
```

Returns

true if the object is a syntax node.

Definition at line 34 of file [SyntaxNode.h](#).

7.17.4.2 isToken()

```
bool slang::syntax::PtrTokenOrSyntax::isToken ( ) const [inline]
```

Returns

true if the object is a token.

Definition at line 31 of file [SyntaxNode.h](#).

7.17.4.3 node()

```
SyntaxNode * slang::syntax::PtrTokenOrSyntax::node ( ) const [inline]
```

Gets access to the object as a syntax node (throwing an exception if it's not actually a syntax node).

Definition at line 42 of file [SyntaxNode.h](#).

Here is the caller graph for this function:



7.17.4.4 range()

```
SourceRange slang::syntax::PtrTokenOrSyntax::range ( ) const
```

Gets the source range for the token or syntax node or NoLocation if it points to nullptr.

7.17.4.5 token()

```
parsing::Token * slang::syntax::PtrTokenOrSyntax::token ( ) const [inline]
```

Gets access to the object as a token (throwing an exception if it's not actually a token).

Definition at line 38 of file [SyntaxNode.h](#).

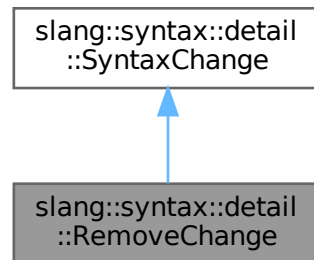
The documentation for this struct was generated from the following file:

- `include_3rd_party/SyntaxNode.h`

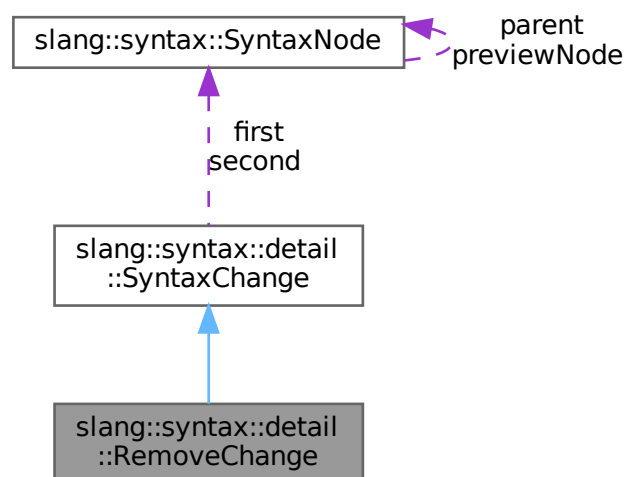
7.18 slang::syntax::detail::RemoveChange Struct Reference

```
#include <SyntaxVisitor.h>
```

Inheritance diagram for slang::syntax::detail::RemoveChange:



Collaboration diagram for slang::syntax::detail::RemoveChange:



Additional Inherited Members

7.18.1 Detailed Description

Definition at line 65 of file [SyntaxVisitor.h](#).

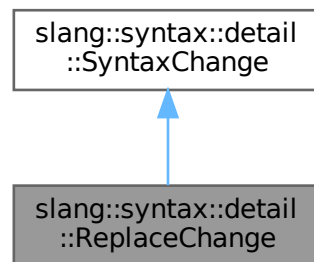
The documentation for this struct was generated from the following file:

- `include_3rd_party/SyntaxVisitor.h`

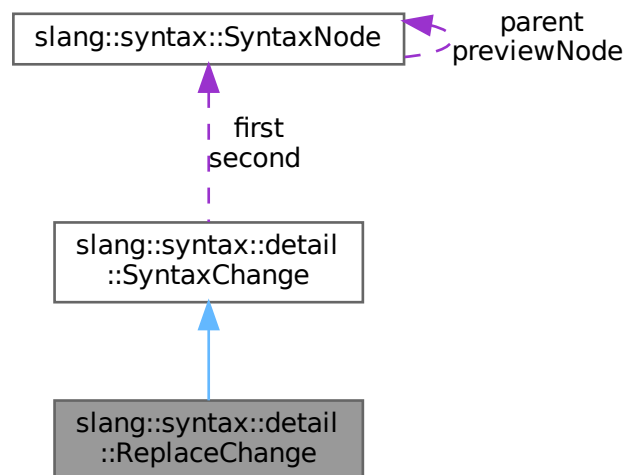
7.19 slang::syntax::detail::ReplaceChange Struct Reference

```
#include <SyntaxVisitor.h>
```

Inheritance diagram for slang::syntax::detail::ReplaceChange:



Collaboration diagram for slang::syntax::detail::ReplaceChange:



Additional Inherited Members

7.19.1 Detailed Description

Definition at line 66 of file [SyntaxVisitor.h](#).

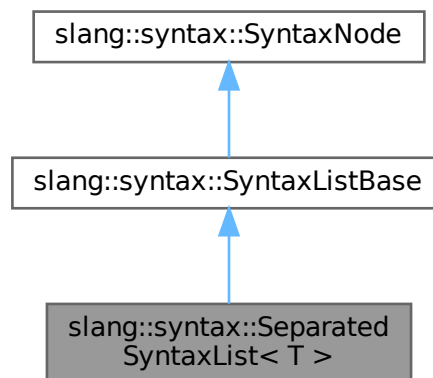
The documentation for this struct was generated from the following file:

- include_3rd_party/[SyntaxVisitor.h](#)

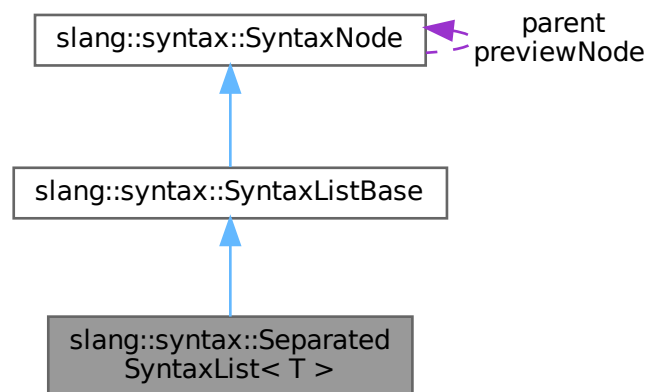
7.20 slang::syntax::SeparatedSyntaxList< T > Class Template Reference

```
#include <SyntaxNode.h>
```

Inheritance diagram for slang::syntax::SeparatedSyntaxList< T >:



Collaboration diagram for slang::syntax::SeparatedSyntaxList< T >:



Classes

- class [iterator_base](#)

Public Types

- using [iterator](#) = [iterator_base](#)< T * >
- using [const_iterator](#) = [iterator_base](#)< const T * >

Public Member Functions

- [SeparatedSyntaxList](#) (nullptr_t)
- [SeparatedSyntaxList](#) (std::span< [TokenOrSyntax](#) > [elements](#))
- std::span< const [ConstTokenOrSyntax](#) > [elems](#) () const
- std::span< [TokenOrSyntax](#) > [elems](#) ()
- bool [empty](#) () const
- size_t [size](#) () const noexcept
- const T * [operator\[\]](#) (size_t index) const
- T * [operator\[\]](#) (size_t index)
- [iterator](#) [begin](#) ()
- [iterator](#) [end](#) ()
- [const_iterator](#) [begin](#) () const
- [const_iterator](#) [end](#) () const
- [const_iterator](#) [cbegin](#) () const
- [const_iterator](#) [cend](#) () const

Private Member Functions

- [TokenOrSyntax getChild](#) (size_t index) final
Gets the child (token or node) at the given index.
- [ConstTokenOrSyntax getChild](#) (size_t index) const final
Gets the child (token or node) at the given index.
- [PtrTokenOrSyntax getChildPtr](#) (size_t index) final
- void [setChild](#) (size_t index, [TokenOrSyntax](#) child) final
Sets the child (token or node) at the given index.
- [SyntaxListBase](#) * [clone](#) (BumpAllocator &alloc) const final
Clones the list into a new node using the provided allocator.
- void [resetAll](#) (BumpAllocator &alloc, std::span< const [TokenOrSyntax](#) > children) final

Private Attributes

- std::span< [TokenOrSyntax](#) > [elements](#)

Additional Inherited Members

7.20.1 Detailed Description

```
template<typename T>
class slang::syntax::SeparatedSyntaxList< T >
```

A syntax node that represents a token-separated list of child syntax nodes. The stored children are assumed to alternate between delimiters (such as a comma) and nodes.

Definition at line 381 of file [SyntaxNode.h](#).

7.20.2 Member Typedef Documentation

7.20.2.1 const_iterator

```
template<typename T >
using slang::syntax::SeparatedSyntaxList< T >::const_iterator = iterator\_base<const T*>
```

Definition at line 412 of file [SyntaxNode.h](#).

7.20.2.2 iterator

```
template<typename T >
using slang::syntax::SeparatedSyntaxList< T >::iterator = iterator_base<T*>
```

Definition at line 411 of file [SyntaxNode.h](#).

7.20.3 Constructor & Destructor Documentation

7.20.3.1 SeparatedSyntaxList() [1/2]

```
template<typename T >
slang::syntax::SeparatedSyntaxList< T >::SeparatedSyntaxList (
    nullptr_t ) [inline]
```

Definition at line 414 of file [SyntaxNode.h](#).

7.20.3.2 SeparatedSyntaxList() [2/2]

```
template<typename T >
slang::syntax::SeparatedSyntaxList< T >::SeparatedSyntaxList (
    std::span< TokenOrSyntax > elements ) [inline]
```

Definition at line 415 of file [SyntaxNode.h](#).

7.20.4 Member Function Documentation

7.20.4.1 begin() [1/2]

```
template<typename T >
iterator slang::syntax::SeparatedSyntaxList< T >::begin ( ) [inline]
```

Definition at line 443 of file [SyntaxNode.h](#).

7.20.4.2 begin() [2/2]

```
template<typename T >
const_iterator slang::syntax::SeparatedSyntaxList< T >::begin ( ) const [inline]
```

Definition at line 445 of file [SyntaxNode.h](#).

7.20.4.3 cbegin()

```
template<typename T >
const_iterator slang::syntax::SeparatedSyntaxList< T >::cbegin ( ) const [inline]
```

Definition at line 447 of file [SyntaxNode.h](#).

7.20.4.4 cend()

```
template<typename T >
const_iterator slang::syntax::SeparatedSyntaxList< T >::cend ( ) const [inline]
```

Definition at line 448 of file [SyntaxNode.h](#).

7.20.4.5 clone()

```
template<typename T >
SyntaxListBase * slang::syntax::SeparatedSyntaxList< T >::clone (
    BumpAllocator & alloc ) const [inline], [final], [private], [virtual]
```

Clones the list into a new node using the provided allocator.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 461 of file [SyntaxNode.h](#).

7.20.4.6 elems() [1/2]

```
template<typename T >
std::span< TokenOrSyntax > slang::syntax::SeparatedSyntaxList< T >::elems ( ) [inline]
```

Returns

the elements of nodes in the list

Definition at line 425 of file [SyntaxNode.h](#).

7.20.4.7 elems() [2/2]

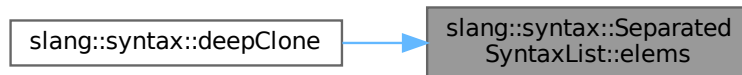
```
template<typename T >
std::span< const ConstTokenOrSyntax > slang::syntax::SeparatedSyntaxList< T >::elems ( ) const [inline]
```

Returns

the elements of nodes in the list

Definition at line 419 of file [SyntaxNode.h](#).

Here is the caller graph for this function:

**7.20.4.8 empty()**

```
template<typename T >
bool slang::syntax::SeparatedSyntaxList< T >::empty ( ) const [inline]
```

Returns

true if the list is empty, and false if it has elements.

Definition at line 428 of file [SyntaxNode.h](#).

7.20.4.9 end() [1/2]

```
template<typename T >
iterator slang::syntax::SeparatedSyntaxList< T >::end ( ) [inline]
```

Definition at line 444 of file [SyntaxNode.h](#).

7.20.4.10 end() [2/2]

```
template<typename T >
const_iterator slang::syntax::SeparatedSyntaxList< T >::end ( ) const [inline]
```

Definition at line 446 of file [SyntaxNode.h](#).

7.20.4.11 getChild() [1/2]

```
template<typename T >
ConstTokenOrSyntax slang::syntax::SeparatedSyntaxList< T >::getChild (
    size_t index ) const [inline], [final], [private], [virtual]
```

Gets the child (token or node) at the given index.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 452 of file [SyntaxNode.h](#).

7.20.4.12 getChild() [2/2]

```
template<typename T >
TokenOrSyntax slang::syntax::SeparatedSyntaxList< T >::getChild (
    size_t index ) [inline], [final], [private], [virtual]
```

Gets the child (token or node) at the given index.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 451 of file [SyntaxNode.h](#).

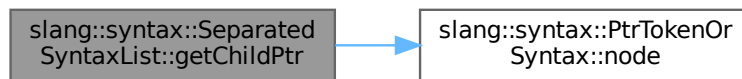
7.20.4.13 getChildPtr()

```
template<typename T >
PtrTokenOrSyntax slang::syntax::SeparatedSyntaxList< T >::getChildPtr (
    size_t index ) [inline], [final], [private], [virtual]
```

Implements [slang::syntax::SyntaxListBase](#).

Definition at line [453](#) of file [SyntaxNode.h](#).

Here is the call graph for this function:



7.20.4.14 operator[]() [1/2]

```
template<typename T >
T * slang::syntax::SeparatedSyntaxList< T >::operator[] (
    size_t index ) [inline]
```

Definition at line [438](#) of file [SyntaxNode.h](#).

7.20.4.15 operator[]() [2/2]

```
template<typename T >
const T * slang::syntax::SeparatedSyntaxList< T >::operator[] (
    size_t index ) const [inline]
```

Definition at line [433](#) of file [SyntaxNode.h](#).

7.20.4.16 resetAll()

```
template<typename T >
void slang::syntax::SeparatedSyntaxList< T >::resetAll (
    BumpAllocator & alloc,
    std::span< const TokenOrSyntax > children ) [inline], [final], [private], [virtual]
```

Overwrites all children with the new set of provided *children* (and making a copy with the provided allocator).

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 465 of file [SyntaxNode.h](#).

7.20.4.17 setChild()

```
template<typename T >
void slang::syntax::SeparatedSyntaxList< T >::setChild (
    size_t index,
    TokenOrSyntax child ) [inline], [final], [private], [virtual]
```

Sets the child (token or node) at the given index.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 459 of file [SyntaxNode.h](#).

7.20.4.18 size()

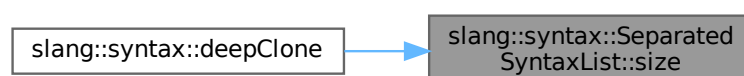
```
template<typename T >
size_t slang::syntax::SeparatedSyntaxList< T >::size ( ) const [inline], [noexcept]
```

Returns

the number of nodes in the list (doesn't include delimiter tokens in the count).

Definition at line 431 of file [SyntaxNode.h](#).

Here is the caller graph for this function:



7.20.5 Member Data Documentation

7.20.5.1 elements

```
template<typename T >
std::span<TokenOrSyntax> slang::syntax::SeparatedSyntaxList< T >::elements [private]
```

Definition at line 473 of file [SyntaxNode.h](#).

The documentation for this class was generated from the following file:

- include_3rd_party/[SyntaxNode.h](#)

7.21 si2drExprT Struct Reference

```
#include <si2dr_liberty.h>
```

Collaboration diagram for si2drExprT:



Public Attributes

- [si2drExprTypeT](#) type
- union {
 - [si2drInt32T](#) i
 - [si2drFloat64T](#) d
 - [si2drStringT](#) s
 - [si2drBooleanT](#) b
 } u
- [si2drValueTypeT](#) valuetype
- struct [si2drExprT](#) * left
- struct [si2drExprT](#) * right

7.21.1 Detailed Description

Definition at line 172 of file [si2dr_liberty.h](#).

7.21.2 Member Data Documentation

7.21.2.1 b

[si2drBooleanT](#) si2drExprT::b

Definition at line 180 of file [si2dr_liberty.h](#).

7.21.2.2 d

[si2drFloat64T](#) si2drExprT::d

Definition at line 178 of file [si2dr_liberty.h](#).

7.21.2.3 i

[si2drInt32T](#) si2drExprT::i

Definition at line 177 of file [si2dr_liberty.h](#).

7.21.2.4 left

struct [si2drExprT](#)* si2drExprT::left

Definition at line 185 of file [si2dr_liberty.h](#).

7.21.2.5 right

```
struct si2drExprT* si2drExprT::right
```

Definition at line [186](#) of file [si2dr_liberty.h](#).

7.21.2.6 s

```
si2drStringT si2drExprT::s
```

Definition at line [179](#) of file [si2dr_liberty.h](#).

7.21.2.7 type

```
si2drExprTypeT si2drExprT::type
```

Definition at line [174](#) of file [si2dr_liberty.h](#).

7.21.2.8

```
union { ... } si2drExprT::u
```

7.21.2.9 valuetype

```
si2drValueTypeT si2drExprT::valuetype
```

Definition at line [183](#) of file [si2dr_liberty.h](#).

The documentation for this struct was generated from the following file:

- `include_3rd_party/si2dr_liberty.h`

7.22 si2OID Struct Reference

```
#include <si2dr_liberty.h>
```

Public Attributes

- void * [v1](#)
- void * [v2](#)

7.22.1 Detailed Description

Definition at line [79](#) of file [si2dr_liberty.h](#).

7.22.2 Member Data Documentation

7.22.2.1 v1

```
void* si2OID::v1
```

Definition at line [79](#) of file [si2dr_liberty.h](#).

7.22.2.2 v2

```
void * si2OID::v2
```

Definition at line [79](#) of file [si2dr_liberty.h](#).

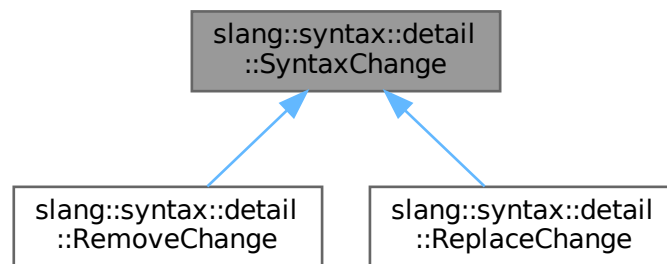
The documentation for this struct was generated from the following file:

- [include_3rd_party/si2dr_liberty.h](#)

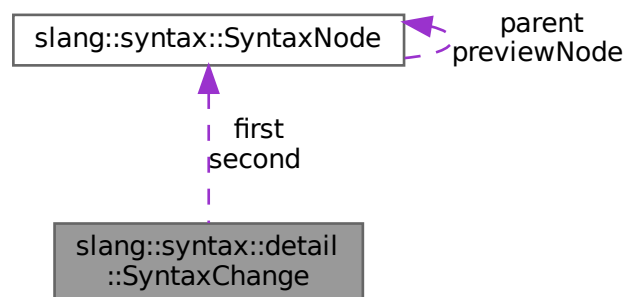
7.23 slang::syntax::detail::SyntaxChange Struct Reference

```
#include <SyntaxVisitor.h>
```

Inheritance diagram for slang::syntax::detail::SyntaxChange:



Collaboration diagram for slang::syntax::detail::SyntaxChange:



Public Attributes

- `const SyntaxNode * first = nullptr`
- `SyntaxNode * second = nullptr`
- `parsing::Token separator = {}`

7.23.1 Detailed Description

Definition at line 59 of file [SyntaxVisitor.h](#).

7.23.2 Member Data Documentation

7.23.2.1 first

```
const SyntaxNode* slang::syntax::detail::SyntaxChange::first = nullptr
```

Definition at line 60 of file [SyntaxVisitor.h](#).

7.23.2.2 second

```
SyntaxNode* slang::syntax::detail::SyntaxChange::second = nullptr
```

Definition at line 61 of file [SyntaxVisitor.h](#).

7.23.2.3 separator

```
parsing::Token slang::syntax::detail::SyntaxChange::separator = {}
```

Definition at line 62 of file [SyntaxVisitor.h](#).

The documentation for this struct was generated from the following file:

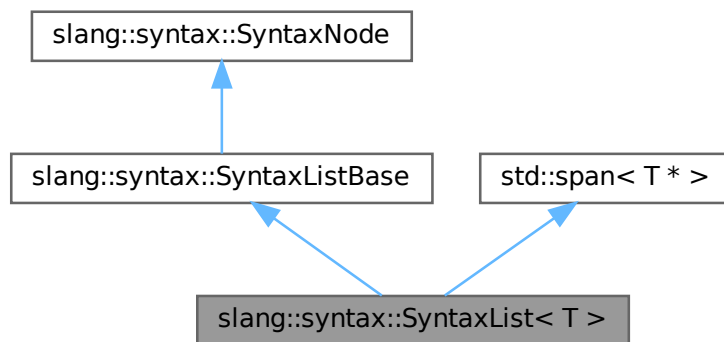
- include_3rd_party/[SyntaxVisitor.h](#)

7.24 slang::syntax::SyntaxList< T > Class Template Reference

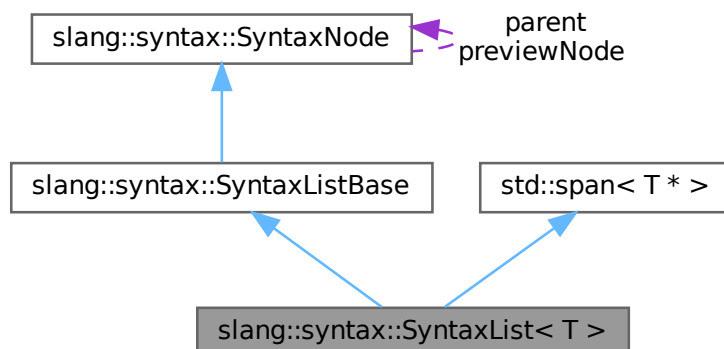
A syntax node that represents a list of child syntax nodes.

```
#include <SyntaxNode.h>
```

Inheritance diagram for slang::syntax::SyntaxList< T >:



Collaboration diagram for slang::syntax::SyntaxList< T >:



Public Member Functions

- [SyntaxList](#) (nullptr_t)
- [SyntaxList](#) (std::span< T * > elements)

Private Member Functions

- [TokenOrSyntax getChild](#) (size_t index) final
Gets the child (token or node) at the given index.
- [ConstTokenOrSyntax getChild](#) (size_t index) const final
Gets the child (token or node) at the given index.
- [PtrTokenOrSyntax getChildPtr](#) (size_t index) final
- void [setChild](#) (size_t index, [TokenOrSyntax](#) child) final
Sets the child (token or node) at the given index.
- [SyntaxListBase * clone](#) (BumpAllocator &alloc) const final
Clones the list into a new node using the provided allocator.
- void [resetAll](#) (BumpAllocator &alloc, std::span< const [TokenOrSyntax](#) > children) final

Additional Inherited Members

7.24.1 Detailed Description

```
template<typename T>
class slang::syntax::SyntaxList< T >
```

A syntax node that represents a list of child syntax nodes.

Definition at line 313 of file [SyntaxNode.h](#).

7.24.2 Constructor & Destructor Documentation

7.24.2.1 SyntaxList() ^[1/2]

```
template<typename T >
slang::syntax::SyntaxList< T >::SyntaxList (
    nullptr_t ) [inline]
```

Definition at line 315 of file [SyntaxNode.h](#).

7.24.2.2 SyntaxList() [2/2]

```
template<typename T >
slang::syntax::SyntaxList< T >::SyntaxList (
    std::span< T * > elements ) [inline]
```

Definition at line 316 of file [SyntaxNode.h](#).

7.24.3 Member Function Documentation

7.24.3.1 clone()

```
template<typename T >
SyntaxListBase * slang::syntax::SyntaxList< T >::clone (
    BumpAllocator & alloc ) const [inline], [final], [private], [virtual]
```

Clones the list into a new node using the provided allocator.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 328 of file [SyntaxNode.h](#).

7.24.3.2 getChild() [1/2]

```
template<typename T >
ConstTokenOrSyntax slang::syntax::SyntaxList< T >::getChild (
    size_t index ) const [inline], [final], [private], [virtual]
```

Gets the child (token or node) at the given index.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 321 of file [SyntaxNode.h](#).

7.24.3.3 getChild() [2/2]

```
template<typename T >
TokenOrSyntax slang::syntax::SyntaxList< T >::getChild (
    size_t index ) [inline], [final], [private], [virtual]
```

Gets the child (token or node) at the given index.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 320 of file [SyntaxNode.h](#).

7.24.3.4 getChildPtr()

```
template<typename T >
PtrTokenOrSyntax slang::syntax::SyntaxList< T >::getChildPtr (
    size_t index ) [inline], [final], [private], [virtual]
```

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 322 of file [SyntaxNode.h](#).

7.24.3.5 resetAll()

```
template<typename T >
void slang::syntax::SyntaxList< T >::resetAll (
    BumpAllocator & alloc,
    std::span< const TokenOrSyntax > children ) [inline], [final], [private], [virtual]
```

Overwrites all children with the new set of provided *children* (and making a copy with the provided allocator).

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 332 of file [SyntaxNode.h](#).

7.24.3.6 setChild()

```
template<typename T >
void slang::syntax::SyntaxList< T >::setChild (
    size_t index,
    TokenOrSyntax child ) [inline], [final], [private], [virtual]
```

Sets the child (token or node) at the given index.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 324 of file [SyntaxNode.h](#).

The documentation for this class was generated from the following file:

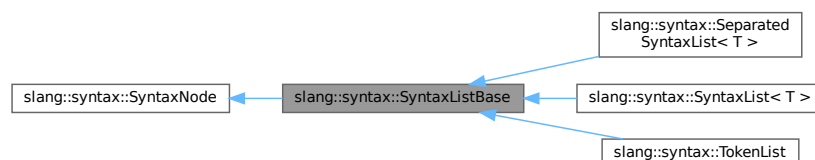
- include_3rd_party/[SyntaxNode.h](#)

7.25 slang::syntax::SyntaxListBase Class Reference

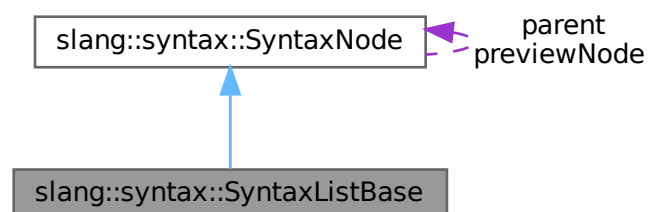
A base class for syntax nodes that represent a list of items.

```
#include <SyntaxNode.h>
```

Inheritance diagram for slang::syntax::SyntaxListBase:



Collaboration diagram for slang::syntax::SyntaxListBase:



Public Member Functions

- `size_t getChildCount () const`
Gets the number of child items in the node.
- `virtual TokenOrSyntax getChild (size_t index)=0`
Gets the child (token or node) at the given index.
- `virtual ConstTokenOrSyntax getChild (size_t index) const =0`
Gets the child (token or node) at the given index.
- `virtual PtrTokenOrSyntax getChildPtr (size_t index)=0`
- `virtual void setChild (size_t index, TokenOrSyntax child)=0`
Sets the child (token or node) at the given index.
- `virtual SyntaxListBase * clone (BumpAllocator &alloc) const =0`
Clones the list into a new node using the provided allocator.
- `virtual void resetAll (BumpAllocator &alloc, std::span< const TokenOrSyntax > children)=0`

Static Public Member Functions

- `static bool isKind (SyntaxKind kind)`
- `static bool isChildOptional (size_t index)`

Protected Member Functions

- `SyntaxListBase (SyntaxKind kind, size_t childCount)`

Protected Attributes

- `size_t childCount`

Additional Inherited Members

7.25.1 Detailed Description

A base class for syntax nodes that represent a list of items.

Definition at line 277 of file [SyntaxNode.h](#).

7.25.2 Constructor & Destructor Documentation

7.25.2.1 SyntaxListBase()

```
slang::syntax::SyntaxListBase::SyntaxListBase (
    SyntaxKind kind,
    size_t childCount ) [inline], [protected]
```

Definition at line 306 of file [SyntaxNode.h](#).

7.25.3 Member Function Documentation

7.25.3.1 clone()

```
virtual SyntaxListBase * slang::syntax::SyntaxListBase::clone (
    BumpAllocator & alloc ) const [pure virtual]
```

Clones the list into a new node using the provided allocator.

Implemented in [slang::syntax::SyntaxList< T >](#), [slang::syntax::TokenList](#), and [slang::syntax::SeparatedSyntaxList< T >](#).

7.25.3.2 getChild() [1/2]

```
virtual ConstTokenOrSyntax slang::syntax::SyntaxListBase::getChild (
    size_t index ) const [pure virtual]
```

Gets the child (token or node) at the given index.

Implemented in [slang::syntax::SyntaxList< T >](#), [slang::syntax::TokenList](#), and [slang::syntax::SeparatedSyntaxList< T >](#).

7.25.3.3 getChild() [2/2]

```
virtual TokenOrSyntax slang::syntax::SyntaxListBase::getChild (
    size_t index ) [pure virtual]
```

Gets the child (token or node) at the given index.

Implemented in [slang::syntax::SyntaxList< T >](#), [slang::syntax::TokenList](#), and [slang::syntax::SeparatedSyntaxList< T >](#).

7.25.3.4 getChildCount()

```
size_t slang::syntax::SyntaxListBase::getChildCount ( ) const [inline]
```

Gets the number of child items in the node.

Definition at line 280 of file [SyntaxNode.h](#).

7.25.3.5 getChildPtr()

```
virtual PtrTokenOrSyntax slang::syntax::SyntaxListBase::getChildPtr (
    size_t index ) [pure virtual]
```

Implemented in [slang::syntax::SyntaxList< T >](#), [slang::syntax::TokenList](#), and [slang::syntax::SeparatedSyntaxList< T >](#).

7.25.3.6 isChildOptional()

```
static bool slang::syntax::SyntaxListBase::isChildOptional (
    size_t index ) [static]
```

7.25.3.7 isKind()

```
static bool slang::syntax::SyntaxListBase::isKind (
    SyntaxKind kind ) [static]
```

7.25.3.8 resetAll()

```
virtual void slang::syntax::SyntaxListBase::resetAll (
    BumpAllocator & alloc,
    std::span< const TokenOrSyntax > children ) [pure virtual]
```

Overwrites all children with the new set of provided *children* (and making a copy with the provided allocator).

Implemented in [slang::syntax::SyntaxList< T >](#), [slang::syntax::TokenList](#), and [slang::syntax::SeparatedSyntaxList< T >](#).

7.25.3.9 setChild()

```
virtual void slang::syntax::SyntaxListBase::setChild (
    size_t index,
    TokenOrSyntax child ) [pure virtual]
```

Sets the child (token or node) at the given index.

Implemented in [slang::syntax::SyntaxList< T >](#), [slang::syntax::TokenList](#), and [slang::syntax::SeparatedSyntaxList< T >](#).

7.25.4 Member Data Documentation

7.25.4.1 childCount

```
size_t slang::syntax::SyntaxListBase::childCount [protected]
```

Definition at line 308 of file [SyntaxNode.h](#).

The documentation for this class was generated from the following file:

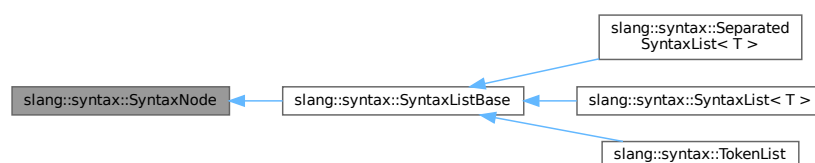
- include_3rd_party/[SyntaxNode.h](#)

7.26 slang::syntax::SyntaxNode Class Reference

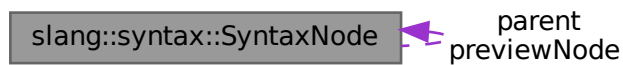
Base class for all syntax nodes.

```
#include <SyntaxNode.h>
```

Inheritance diagram for slang::syntax::SyntaxNode:



Collaboration diagram for slang::syntax::SyntaxNode:



Public Types

- using `Token` = `parsing::Token`

Public Member Functions

- `std::string toString () const`
Print the node and all of its children to a string.
- `Token getFirstToken () const`
Get the first leaf token in this subtree.
- `Token getLastToken () const`
Get the last leaf token in this subtree.
- `Token * getFirstTokenPtr ()`
Get the first leaf token as a mutable pointer in this subtree.
- `Token * getLastTokenPtr ()`
Get the last leaf token a mutable pointer in this subtree.
- `SourceRange sourceRange () const`
Get the source range of the node.
- `const SyntaxNode * childNode (size_t index) const`
- `SyntaxNode * childNode (size_t index)`
- `Token childToken (size_t index) const`
- `Token * childTokenPtr (size_t index)`
- `size_t getChildCount () const`
Gets the number of (direct) children underneath this node in the tree.
- `bool isEquivalentTo (const SyntaxNode &other) const`
- `template<typename T >`
`T & as ()`
- `template<typename T >`
`const T & as () const`
- `template<typename T >`
`T * as_if ()`

- `template<typename T >`
`const T * as\_if () const`
- `template<typename TVisitor , typename... Args>`
`decltype(auto) visit (TVisitor &visitor, Args &&... args)`
- `template<typename TVisitor , typename... Args>`
`decltype(auto) visit (TVisitor &visitor, Args &&... args) const`

Static Public Member Functions

- static bool [isKind](#) (SyntaxKind)
- static bool [isChildOptional](#) (size_t)

Public Attributes

- SyntaxKind [kind](#)
The kind of syntax node.
- [SyntaxNode](#) * [parent](#) = nullptr
- const [SyntaxNode](#) * [previewNode](#) = nullptr
An optional pointer to a syntax node that can be useful to know ahead of time when visiting this node.

Protected Member Functions

- [SyntaxNode](#) (SyntaxKind [kind](#))

Private Member Functions

- [ConstTokenOrSyntax](#) [getChild](#) (size_t index) const
- [TokenOrSyntax](#) [getChild](#) (size_t index)
- [PtrTokenOrSyntax](#) [getChildPtr](#) (size_t index)

7.26.1 Detailed Description

Base class for all syntax nodes.

Definition at line 90 of file [SyntaxNode.h](#).

7.26.2 Member Typedef Documentation

7.26.2.1 Token

```
using slang::syntax::SyntaxNode::Token = parsing::Token
```

Definition at line 92 of file [SyntaxNode.h](#).

7.26.3 Constructor & Destructor Documentation

7.26.3.1 SyntaxNode()

```
slang::syntax::SyntaxNode::SyntaxNode (  
    SyntaxKind kind ) [inline], [explicit], [protected]
```

Definition at line 207 of file [SyntaxNode.h](#).

7.26.4 Member Function Documentation

7.26.4.1 as() [1/2]

```
template<typename T >  
T & slang::syntax::SyntaxNode::as ( ) [inline]
```

Reinterprets this node as being of type T. In debug this will assert that the dynamic kind is appropriate for the specified static type.

Definition at line 158 of file [SyntaxNode.h](#).

7.26.4.2 as() [2/2]

```
template<typename T >  
const T & slang::syntax::SyntaxNode::as ( ) const [inline]
```

Reinterprets this node as being of type T. In debug this will assert that the dynamic kind is appropriate for the specified static type.

Definition at line 166 of file [SyntaxNode.h](#).

7.26.4.3 `as_if()` [1/2]

```
template<typename T >
T * slang::syntax::SyntaxNode::as_if ( ) [inline]
```

Definition at line 173 of file [SyntaxNode.h](#).

7.26.4.4 `as_if()` [2/2]

```
template<typename T >
const T * slang::syntax::SyntaxNode::as_if ( ) const [inline]
```

Definition at line 181 of file [SyntaxNode.h](#).

7.26.4.5 `childNode()` [1/2]

```
SyntaxNode * slang::syntax::SyntaxNode::childNode (
    size_t index )
```

Gets the child syntax node at the specified index. If the child at the given index is not a node (probably a token) then this returns null.

7.26.4.6 `childNode()` [2/2]

```
const SyntaxNode * slang::syntax::SyntaxNode::childNode (
    size_t index ) const
```

Gets the child syntax node at the specified index. If the child at the given index is not a node (probably a token) then this returns null.

7.26.4.7 `childToken()`

```
Token slang::syntax::SyntaxNode::childToken (
    size_t index ) const
```

Gets the child token at the specified index. If the child at the given index is not a token (probably a node) then this returns an empty Token.

7.26.4.8 childTokenPtr()

```
Token * slang::syntax::SyntaxNode::childTokenPtr (
    size_t index )
```

Gets a pointer to the child token at the specified index. If the child at the given index is not a token (probably a node) then this returns null.

7.26.4.9 getChild() [1/2]

```
TokenOrSyntax slang::syntax::SyntaxNode::getChild (
    size_t index ) [private]
```

7.26.4.10 getChild() [2/2]

```
ConstTokenOrSyntax slang::syntax::SyntaxNode::getChild (
    size_t index ) const [private]
```

7.26.4.11 getChildCount()

```
size_t slang::syntax::SyntaxNode::getChildCount ( ) const
```

Gets the number of (direct) children underneath this node in the tree.

7.26.4.12 getChildPtr()

```
PtrTokenOrSyntax slang::syntax::SyntaxNode::getChildPtr (
    size_t index ) [private]
```

7.26.4.13 getFirstToken()

```
Token slang::syntax::SyntaxNode::getFirstToken ( ) const
```

Get the first leaf token in this subtree.

7.26.4.14 `getFirstTokenPtr()`

```
Token * slang::syntax::SyntaxNode::getFirstTokenPtr ( )
```

Get the first leaf token as a mutable pointer in this subtree.

7.26.4.15 `getLastToken()`

```
Token slang::syntax::SyntaxNode::getLastToken ( ) const
```

Get the last leaf token in this subtree.

7.26.4.16 `getLastTokenPtr()`

```
Token * slang::syntax::SyntaxNode::getLastTokenPtr ( )
```

Get the last leaf token a mutable pointer in this subtree.

7.26.4.17 `isChildOptional()`

```
static bool slang::syntax::SyntaxNode::isChildOptional (
    size_t ) [inline], [static]
```

Derived nodes should implement this and return true if child at provided index is pointer not wrapped in `not_null`.

Definition at line 204 of file [SyntaxNode.h](#).

7.26.4.18 `isEquivalentTo()`

```
bool slang::syntax::SyntaxNode::isEquivalentTo (
    const SyntaxNode & other ) const
```

Returns true if this syntax node is "equivalent" to the other provided syntax node. Equivalence here is determined by the entire subtrees having the same kinds of syntax nodes in the same order and all leaf tokens having the same kinds and value text.

7.26.4.19 isKind()

```
static bool slang::syntax::SyntaxNode::isKind (
    SyntaxKind ) [inline], [static]
```

A base implementation of the method that checks correctness of dynamic casting. Derived nodes should reimplement this and return true if the provided syntax kind is compatible with the static type of the object.

Definition at line 200 of file [SyntaxNode.h](#).

7.26.4.20 sourceRange()

```
SourceRange slang::syntax::SyntaxNode::sourceRange ( ) const
```

Get the source range of the node.

7.26.4.21 toString()

```
std::string slang::syntax::SyntaxNode::toString ( ) const
```

Print the node and all of its children to a string.

7.26.4.22 visit() ^[1/2]

```
template<typename TVisitor , typename... Args>
decltype(auto) slang::syntax::SyntaxNode::visit (
    TVisitor & visitor,
    Args &&... args )
```

Applies a visitor object to this node by dispatching based on the dynamic kind. The given *args* are forwarded to the visitor.

7.26.4.23 visit() [2/2]

```
template<typename TVisitor , typename... Args>
decltype(auto) slang::syntax::SyntaxNode::visit (
    TVisitor & visitor,
    Args &&... args ) const
```

Applies a visitor object to this node by dispatching based on the dynamic kind. The given *args* are forwarded to the visitor.

7.26.5 Member Data Documentation

7.26.5.1 kind

```
SyntaxKind slang::syntax::SyntaxNode::kind
```

The kind of syntax node.

Definition at line 95 of file [SyntaxNode.h](#).

7.26.5.2 parent

```
SyntaxNode\* slang::syntax::SyntaxNode::parent = nullptr
```

The parent node of this syntax node. The root of the syntax tree does not have a parent (will be nullptr).

Definition at line 99 of file [SyntaxNode.h](#).

7.26.5.3 previewNode

```
const SyntaxNode\* slang::syntax::SyntaxNode::previewNode = nullptr
```

An optional pointer to a syntax node that can be useful to know ahead of time when visiting this node.

The node, if set, is underneath this node in the syntax tree.

For example, an enum declaration deep inside an expression tree needs to be known up front to add its members to its parent scope.

Definition at line 108 of file [SyntaxNode.h](#).

The documentation for this class was generated from the following file:

- [include_3rd_party/SyntaxNode.h](#)

7.27 slang::syntax::SyntaxPrinter Class Reference

```
#include <SyntaxPrinter.h>
```

Public Member Functions

- [SyntaxPrinter](#) ()=default
- [SyntaxPrinter](#) (const SourceManager &[sourceManager](#))
- [SyntaxPrinter](#) & [append](#) (std::string_view text)
- [SyntaxPrinter](#) & [print](#) (parsing::Trivia trivia)
- [SyntaxPrinter](#) & [print](#) (parsing::Token token)
- [SyntaxPrinter](#) & [print](#) (const [SyntaxNode](#) &node)
- [SyntaxPrinter](#) & [print](#) (const [SyntaxTree](#) &tree)
- [SyntaxPrinter](#) & [setIncludeTrivia](#) (bool include)
- [SyntaxPrinter](#) & [setIncludeMissing](#) (bool include)
- [SyntaxPrinter](#) & [setIncludeSkipped](#) (bool include)
- [SyntaxPrinter](#) & [setIncludeDirectives](#) (bool include)
- [SyntaxPrinter](#) & [setIncludePreprocessed](#) (bool include)
- [SyntaxPrinter](#) & [setIncludeComments](#) (bool include)
- [SyntaxPrinter](#) & [setSquashNewlines](#) (bool include)
- std::string [str](#) () const

Static Public Member Functions

- static std::string [printFile](#) (const [SyntaxTree](#) &tree)

Private Attributes

- std::string [buffer](#)
- const SourceManager * [sourceManager](#) = nullptr
- bool [includeTrivia](#) = true
- bool [includeMissing](#) = false
- bool [includeSkipped](#) = false
- bool [includeDirectives](#) = false
- bool [includePreprocessed](#) = true
- bool [includeComments](#) = true
- bool [squashNewlines](#) = true

7.27.1 Detailed Description

Provides support for printing tokens, trivia, or whole syntax trees back to source code.

Definition at line 25 of file [SyntaxPrinter.h](#).

7.27.2 Constructor & Destructor Documentation

7.27.2.1 SyntaxPrinter() [1/2]

```
slang::syntax::SyntaxPrinter::SyntaxPrinter ( ) [default]
```

7.27.2.2 SyntaxPrinter() [2/2]

```
slang::syntax::SyntaxPrinter::SyntaxPrinter (
    const SourceManager & sourceManager ) [explicit]
```

7.27.3 Member Function Documentation

7.27.3.1 append()

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::append (
    std::string_view text )
```

Append raw text to the buffer.

Returns

a reference to this object, to allow chaining additional method calls.

7.27.3.2 print() [1/4]

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::print (
    const SyntaxNode & node )
```

Print the provided *node* to the internal buffer.

Returns

a reference to this object, to allow chaining additional method calls.

7.27.3.3 print() [2/4]

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::print (
    const SyntaxTree & tree )
```

Print the provided *tree* to the internal buffer.

Returns

a reference to this object, to allow chaining additional method calls.

7.27.3.4 print() [3/4]

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::print (
    parsing::Token token )
```

Print the provided *token* to the internal buffer.

Returns

a reference to this object, to allow chaining additional method calls.

7.27.3.5 print() [4/4]

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::print (
    parsing::Trivia trivia )
```

Print the provided *trivia* to the internal buffer.

Returns

a reference to this object, to allow chaining additional method calls.

7.27.3.6 printFile()

```
static std::string slang::syntax::SyntaxPrinter::printFile (
    const SyntaxTree & tree ) [static]
```

A helper method that assists in printing an entire syntax tree back to source text. A [SyntaxPrinter](#) with useful defaults is constructed, the tree is printed, and the resulting text is returned. Here is the caller graph for this function:



7.27.3.7 setIncludeComments()

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::setIncludeComments (
    bool include ) [inline]
```

Sets whether to include comments when printing syntax.

Returns

a reference to this object, to allow chaining additional method calls.

Definition at line 87 of file [SyntaxPrinter.h](#).

7.27.3.8 setIncludeDirectives()

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::setIncludeDirectives (  
    bool include ) [inline]
```

Sets whether to include preprocessor directives when printing syntax.

Returns

a reference to this object, to allow chaining additional method calls.

Definition at line 73 of file [SyntaxPrinter.h](#).

7.27.3.9 setIncludeMissing()

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::setIncludeMissing (  
    bool include ) [inline]
```

Sets whether to include missing (automatically inserted) nodes when printing syntax.

Returns

a reference to this object, to allow chaining additional method calls.

Definition at line 59 of file [SyntaxPrinter.h](#).

7.27.3.10 setIncludePreprocessed()

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::setIncludePreprocessed (  
    bool include ) [inline]
```

Sets whether to include preprocessor-expanded tokens when printing syntax.

Returns

a reference to this object, to allow chaining additional method calls.

Definition at line 80 of file [SyntaxPrinter.h](#).

7.27.3.11 setIncludeSkipped()

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::setIncludeSkipped (  
    bool include ) [inline]
```

Sets whether to include skipped (due to some sort of error) nodes when printing syntax.

Returns

a reference to this object, to allow chaining additional method calls.

Definition at line 66 of file [SyntaxPrinter.h](#).

7.27.3.12 setIncludeTrivia()

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::setIncludeTrivia (  
    bool include ) [inline]
```

Sets whether to include trivia when printing syntax.

Returns

a reference to this object, to allow chaining additional method calls.

Definition at line 52 of file [SyntaxPrinter.h](#).

7.27.3.13 setSquashNewlines()

```
SyntaxPrinter & slang::syntax::SyntaxPrinter::setSquashNewlines (  
    bool include ) [inline]
```

Sets whether to squash adjacent newlines down into one when printing syntax.

Returns

a reference to this object, to allow chaining additional method calls.

Definition at line 94 of file [SyntaxPrinter.h](#).

7.27.3.14 str()

```
std::string slang::syntax::SyntaxPrinter::str ( ) const [inline]
```

Returns

a copy of the internal text buffer.

Definition at line 100 of file [SyntaxPrinter.h](#).

7.27.4 Member Data Documentation

7.27.4.1 buffer

```
std::string slang::syntax::SyntaxPrinter::buffer [private]
```

Definition at line 108 of file [SyntaxPrinter.h](#).

7.27.4.2 includeComments

```
bool slang::syntax::SyntaxPrinter::includeComments = true [private]
```

Definition at line 115 of file [SyntaxPrinter.h](#).

7.27.4.3 includeDirectives

```
bool slang::syntax::SyntaxPrinter::includeDirectives = false [private]
```

Definition at line 113 of file [SyntaxPrinter.h](#).

7.27.4.4 includeMissing

```
bool slang::syntax::SyntaxPrinter::includeMissing = false [private]
```

Definition at line 111 of file [SyntaxPrinter.h](#).

7.27.4.5 includePreprocessed

```
bool slang::syntax::SyntaxPrinter::includePreprocessed = true [private]
```

Definition at line 114 of file [SyntaxPrinter.h](#).

7.27.4.6 includeSkipped

```
bool slang::syntax::SyntaxPrinter::includeSkipped = false [private]
```

Definition at line 112 of file [SyntaxPrinter.h](#).

7.27.4.7 includeTrivia

```
bool slang::syntax::SyntaxPrinter::includeTrivia = true [private]
```

Definition at line 110 of file [SyntaxPrinter.h](#).

7.27.4.8 sourceManager

```
const SourceManager* slang::syntax::SyntaxPrinter::sourceManager = nullptr [private]
```

Definition at line 109 of file [SyntaxPrinter.h](#).

7.27.4.9 squashNewlines

```
bool slang::syntax::SyntaxPrinter::squashNewlines = true [private]
```

Definition at line 116 of file [SyntaxPrinter.h](#).

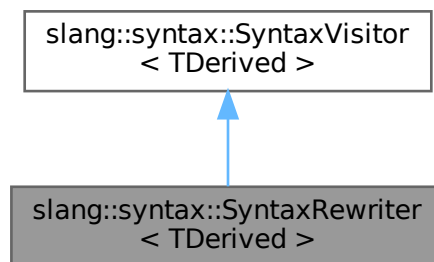
The documentation for this class was generated from the following file:

- include_3rd_party/[SyntaxPrinter.h](#)

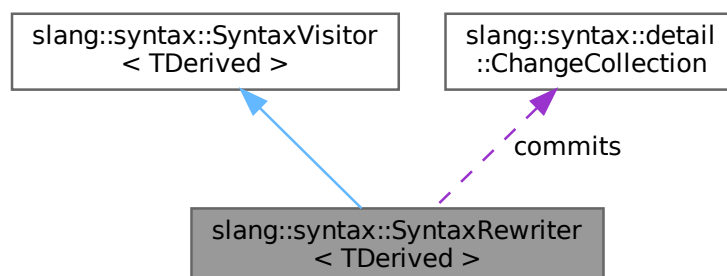
7.28 slang::syntax::SyntaxRewriter< TDerived > Class Template Reference

```
#include <SyntaxVisitor.h>
```

Inheritance diagram for slang::syntax::SyntaxRewriter< TDerived >:



Collaboration diagram for slang::syntax::SyntaxRewriter< TDerived >:



Public Member Functions

- [SyntaxRewriter](#) ()
- `std::shared_ptr< SyntaxTree > transform` (const `std::shared_ptr< SyntaxTree >` &tree, const `SourceLibrary *library=nullptr`)

Protected Types

- using [Token](#) = `parsing::Token`

Protected Member Functions

- [SyntaxNode](#) & [parse](#) (std::string_view text)
A helper for derived classes that parses some text into syntax nodes.
- void [remove](#) (const [SyntaxNode](#) &oldNode)
Register a removal for the given syntax node from the tree.
- void [replace](#) (const [SyntaxNode](#) &oldNode, [SyntaxNode](#) &newNode)
Replace the given oldNode with newNode in the rewritten tree.
- void [insertBefore](#) (const [SyntaxNode](#) &oldNode, [SyntaxNode](#) &newNode)
Insert newNode before oldNode in the rewritten tree.
- void [insertAfter](#) (const [SyntaxNode](#) &oldNode, [SyntaxNode](#) &newNode)
Insert newNode after oldNode in the rewritten tree.
- void [insertAtFront](#) (const [SyntaxListBase](#) &list, [SyntaxNode](#) &newNode, [Token](#) separator={})
Insert newNode at the front of list in the rewritten tree.
- void [insertAtBack](#) (const [SyntaxListBase](#) &list, [SyntaxNode](#) &newNode, [Token](#) separator={})
Insert newNode at the back of list in the rewritten tree.
- [Token](#) [makeToken](#) (parsing::TokenKind kind, std::string_view text, std::span< const parsing::Trivia > trivia={})
- [Token](#) [makeId](#) (std::string_view text, std::span< const parsing::Trivia > trivia={})
- [Token](#) [makeId](#) (std::string_view text, parsing::Trivia trivia)
- [Token](#) [makeComma](#) ()

Protected Attributes

- BumpAllocator [alloc](#)
- SyntaxFactory [factory](#)

Static Protected Attributes

- static const parsing::Trivia [SingleSpace](#) {parsing::TriviaKind::Whitespace, " "sv}

Private Attributes

- SourceManager * [sourceManager](#) = nullptr
- [detail::ChangeCollection](#) commits
- std::vector< std::shared_ptr< [SyntaxTree](#) > > [tempTrees](#)

7.28.1 Detailed Description

```
template<typename TDerived>
class slang::syntax::SyntaxRewriter< TDerived >
```

A helper class that assists in rewriting syntax trees – useful for automated refactoring tools.

Definition at line [102](#) of file [SyntaxVisitor.h](#).

7.28.2 Member Typedef Documentation

7.28.2.1 Token

```
template<typename TDerived >
using slang::syntax::SyntaxRewriter< TDerived >::Token = parsing::Token [protected]
```

Definition at line [131](#) of file [SyntaxVisitor.h](#).

7.28.3 Constructor & Destructor Documentation

7.28.3.1 SyntaxRewriter()

```
template<typename TDerived >
slang::syntax::SyntaxRewriter< TDerived >::SyntaxRewriter ( ) [inline]
```

Definition at line [104](#) of file [SyntaxVisitor.h](#).

7.28.4 Member Function Documentation

7.28.4.1 insertAfter()

```
template<typename TDerived >
void slang::syntax::SyntaxRewriter< TDerived >::insertAfter (
    const SyntaxNode & oldNode,
    SyntaxNode & newNode ) [inline], [protected]
```

Insert *newNode* after *oldNode* in the rewritten tree.

Definition at line 163 of file [SyntaxVisitor.h](#).

7.28.4.2 insertAtBack()

```
template<typename TDerived >
void slang::syntax::SyntaxRewriter< TDerived >::insertAtBack (
    const SyntaxListBase & list,
    SyntaxNode & newNode,
    Token separator = {} ) [inline], [protected]
```

Insert *newNode* at the back of *list* in the rewritten tree.

Definition at line 173 of file [SyntaxVisitor.h](#).

7.28.4.3 insertAtFront()

```
template<typename TDerived >
void slang::syntax::SyntaxRewriter< TDerived >::insertAtFront (
    const SyntaxListBase & list,
    SyntaxNode & newNode,
    Token separator = {} ) [inline], [protected]
```

Insert *newNode* at the front of *list* in the rewritten tree.

Definition at line 168 of file [SyntaxVisitor.h](#).

7.28.4.4 insertBefore()

```
template<typename TDerived >
void slang::syntax::SyntaxRewriter< TDerived >::insertBefore (
    const SyntaxNode & oldNode,
    SyntaxNode & newNode ) [inline], [protected]
```

Insert *newNode* before *oldNode* in the rewritten tree.

Definition at line 158 of file [SyntaxVisitor.h](#).

7.28.4.5 makeComma()

```
template<typename TDerived >
Token slang::syntax::SyntaxRewriter< TDerived >::makeComma ( ) [inline], [protected]
```

Definition at line 191 of file [SyntaxVisitor.h](#).

Here is the call graph for this function:



7.28.4.6 makeId() ^[1/2]

```
template<typename TDerived >
Token slang::syntax::SyntaxRewriter< TDerived >::makeId (
    std::string_view text,
    parsing::Trivia trivia ) [inline], [protected]
```

Definition at line 186 of file [SyntaxVisitor.h](#).

Here is the call graph for this function:



7.28.4.7 makeId() [2/2]

```

template<typename TDerived >
Token slang::syntax::SyntaxRewriter< TDerived >::makeId (
    std::string_view text,
    std::span< const parsing::Trivia > trivia = {} ) [inline], [protected]
  
```

Definition at line 182 of file [SyntaxVisitor.h](#).

Here is the caller graph for this function:



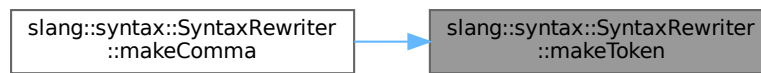
7.28.4.8 makeToken()

```

template<typename TDerived >
Token slang::syntax::SyntaxRewriter< TDerived >::makeToken (
    parsing::TokenKind kind,
    std::string_view text,
    std::span< const parsing::Trivia > trivia = {} ) [inline], [protected]
  
```

Definition at line 177 of file [SyntaxVisitor.h](#).

Here is the caller graph for this function:



7.28.4.9 parse()

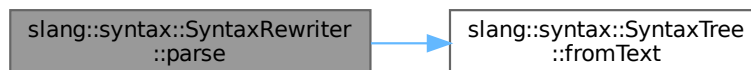
```

template<typename TDerived >
SyntaxNode & slang::syntax::SyntaxRewriter< TDerived >::parse (
    std::string_view text ) [inline], [protected]
  
```

A helper for derived classes that parses some text into syntax nodes.

Definition at line 134 of file [SyntaxVisitor.h](#).

Here is the call graph for this function:



7.28.4.10 remove()

```

template<typename TDerived >
void slang::syntax::SyntaxRewriter< TDerived >::remove (
    const SyntaxNode & oldNode ) [inline], [protected]
  
```

Register a removal for the given syntax node from the tree.

Definition at line 140 of file [SyntaxVisitor.h](#).

7.28.4.11 replace()

```
template<typename TDerived >
void slang::syntax::SyntaxRewriter< TDerived >::replace (
    const SyntaxNode & oldNode,
    SyntaxNode & newNode ) [inline], [protected]
```

Replace the given *oldNode* with *newNode* in the rewritten tree.

Definition at line 149 of file [SyntaxVisitor.h](#).

7.28.4.12 transform()

```
template<typename TDerived >
std::shared_ptr< SyntaxTree > slang::syntax::SyntaxRewriter< TDerived >::transform (
    const std::shared_ptr< SyntaxTree > & tree,
    const SourceLibrary * library = nullptr ) [inline]
```

Transforms the given syntax tree using the rewriter. The tree will be visited in order and each node will be given a chance to be rewritten (by providing [visit\(\)](#) implementations in the derived class).

Returns

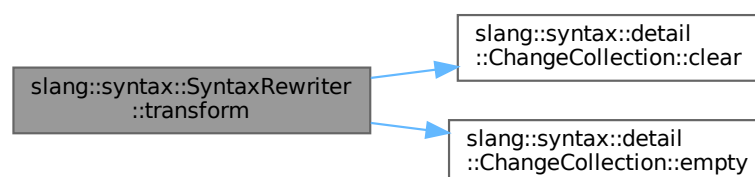
if no changes are requested, returns the original syntax tree. Otherwise, the changes are applied and the newly rewritten syntax tree is returned.

Note

lifetimes of new and old trees are independent from each other. If you shallow clone something from old tree into new one, you have to make sure that `shared_ptr` to original doesn't go out of scope too early.

Definition at line 116 of file [SyntaxVisitor.h](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.28.5 Member Data Documentation

7.28.5.1 alloc

```
template<typename TDerived >
BumpAllocator slang::syntax::SyntaxRewriter< TDerived >::alloc [protected]
```

Definition at line 193 of file [SyntaxVisitor.h](#).

7.28.5.2 commits

```
template<typename TDerived >
detail::ChangeCollection slang::syntax::SyntaxRewriter< TDerived >::commits [private]
```

Definition at line 200 of file [SyntaxVisitor.h](#).

7.28.5.3 factory

```
template<typename TDerived >
SyntaxFactory slang::syntax::SyntaxRewriter< TDerived >::factory [protected]
```

Definition at line 194 of file [SyntaxVisitor.h](#).

7.28.5.4 SingleSpace

```
template<typename TDerived >
const parsing::Trivia slang::syntax::SyntaxRewriter< TDerived >::SingleSpace {parsing::TriviaKind::Whitespace, "
"sv}  [static], [protected]
```

Definition at line 196 of file [SyntaxVisitor.h](#).

7.28.5.5 sourceManager

```
template<typename TDerived >
SourceManager* slang::syntax::SyntaxRewriter< TDerived >::sourceManager = nullptr  [private]
```

Definition at line 199 of file [SyntaxVisitor.h](#).

7.28.5.6 tempTrees

```
template<typename TDerived >
std::vector<std::shared_ptr<SyntaxTree> > slang::syntax::SyntaxRewriter< TDerived >::tempTrees  [private]
```

Definition at line 201 of file [SyntaxVisitor.h](#).

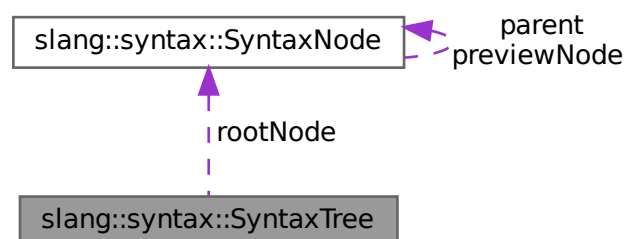
The documentation for this class was generated from the following file:

- [include_3rd_party/SyntaxVisitor.h](#)

7.29 slang::syntax::SyntaxTree Class Reference

```
#include <SyntaxTree.h>
```

Collaboration diagram for slang::syntax::SyntaxTree:



Public Types

- using [TreeOrError](#) = nonstd::expected< std::shared_ptr< [SyntaxTree](#) >, std::pair< std::error_code, std::string_view > >
- using [MacroList](#) = std::span< const DefineDirectiveSyntax *const >

Public Member Functions

- [SyntaxTree](#) ([SyntaxNode](#) *root, SourceManager &[sourceManager](#), BumpAllocator &&[alloc](#), const SourceLibrary *library, const std::shared_ptr< [SyntaxTree](#) > &parent=nullptr)
- [SyntaxTree](#) ([SyntaxTree](#) &&other)=default
- ~[SyntaxTree](#) ()
- Diagnostics & [diagnostics](#) ()
Gets any diagnostics generated while parsing.
- BumpAllocator & [allocator](#) ()
Gets the allocator containing the memory for the parse tree.
- SourceManager & [sourceManager](#) ()
Gets the source manager used to build the syntax tree.
- const SourceManager & [sourceManager](#) () const
Gets the source manager used to build the syntax tree.
- const SourceLibrary * [getSourceLibrary](#) () const
Gets the source library with which the syntax tree is associated.
- [SyntaxNode](#) & [root](#) ()
Gets the root of the syntax tree.
- const [SyntaxNode](#) & [root](#) () const
Gets the root of the syntax tree.
- const Bag & [options](#) () const
The options used to construct the syntax tree.
- const parsing::ParserMetadata & [getMetadata](#) () const
Gets various bits of metadata collected during parsing.
- [MacroList](#) [getDefinedMacros](#) () const
Gets the list of macros that were defined at the end of the loaded source file.

Static Public Member Functions

- static [TreeOrError](#) [fromFile](#) (std::string_view path)
- static [TreeOrError](#) [fromFile](#) (std::string_view path, SourceManager &[sourceManager](#), const Bag &[options](#)={})
- static [TreeOrError](#) [fromFiles](#) (std::span< const std::string_view > paths)
- static [TreeOrError](#) [fromFiles](#) (std::span< const std::string_view > paths, SourceManager &[sourceManager](#), const Bag &[options](#)={})

- static std::shared_ptr< [SyntaxTree](#) > [fromText](#) (std::string_view text, std::string_view name="source", std::string_view path="")
- static std::shared_ptr< [SyntaxTree](#) > [fromText](#) (std::string_view text, const Bag &[options](#), std::string_view name="source"sv, std::string_view path="")
- static std::shared_ptr< [SyntaxTree](#) > [fromText](#) (std::string_view text, SourceManager &[sourceManager](#), std::string_view name="source"sv, std::string_view path="", const Bag &[options](#)={}, const SourceLibrary *[library](#)=nullptr)
- static std::shared_ptr< [SyntaxTree](#) > [fromFileInMemory](#) (std::string_view text, SourceManager &[sourceManager](#), std::string_view name="source"sv, std::string_view path="", const Bag &[options](#)={})
- static std::shared_ptr< [SyntaxTree](#) > [fromBuffer](#) (const SourceBuffer &buffer, SourceManager &[sourceManager](#), const Bag &[options](#)={}, [MacroList](#) inheritedMacros={})
- static std::shared_ptr< [SyntaxTree](#) > [fromBuffers](#) (std::span< const SourceBuffer > buffers, SourceManager &[sourceManager](#), const Bag &[options](#)={}, [MacroList](#) inheritedMacros={})
- static std::shared_ptr< [SyntaxTree](#) > [fromLibraryMapFile](#) (std::string_view path, SourceManager &[sourceManager](#), const Bag &[options](#)={})
- static std::shared_ptr< [SyntaxTree](#) > [fromLibraryMapText](#) (std::string_view text, SourceManager &[sourceManager](#), std::string_view name="source"sv, std::string_view path="", const Bag &[options](#)={})
- static std::shared_ptr< [SyntaxTree](#) > [fromLibraryMapBuffer](#) (const SourceBuffer &buffer, SourceManager &[sourceManager](#), const Bag &[options](#)={})
- static SourceManager & [getDefaultSourceManager](#) ()

Public Attributes

- bool [isLibraryUnit](#) = false

Private Member Functions

- [SyntaxTree](#) ([SyntaxNode](#) *[root](#), const SourceLibrary *[library](#), SourceManager &[sourceManager](#), BumpAllocator &&[alloc](#), Diagnostics &&[diagnostics](#), parsing::ParserMetadata &&[metadata](#), std::vector< const DefinedDirectiveSyntax * > &&[macros](#), Bag [options](#))

Static Private Member Functions

- static std::shared_ptr< [SyntaxTree](#) > [create](#) (SourceManager &[sourceManager](#), std::span< const SourceBuffer > source, const Bag &[options](#), [MacroList](#) inheritedMacros, bool guess)

Private Attributes

- [SyntaxNode](#) * [rootNode](#)
- const SourceLibrary * [library](#)
- SourceManager & [sourceMan](#)
- BumpAllocator [alloc](#)
- Diagnostics [diagnosticsBuffer](#)
- Bag [options_](#)
- std::unique_ptr< parsing::ParserMetadata > [metadata](#)
- std::vector< const DefineDirectiveSyntax * > [macros](#)

7.29.1 Detailed Description

The [SyntaxTree](#) is the easiest way to interface with the lexer / preprocessor / parser stack. Give it some source text and it produces a parse tree.

The [SyntaxTree](#) object owns all of the memory for the parse tree, so it must live for as long as you need to access its syntax nodes.

Definition at line 39 of file [SyntaxTree.h](#).

7.29.2 Member Typedef Documentation

7.29.2.1 MacroList

```
using slang::syntax::SyntaxTree::MacroList = std::span<const DefineDirectiveSyntax* const>
```

Definition at line 43 of file [SyntaxTree.h](#).

7.29.2.2 TreeOrError

```
using slang::syntax::SyntaxTree::TreeOrError = nonstd::expected<std::shared_ptr<SyntaxTree>, std::pair<std::error_code, std::string_view> >
```

Definition at line 41 of file [SyntaxTree.h](#).

7.29.3 Constructor & Destructor Documentation

7.29.3.1 SyntaxTree() [1/3]

```
slang::syntax::SyntaxTree::SyntaxTree (
    SyntaxNode * root,
    SourceManager & sourceManager,
    BumpAllocator && alloc,
    const SourceLibrary * library,
    const std::shared_ptr< SyntaxTree > & parent = nullptr )
```

7.29.3.2 SyntaxTree() [2/3]

```
slang::syntax::SyntaxTree::SyntaxTree (
    SyntaxTree && other ) [default]
```

7.29.3.3 ~SyntaxTree()

```
slang::syntax::SyntaxTree::~~SyntaxTree ( )
```

7.29.3.4 SyntaxTree() [3/3]

```
slang::syntax::SyntaxTree::SyntaxTree (
    SyntaxNode * root,
    const SourceLibrary * library,
    SourceManager & sourceManager,
    BumpAllocator && alloc,
    Diagnostics && diagnostics,
    parsing::ParserMetadata && metadata,
    std::vector< const DefineDirectiveSyntax * > && macros,
    Bag options ) [private]
```

7.29.4 Member Function Documentation

7.29.4.1 allocator()

```
BumpAllocator & slang::syntax::SyntaxTree::allocator ( ) [inline]
```

Gets the allocator containing the memory for the parse tree.

Definition at line 187 of file [SyntaxTree.h](#).

7.29.4.2 create()

```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::create (
    SourceManager & sourceManager,
    std::span< const SourceBuffer > source,
    const Bag & options,
    MacroList inheritedMacros,
    bool guess ) [static], [private]
```

7.29.4.3 diagnostics()

```
Diagnostics & slang::syntax::SyntaxTree::diagnostics ( ) [inline]
```

Gets any diagnostics generated while parsing.

Definition at line 184 of file [SyntaxTree.h](#).

7.29.4.4 fromBuffer()

```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::fromBuffer (
    const SourceBuffer & buffer,
    SourceManager & sourceManager,
    const Bag & options = {},
    MacroList inheritedMacros = {} ) [static]
```

Creates a syntax tree from an already loaded source buffer. *buffer* is the loaded source buffer. *sourceManager* is the manager that owns the buffer. *options* is an optional bag of lexer, preprocessor, and parser options. *inheritedMacros* is a list of macros to predefine in the new syntax tree.

Returns

the created and parsed syntax tree.

7.29.4.5 fromBuffers()

```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::fromBuffers (
    std::span< const SourceBuffer > buffers,
    SourceManager & sourceManager,
    const Bag & options = {},
    MacroList inheritedMacros = {} ) [static]
```

Creates a syntax tree by concatenating several loaded source buffers. *buffers* is the list of buffers that should be concatenated to form the compilation unit to parse. *sourceManager* is the manager that owns the buffers. *options* is an optional bag of lexer, preprocessor, and parser options. *inheritedMacros* is a list of macros to predefine in the new syntax tree.

Returns

the created and parsed syntax tree.

7.29.4.6 fromFile() [1/2]

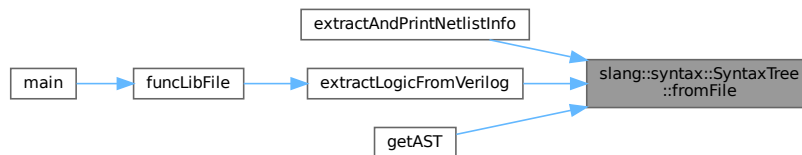
```
static TreeOrError slang::syntax::SyntaxTree::fromFile (
    std::string_view path ) [static]
```

Creates a syntax tree from a full compilation unit. *path* is the path to the source file on disk.

Returns

the created and parsed syntax tree.

Here is the caller graph for this function:



7.29.4.7 fromFile() [2/2]

```
static TreeOrError slang::syntax::SyntaxTree::fromFile (
    std::string_view path,
    SourceManager & sourceManager,
    const Bag & options = {} ) [static]
```

Creates a syntax tree from a full compilation unit. *path* is the path to the source file on disk. *sourceManager* is the manager that owns all of the loaded source code. *options* is an optional bag of lexer, preprocessor, and parser options.

Returns

the created and parsed syntax tree on success, or an OS error code if the file fails to load.

7.29.4.8 fromFileInMemory()

```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::fromFileInMemory (
    std::string_view text,
    SourceManager & sourceManager,
    std::string_view name = "source"sv,
    std::string_view path = "",
    const Bag & options = {} ) [static]
```

Creates a syntax tree from a full compilation unit already in memory. *text* is the actual source code text. *sourceManager* is the manager that owns all of the loaded source code. *name* is an optional name to give to the loaded source buffer. *path* is an optional path to give to the loaded source buffer. *options* is an optional bag of lexer, preprocessor, and parser options.

Returns

the created and parsed syntax tree.

7.29.4.9 fromFiles() [1/2]

```
static TreeOrError slang::syntax::SyntaxTree::fromFiles (
    std::span< const std::string_view > paths ) [static]
```

Creates a syntax tree by concatenating several files loaded from disk. *paths* is the list of paths to the source files on disk.

Returns

the created and parsed syntax tree on success, or an OS error code if the file fails to load.

7.29.4.10 fromFiles() [2/2]

```
static TreeOrError slang::syntax::SyntaxTree::fromFiles (
    std::span< const std::string_view > paths,
    SourceManager & sourceManager,
    const Bag & options = {} ) [static]
```

Creates a syntax tree by concatenating several files loaded from disk. *paths* is the list of paths to the source files on disk. *sourceManager* is the manager that owns all of the loaded source code. *options* is an optional bag of lexer, preprocessor, and parser options.

Returns

the created and parsed syntax tree on success, or an OS error code if the file fails to load.

7.29.4.11 fromLibraryMapBuffer()

```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::fromLibraryMapBuffer (
    const SourceBuffer & buffer,
    SourceManager & sourceManager,
    const Bag & options = {} ) [static]
```

Creates a syntax tree from a library map already loaded into a source buffer. *buffer* is the loaded source buffer. *sourceManager* is the manager that owns the buffer. *options* is an optional bag of lexer, preprocessor, and parser options.

Returns

the created and parsed syntax tree.

7.29.4.12 fromLibraryMapFile()

```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::fromLibraryMapFile (
    std::string_view path,
    SourceManager & sourceManager,
    const Bag & options = {} ) [static]
```

Creates a syntax tree from a library map file. *path* is the path to the source file on disk. *sourceManager* is the manager that owns all of the loaded source code. *options* is an optional bag of lexer, preprocessor, and parser options.

Returns

the created and parsed syntax tree.

7.29.4.13 fromLibraryMapText()

```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::fromLibraryMapText (
    std::string_view text,
    SourceManager & sourceManager,
    std::string_view name = "source"sv,
    std::string_view path = "",
    const Bag & options = {} ) [static]
```

Creates a syntax tree from a library map located in memory. *text* is the actual source code text. *sourceManager* is the manager that owns all of the loaded source code. *name* is an optional name to give to the loaded source buffer. *path* is an optional path to give to the loaded source buffer. *options* is an optional bag of lexer, preprocessor, and parser options.

Returns

the created and parsed syntax tree.

7.29.4.14 fromText() [1/3]

```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::fromText (
    std::string_view text,
    const Bag & options,
    std::string_view name = "source"sv,
    std::string_view path = "" ) [static]
```

Creates a syntax tree by guessing at what might be in the given source snippet. *text* is the actual source code text. *options* is a bag of lexer, preprocessor, and parser options. *name* is an optional name to give to the loaded source buffer. *path* is an optional path to give to the loaded source buffer.

Returns

the created and parsed syntax tree.

7.29.4.15 fromText() [2/3]

```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::fromText (
    std::string_view text,
    SourceManager & sourceManager,
    std::string_view name = "source"sv,
    std::string_view path = "",
    const Bag & options = {},
    const SourceLibrary * library = nullptr ) [static]
```

Creates a syntax tree by guessing at what might be in the given source snippet. *text* is the actual source code text. *sourceManager* is the manager that owns all of the loaded source code. *name* is an optional name to give to the loaded source buffer. *path* is an optional path to give to the loaded source buffer. *options* is an optional bag of lexer, preprocessor, and parser options. *library* the source library to associated with the parsed tree

Returns

the created and parsed syntax tree.

7.29.4.16 fromText() [3/3]

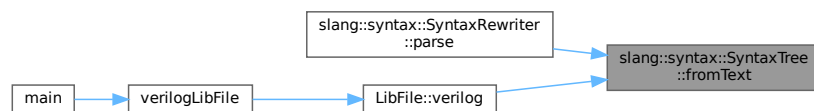
```
static std::shared_ptr< SyntaxTree > slang::syntax::SyntaxTree::fromText (
    std::string_view text,
    std::string_view name = "source",
    std::string_view path = "" ) [static]
```

Creates a syntax tree by guessing at what might be in the given source snippet. *text* is the actual source code text. *name* is an optional name to give to the loaded source buffer. *path* is an optional path to give to the loaded source buffer.

Returns

the created and parsed syntax tree.

Here is the caller graph for this function:

**7.29.4.17 getDefaultSourceManager()**

```
static SourceManager & slang::syntax::SyntaxTree::getDefaultSourceManager ( ) [static]
```

This is a shared default source manager for cases where the user doesn't care about managing the lifetime of loaded source. Note that all of the source loaded by this thing will live in memory for the lifetime of the process.

7.29.4.18 getDefinedMacros()

```
MacroList slang::syntax::SyntaxTree::getDefinedMacros ( ) const [inline]
```

Gets the list of macros that were defined at the end of the loaded source file.

Definition at line 211 of file [SyntaxTree.h](#).

7.29.4.19 getMetadata()

```
const parsing::ParserMetadata & slang::syntax::SyntaxTree::getMetadata ( ) const [inline]
```

Gets various bits of metadata collected during parsing.

Definition at line 208 of file [SyntaxTree.h](#).

7.29.4.20 getSourceLibrary()

```
const SourceLibrary * slang::syntax::SyntaxTree::getSourceLibrary ( ) const [inline]
```

Gets the source library with which the syntax tree is associated.

Definition at line 196 of file [SyntaxTree.h](#).

7.29.4.21 options()

```
const Bag & slang::syntax::SyntaxTree::options ( ) const [inline]
```

The options used to construct the syntax tree.

Definition at line 205 of file [SyntaxTree.h](#).

7.29.4.22 root() [1/2]

```
SyntaxNode & slang::syntax::SyntaxTree::root ( ) [inline]
```

Gets the root of the syntax tree.

Definition at line 199 of file [SyntaxTree.h](#).

7.29.4.23 root() [2/2]

```
const SyntaxNode & slang::syntax::SyntaxTree::root ( ) const [inline]
```

Gets the root of the syntax tree.

Definition at line 202 of file [SyntaxTree.h](#).

7.29.4.24 sourceManager() [1/2]

```
SourceManager & slang::syntax::SyntaxTree::sourceManager ( ) [inline]
```

Gets the source manager used to build the syntax tree.

Definition at line 190 of file [SyntaxTree.h](#).

7.29.4.25 sourceManager() [2/2]

```
const SourceManager & slang::syntax::SyntaxTree::sourceManager ( ) const [inline]
```

Gets the source manager used to build the syntax tree.

Definition at line 193 of file [SyntaxTree.h](#).

7.29.5 Member Data Documentation

7.29.5.1 alloc

```
BumpAllocator slang::syntax::SyntaxTree::alloc [private]
```

Definition at line 232 of file [SyntaxTree.h](#).

7.29.5.2 diagnosticsBuffer

```
Diagnostics slang::syntax::SyntaxTree::diagnosticsBuffer [private]
```

Definition at line 233 of file [SyntaxTree.h](#).

7.29.5.3 isLibraryUnit

```
bool slang::syntax::SyntaxTree::isLibraryUnit = false
```

Indicates whether this syntax tree represents a "library" compilation unit, which means that modules declared within it are not automatically instantiated.

Definition at line 47 of file [SyntaxTree.h](#).

7.29.5.4 library

```
const SourceLibrary* slang::syntax::SyntaxTree::library [private]
```

Definition at line 230 of file [SyntaxTree.h](#).

7.29.5.5 macros

```
std::vector<const DefineDirectiveSyntax*> slang::syntax::SyntaxTree::macros [private]
```

Definition at line 236 of file [SyntaxTree.h](#).

7.29.5.6 metadata

```
std::unique_ptr<parsing::ParserMetadata> slang::syntax::SyntaxTree::metadata [private]
```

Definition at line 235 of file [SyntaxTree.h](#).

7.29.5.7 options_

Bag slang::syntax::SyntaxTree::options_ [private]

Definition at line 234 of file [SyntaxTree.h](#).

7.29.5.8 rootNode

[SyntaxNode*](#) slang::syntax::SyntaxTree::rootNode [private]

Definition at line 229 of file [SyntaxTree.h](#).

7.29.5.9 sourceMan

SourceManager& slang::syntax::SyntaxTree::sourceMan [private]

Definition at line 231 of file [SyntaxTree.h](#).

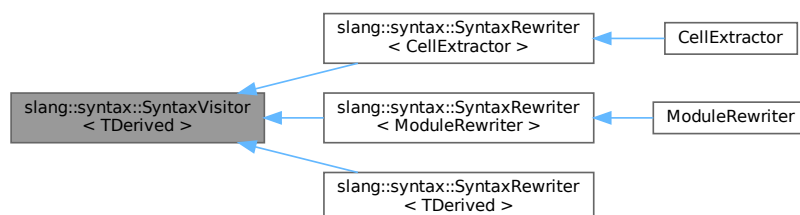
The documentation for this class was generated from the following file:

- include_3rd_party/[SyntaxTree.h](#)

7.30 slang::syntax::SyntaxVisitor< TDerived > Class Template Reference

```
#include <SyntaxVisitor.h>
```

Inheritance diagram for slang::syntax::SyntaxVisitor< TDerived >:



Public Member Functions

- `template<typename T >`
`void visit (T &&t)`
Visit the provided node, of static type T.
- `template<typename T >`
`void visitDefault (T &&node)`

Private Member Functions

- `void visitToken (parsing::Token)`

7.30.1 Detailed Description

```
template<typename TDerived>
class slang::syntax::SyntaxVisitor< TDerived >
```

Use this type as a base class for syntax tree visitors. It will default to traversing all children of each node. Add implementations for any specific node types you want to handle.

Definition at line 25 of file [SyntaxVisitor.h](#).

7.30.2 Member Function Documentation

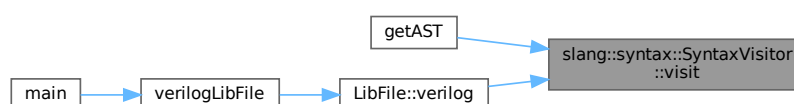
7.30.2.1 visit()

```
template<typename TDerived >
template<typename T >
void slang::syntax::SyntaxVisitor< TDerived >::visit (
    T && t ) [inline]
```

Visit the provided node, of static type T.

Definition at line 29 of file [SyntaxVisitor.h](#).

Here is the caller graph for this function:



7.30.2.2 visitDefault()

```
template<typename TDerived >
template<typename T >
void slang::syntax::SyntaxVisitor< TDerived >::visitDefault (
    T && node ) [inline]
```

The default handler invoked when no [visit\(\)](#) method is overridden for a particular type. Will visit all child nodes by default.

Definition at line 39 of file [SyntaxVisitor.h](#).

Here is the caller graph for this function:



7.30.2.3 visitToken()

```
template<typename TDerived >
void slang::syntax::SyntaxVisitor< TDerived >::visitToken (
    parsing::Token ) [inline], [private]
```

Definition at line 54 of file [SyntaxVisitor.h](#).

The documentation for this class was generated from the following file:

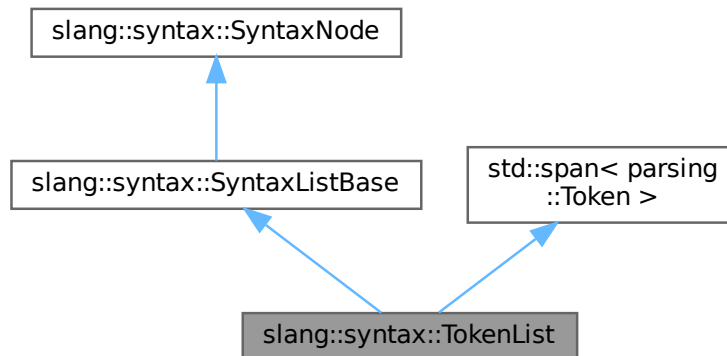
- `include_3rd_party/SyntaxVisitor.h`

7.31 slang::syntax::TokenList Class Reference

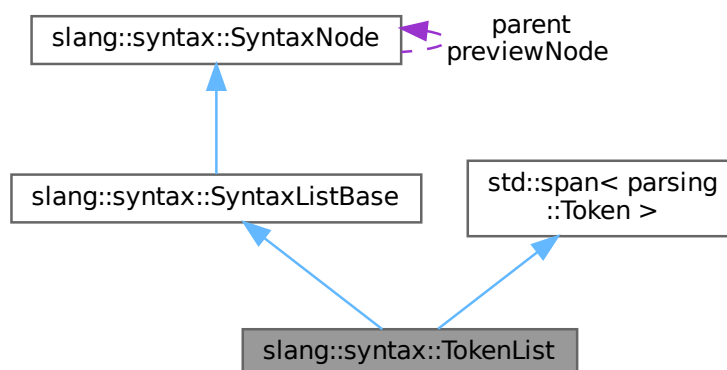
A syntax node that represents a list of child tokens.

```
#include <SyntaxNode.h>
```

Inheritance diagram for slang::syntax::TokenList:



Collaboration diagram for slang::syntax::TokenList:



Public Member Functions

- [TokenList](#) (nullptr_t)
- [TokenList](#) (std::span< [Token](#) > elements)

Private Member Functions

- [TokenOrSyntax getChild](#) (size_t index) final
Gets the child (token or node) at the given index.
- [ConstTokenOrSyntax getChild](#) (size_t index) const final
Gets the child (token or node) at the given index.
- [PtrTokenOrSyntax getChildPtr](#) (size_t index) final
- void [setChild](#) (size_t index, [TokenOrSyntax](#) child) final
Sets the child (token or node) at the given index.
- [SyntaxListBase * clone](#) (BumpAllocator &alloc) const final
Clones the list into a new node using the provided allocator.
- void [resetAll](#) (BumpAllocator &alloc, std::span< const [TokenOrSyntax](#) > children) final

Additional Inherited Members

7.31.1 Detailed Description

A syntax node that represents a list of child tokens.

Definition at line 352 of file [SyntaxNode.h](#).

7.31.2 Constructor & Destructor Documentation

7.31.2.1 TokenList() [1/2]

```
slang::syntax::TokenList::TokenList (
    nullptr_t ) [inline]
```

Definition at line 354 of file [SyntaxNode.h](#).

7.31.2.2 TokenList() [2/2]

```
slang::syntax::TokenList::TokenList (
    std::span< Token > elements ) [inline]
```

Definition at line 355 of file [SyntaxNode.h](#).

7.31.3 Member Function Documentation

7.31.3.1 clone()

```
SyntaxListBase * slang::syntax::TokenList::clone (  
    BumpAllocator & alloc ) const    [inline], [final], [private], [virtual]
```

Clones the list into a new node using the provided allocator.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 364 of file [SyntaxNode.h](#).

7.31.3.2 getChild() [1/2]

```
ConstTokenOrSyntax slang::syntax::TokenList::getChild (  
    size_t index ) const    [inline], [final], [private], [virtual]
```

Gets the child (token or node) at the given index.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 360 of file [SyntaxNode.h](#).

7.31.3.3 getChild() [2/2]

```
TokenOrSyntax slang::syntax::TokenList::getChild (  
    size_t index )    [inline], [final], [private], [virtual]
```

Gets the child (token or node) at the given index.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 359 of file [SyntaxNode.h](#).

7.31.3.4 getChildPtr()

```
PtrTokenOrSyntax slang::syntax::TokenList::getChildPtr (
    size_t index ) [inline], [final], [private], [virtual]
```

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 361 of file [SyntaxNode.h](#).

7.31.3.5 resetAll()

```
void slang::syntax::TokenList::resetAll (
    BumpAllocator & alloc,
    std::span< const TokenOrSyntax > children ) [inline], [final], [private], [virtual]
```

Overwrites all children with the new set of provided *children* (and making a copy with the provided allocator).

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 368 of file [SyntaxNode.h](#).

7.31.3.6 setChild()

```
void slang::syntax::TokenList::setChild (
    size_t index,
    TokenOrSyntax child ) [inline], [final], [private], [virtual]
```

Sets the child (token or node) at the given index.

Implements [slang::syntax::SyntaxListBase](#).

Definition at line 362 of file [SyntaxNode.h](#).

The documentation for this class was generated from the following file:

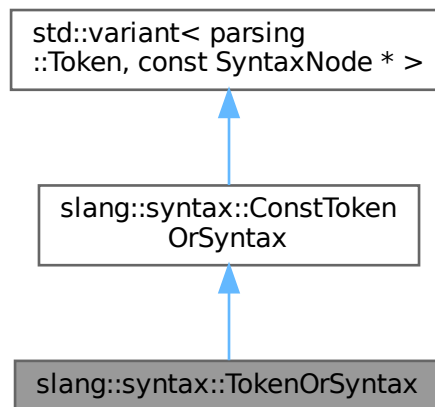
- [include_3rd_party/SyntaxNode.h](#)

7.32 slang::syntax::TokenOrSyntax Struct Reference

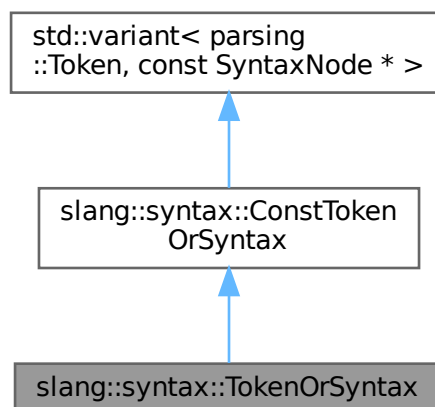
A token or a syntax node.

```
#include <SyntaxNode.h>
```

Inheritance diagram for slang::syntax::TokenOrSyntax:



Collaboration diagram for slang::syntax::TokenOrSyntax:



Public Member Functions

- [TokenOrSyntax](#) (parsing::Token *token*)
- [TokenOrSyntax](#) (SyntaxNode **node*)
- [TokenOrSyntax](#) (nullptr_t)
- [SyntaxNode](#) * *node* () const

Additional Inherited Members

7.32.1 Detailed Description

A token or a syntax node.

Definition at line 75 of file [SyntaxNode.h](#).

7.32.2 Constructor & Destructor Documentation

7.32.2.1 TokenOrSyntax() [1/3]

```
slang::syntax::TokenOrSyntax::TokenOrSyntax (  
    parsing::Token token ) [inline]
```

Definition at line 76 of file [SyntaxNode.h](#).

7.32.2.2 TokenOrSyntax() [2/3]

```
slang::syntax::TokenOrSyntax::TokenOrSyntax (  
    SyntaxNode * node ) [inline]
```

Definition at line 77 of file [SyntaxNode.h](#).

7.32.2.3 TokenOrSyntax() [3/3]

```
slang::syntax::TokenOrSyntax::TokenOrSyntax (  
    nullptr_t ) [inline]
```

Definition at line 78 of file [SyntaxNode.h](#).

7.32.3 Member Function Documentation

7.32.3.1 node()

```
SyntaxNode * slang::syntax::TokenOrSyntax::node ( ) const [inline]
```

Gets access to the object as a syntax node (throwing an exception if it's not actually a syntax node).

Definition at line 82 of file [SyntaxNode.h](#).

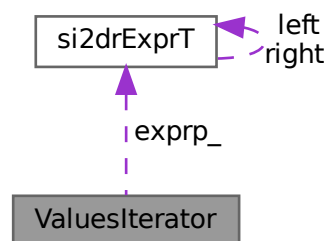
The documentation for this struct was generated from the following file:

- [include_3rd_party/SyntaxNode.h](#)

7.33 ValuesIterator Class Reference

```
#include <iterators.hpp>
```

Collaboration diagram for ValuesIterator:



Public Member Functions

- [ValuesIterator](#) ([si2drValuesIdT](#) values, [si2drErrorT](#) &err)
- [~ValuesIterator](#) ()
- void [next](#) ()
- bool [end](#) ()

Public Attributes

- [si2drValuesIdT values_](#)
- [si2drValueTypeT vtype_](#)
- [si2drInt32T int_](#)
- [si2drFloat64T float_](#)
- [si2drStringT str_](#)
- [si2drBooleanT bool_](#)
- [si2drExprT * exprp_](#)
- [si2drErrorT & err_](#)

7.33.1 Detailed Description

Definition at line 37 of file [iterators.hpp](#).

7.33.2 Constructor & Destructor Documentation

7.33.2.1 ValuesIterator()

```
ValuesIterator::ValuesIterator (
    si2drValuesIdT values,
    si2drErrorT & err )
```

Definition at line 24 of file [iterators.cpp](#).

7.33.2.2 ~ValuesIterator()

```
ValuesIterator::~ValuesIterator ( )
```

Definition at line 27 of file [iterators.cpp](#).

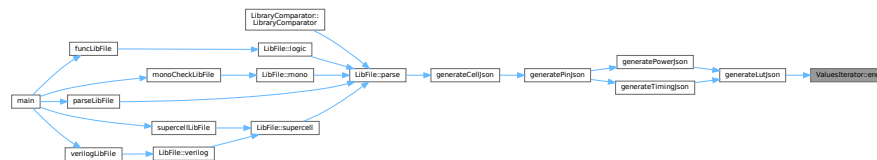
7.33.3 Member Function Documentation

7.33.3.1 end()

```
bool ValuesIterator::end ( )
```

Definition at line 32 of file [iterators.cpp](#).

Here is the caller graph for this function:

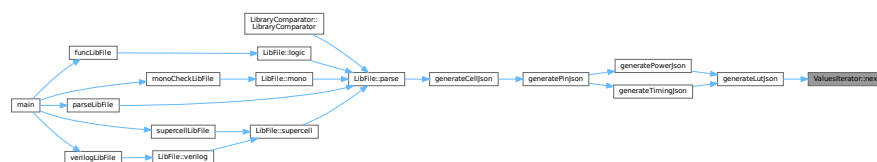


7.33.3.2 next()

```
void ValuesIterator::next ( )
```

Definition at line 29 of file [iterators.cpp](#).

Here is the caller graph for this function:



7.33.4 Member Data Documentation

7.33.4.1 bool_

```
si2drBooleanT ValuesIterator::bool_
```

Definition at line 49 of file [iterators.hpp](#).

7.33.4.2 err_

`si2drErrorT& ValuesIterator::err_`

Definition at line 51 of file [iterators.hpp](#).

7.33.4.3 exprp_

`si2drExprT* ValuesIterator::exprp_`

Definition at line 50 of file [iterators.hpp](#).

7.33.4.4 float_

`si2drFloat64T ValuesIterator::float_`

Definition at line 47 of file [iterators.hpp](#).

7.33.4.5 int_

`si2drInt32T ValuesIterator::int_`

Definition at line 46 of file [iterators.hpp](#).

7.33.4.6 str_

`si2drStringT ValuesIterator::str_`

Definition at line 48 of file [iterators.hpp](#).

7.33.4.7 values_

[si2drValuesIdT](#) ValuesIterator::values_

Definition at line 44 of file [iterators.hpp](#).

7.33.4.8 vtype_

[si2drValueTypeT](#) ValuesIterator::vtype_

Definition at line 45 of file [iterators.hpp](#).

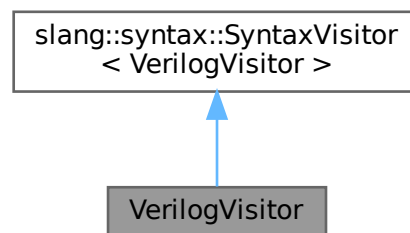
The documentation for this class was generated from the following files:

- [include/iterators.hpp](#)
- [src/iterators.cpp](#)

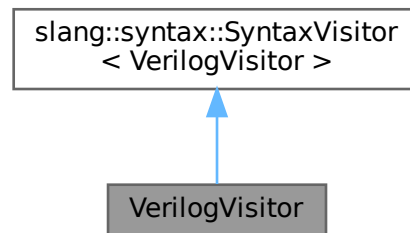
7.34 VerilogVisitor Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for VerilogVisitor:



Collaboration diagram for VerilogVisitor:



Public Member Functions

- [VerilogVisitor](#) (const std::string &targetCell)
- void [handle](#) (const [slang::syntax::SyntaxNode](#) &node)
- void [handle](#) (const slang::syntax::ModuleDeclarationSyntax &module)
- void [handle](#) (const slang::syntax::PortDeclarationSyntax &portDecl)
- void [handle](#) (const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst)
- void [handle](#) (const slang::syntax::SpecifyBlockSyntax &specifyBlock)

Private Attributes

- const std::string & [targetCell_](#)
- int [depth_](#)
- bool [inTargetModule_](#)

7.34.1 Detailed Description

Definition at line 25 of file [verilog_utils.hpp](#).

7.34.2 Constructor & Destructor Documentation

7.34.2.1 VerilogVisitor()

```
VerilogVisitor::VerilogVisitor (
    const std::string & targetCell ) [inline], [explicit]
```

Definition at line 27 of file [verilog_utils.hpp](#).

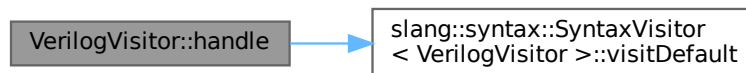
7.34.3 Member Function Documentation

7.34.3.1 handle() [1/5]

```
void VerilogVisitor::handle (
    const slang::syntax::HierarchyInstantiationSyntax & hierarchyInst )
```

Definition at line 80 of file [verilog_utils.cpp](#).

Here is the call graph for this function:

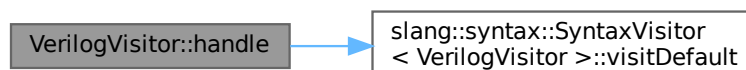


7.34.3.2 handle() [2/5]

```
void VerilogVisitor::handle (
    const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 19 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



7.34.3.3 handle() [3/5]

```
void VerilogVisitor::handle (
    const slang::syntax::PortDeclarationSyntax & portDecl )
```

Definition at line 48 of file [verilog_utils.cpp](#).

7.34.3.4 handle() [4/5]

```
void VerilogVisitor::handle (
    const slang::syntax::SpecifyBlockSyntax & specifyBlock )
```

Definition at line 260 of file [verilog_utils.cpp](#).

Here is the call graph for this function:

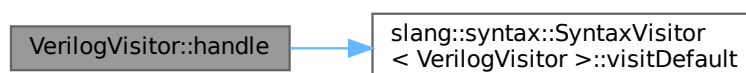


7.34.3.5 handle() [5/5]

```
void VerilogVisitor::handle (
    const slang::syntax::SyntaxNode & node )
```

Definition at line 8 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.34.4 Member Data Documentation

7.34.4.1 depth_

```
int VerilogVisitor::depth_ [private]
```

Definition at line 38 of file [verilog_utils.hpp](#).

7.34.4.2 inTargetModule_

```
bool VerilogVisitor::inTargetModule_ [private]
```

Definition at line 39 of file [verilog_utils.hpp](#).

7.34.4.3 targetCell_

```
const std::string& VerilogVisitor::targetCell_ [private]
```

Definition at line 37 of file [verilog_utils.hpp](#).

The documentation for this class was generated from the following files:

- include/[verilog_utils.hpp](#)
- src/[verilog_utils.cpp](#)

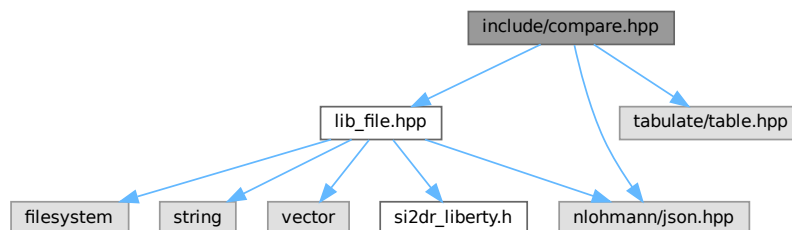
Chapter 8

File Documentation

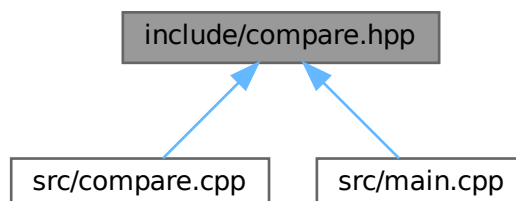
8.1 include/compare.hpp File Reference

```
#include "lib_file.hpp"
#include "nlohmann/json.hpp"
#include "tabulate/table.hpp"
```

Include dependency graph for compare.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [LibraryComparator](#)

Typedefs

- using [json](#) = nlohmann::json

8.1.1 Typedef Documentation

8.1.1.1 json

using [json](#) = nlohmann::json

Definition at line 9 of file [compare.hpp](#).

8.2 compare.hpp

[Go to the documentation of this file.](#)

```

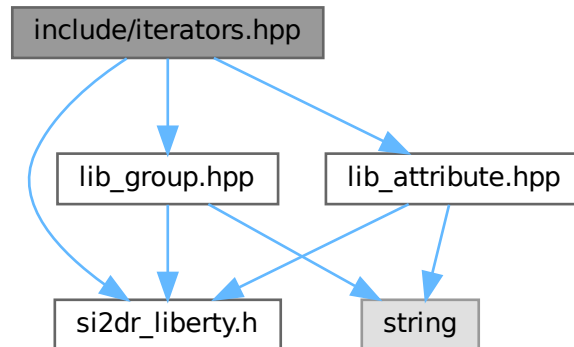
00001 #ifndef COMPARE_HPP
00002 #define COMPARE_HPP
00003
00004 #include "lib_file.hpp"
00005
00006 #include "nlohmann/json.hpp"
00007 #include "tabulate/table.hpp"
00008
00009 using json = nlohmann::json;
00010 using namespace tabulate;
00011
00012 class LibraryComparator {
00013 public:
00014     LibraryComparator(LibFile &ref_libfile, LibFile &comp_libfile, double reltol, double abstol);
00015     std::filesystem::path ref_lib_path_;
00016     std::filesystem::path comp_lib_path_;
00017     void generateReport(const std::string &output_file);
00018
00019 private:
00020     json ref_json_;
00021     json comp_json_;
00022     double reltol_;
00023     double abstol_;
00024
00025     void compareCell(const std::string &cell_name, const json &ref_cell, const json &comp_cell,
00026                     Table &table);
00027     void comparePin(const std::string &cell_name, const std::string &pin_name, const json &ref_pin,
00028                    const json &comp_pin, Table &table);
00029     void compareTimingArc(const std::string &cell_name, const std::string &pin_name,
00030                           const std::string &timing_type, const json &ref_timing_arc,
00031                           const json &comp_timing_arc, Table &table);
00032     void compareLut(const std::string &cell_name, const std::string &pin_name,
00033                     const std::string &timing_type, const std::string &related_pin,
00034                     const std::string &arc_name, const json &ref_lut, const json &comp_lut, Table &table);
00035     // void configureTableFormat(Table &table);
00036 };
00037
00038 #endif // COMPARE_HPP

```

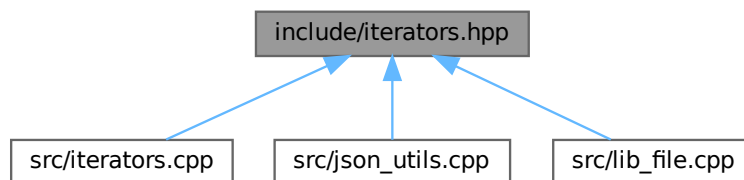
8.3 include/iterators.hpp File Reference

```
#include "si2dr_liberty.h"  
#include "lib_attribute.hpp"  
#include "lib_group.hpp"
```

Include dependency graph for iterators.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [GroupsIterator](#)
- class [AttributesIterator](#)
- class [ValuesIterator](#)

8.4 iterators.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef ITERATORS_H
00002 #define ITERATORS_H
00003
00004 #include "si2dr_liberty.h"
00005
00006 #include "lib_attribute.hpp"
00007 #include "lib_group.hpp"
00008
00009 class GroupsIterator {
00010 public:
00011     GroupsIterator(si2drGroupsIdT groups, si2drErrorT &err);
00012     ~GroupsIterator();
00013     void next();
00014     bool end();
00015     LibGroup get();
00016
00017 private:
00018     si2drGroupsIdT groups_;
00019     si2drGroupIdT group_;
00020     si2drErrorT &err_;
00021 };
00022
00023 class AttributesIterator {
00024 public:
00025     AttributesIterator(si2drAttrsIdT attrs, si2drErrorT &err);
00026     ~AttributesIterator();
00027     void next();
00028     bool end();
00029     LibAttribute get();
00030
00031 private:
00032     si2drAttrsIdT attrs_;
00033     si2drAttrIdT attr_;
00034     si2drErrorT &err_;
00035 };
00036
00037 class ValuesIterator {
00038 public:
00039     ValuesIterator(si2drValuesIdT values, si2drErrorT &err);
00040     ~ValuesIterator();
00041     void next();
00042     bool end();
00043
00044     si2drValuesIdT values_;
00045     si2drValueTypeT vtype_;
00046     si2drInt32T int_;
00047     si2drFloat64T float_;
00048     si2drStringT str_;
00049     si2drBooleanT bool_;
00050     si2drExprT *exprp_;
00051     si2drErrorT &err_;
00052 };
00053
00054 #endif // ITERATORS_H

```

8.5 include/json_utils.hpp File Reference

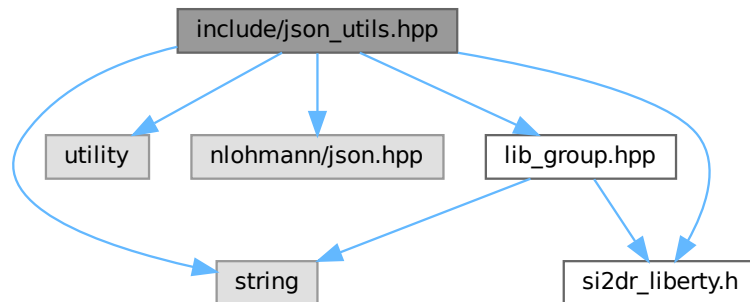
```

#include <string>
#include <utility>

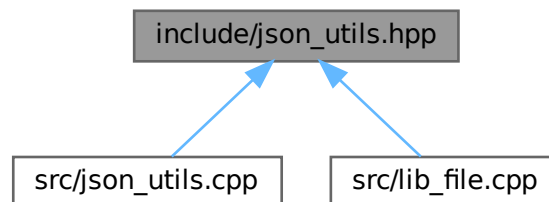
```

```
#include "nlohmann/json.hpp"
#include "si2dr_liberty.h"
#include "lib_group.hpp"
```

Include dependency graph for json_utils.hpp:



This graph shows which files directly or indirectly include this file:



Typedefs

- using `json` = `nlohmann::json`

Functions

- `json generateLutJson (LibGroup &lib_lut_group, si2drErrorT &err)`
Generates a JSON object representation of a look-up table (LUT) from a `LibGroup` object.
- `json generatePowerJson (LibGroup &lib_power_group, si2drErrorT &err)`
Converts a Liberty power group into a JSON representation.
- `std::pair< std::string, json > generatePinJson (LibGroup &lib_pin_group, si2drErrorT &err)`

Generates a JSON representation of a Liberty pin group.

- `json generateCellJson (LibGroup &lib_cell_group, si2drErrorT &err)`

Generates a JSON representation of a cell from a [LibGroup](#) object.

8.5.1 Typedef Documentation

8.5.1.1 json

```
using json = nlohmann::json
```

Definition at line 12 of file [json_utils.hpp](#).

8.5.2 Function Documentation

8.5.2.1 generateCellJson()

```
json generateCellJson (
    LibGroup & lib_cell_group,
    si2drErrorT & err )
```

Generates a JSON representation of a cell from a [LibGroup](#) object.

This function processes a [LibGroup](#) representing a cell and converts it into a JSON object. It extracts the cell name and processes specific attributes such as area, cell_leakage_power, and cell_footprint. It also processes pins within the cell by categorizing them based on their direction (input, output, internal, inout).

Parameters

<i>lib_cell_group</i>	The LibGroup object representing the cell
<i>err</i>	Reference to a si2drErrorT object for error handling

Returns

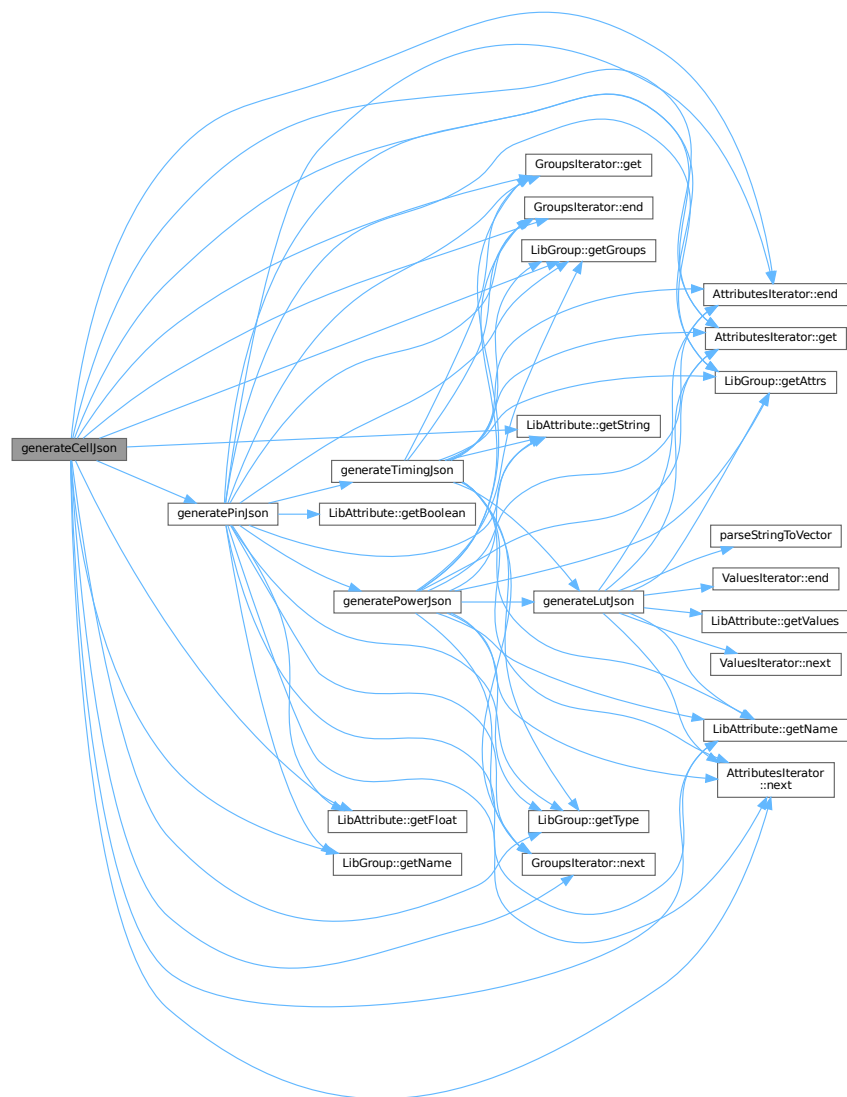
json A JSON object containing the cell's information with the following structure:

- "cell_name": string - name of the cell

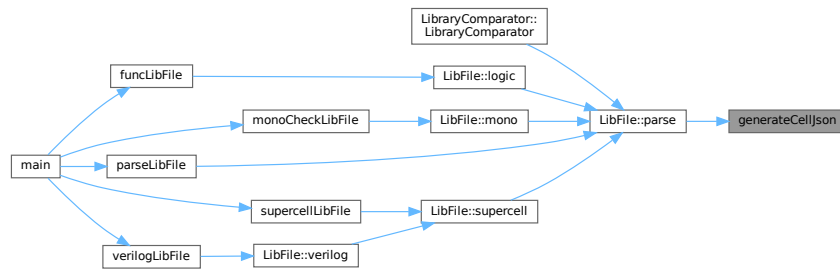
- "area": float (optional) - cell area if present
- "cell_leakage_power": float (optional) - cell leakage power if present
- "cell_footprint": string (optional) - cell footprint if present
- "input_pins": array - list of input pins
- "output_pins": array - list of output pins
- "internal_pins": array - list of internal pins
- "inout_pins": array - list of inout pins

Definition at line 272 of file `json_utils.cpp`.

Here is the call graph for this function:



Here is the caller graph for this function:



8.5.2.2 generateLutJson()

```

json generateLutJson (
    LibGroup & lib_lut_group,
    si2drErrorT & err )

```

Generates a JSON object representation of a look-up table (LUT) from a [LibGroup](#) object.

This function iterates through the attributes of the provided [LibGroup](#) object representing a LUT and converts them into a JSON structure. It specifically handles the following attributes:

- "index_1": Converted to a vector and stored in the JSON
- "index_2": Converted to a vector and stored in the JSON
- "values": Each value is parsed into a vector and added to an array in the JSON

Any other attributes encountered will trigger a warning message.

Parameters

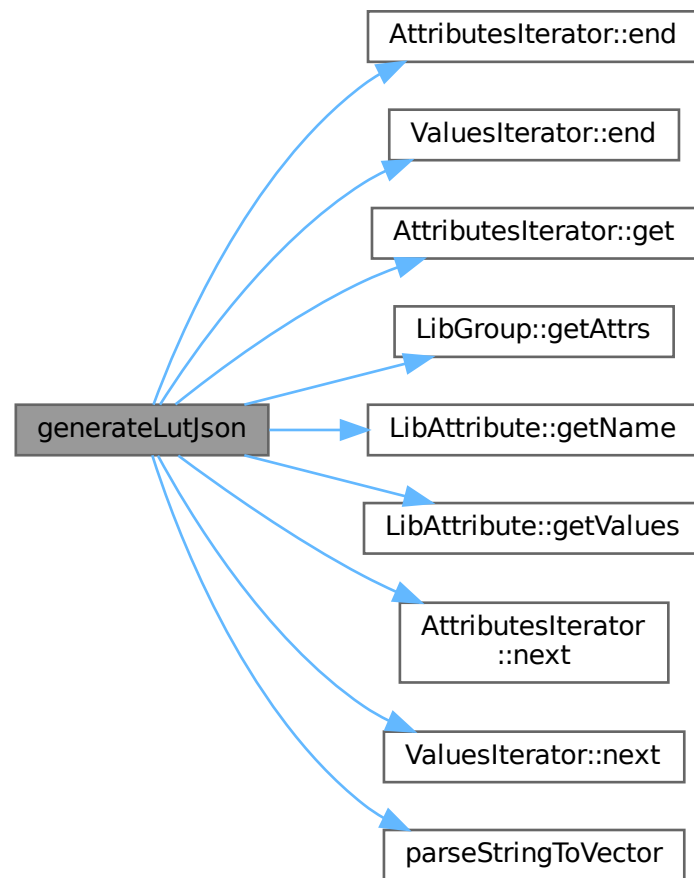
<i>lib_lut_group</i>	The LibGroup object containing the LUT data to be converted
<i>err</i>	Reference to an <code>si2drErrorT</code> object to track any errors during processing

Returns

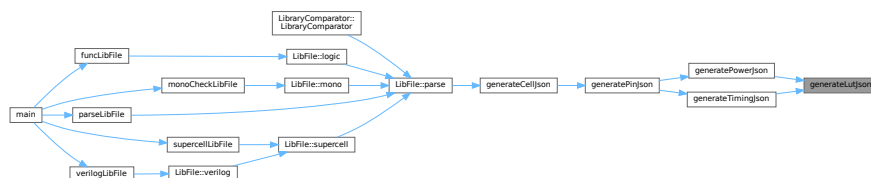
json A JSON object representing the LUT data with keys for "index_1", "index_2", and "values"

Definition at line 62 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.5.2.3 generatePinJson()

```
std::pair< std::string, json > generatePinJson (
```

```
LibGroup & lib_pin_group,  
si2drErrorT & err )
```

Generates a JSON representation of a Liberty pin group.

This function traverses a Liberty pin group, extracts relevant attributes and sub-groups, and converts them into a JSON object. It also determines the pin's direction.

Processes the following pin attributes:

- direction
- max_transition, capacitance, rise_capacitance, fall_capacitance, max_capacitance (float types)
- function, power_down_function, related_ground_pin, related_power_pin, three_state (string types)
- clock (boolean type)

Handles the following sub-groups:

- internal_power: Converted using [generatePowerJson\(\)](#)
- timing: Converted using [generateTimingJson\(\)](#)

Parameters

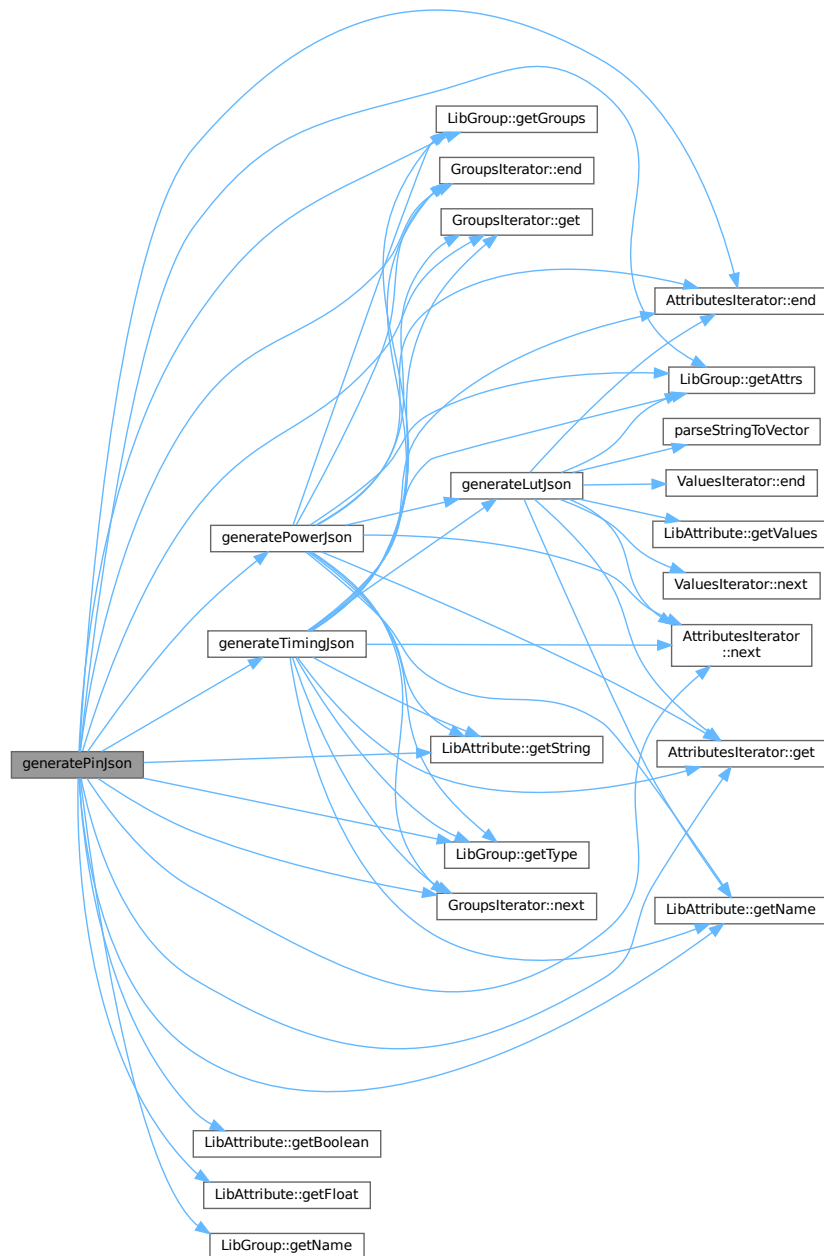
<i>lib_pin_group</i>	The Liberty pin group to process
<i>err</i>	Reference to an error object for tracking Liberty API errors

Returns

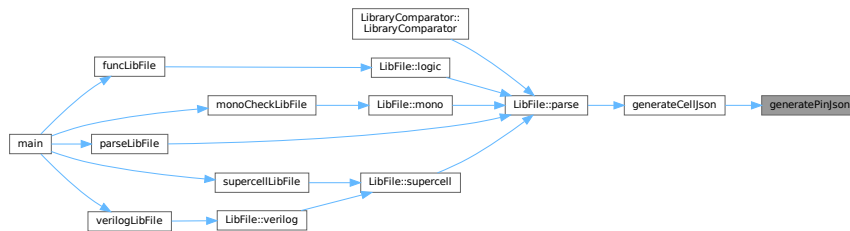
A pair containing the pin direction (string) and the JSON representation of the pin

Definition at line 206 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.5.2.4 generatePowerJson()

```

json generatePowerJson (
    LibGroup & lib_power_group,
    si2drErrorT & err )

```

Converts a Liberty power group into a JSON representation.

This function processes a Liberty power group and converts its attributes and subgroups into a JSON object. It handles attributes like "when", "related_pin", and "related_pg_pin", as well as "rise_power" and "fall_power" subgroups.

Parameters

<i>lib_power_group</i>	The Liberty power group to convert
<i>err</i>	Reference to an si2drErrorT object for error handling

Returns

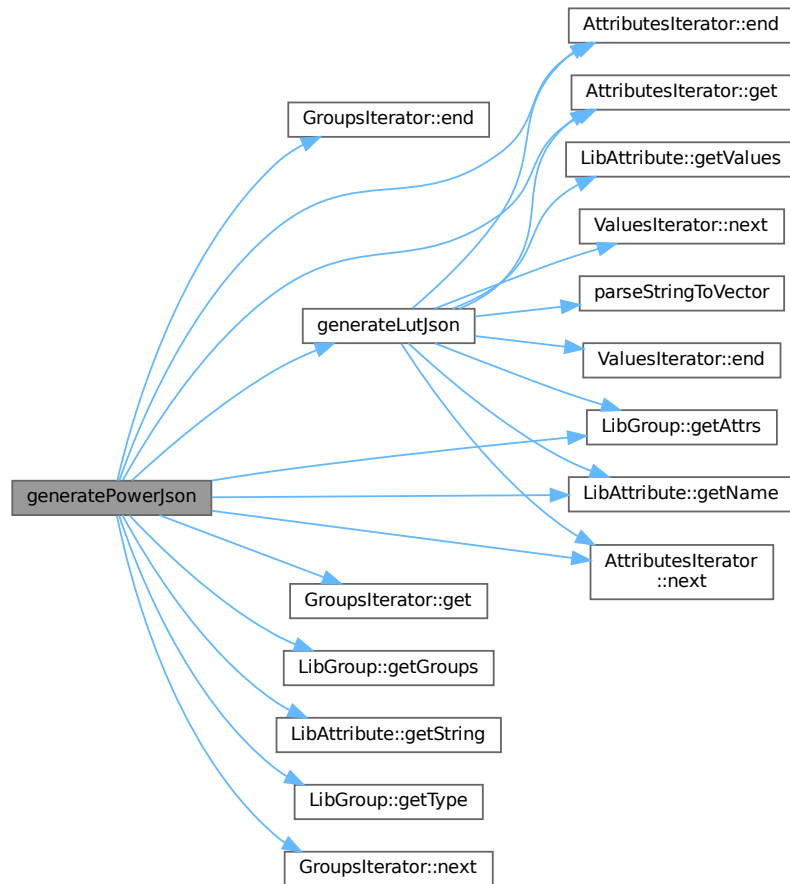
json A JSON object representing the power group data

The function:

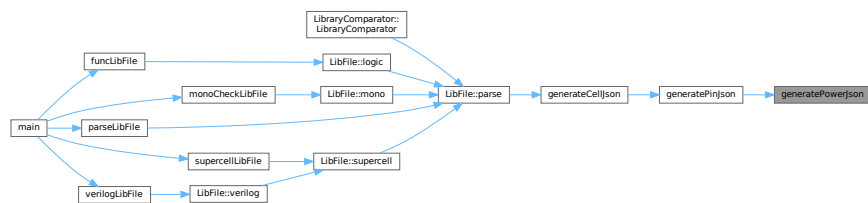
- Extracts string attributes (when, related_pin, related_pg_pin)
- Processes rise_power and fall_power subgroups by converting them to LUT JSON format
- Logs warnings for unknown power subgroup types

Definition at line 154 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.6 json_utils.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef JSON_UTILS_HPP
00002 #define JSON_UTILS_HPP

```

```

00003
00004 #include <string>
00005 #include <utility>
00006
00007 #include "nlohmann/json.hpp"
00008 #include "si2dr_liberty.h"
00009
00010 #include "lib_group.hpp"
00011
00012 using json = nlohmann::json;
00013
00014 json generateLutJson(LibGroup &lib_lut_group, si2drErrorT &err);
00015 json generatePowerJson(LibGroup &lib_power_group, si2drErrorT &err);
00016 std::pair<std::string, json> generatePinJson(LibGroup &lib_pin_group, si2drErrorT &err);
00017 json generateCellJson(LibGroup &lib_cell_group, si2drErrorT &err);
00018
00019 #endif // JSON_UTILS_HPP

```

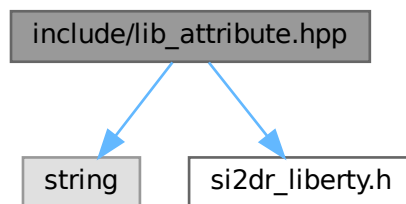
8.7 include/lib_attribute.hpp File Reference

```

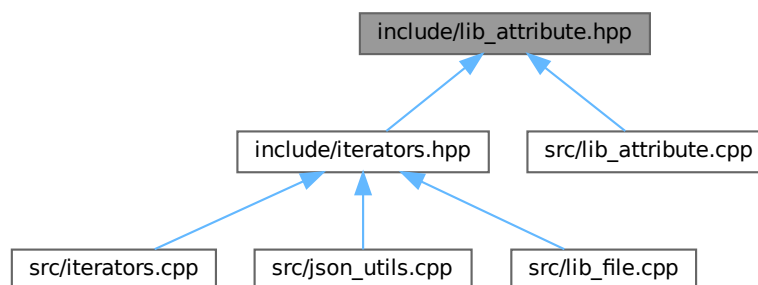
#include <string>
#include "si2dr_liberty.h"

```

Include dependency graph for lib_attribute.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [LibAttribute](#)

8.8 lib_attribute.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef LIB_ATTRIBUTE_H
00002 #define LIB_ATTRIBUTE_H
00003
00004 #include <string>
00005
00006 #include "si2dr_liberty.h"
00007
00008 class LibAttribute {
00009 public:
00010     LibAttribute(si2drAttrIdT attr, si2drErrorT &err);
00011     ~LibAttribute();
00012     std::string getName();
00013     bool isComplex();
00014     si2drValuesIdT getValues();
00015     long int getInt();
00016     double getFloat();
00017     std::string getString();
00018     bool getBoolean();
00019
00020 private:
00021     si2drAttrIdT attr_;
00022     si2drErrorT &err_;
00023 };
00024
00025 #endif // LIB_ATTRIBUTE_H

```

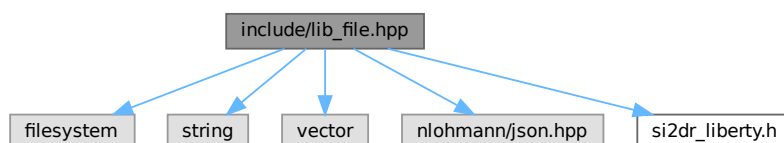
8.9 include/lib_file.hpp File Reference

```

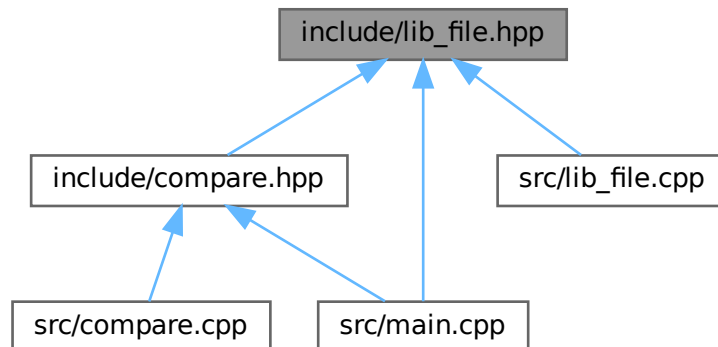
#include <filesystem>
#include <string>
#include <vector>
#include "nlohmann/json.hpp"
#include "si2dr_liberty.h"

```

Include dependency graph for lib_file.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [LibFile](#)

Typedefs

- using [json](#) = `nlohmann::json`

8.9.1 Typedef Documentation

8.9.1.1 json

```
using json = nlohmann::json
```

Definition at line 11 of file [lib_file.hpp](#).

8.10 lib_file.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef LIB_FILE_H
00002 #define LIB_FILE_H
00003
00004 #include <filesystem>
00005 #include <string>
00006 #include <vector>
00007
00008 #include "nlohmann/json.hpp"
00009 #include "si2dr_liberty.h"
00010
00011 using json = nlohmann::json;
00012
00013 class LibFile {
00014 public:
00015     LibFile(const std::string &filepath, const std::string &loggername);
00016     ~LibFile();
00017     std::shared_ptr<spdlog::logger> logger_;
00018     std::filesystem::path filepath_; // full path to the file
00019     std::string basename_;          // file name without extension
00020     std::string filename_;          // full file name with extension
00021     std::string libname_ = "";      // library name obtained through parsing
00022     std::string jsonname_ = "";     // json file name to store parsed data
00023     std::string loggername_ = "";   // log file name
00024     json lib_json_ = json::object();
00025     void writeJsonToFile();
00026     void parse();
00027     void modify();
00028     void mono(const bool is_slew);
00029     void supercell(const int chain_length, const std::vector<std::string> &cell_names);
00030     void verilog(const int chain_length, const std::vector<std::string> &cell_names);
00031     std::map<std::string, std::string> logic(const std::string &cell_name);
00032
00033 private:
00034     si2drErrorT err_;
00035     int process_ = 0;
00036     float voltage_ = 0.0;
00037     int temperature_ = 0;
00038     void read();
00039     bool checkTimingArcMonotonicity(const json &cell, const json &pin, const json &arc,
00040                                     const std::string &type, bool is_slew);
00041 };
00042
00043 #endif // LIB_FILE_H

```

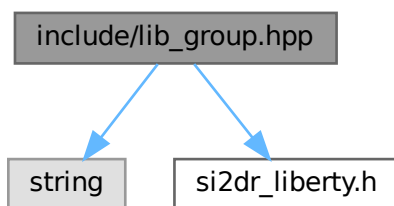
8.11 include/lib_group.hpp File Reference

```

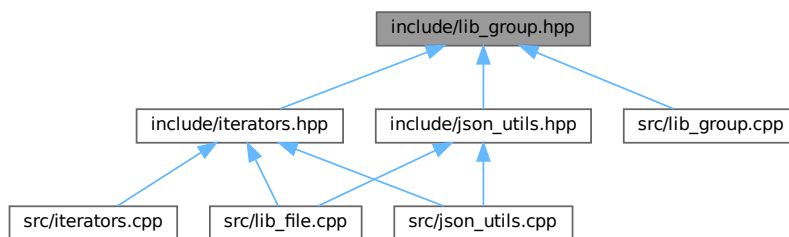
#include <string>
#include "si2dr_liberty.h"

```

Include dependency graph for lib_group.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [LibGroup](#)

8.12 lib_group.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef LIB_GROUP_H
00002 #define LIB_GROUP_H
00003
00004 #include <string>
00005
00006 #include "si2dr_liberty.h"
00007
00008 class LibGroup {
00009 public:
00010     LibGroup(si2drGroupIdT group, si2drErrorT &err);
00011     ~LibGroup();
00012     std::string getName();
00013     std::string getType();
00014     si2drAttrsIdT getAttrs();
  
```

```

00015     si2drGroupsIdT getGroups();
00016
00017 private:
00018     si2drGroupIdT group_;
00019     si2drErrorT &err_;
00020 };
00021
00022 #endif // LIB_GROUP_H

```

8.13 include/verilog_utils.hpp File Reference

```

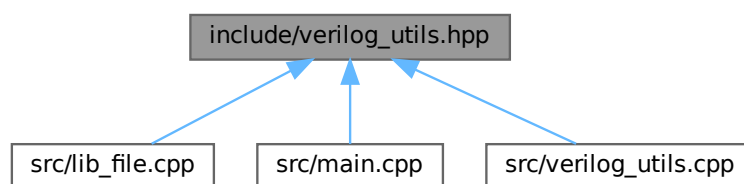
#include <algorithm>
#include <fstream>
#include <iostream>
#include <map>
#include <memory>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>
#include "slang/parsing/Token.h"
#include "slang/syntax/AllSyntax.h"
#include "slang/syntax/SyntaxPrinter.h"
#include "slang/syntax/SyntaxTree.h"
#include "slang/syntax/SyntaxVisitor.h"
#include "spdlog/spdlog.h"

```

Include dependency graph for verilog_utils.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [VerilogVisitor](#)
- class [CellExtractor](#)
- class [CellPrinter](#)
- class [ModuleRewriter](#)
- struct [GateInfo](#)
- class [LogicExtractor](#)

Functions

- void [getAST](#) (const std::string &verilog_file, const std::string &cell)
- void [extractAndPrintNetlistInfo](#) (const std::string &verilog_file, const std::string &cell)
- std::map< std::string, std::string > [extractLogicFromVerilog](#) (const std::string &verilog_file, const std::string &cell)

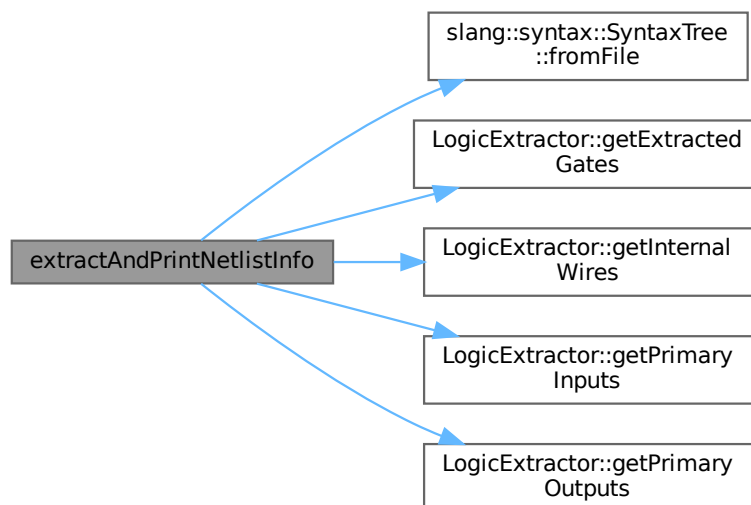
8.13.1 Function Documentation

8.13.1.1 extractAndPrintNetlistInfo()

```
void extractAndPrintNetlistInfo (
    const std::string & verilog_file,
    const std::string & cell )
```

Definition at line 1123 of file [verilog_utils.cpp](#).

Here is the call graph for this function:

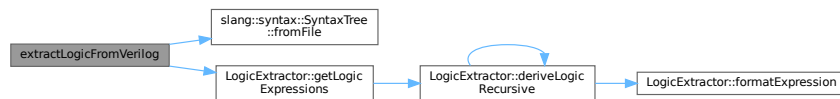


8.13.1.2 extractLogicFromVerilog()

```
std::map< std::string, std::string > extractLogicFromVerilog (
    const std::string & verilog_file,
    const std::string & cell )
```

Definition at line 1184 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:

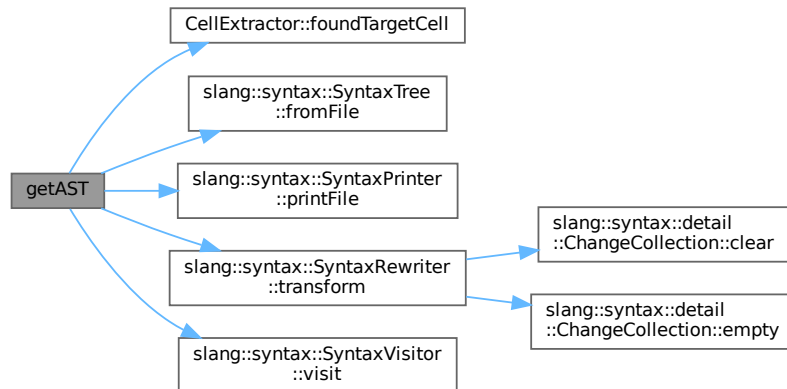


8.13.1.3 getAST()

```
void getAST (
    const std::string & verilog_file,
    const std::string & cell )
```

Definition at line 546 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



8.14 verilog_utils.hpp

[Go to the documentation of this file.](#)

```

00001 // verilog_utils.hpp
00002
00003 #ifndef VERILOG_UTILS_H
00004 #define VERILOG_UTILS_H
00005
00006 #include <algorithm> // For std::find
00007 #include <fstream>
00008 #include <iostream>
00009 #include <map>
00010 #include <memory>    // For shared_ptr
00011 #include <stdexcept> // For exceptions
00012 #include <string>
00013 #include <unordered_map>
00014 #include <unordered_set>
00015 #include <vector>
00016
00017 #include "slang/parsing/Token.h" // Include Token header for TokenKind
00018 #include "slang/syntax/AllSyntax.h" // Include for specific syntax nodes like NonAnsiPortListSyntax etc.
00019 #include "slang/syntax/SyntaxPrinter.h"
00020 #include "slang/syntax/SyntaxTree.h" // Include SyntaxTree header
00021 #include "slang/syntax/SyntaxVisitor.h"
00022 #include "spdlog/spdlog.h" // Include spdlog
00023
00024 // Creating custom visitor class
00025 class VerilogVisitor : public slang::syntax::SyntaxVisitor<VerilogVisitor> {
00026 public:
00027     explicit VerilogVisitor(const std::string &targetCell)
00028         : targetCell_(targetCell), depth_(0), inTargetModule_(false) {}
00029     void handle(const slang::syntax::SyntaxNode &node);
00030     void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00031     void handle(const slang::syntax::PortDeclarationSyntax &portDecl);
00032     void handle(const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst);
00033     // void handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst);
00034     void handle(const slang::syntax::SpecifyBlockSyntax &specifyBlock);
00035
00036 private:
00037     const std::string &targetCell_;

```

```

00038     int depth_;
00039     bool inTargetModule_;
00040 };
00041
00042 // Creating a Rewriter to extract a specific cell
00043 class CellExtractor : public slang::syntax::SyntaxRewriter<CellExtractor> {
00044 public:
00045     explicit CellExtractor(const std::string &targetCell)
00046         : targetCell_(targetCell), foundTarget_(false) {}
00047     void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00048     bool foundTargetCell() const;
00049
00050 private:
00051     const std::string &targetCell_;
00052     bool foundTarget_;
00053 };
00054
00055 // Print specific cell when visiting SyntaxTree
00056 class CellPrinter : public slang::syntax::SyntaxVisitor<CellPrinter> {
00057 public:
00058     explicit CellPrinter(const std::string &targetCell, std::ostream &out)
00059         : targetCell_(targetCell), out_(out), foundTarget_(false) {}
00060     void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00061
00062 private:
00063     const std::string &targetCell_;
00064     std::ostream &out_;
00065     bool foundTarget_;
00066 };
00067
00068 // Comprehensive module rewriter for adding ports, instances, and connections
00069 class ModuleRewriter : public slang::syntax::SyntaxRewriter<ModuleRewriter> {
00070 public:
00071     explicit ModuleRewriter(const std::vector<std::string> &inputPins,
00072                             const std::vector<std::string> &outputPins,
00073                             const std::pair<std::string, std::string> &supercell_entry,
00074                             int instance_count, std::shared_ptr<spdlog::logger> logger)
00075         : inputPins_(inputPins), outputPins_(outputPins), cellName_(supercell_entry.first),
00076           moduleName_(supercell_entry.second), logger_(logger), depth_(0),
00077           instance_count_(instance_count) {}
00078     std::shared_ptr<spdlog::logger> logger_;
00079     void handle(const slang::syntax::SyntaxNode &node);
00080     void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00081
00082 private:
00083     const std::vector<std::string> &inputPins_;
00084     const std::vector<std::string> &outputPins_;
00085     const std::string cellName_;
00086     const std::string moduleName_;
00087     std::map<std::string, std::string> portInfoMap_; // Map from port name to direction
00088     int depth_;
00089     int instance_count_;
00090 };
00091
00092 void getAST(const std::string &verilog_file, const std::string &cell);
00093
00094 // --- New Class for Logic Extraction ---
00095
00096 // Structure to represent a gate instance's information
00097 struct GateInfo {
00098     slang::parsing::TokenKind kind; // Store the TokenKind for reliable checks
00099     std::string gateTypeName;        // Store the string name like "and", "xor"
00100     std::vector<std::string> inputSignals;
00101     std::string outputSignal;
00102 };
00103
00104 // Visitor class to extract netlist information and derive logic expressions
00105 class LogicExtractor : public slang::syntax::SyntaxVisitor<LogicExtractor> {
00106 public:

```

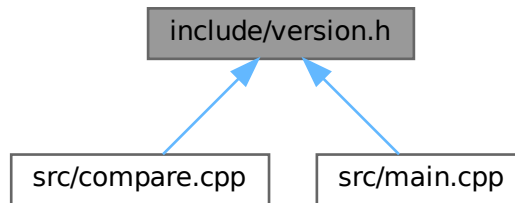
```

00107 // Constructor takes the target cell name
00108 explicit LogicExtractor(const std::string &targetCell)
00109     : targetCell_(targetCell), inTargetModule_(false), parsingComplete_(false) {}
00110
00111 // --- Visitor Handlers ---
00112 void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00113 void handle(
00114     const slang::syntax::PortDeclarationSyntax &portDecl); // Handles direction/type declaration
00115 void
00116 handle(const slang::syntax::NonAnsiPortListSyntax &portList); // Handles port names list in header
00117 void
00118 handle(const slang::syntax::NetDeclarationSyntax &netDecl); // To find explicitly declared wires
00119 void handle(const slang::syntax::PrimitiveInstantiationSyntax
00120     &primitiveInst); // Handles gate instantiation
00121 // Potentially handle ContinuousAssignSyntax if needed later:
00122 // void handle(const slang::syntax::ContinuousAssignSyntax& assign);
00123
00124 // --- Access Extracted Info (for Step 1 debugging) ---
00125 const std::unordered_map<std::string, GateInfo> &getExtractedGates()const {
00126     return gateOutputDrivers_;
00127 }
00128 const std::unordered_set<std::string> &getPrimaryInputs()const { return primaryInputs_; }
00129 const std::unordered_set<std::string> &getPrimaryOutputs()const { return primaryOutputs_; }
00130 const std::unordered_set<std::string> &getInternalWires()const {
00131     return internalWires_;
00132 }
00133
00134 // --- Logic Derivation (Commented out for Step 1) ---
00135 std::map<std::string, std::string> getLogicExpressions();
00136
00137 private:
00138 // --- Internal State ---
00139 const std::string &targetCell_;
00140 bool inTargetModule_;
00141 bool parsingComplete_; // Flag to indicate AST traversal is done
00142
00143 // Netlist Information
00144 std::unordered_set<std::string> primaryInputs_;
00145 std::unordered_set<std::string> primaryOutputs_;
00146 std::unordered_set<std::string> internalWires_; // Includes gate outputs
00147
00148 // Temporary map to store directions found in PortDeclarationSyntax
00149 // Key: port name, Value: direction ("input", "output", "inout", "unknown")
00150 std::unordered_map<std::string, std::string> portDirections_;
00151
00152 // Map: output signal name -> GateInfo driving it
00153 std::unordered_map<std::string, GateInfo> gateOutputDrivers_;
00154
00155 // Map: signal name -> Logic expression string (Memoization Cache) - (Used in Step 2)
00156 std::unordered_map<std::string, std::string> logicCache_;
00157
00158 // --- Helper Methods ---
00159 // Recursive function to derive logic for a given signal (Used in Step 2)
00160 std::string deriveLogicRecursive(const std::string &signalName);
00161 // Helper to format expressions based on gate type (Used in Step 2)
00162 std::string formatExpression(const GateInfo &gateInfo,
00163     const std::vector<std::string> &inputExprs);
00164 };
00165
00166 // --- Function Declaration ---
00167 // Modify getAST or create a new function to use LogicExtractor
00168 // For Step 1, this function will just run the visitor and maybe print summary
00169 void extractAndPrintNetlistInfo(const std::string &verilog_file, const std::string &cell);
00170
00171 // Function to be implemented in Step 2
00172 std::map<std::string, std::string> extractLogicFromVerilog(const std::string &verilog_file,
00173     const std::string &cell);
00174
00175 #endif // VERILOG_UTILS_H

```


8.15 include/version.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- `#define APP_NAME "ZlibValidation"`
- `#define APP_VERSION_MAJOR 0`
- `#define APP_VERSION_MINOR 1`
- `#define APP_VERSION_PATCH 0`
- `#define APP_VERSION "0.1.0"`
- `#define APP_AUTHOR "Song Zixuan"`
- `#define APP_CONTACT "cedar@zju.edu.cn"`
- `#define BUILD_TIMESTAMP "2025-03-30 23:38:12"`

8.15.1 Macro Definition Documentation

8.15.1.1 APP_AUTHOR

```
#define APP_AUTHOR "Song Zixuan"
```

Definition at line 10 of file [version.h](#).

8.15.1.2 APP_CONTACT

```
#define APP_CONTACT "cedar@zju.edu.cn"
```

Definition at line 11 of file [version.h](#).

8.15.1.3 APP_NAME

```
#define APP_NAME "ZlibValidation"
```

Definition at line 5 of file [version.h](#).

8.15.1.4 APP_VERSION

```
#define APP_VERSION "0.1.0"
```

Definition at line 9 of file [version.h](#).

8.15.1.5 APP_VERSION_MAJOR

```
#define APP_VERSION_MAJOR 0
```

Definition at line 6 of file [version.h](#).

8.15.1.6 APP_VERSION_MINOR

```
#define APP_VERSION_MINOR 1
```

Definition at line 7 of file [version.h](#).

8.15.1.7 APP_VERSION_PATCH

```
#define APP_VERSION_PATCH 0
```

Definition at line 8 of file [version.h](#).

8.15.1.8 BUILD_TIMESTAMP

```
#define BUILD_TIMESTAMP "2025-03-30 23:38:12"
```

Definition at line 12 of file [version.h](#).

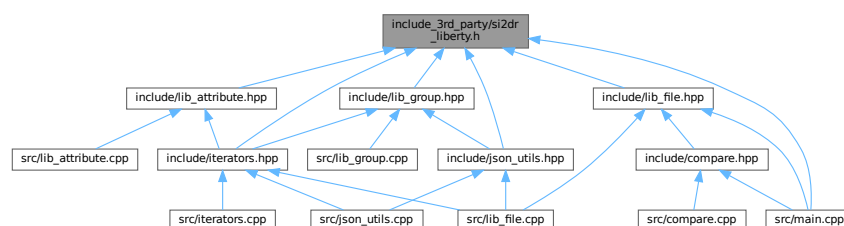
8.16 version.h

[Go to the documentation of this file.](#)

```
00001 // version.h.in
00002 #pragma once
00003
00004 // Auto-generated by CMake - DO NOT EDIT
00005 #define APP_NAME          "ZlibValidation"
00006 #define APP_VERSION_MAJOR 0
00007 #define APP_VERSION_MINOR 1
00008 #define APP_VERSION_PATCH 0
00009 #define APP_VERSION       "0.1.0"
00010 #define APP_AUTHOR        "Song Zixuan"
00011 #define APP_CONTACT       "cedar@zju.edu.cn"
00012 #define BUILD_TIMESTAMP   "2025-03-30 23:38:12"
```

8.17 include_3rd_party/si2dr_liberty.h File Reference

This graph shows which files directly or indirectly include this file:



Classes

- struct [si2OID](#)
- struct [si2drExprT](#)
- struct [liberty_value_data](#)

Macros

- `#define` [SI2_ARGS](#)(args) ()
- `#define` [SI2_LONG_MAX](#) 2147483647
- `#define` [SI2_LONG_MIN](#) -2147483647
- `#define` [SI2_ULONG_MAX](#) 4294967295
- `#define` [SI2DR_MIN_FLOAT32](#) -1E+37
- `#define` [SI2DR_MAX_FLOAT32](#) 1E+37
- `#define` [SI2DR_MIN_FLOAT64](#) -1.797693e+308
- `#define` [SI2DR_MAX_FLOAT64](#) 1.797693e+308
- `#define` [SI2DR_MIN_INT32](#) -2147483647
- `#define` [SI2DR_MAX_INT32](#) 2147483647
- `#define` [SI2DR_MAX_STRING_LEN](#) 1048576 /*1024*1024*/
- `#define` [SI2DR_TRUE](#) SI2_TRUE
- `#define` [SI2DR_FALSE](#) SI2_FALSE
- `#define` [LONG_DOUBLE](#) long double

Typedefs

- `typedef enum` [si2BooleanEnum](#) [si2BooleanT](#)
- `typedef long int` [si2LongT](#)
- `typedef float` [si2FloatT](#)
- `typedef double` [si2DoubleT](#)
- `typedef char *` [si2StringT](#)
- `typedef void` [si2VoidT](#)
- `typedef enum` [si2TypeEnum](#) [si2TypeT](#)
- `typedef struct` [si2OID](#) [si2ObjectIdT](#)
- `typedef void *` [si2IteratorIdT](#)
- `typedef` [si2BooleanT](#) [si2drBooleanT](#)
- `typedef` [si2LongT](#) [si2drInt32T](#)
- `typedef` [si2FloatT](#) [si2drFloat32T](#)
- `typedef` [si2DoubleT](#) [si2drFloat64T](#)
- `typedef` [si2StringT](#) [si2drStringT](#)
- `typedef` [si2VoidT](#) [si2drVoidT](#)
- `typedef` [si2IteratorIdT](#) [si2drIterIdT](#)

- typedef [si2ObjectIdT](#) [si2drObjectIdT](#)
- typedef [si2drObjectIdT](#) [si2drGroupIdT](#)
- typedef [si2drObjectIdT](#) [si2drAttrIdT](#)
- typedef [si2drObjectIdT](#) [si2drAttrComplexValIdT](#)
- typedef [si2drObjectIdT](#) [si2drDefinIdT](#)
- typedef [si2drIterIdT](#) [si2drObjectsIdT](#)
- typedef [si2drIterIdT](#) [si2drGroupsIdT](#)
- typedef [si2drIterIdT](#) [si2drAttrIdT](#)
- typedef [si2drIterIdT](#) [si2drDefinesIdT](#)
- typedef [si2drIterIdT](#) [si2drNamesIdT](#)
- typedef [si2drIterIdT](#) [si2drValuesIdT](#)
- typedef enum [si2drSeverityT](#) [si2drSeverityT](#)
- typedef enum [si2drObjectTypeT](#) [si2drObjectTypeT](#)
- typedef enum [si2drAttrTypeT](#) [si2drAttrTypeT](#)
- typedef enum [si2drValueTypeT](#) [si2drValueTypeT](#)
- typedef enum [si2drExprTypeT](#) [si2drExprTypeT](#)
- typedef struct [si2drExprT](#) [si2drExprT](#)
- typedef enum [si2drErrorT](#) [si2drErrorT](#)
- typedef [si2drVoidT](#)(* [si2drMessageHandlerT](#)) ([si2drSeverityT](#) sev, [si2drErrorT](#) errToPrint, [si2drStringT](#) auxText, [si2drErrorT](#) *err)

Enumerations

- enum [si2BooleanEnum](#) { [SI2_FALSE](#) = 0 , [SI2_TRUE](#) = 1 }
- enum [si2TypeEnum](#) {
[SI2_UNDEFINED_VALUETYPE](#) = 0 , [SI2_BOOLEAN](#) = 1 , [SI2_LONG](#) = 2 , [SI2_FLOAT](#) = 3
,
[SI2_DOUBLE](#) = 4 , [SI2_STRING](#) = 5 , [SI2_VOID](#) = 6 , [SI2_OID](#) = 7 ,
[SI2_ITER](#) = 8 , [SI2_EXPR](#) = 9 , [SI2_MAX_VALUETYPE](#) = 10 }
- enum [si2drSeverityT](#) { [SI2DR_SEVERITY_NOTE](#) = 0 , [SI2DR_SEVERITY_WARN](#) = 1 ,
[SI2DR_SEVERITY_ERR](#) = 2 }
- enum [si2drObjectTypeT](#) {
[SI2DR_UNDEFINED_OBJECTTYPE](#) = 0 , [SI2DR_GROUP](#) = 1 , [SI2DR_ATTR](#) = 2 ,
[SI2DR_DEFINE](#) = 3 ,
[SI2DR_COMPLEXVAL](#) = 4 , [SI2DR_MAX_OBJECTTYPE](#) = 5 }
- enum [si2drAttrTypeT](#) { [SI2DR_SIMPLE](#) , [SI2DR_COMPLEX](#) }
- enum [si2drValueTypeT](#) {
[SI2DR_UNDEFINED_VALUETYPE](#) = [SI2_UNDEFINED_VALUETYPE](#) , [SI2DR_INT32](#) = [SI2_](#)
[_LONG](#) , [SI2DR_STRING](#) = [SI2_STRING](#) , [SI2DR_FLOAT64](#) = [SI2_DOUBLE](#) ,
[SI2DR_BOOLEAN](#) = [SI2_BOOLEAN](#) , [SI2DR_EXPR](#) = [SI2_EXPR](#) , [SI2DR_MAX_VALUETYPE](#)
= [SI2_MAX_VALUETYPE](#) }

- enum `si2drExprTypeT` {
`SI2DR_EXPR_VAL` = 0 , `SI2DR_EXPR_OP_ADD` = 1 , `SI2DR_EXPR_OP_SUB` = 2 ,
`SI2DR_EXPR_OP_MUL` = 3 ,
`SI2DR_EXPR_OP_DIV` = 4 , `SI2DR_EXPR_OP_PAREN` = 5 , `SI2DR_EXPR_OP_LOG2` = 6
, `SI2DR_EXPR_OP_LOG10` = 7 ,
`SI2DR_EXPR_OP_EXP` = 8 }
- enum `si2drErrorT` {
`SI2DR_NO_ERROR` = 0 , `SI2DR_INTERNAL_SYSTEM_ERROR` = 1 , `SI2DR_INVALID_VALUE`
= 4 , `SI2DR_INVALID_NAME` = 5 ,
`SI2DR_INVALID_OBJECTTYPE` = 6 , `SI2DR_INVALID_ATTRTYPE` = 10 , `SI2DR_UNUSABLE_OID`
= 7 , `SI2DR_OBJECT_ALREADY_EXISTS` = 8 ,
`SI2DR_OBJECT_NOT_FOUND` = 9 , `SI2DR_SYNTAX_ERROR` = 2 , `SI2DR_TRACE_FILES_CANNOT_BE_C`
= 3 , `SI2DR_PIINIT_NOT_CALLED` = 11 ,
`SI2DR_SEMANTIC_ERROR` = 12 , `SI2DR_REFERENCE_ERROR` = 13 , `SI2DR_MAX_ERROR`
= 14 }

Functions

- `si2drVoidT si2drDefaultMessageHandler` (`si2drSeverityT` sev, `si2drErrorT` errToPrint, `si2drStringT` auxText, `si2drErrorT` *err)
- `si2drMessageHandlerT si2drPIGetMessageHandler` (`si2drErrorT` *err)
- `si2drVoidT si2drPISetMessageHandler` (`si2drMessageHandlerT` MsgHandFuncPtr, `si2drErrorT` *err)
- `si2drGroupIdT si2drPICreateGroup` `SI2_ARGS` ((`si2drStringT` name, `si2drStringT` group_type, `si2drErrorT` *err))
- `si2drAttrIdT si2drGroupCreateAttr` `SI2_ARGS` ((`si2drGroupIdT` group, `si2drStringT` name, `si2drAttrTypeT` type, `si2drErrorT` *err))
- `si2drAttrTypeT si2drAttrGetAttrType` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drErrorT` *err))
- `si2drVoidT si2drComplexAttrAddInt32Value` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drInt32T` integr, `si2drErrorT` *err))
- `si2drVoidT si2drComplexAttrAddStringValue` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drStringT` string, `si2drErrorT` *err))
- `si2drVoidT si2drComplexAttrAddBooleanValue` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drBooleanT` boolval, `si2drErrorT` *err))
- `si2drVoidT si2drComplexAttrAddFloat64Value` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drFloat64T` float64, `si2drErrorT` *err))
- `si2drVoidT si2drComplexAttrAddExprValue` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drExprT` *expr, `si2drErrorT` *err))
- `si2drVoidT si2drIterNextComplexValue` `SI2_ARGS` ((`si2drValuesIdT` iter, `si2drValueTypeT` *type, `si2drInt32T` *integr, `si2drFloat64T` *float64, `si2drStringT` *string, `si2drBooleanT` *boolval, `si2drExprT` **expr, `si2drErrorT` *err))
- `si2drAttrComplexValIdT si2drIterNextComplex` `SI2_ARGS` ((`si2drValuesIdT` iter, `si2drErrorT` *err))

- `si2drVoidT` `si2drSimpleAttrSetBooleanValue SI2_ARGS ((si2drAttrIdT attr, si2drBooleanT integr, si2drErrorT *err))`
- `si2drVoidT` `si2drExprDestroy SI2_ARGS ((si2drExprT *expr, si2drErrorT *err))`
- `si2drExprT` `*si2drCreateExpr SI2_ARGS ((si2drExprTypeT type, si2drErrorT *err))`
- `si2drExprT` `*si2drCreateBooleanValExpr SI2_ARGS ((si2drBooleanT b, si2drErrorT *err))`
- `si2drExprT` `*si2drCreateDoubleValExpr SI2_ARGS ((si2drFloat64T d, si2drErrorT *err))`
- `si2drExprT` `*si2drCreateStringValExpr SI2_ARGS ((si2drStringT s, si2drErrorT *err))`
- `si2drExprT` `*si2drCreateIntValExpr SI2_ARGS ((si2drInt32T i, si2drErrorT *err))`
- `si2drExprT` `*si2drCreateBinaryOpExpr SI2_ARGS ((si2drExprT *left, si2drExprTypeT optype, si2drExprT *right, si2drErrorT *err))`
- `si2drExprT` `*si2drCreateUnaryOpExpr SI2_ARGS ((si2drExprTypeT optype, si2drExprT *expr, si2drErrorT *err))`
- `si2drDefinIdT` `si2drGroupCreateDefine SI2_ARGS ((si2drGroupIdT group, si2drStringT name, si2drStringT allowed_group_name, si2drValueTypeT valtype, si2drErrorT *err))`
- `si2drVoidT` `si2drDefineGetInfo SI2_ARGS ((si2drDefinIdT def, si2drStringT *name, si2drStringT *allowed_group_name, si2drValueTypeT *valtype, si2drErrorT *err))`
- `si2drStringT` `si2drDefineGetName SI2_ARGS ((si2drDefinIdT def, si2drErrorT *err))`
- `si2drGroupIdT` `si2drGroupCreateGroup SI2_ARGS ((si2drGroupIdT group, si2drStringT name, si2drStringT group_type, si2drErrorT *err))`
- `si2drStringT` `si2drGroupGetGroupType SI2_ARGS ((si2drGroupIdT group, si2drErrorT *err))`
- `si2drVoidT` `si2drGroupSetComment SI2_ARGS ((si2drGroupIdT group, si2drStringT comment, si2drErrorT *err))`
- `si2drVoidT` `si2drAttrSetComment SI2_ARGS ((si2drAttrIdT attr, si2drStringT comment, si2drErrorT *err))`
- `si2drVoidT` `si2drDefineSetComment SI2_ARGS ((si2drDefinIdT def, si2drStringT comment, si2drErrorT *err))`
- `si2drVoidT` `si2drGroupAddName SI2_ARGS ((si2drGroupIdT group, si2drStringT name, si2drErrorT *err))`
- `si2drGroupIdT` `si2drPIFindGroupByName SI2_ARGS ((si2drStringT name, si2drStringT type, si2drErrorT *err))`
- `si2drGroupIdT` `si2drGroupFindGroupByName SI2_ARGS ((si2drGroupIdT group, si2drStringT name, si2drStringT type, si2drErrorT *err))`
- `si2drDefinIdT` `si2drPIFindDefineByName SI2_ARGS ((si2drStringT name, si2drErrorT *err))`
- `si2drGroupsIdT` `si2drPIGetGroups SI2_ARGS ((si2drErrorT *err))`
- `si2drVoidT` `si2drObjectDelete SI2_ARGS ((si2drObjectIdT object, si2drErrorT *err))`
- `si2drStringT` `si2drPIGetErrorText SI2_ARGS ((si2drErrorT errorCode, si2drErrorT *err))`
- `si2drBooleanT` `si2drObjectIsSame SI2_ARGS ((si2drObjectIdT object1, si2drObjectIdT object2, si2drErrorT *err))`
- `si2drVoidT` `si2drObjectSetFileName SI2_ARGS ((si2drObjectIdT object, si2drStringT filename, si2drErrorT *err))`
- `si2drVoidT` `si2drObjectSetLineNo SI2_ARGS ((si2drObjectIdT object, si2drInt32T lineno, si2drErrorT *err))`

- `si2drVoidT` `si2drReadLibertyFile` `SI2_ARGS` ((char *filename, `si2drErrorT` *err))
- `si2drVoidT` `si2drWriteLibertyFile` `SI2_ARGS` ((char *filename, `si2drGroupIdT` group, char *cell-name, `si2drErrorT` *err))
- `si2drVoidT` `si2drPISetTraceMode` `SI2_ARGS` ((`si2drStringT` fname, `si2drErrorT` *err))
- `si2drVoidT` `si2drGroupMoveAfter` `SI2_ARGS` ((`si2drGroupIdT` groupToMove, `si2drGroupIdT` targetGroup, `si2drErrorT` *err))
- `LONG_DOUBLE` `liberty_get_element` (struct `liberty_value_data` *vd,...)
- void `liberty_destroy_value_data` (struct `liberty_value_data` *vd)
- struct `liberty_value_data` * `liberty_get_values_data` (`si2drGroupIdT` table_group)
- void `print_version` ()
- void `usage` ()
- int `main_liberty_parser` (int argc, char *argv[])
- void `get_function` (char *filename)

8.17.1 Macro Definition Documentation

8.17.1.1 LONG_DOUBLE

```
#define LONG_DOUBLE long double
```

Definition at line 609 of file `si2dr_liberty.h`.

8.17.1.2 SI2_ARGS

```
#define SI2_ARGS(  
    args ) ()
```

Definition at line 34 of file `si2dr_liberty.h`.

8.17.1.3 SI2_LONG_MAX

```
#define SI2_LONG_MAX 2147483647
```

Definition at line 74 of file `si2dr_liberty.h`.

8.17.1.4 SI2_LONG_MIN

```
#define SI2_LONG_MIN -2147483647
```

Definition at line 75 of file [si2dr_liberty.h](#).

8.17.1.5 SI2_ULONG_MAX

```
#define SI2_ULONG_MAX 4294967295
```

Definition at line 76 of file [si2dr_liberty.h](#).

8.17.1.6 SI2DR_FALSE

```
#define SI2DR_FALSE SI2_FALSE
```

Definition at line 96 of file [si2dr_liberty.h](#).

8.17.1.7 SI2DR_MAX_FLOAT32

```
#define SI2DR_MAX_FLOAT32 1E+37
```

Definition at line 87 of file [si2dr_liberty.h](#).

8.17.1.8 SI2DR_MAX_FLOAT64

```
#define SI2DR_MAX_FLOAT64 1.797693e+308
```

Definition at line 89 of file [si2dr_liberty.h](#).

8.17.1.9 SI2DR_MAX_INT32

```
#define SI2DR_MAX_INT32 2147483647
```

Definition at line 91 of file [si2dr_liberty.h](#).

8.17.1.10 SI2DR_MAX_STRING_LEN

```
#define SI2DR_MAX_STRING_LEN 1048576 /*1024*1024*/
```

Definition at line 93 of file [si2dr_liberty.h](#).

8.17.1.11 SI2DR_MIN_FLOAT32

```
#define SI2DR_MIN_FLOAT32 -1E+37
```

Definition at line 86 of file [si2dr_liberty.h](#).

8.17.1.12 SI2DR_MIN_FLOAT64

```
#define SI2DR_MIN_FLOAT64 -1.797693e+308
```

Definition at line 88 of file [si2dr_liberty.h](#).

8.17.1.13 SI2DR_MIN_INT32

```
#define SI2DR_MIN_INT32 -2147483647
```

Definition at line 90 of file [si2dr_liberty.h](#).

8.17.1.14 SI2DR_TRUE

```
#define SI2DR_TRUE SI2_TRUE
```

Definition at line 95 of file [si2dr_liberty.h](#).

8.17.2 Typedef Documentation

8.17.2.1 si2BooleanT

```
typedef enum si2BooleanEnum si2BooleanT
```

8.17.2.2 si2DoubleT

```
typedef double si2DoubleT
```

Definition at line 52 of file [si2dr_liberty.h](#).

8.17.2.3 si2drAttrComplexValIdT

```
typedef si2drObjectIdT si2drAttrComplexValIdT
```

Definition at line 111 of file [si2dr_liberty.h](#).

8.17.2.4 si2drAttrIdT

```
typedef si2drObjectIdT si2drAttrIdT
```

Definition at line 110 of file [si2dr_liberty.h](#).

8.17.2.5 si2drAttrsIdT

```
typedef si2drIterIdT si2drAttrsIdT
```

Definition at line 116 of file [si2dr_liberty.h](#).

8.17.2.6 si2drAttrTypeT

```
typedef enum si2drAttrTypeT si2drAttrTypeT
```

8.17.2.7 si2drBooleanT

```
typedef si2BooleanT si2drBooleanT
```

Definition at line 98 of file [si2dr_liberty.h](#).

8.17.2.8 si2drDefineIdT

```
typedef si2drObjectIdT si2drDefineIdT
```

Definition at line 112 of file [si2dr_liberty.h](#).

8.17.2.9 si2drDefinesIdT

```
typedef si2drIterIdT si2drDefinesIdT
```

Definition at line 117 of file [si2dr_liberty.h](#).

8.17.2.10 si2drErrorT

```
typedef enum si2drErrorT si2drErrorT
```

8.17.2.11 si2drExprT

```
typedef struct si2drExprT si2drExprT
```

8.17.2.12 si2drExprTypeT

```
typedef enum si2drExprTypeT si2drExprTypeT
```

8.17.2.13 si2drFloat32T

```
typedef si2FloatT si2drFloat32T
```

Definition at line 101 of file [si2dr_liberty.h](#).

8.17.2.14 si2drFloat64T

```
typedef si2DoubleT si2drFloat64T
```

Definition at line 102 of file [si2dr_liberty.h](#).

8.17.2.15 si2drGroupIdT

```
typedef si2drObjectIdT si2drGroupIdT
```

Definition at line 109 of file [si2dr_liberty.h](#).

8.17.2.16 si2drGroupsIdT

```
typedef si2drIterIdT si2drGroupsIdT
```

Definition at line 115 of file [si2dr_liberty.h](#).

8.17.2.17 si2drInt32T

```
typedef si2LongT si2drInt32T
```

Definition at line 100 of file [si2dr_liberty.h](#).

8.17.2.18 si2drIterIdT

```
typedef si2IteratorIdT si2drIterIdT
```

Definition at line 106 of file [si2dr_liberty.h](#).

8.17.2.19 si2drMessageHandlerT

```
typedef si2drVoidT(* si2drMessageHandlerT) (si2drSeverityT sev, si2drErrorT errToPrint, si2drStringT auxText,  
si2drErrorT *err)
```

Definition at line 215 of file [si2dr_liberty.h](#).

8.17.2.20 si2drNamesIdT

```
typedef si2drIterIdT si2drNamesIdT
```

Definition at line 118 of file [si2dr_liberty.h](#).

8.17.2.21 si2drObjectIdT

```
typedef si2ObjectIdT si2drObjectIdT
```

Definition at line 107 of file [si2dr_liberty.h](#).

8.17.2.22 si2drObjectsIdT

```
typedef si2drIterIdT si2drObjectsIdT
```

Definition at line 114 of file [si2dr_liberty.h](#).

8.17.2.23 si2drObjectTypeT

```
typedef enum si2drObjectTypeT si2drObjectTypeT
```

8.17.2.24 si2drSeverityT

```
typedef enum si2drSeverityT si2drSeverityT
```

8.17.2.25 si2drStringT

```
typedef si2StringT si2drStringT
```

Definition at line 103 of file [si2dr_liberty.h](#).

8.17.2.26 si2drValuesIdT

```
typedef si2drIterIdT si2drValuesIdT
```

Definition at line 119 of file [si2dr_liberty.h](#).

8.17.2.27 si2drValueTypeT

```
typedef enum si2drValueTypeT si2drValueTypeT
```

8.17.2.28 si2drVoidT

```
typedef si2VoidT si2drVoidT
```

Definition at line 104 of file [si2dr_liberty.h](#).

8.17.2.29 si2FloatT

```
typedef float si2FloatT
```

Definition at line 51 of file [si2dr_liberty.h](#).

8.17.2.30 si2IteratorIdT

```
typedef void* si2IteratorIdT
```

Definition at line 83 of file [si2dr_liberty.h](#).

8.17.2.31 si2LongT

```
typedef long int si2LongT
```

Definition at line 50 of file [si2dr_liberty.h](#).

8.17.2.32 si2ObjectIdT

```
typedef struct si2OID si2ObjectIdT
```

8.17.2.33 si2StringT

```
typedef char* si2StringT
```

Definition at line 53 of file [si2dr_liberty.h](#).

8.17.2.34 si2TypeT

```
typedef enum si2TypeEnum si2TypeT
```

8.17.2.35 si2VoidT

```
typedef void si2VoidT
```

Definition at line [54](#) of file [si2dr_liberty.h](#).

8.17.3 Enumeration Type Documentation

8.17.3.1 si2BooleanEnum

```
enum si2BooleanEnum
```

Enumerator

SI2_FALSE	
SI2_TRUE	

Definition at line [49](#) of file [si2dr_liberty.h](#).

8.17.3.2 si2drAttrTypeT

```
enum si2drAttrTypeT
```

Enumerator

SI2DR_SIMPLE	
SI2DR_COMPLEX	

Definition at line [139](#) of file [si2dr_liberty.h](#).

8.17.3.3 si2drErrorT

enum [si2drErrorT](#)

Enumerator

SI2DR_NO_ERROR	
SI2DR_INTERNAL_SYSTEM_ERROR	
SI2DR_INVALID_VALUE	
SI2DR_INVALID_NAME	
SI2DR_INVALID_OBJECTTYPE	
SI2DR_INVALID_ATTRTYPE	
SI2DR_UNUSABLE_OID	
SI2DR_OBJECT_ALREADY_EXISTS	
SI2DR_OBJECT_NOT_FOUND	
SI2DR_SYNTAX_ERROR	
SI2DR_TRACE_FILES_CANNOT_BE_OPENED	
SI2DR_PIINIT_NOT_CALLED	
SI2DR_SEMANTIC_ERROR	
SI2DR_REFERENCE_ERROR	
SI2DR_MAX_ERROR	

Definition at line 190 of file [si2dr_liberty.h](#).

8.17.3.4 si2drExprTypeT

enum [si2drExprTypeT](#)

Enumerator

SI2DR_EXPR_VAL	
SI2DR_EXPR_OP_ADD	
SI2DR_EXPR_OP_SUB	
SI2DR_EXPR_OP_MUL	
SI2DR_EXPR_OP_DIV	
SI2DR_EXPR_OP_PAREN	
SI2DR_EXPR_OP_LOG2	
SI2DR_EXPR_OP_LOG10	
SI2DR_EXPR_OP_EXP	

Definition at line 157 of file [si2dr_liberty.h](#).

8.17.3.5 si2drObjectTypeT

enum [si2drObjectTypeT](#)

Enumerator

SI2DR_UNDEFINED_OBJECTTYPE	
SI2DR_GROUP	
SI2DR_ATTR	
SI2DR_DEFINE	
SI2DR_COMPLEXVAL	
SI2DR_MAX_OBJECTTYPE	

Definition at line 129 of file [si2dr_liberty.h](#).

8.17.3.6 si2drSeverityT

enum [si2drSeverityT](#)

Enumerator

SI2DR_SEVERITY_NOTE	
SI2DR_SEVERITY_WARN	
SI2DR_SEVERITY_ERR	

Definition at line 121 of file [si2dr_liberty.h](#).

8.17.3.7 si2drValueTypeT

enum [si2drValueTypeT](#)

Enumerator

SI2DR_UNDEFINED_VALUETYPE	
---------------------------	--

Enumerator

SI2DR_INT32	
SI2DR_STRING	
SI2DR_FLOAT64	
SI2DR_BOOLEAN	
SI2DR_EXPR	
SI2DR_MAX_VALUETYPE	

Definition at line 146 of file [si2dr_liberty.h](#).

8.17.3.8 si2TypeEnum

enum [si2TypeEnum](#)

Enumerator

SI2_UNDEFINED_VALUETYPE	
SI2_BOOLEAN	
SI2_LONG	
SI2_FLOAT	
SI2_DOUBLE	
SI2_STRING	
SI2_VOID	
SI2_OID	
SI2_ITER	
SI2_EXPR	
SI2_MAX_VALUETYPE	

Definition at line 56 of file [si2dr_liberty.h](#).

8.17.4 Function Documentation

8.17.4.1 get_function()

```
void get_function (
    char * filename )
```

8.17.4.2 liberty_destroy_value_data()

```
void liberty_destroy_value_data (
    struct liberty_value_data * vd )
```

8.17.4.3 liberty_get_element()

```
LONG_DOUBLE liberty_get_element (
    struct liberty_value_data * vd,
    ... )
```

8.17.4.4 liberty_get_values_data()

```
struct liberty_value_data * liberty_get_values_data (
    si2drGroupIdT table_group )
```

8.17.4.5 main_liberty_parser()

```
int main_liberty_parser (
    int argc,
    char * argv[] )
```

8.17.4.6 print_version()

```
void print_version ( )
```

8.17.4.7 SI2_ARGS() [1/41]

```
si2drVoidT si2drReadLibertyFile SI2_ARGS (
    (char *filename, si2drErrorT *err) )
```

8.17.4.8 SI2_ARGS() [2/41]

```
si2drVoidT si2drWriteLibertyFile SI2_ARGS (  
    (char *filename, si2drGroupIdT group, char *cellname, si2drErrorT *err) )
```

8.17.4.9 SI2_ARGS() [3/41]

```
si2drVoidT si2drComplexAttrAddBooleanValue SI2_ARGS (  
    (si2drAttrIdT attr, si2drBooleanT boolval, si2drErrorT *err) )
```

8.17.4.10 SI2_ARGS() [4/41]

```
si2drVoidT si2drSimpleAttrSetBooleanValue SI2_ARGS (  
    (si2drAttrIdT attr, si2drBooleanT intgr, si2drErrorT *err) )
```

8.17.4.11 SI2_ARGS() [5/41]

```
si2drStringT si2drAttrGetComment SI2_ARGS (  
    (si2drAttrIdT attr, si2drErrorT *err) )
```

8.17.4.12 SI2_ARGS() [6/41]

```
si2drVoidT si2drSimpleAttrSetExprValue SI2_ARGS (  
    (si2drAttrIdT attr, si2drExprT *expr, si2drErrorT *err) )
```

8.17.4.13 SI2_ARGS() [7/41]

```
si2drVoidT si2drSimpleAttrSetFloat64Value SI2_ARGS (  
    (si2drAttrIdT attr, si2drFloat64T float64, si2drErrorT *err) )
```

8.17.4.14 SI2_ARGS() [8/41]

```
si2drVoidT si2drSimpleAttrSetInt32Value SI2_ARGS (  
    (si2drAttrIdT attr, si2drInt32T intgr, si2drErrorT *err) )
```

8.17.4.15 SI2_ARGS() [9/41]

```
si2drVoidT si2drAttrSetComment SI2_ARGS (  
    (si2drAttrIdT attr, si2drStringT comment, si2drErrorT *err) )
```

8.17.4.16 SI2_ARGS() [10/41]

```
si2drVoidT si2drSimpleAttrSetStringValue SI2_ARGS (  
    (si2drAttrIdT attr, si2drStringT string, si2drErrorT *err) )
```

8.17.4.17 SI2_ARGS() [11/41]

```
si2drExprT *si2drCreateBooleanValExpr SI2_ARGS (  
    (si2drBooleanT b, si2drErrorT *err) )
```

8.17.4.18 SI2_ARGS() [12/41]

```
si2drStringT si2drDefineGetComment SI2_ARGS (  
    (si2drDefineIdT def, si2drErrorT *err) )
```

8.17.4.19 SI2_ARGS() [13/41]

```
si2drVoidT si2drDefineGetInfo SI2_ARGS (  
    (si2drDefineIdT def, si2drStringT *name, si2drStringT *allowed_group_name, si2drValueType *valtype,  
    si2drErrorT *err) )
```

8.17.4.20 SI2_ARGS() [14/41]

```
si2drVoidT si2drDefineSetComment SI2_ARGS (  
    (si2drDefineIdT def, si2drStringT comment, si2drErrorT *err) )
```

8.17.4.21 SI2_ARGS() [15/41]

```
si2drBooleanT si2drPIGetNocheckMode SI2_ARGS (  
    (si2drErrorT *err) )
```

8.17.4.22 SI2_ARGS() [16/41]

```
si2drStringT si2drPIGetErrorText SI2_ARGS (  
    (si2drErrorT errorCode, si2drErrorT *err) )
```

8.17.4.23 SI2_ARGS() [17/41]

```
si2drExprT *si2drOpExprGetRightExpr SI2_ARGS (  
    (si2drExprT *expr, si2drErrorT *err) )
```

8.17.4.24 SI2_ARGS() [18/41]

```
si2drExprT *si2drCreateBinaryOpExpr SI2_ARGS (  
    (si2drExprT *left, si2drExprTypeT optype, si2drExprT *right, si2drErrorT *err) )
```

8.17.4.25 SI2_ARGS() [19/41]

```
si2drExprT *si2drCreateUnaryOpExpr SI2_ARGS (  
    (si2drExprTypeT optype, si2drExprT *expr, si2drErrorT *err) )
```


8.17.4.26 SI2_ARGS() [20/41]

```
si2drExprT *si2drCreateExpr SI2_ARGS (  
    (si2drExprTypeT type, si2drErrorT *err) )
```

8.17.4.27 SI2_ARGS() [21/41]

```
si2drExprT *si2drCreateDoubleValExpr SI2_ARGS (  
    (si2drFloat64T d, si2drErrorT *err) )
```

8.17.4.28 SI2_ARGS() [22/41]

```
si2drVoidT si2drCheckLibertyLibrary SI2_ARGS (  
    (si2drGroupIdT group, si2drErrorT *err) )
```

8.17.4.29 SI2_ARGS() [23/41]

```
si2drVoidT si2drGroupSetComment SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT comment, si2drErrorT *err) )
```

8.17.4.30 SI2_ARGS() [24/41]

```
si2drAttrIdT si2drGroupCreateAttr SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drAttrTypeT type, si2drErrorT *err) )
```

8.17.4.31 SI2_ARGS() [25/41]

```
si2drDefineIdT si2drGroupFindDefineByName SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drErrorT *err) )
```

8.17.4.32 SI2_ARGS() [26/41]

```
si2drDefineIdT si2drGroupCreateDefine SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drStringT allowed_group_name, si2drValueTypeT valtype,  
    si2drErrorT *err) )
```

8.17.4.33 SI2_ARGS() [27/41]

```
si2drGroupIdT si2drGroupCreateGroup SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drStringT group_type, si2drErrorT *err) )
```

8.17.4.34 SI2_ARGS() [28/41]

```
si2drGroupIdT si2drGroupFindGroupByName SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drStringT type, si2drErrorT *err) )
```

8.17.4.35 SI2_ARGS() [29/41]

```
si2drVoidT si2drGroupMoveBefore SI2_ARGS (  
    (si2drGroupIdT groupToMove, si2drGroupIdT targetGroup, si2drErrorT *err) )
```

8.17.4.36 SI2_ARGS() [30/41]

```
si2drExprT *si2drCreateIntValExpr SI2_ARGS (  
    (si2drInt32T i, si2drErrorT *err) )
```

8.17.4.37 SI2_ARGS() [31/41]

```
si2drStringT si2drObjectGetFileName SI2_ARGS (  
    (si2drObjectIdT object, si2drErrorT *err) )
```

8.17.4.38 SI2_ARGS() [32/41]

```
si2drVoidT si2drObjectSetLineNo SI2_ARGS (  
    (si2drObjectIdT object, si2drInt32T lineno, si2drErrorT *err) )
```

8.17.4.39 SI2_ARGS() [33/41]

```
si2drVoidT si2drObjectSetFileName SI2_ARGS (  
    (si2drObjectIdT object, si2drStringT filename, si2drErrorT *err) )
```

8.17.4.40 SI2_ARGS() [34/41]

```
si2drBooleanT si2drObjectIsSame SI2_ARGS (  
    (si2drObjectIdT object1, si2drObjectIdT object2, si2drErrorT *err) )
```

8.17.4.41 SI2_ARGS() [35/41]

```
si2drVoidT si2drPISetTraceMode SI2_ARGS (  
    (si2drStringT fname, si2drErrorT *err) )
```

8.17.4.42 SI2_ARGS() [36/41]

```
si2drDefineIdT si2drPIFindDefineByName SI2_ARGS (  
    (si2drStringT name, si2drErrorT *err) )
```

8.17.4.43 SI2_ARGS() [37/41]

```
si2drGroupIdT si2drPICreateGroup SI2_ARGS (  
    (si2drStringT name, si2drStringT group_type, si2drErrorT *err) )
```

8.17.4.44 SI2_ARGS() [38/41]

```
si2drGroupIdT si2drPIFindGroupByName SI2_ARGS (
    (si2drStringT name, si2drStringT type, si2drErrorT *err) )
```

8.17.4.45 SI2_ARGS() [39/41]

```
si2drExprT *si2drCreateStringValExpr SI2_ARGS (
    (si2drStringT s, si2drErrorT *err) )
```

8.17.4.46 SI2_ARGS() [40/41]

```
si2drVoidT si2drIterQuit SI2_ARGS (
    (si2drValuesIdT iter, si2drErrorT *err) )
```

8.17.4.47 SI2_ARGS() [41/41]

```
si2drVoidT si2drIterNextComplexValue SI2_ARGS (
    (si2drValuesIdT iter, si2drValueTypeT *type, si2drInt32T *intgr, si2drFloat64T *float64, si2drStringT
    *string, si2drBooleanT *boolval, si2drExprT **expr, si2drErrorT *err) )
```

8.17.4.48 si2drDefaultMessageHandler()

```
si2drVoidT si2drDefaultMessageHandler (
    si2drSeverityT sev,
    si2drErrorT errToPrint,
    si2drStringT auxText,
    si2drErrorT * err )
```

8.17.4.49 si2drPIGetMessageHandler()

```
si2drMessageHandlerT si2drPIGetMessageHandler (
    si2drErrorT * err )
```

8.17.4.50 si2drPISetMessageHandler()

```
si2drVoidT si2drPISetMessageHandler (
    si2drMessageHandlerT MsgHandFuncPtr,
    si2drErrorT * err )
```

8.17.4.51 usage()

```
void usage ( )
```

8.18 si2dr_liberty.h

[Go to the documentation of this file.](#)

```
00001 /*****
00002 Copyright (c) 1996-2005 Synopsys, Inc.    ALL RIGHTS RESERVED
00003
00004 The contents of this file are subject to the restrictions and limitations
00005 set forth in the SYNOPSYS Open Source License Version 1.0 (the "License");
00006 you may not use this file except in compliance with such restrictions
00007 and limitations. You may obtain instructions on how to receive a copy of
00008 the License at
00009
00010 http://www.synopsys.com/partners/tapin/tapinprogram.html.
00011
00012 Software distributed by Original Contributor under the License is
00013 distributed on an "AS IS" basis, WITHOUT WARRANTY OF ANY KIND, either
00014 expressed or implied. See the License for the specific language governing
00015 rights and limitations under the License.
00016
00017 *****/
00018
00019 /* define a PI for manipulating the liberty data */
00020
00021
00022 /* basic data types */
00023
00024
00025 #ifndef SI2DR_H
00026 #define SI2DR_H
00027
00028 #ifdef SI2_ARGS
00029 #undef SI2_ARGS
00030 #endif
```

```

00031 #if defined(__STDC__) || defined(HPPA) || defined(ibmpoweraix) || defined(_MSC_VER) || defined(__cplusplus)
00032 #define SI2_ARGS(args)  args
00033 #else
00034 #define SI2_ARGS(args)  ()
00035 #endif
00036
00037
00038 /* *****
00039 File contents : LIBERTY-DR type definitions.
00040 ***** */
00041 #ifdef __cplusplus
00042 extern "C" {
00043 #endif
00044
00045 /*
00046 * These primitive types are compatible with the ANSI C Standard (ANSI/
00047 * X3.159-1990 Programming Language C Standard.)
00048 */
00049 typedef enum si2BooleanEnum { SI2_FALSE = 0, SI2_TRUE = 1 } si2BooleanT;
00050 typedef long int    si2LongT;
00051 typedef float       si2FloatT;
00052 typedef double      si2DoubleT;
00053 typedef char*       si2StringT;
00054 typedef void        si2VoidT;
00055
00056 typedef enum si2TypeEnum {
00057     SI2_UNDEFINED_VALUETYPE = 0,
00058     SI2_BOOLEAN = 1,
00059     SI2_LONG = 2,
00060     SI2_FLOAT = 3,
00061     SI2_DOUBLE = 4,
00062     SI2_STRING = 5,
00063     SI2_VOID = 6,
00064     SI2_OID = 7,
00065     SI2_ITER = 8,
00066     SI2_EXPR = 9,
00067     SI2_MAX_VALUETYPE = 10
00068 } si2TypeT;
00069
00070 /*
00071 * Size limits
00072 */
00073
00074 #define SI2_LONG_MAX    2147483647
00075 #define SI2_LONG_MIN    -2147483647
00076 #define SI2_ULONG_MAX   4294967295
00077
00078
00079 typedef struct si2OID {void *v1, *v2;} si2ObjectIdT;
00080 /*
00081 * Iterators are blind handles the size of a single pointer.
00082 */
00083 typedef void *si2IteratorIdT;
00084
00085
00086 #define SI2DR_MIN_FLOAT32  -1E+37
00087 #define SI2DR_MAX_FLOAT32   1E+37
00088 #define SI2DR_MIN_FLOAT64  -1.797693e+308
00089 #define SI2DR_MAX_FLOAT64   1.797693e+308
00090 #define SI2DR_MIN_INT32    -2147483647
00091 #define SI2DR_MAX_INT32    2147483647
00092
00093 #define SI2DR_MAX_STRING_LEN  1048576 /*1024*1024*/
00094
00095 #define SI2DR_TRUE SI2_TRUE
00096 #define SI2DR_FALSE SI2_FALSE
00097
00098 typedef si2BooleanT si2drBooleanT;
00099

```

```

00100 typedef si2LongT    si2drInt32T;
00101 typedef si2FloatT    si2drFloat32T;
00102 typedef si2DoubleT    si2drFloat64T;
00103 typedef si2StringT    si2drStringT;
00104 typedef si2VoidT      si2drVoidT;
00105
00106 typedef si2IteratorIdT si2drIterIdT;
00107 typedef si2ObjectIdT   si2drObjectIdT;
00108
00109 typedef si2drObjectIdT si2drGroupIdT;
00110 typedef si2drObjectIdT si2drAttrIdT;
00111 typedef si2drObjectIdT si2drAttrComplexValIdT;
00112 typedef si2drObjectIdT si2drDefineIdT;
00113
00114 typedef si2drIterIdT si2drObjectsIdT;
00115 typedef si2drIterIdT si2drGroupsIdT;
00116 typedef si2drIterIdT si2drAttrsIdT;
00117 typedef si2drIterIdT si2drDefinesIdT;
00118 typedef si2drIterIdT si2drNamesIdT;
00119 typedef si2drIterIdT si2drValuesIdT;
00120
00121 typedef enum si2drSeverityT
00122 {
00123     SI2DR_SEVERITY_NOTE = 0,
00124     SI2DR_SEVERITY_WARN = 1,
00125     SI2DR_SEVERITY_ERR  = 2
00126 } si2drSeverityT;
00127
00128
00129 typedef enum si2drObjectTypeT
00130 {
00131     SI2DR_UNDEFINED_OBJECTTYPE = 0,
00132     SI2DR_GROUP                = 1,
00133     SI2DR_ATTR                 = 2,
00134     SI2DR_DEFINE               = 3,
00135     SI2DR_COMPLEXVAL           = 4,
00136     SI2DR_MAX_OBJECTTYPE       = 5
00137 } si2drObjectTypeT;
00138
00139 typedef enum si2drAttrTypeT
00140 {
00141     SI2DR_SIMPLE,
00142     SI2DR_COMPLEX
00143 } si2drAttrTypeT;
00144
00145
00146 typedef enum si2drValueTypeT
00147 {
00148     SI2DR_UNDEFINED_VALUETYPE = SI2_UNDEFINED_VALUETYPE,
00149     SI2DR_INT32                = SI2_LONG,
00150     SI2DR_STRING               = SI2_STRING,
00151     SI2DR_FLOAT64              = SI2_DOUBLE,
00152     SI2DR_BOOLEAN              = SI2_BOOLEAN,
00153     SI2DR_EXPR                 = SI2_EXPR,
00154     SI2DR_MAX_VALUETYPE        = SI2_MAX_VALUETYPE
00155 } si2drValueTypeT;
00156
00157 typedef enum si2drExprTypeT
00158 {
00159     SI2DR_EXPR_VAL = 0, /* unary ops just have left arg set: add, sub, logs, paren */
00160     SI2DR_EXPR_OP_ADD = 1,
00161     SI2DR_EXPR_OP_SUB = 2,
00162     SI2DR_EXPR_OP_MUL = 3,
00163     SI2DR_EXPR_OP_DIV = 4,
00164     SI2DR_EXPR_OP_PAREN = 5,
00165     SI2DR_EXPR_OP_LOG2 = 6, /* pretty much flight of fancy from here on? */
00166     SI2DR_EXPR_OP_LOG10 = 7,
00167     SI2DR_EXPR_OP_EXP = 8, /* exponent? */
00168

```

```

00169 } si2drExprTypeT;
00170
00171
00172 typedef struct si2drExprT
00173 {
00174     si2drExprTypeT type;
00175     union
00176     {
00177         si2drInt32T i;
00178         si2drFloat64T d;
00179         si2drStringT s; /* most likely an identifier */
00180         si2drBooleanT b;
00181     } u;
00182
00183     si2drValueTypeT valuetype; /* if the type is a fixed value */
00184
00185     struct si2drExprT *left; /* the exprs form a classic binary tree rep of an arithmetic expression */
00186     struct si2drExprT *right;
00187 } si2drExprT;
00188
00189
00190 typedef enum si2drErrorT
00191 {
00192     SI2DR_NO_ERROR = 0,
00193     SI2DR_INTERNAL_SYSTEM_ERROR = 1,
00194     SI2DR_INVALID_VALUE = 4,
00195     SI2DR_INVALID_NAME = 5,
00196     SI2DR_INVALID_OBJECTTYPE = 6,
00197     SI2DR_INVALID_ATTRTYPE = 10,
00198     SI2DR_UNUSABLE_OID = 7,
00199     SI2DR_OBJECT_ALREADY_EXISTS = 8,
00200     SI2DR_OBJECT_NOT_FOUND = 9,
00201     SI2DR_SYNTAX_ERROR = 2,
00202     SI2DR_TRACE_FILES_CANNOT_BE_OPENED = 3,
00203     SI2DR_PIINIT_NOT_CALLED = 11,
00204     SI2DR_SEMANTIC_ERROR = 12,
00205     SI2DR_REFERENCE_ERROR = 13,
00206     SI2DR_MAX_ERROR = 14
00207 } si2drErrorT;
00208
00209 si2drVoidT si2drDefaultMessageHandler(si2drSeverityT sev,
00210                                       si2drErrorT errToPrint,
00211                                       si2drStringT auxText,
00212                                       si2drErrorT *err);
00213
00214
00215 typedef si2drVoidT (*si2drMessageHandlerT)(si2drSeverityT sev,
00216                                             si2drErrorT errToPrint,
00217                                             si2drStringT auxText,
00218                                             si2drErrorT *err);
00219
00220
00221 si2drMessageHandlerT si2drPIMessageHandler( si2drErrorT *err);
00222
00223
00224 si2drVoidT si2drPISetMessageHandler( si2drMessageHandlerT MsgHandFuncPtr,
00225                                       si2drErrorT *err);
00226
00227
00228 si2drGroupIdT si2drPICreateGroup SI2_ARGS(( si2drStringT name,
00229                                              si2drStringT group_type,
00230                                              si2drErrorT *err));
00231
00232
00233 si2drAttrIdT si2drGroupCreateAttr SI2_ARGS(( si2drGroupIdT group,
00234                                              si2drStringT name,
00235                                              si2drAttrTypeT type,
00236                                              si2drErrorT *err));
00237

```



```

00238     si2drAttrTypeT si2drAttrGetAttrType    SI2_ARGS(( si2drAttrIdT attr,
00239                                                         si2drErrorT *err));
00240
00241     si2drStringT   si2drAttrGetName        SI2_ARGS(( si2drAttrIdT attr,
00242                                                         si2drErrorT *err));
00243
00244     si2drVoidT     si2drComplexAttrAddInt32Value    SI2_ARGS(( si2drAttrIdT attr,
00245                                                         si2drInt32T   intgr,
00246                                                         si2drErrorT   *err ));
00247
00248     si2drVoidT     si2drComplexAttrAddStringValue    SI2_ARGS(( si2drAttrIdT attr,
00249                                                         si2drStringT   string,
00250                                                         si2drErrorT   *err ));
00251
00252     si2drVoidT     si2drComplexAttrAddBooleanValue    SI2_ARGS(( si2drAttrIdT attr,
00253                                                         si2drBooleanT   boolval,
00254                                                         si2drErrorT   *err ));
00255
00256     si2drVoidT     si2drComplexAttrAddFloat64Value    SI2_ARGS(( si2drAttrIdT attr,
00257                                                         si2drFloat64T   float64,
00258                                                         si2drErrorT   *err ));
00259
00260     si2drVoidT     si2drComplexAttrAddExprValue    SI2_ARGS(( si2drAttrIdT attr,
00261                                                         si2drExprT      *expr,
00262                                                         si2drErrorT   *err ));
00263
00264     si2drValuesIdT si2drComplexAttrGetValues    SI2_ARGS(( si2drAttrIdT attr,
00265                                                         si2drErrorT   *err ));
00266
00267     si2drVoidT     si2drIterNextComplexValue    SI2_ARGS(( si2drValuesIdT iter,
00268                                                         si2drValueTypeT *type,
00269                                                         si2drInt32T     *intgr,
00270                                                         si2drFloat64T   *float64,
00271                                                         si2drStringT     *string,
00272                                                         si2drBooleanT   *boolval,
00273                                                         si2drExprT      **expr,
00274                                                         si2drErrorT   *err ));
00275
00276     si2drAttrComplexValIdT si2drIterNextComplex    SI2_ARGS(( si2drValuesIdT iter,
00277                                                         si2drErrorT   *err ));
00278
00279     si2drValueTypeT si2drComplexValGetValueType    SI2_ARGS(( si2drAttrComplexValIdT attr,
00280                                                         si2drErrorT   *err ));
00281
00282     si2drInt32T     si2drComplexValGetInt32Value    SI2_ARGS(( si2drAttrComplexValIdT attr,
00283                                                         si2drErrorT   *err ));
00284
00285     si2drFloat64T   si2drComplexValGetFloat64Value    SI2_ARGS(( si2drAttrComplexValIdT attr,
00286                                                         si2drErrorT   *err ));
00287
00288     si2drStringT     si2drComplexValGetStringValue    SI2_ARGS(( si2drAttrComplexValIdT attr,
00289                                                         si2drErrorT   *err ));
00290
00291     si2drBooleanT    si2drComplexValGetBooleanValue    SI2_ARGS(( si2drAttrComplexValIdT attr,
00292                                                         si2drErrorT   *err ));
00293
00294     si2drExprT      *si2drComplexValGetExprValue    SI2_ARGS(( si2drAttrComplexValIdT attr,
00295                                                         si2drErrorT   *err ));
00296
00297
00298
00299
00300
00301
00302     si2drValueTypeT si2drSimpleAttrGetValueType    SI2_ARGS(( si2drAttrIdT attr,
00303                                                         si2drErrorT   *err ));
00304
00305     si2drInt32T     si2drSimpleAttrGetInt32Value    SI2_ARGS(( si2drAttrIdT attr,
00306                                                         si2drErrorT   *err ));

```

```

00307
00308     si2drFloat64T    si2drSimpleAttrGetFloat64Value    SI2_ARGS(( si2drAttrIdT attr,
00309                                                     si2drErrorT    *err ));
00310
00311     si2drStringT     si2drSimpleAttrGetStringValue     SI2_ARGS(( si2drAttrIdT attr,
00312                                                     si2drErrorT    *err ));
00313
00314     si2drBooleanT    si2drSimpleAttrGetBooleanValue    SI2_ARGS(( si2drAttrIdT attr,
00315                                                     si2drErrorT    *err ));
00316
00317     si2drExprT       *si2drSimpleAttrGetExprValue      SI2_ARGS(( si2drAttrIdT attr,
00318                                                     si2drErrorT    *err ));
00319
00320
00321
00322     si2drVoidT       si2drSimpleAttrSetInt32Value      SI2_ARGS(( si2drAttrIdT attr,
00323                                                     si2drInt32T    intgr,
00324                                                     si2drErrorT    *err ));
00325
00326     si2drVoidT       si2drSimpleAttrSetBooleanValue    SI2_ARGS(( si2drAttrIdT attr,
00327                                                     si2drBooleanT  intgr,
00328                                                     si2drErrorT    *err ));
00329
00330     si2drVoidT       si2drSimpleAttrSetFloat64Value    SI2_ARGS(( si2drAttrIdT attr,
00331                                                     si2drFloat64T  float64,
00332                                                     si2drErrorT    *err ));
00333
00334     si2drVoidT       si2drSimpleAttrSetStringValue     SI2_ARGS(( si2drAttrIdT attr,
00335                                                     si2drStringT   string,
00336                                                     si2drErrorT    *err ));
00337
00338     si2drVoidT       si2drSimpleAttrSetExprValue      SI2_ARGS(( si2drAttrIdT attr,
00339                                                     si2drExprT     *expr,
00340                                                     si2drErrorT    *err ));
00341
00342     si2drBooleanT    si2drSimpleAttrGetIsVar           SI2_ARGS(( si2drAttrIdT attr,
00343                                                     si2drErrorT    *err ));
00344
00345     si2drVoidT       si2drSimpleAttrSetIsVar           SI2_ARGS(( si2drAttrIdT attr,
00346                                                     si2drErrorT    *err ));
00347
00348 /* helper functions for expr creation, building, and destruction */
00349
00350     si2drVoidT       si2drExprDestroy                  SI2_ARGS(( si2drExprT  *expr, /* recursively free the structures */
00351                                                     si2drErrorT    *err));
00352
00353     si2drExprT       *si2drCreateExpr                  SI2_ARGS(( si2drExprTypeT type, /* malloc an Expr and return it */
00354                                                     si2drErrorT    *err));
00355
00356     si2drExprT       *si2drCreateBooleanValExpr        SI2_ARGS(( si2drBooleanT b,
00357                                                     si2drErrorT    *err ));
00358
00359     si2drExprT       *si2drCreateDoubleValExpr         SI2_ARGS(( si2drFloat64T d,
00360                                                     si2drErrorT    *err ));
00361
00362     si2drExprT       *si2drCreateStringValExpr         SI2_ARGS(( si2drStringT s,
00363                                                     si2drErrorT    *err ));
00364
00365     si2drExprT       *si2drCreateIntValExpr            SI2_ARGS(( si2drInt32T i,
00366                                                     si2drErrorT    *err ));
00367
00368     si2drExprT       *si2drCreateBinaryOpExpr          SI2_ARGS(( si2drExprT *left,
00369                                                     si2drExprTypeT optype,
00370                                                     si2drExprT *right,
00371                                                     si2drErrorT    *err ));
00372
00373     si2drExprT       *si2drCreateUnaryOpExpr           SI2_ARGS(( si2drExprTypeT optype,
00374                                                     si2drExprT *expr,
00375                                                     si2drErrorT    *err ));

```

```

00376
00377 si2drStringT    si2drExprToString          SI2_ARGS(( si2drExprT  *expr,
00378                                                         si2drErrorT  *err ));
00379
00380 si2drExprTypeT  si2drExprGetType           SI2_ARGS(( si2drExprT  *expr,
00381                                                         si2drErrorT  *err ));
00382
00383 si2drValueTypeT si2drValExprGetValueType   SI2_ARGS(( si2drExprT  *expr,
00384                                                         si2drErrorT  *err ));
00385
00386 si2drInt32T     si2drIntValExprGetInt      SI2_ARGS(( si2drExprT  *expr,
00387                                                         si2drErrorT  *err ));
00388
00389 si2drFloat64T   si2drDoubleValExprGetDouble SI2_ARGS(( si2drExprT  *expr,
00390                                                         si2drErrorT  *err ));
00391
00392 si2drBooleanT   si2drBooleanValExprGetBoolean SI2_ARGS(( si2drExprT  *expr,
00393                                                         si2drErrorT  *err ));
00394
00395 si2drStringT     si2drStringValExprGetString SI2_ARGS(( si2drExprT  *expr,
00396                                                         si2drErrorT  *err ));
00397
00398 si2drExprT      *si2drOpExprGetLeftExpr    SI2_ARGS(( si2drExprT  *expr, /* will apply to Unary & binary Ops */
00399                                                         si2drErrorT  *err ));
00400
00401 si2drExprT      *si2drOpExprGetRightExpr    SI2_ARGS(( si2drExprT  *expr,
00402                                                         si2drErrorT  *err ));
00403
00404 /* Define related funcs */
00405
00406 si2drDefineIdT  si2drGroupCreateDefine      SI2_ARGS(( si2drGroupIdT group,
00407                                                         si2drStringT  name,
00408                                                         si2drStringT  allowed_group_name,
00409                                                         si2drValueTypeT valtype,
00410                                                         si2drErrorT  *err));
00411
00412 si2drVoidT      si2drDefineGetInfo          SI2_ARGS(( si2drDefineIdT def,
00413                                                         si2drStringT  *name,
00414                                                         si2drStringT  *allowed_group_name,
00415                                                         si2drValueTypeT *valtype,
00416                                                         si2drErrorT  *err));
00417
00418 si2drStringT     si2drDefineGetName          SI2_ARGS(( si2drDefineIdT def,
00419                                                         si2drErrorT  *err));
00420
00421 si2drStringT     si2drDefineGetAllowedGroupName SI2_ARGS(( si2drDefineIdT def,
00422                                                         si2drErrorT  *err));
00423
00424 si2drValueTypeT  si2drDefineGetValueType     SI2_ARGS(( si2drDefineIdT def,
00425                                                         si2drErrorT  *err));
00426
00427 si2drGroupIdT    si2drGroupCreateGroup      SI2_ARGS(( si2drGroupIdT group,
00428                                                         si2drStringT  name,
00429                                                         si2drStringT  group_type,
00430                                                         si2drErrorT  *err));
00431
00432 si2drStringT     si2drGroupGetGroupType      SI2_ARGS(( si2drGroupIdT group,
00433                                                         si2drErrorT  *err));
00434
00435 si2drStringT     si2drGroupGetComment        SI2_ARGS(( si2drGroupIdT group,
00436                                                         si2drErrorT  *err));
00437
00438 si2drVoidT      si2drGroupSetComment        SI2_ARGS(( si2drGroupIdT group,
00439                                                         si2drStringT  comment,
00440                                                         si2drErrorT  *err));
00441
00442 si2drStringT     si2drAttrGetComment         SI2_ARGS(( si2drAttrIdT attr,
00443                                                         si2drErrorT  *err));
00444

```

```

00445     si2drVoidT      si2drAttrSetComment      SI2_ARGS(( si2drAttrIdT attr,
00446                                                    si2drStringT comment,
00447                                                    si2drErrorT *err));
00448
00449     si2drStringT    si2drDefineGetComment      SI2_ARGS(( si2drDefineIdT def,
00450                                                    si2drErrorT *err));
00451
00452     si2drVoidT      si2drDefineSetComment      SI2_ARGS(( si2drDefineIdT def,
00453                                                    si2drStringT comment,
00454                                                    si2drErrorT *err));
00455
00456     si2drVoidT      si2drGroupAddName          SI2_ARGS(( si2drGroupIdT group,
00457                                                    si2drStringT name,
00458                                                    si2drErrorT *err));
00459
00460     si2drVoidT      si2drGroupDeleteName       SI2_ARGS(( si2drGroupIdT group,
00461                                                    si2drStringT name,
00462                                                    si2drErrorT *err));
00463
00464
00465     si2drGroupIdT    si2drPIFindGroupByName     SI2_ARGS(( si2drStringT name,
00466                                                    si2drStringT type,
00467                                                    si2drErrorT *err));
00468
00469     si2drGroupIdT    si2drGroupFindGroupByName SI2_ARGS(( si2drGroupIdT group,
00470                                                    si2drStringT name,
00471                                                    si2drStringT type,
00472                                                    si2drErrorT *err));
00473
00474     si2drAttrIdT     si2drGroupFindAttrByName  SI2_ARGS(( si2drGroupIdT group,
00475                                                    si2drStringT name,
00476                                                    si2drErrorT *err));
00477
00478     si2drDefineIdT   si2drGroupFindDefineByName SI2_ARGS(( si2drGroupIdT group,
00479                                                    si2drStringT name,
00480                                                    si2drErrorT *err));
00481
00482
00483     si2drDefineIdT   si2drPIFindDefineByName   SI2_ARGS(( si2drStringT name,
00484                                                    si2drErrorT *err));
00485
00486
00487
00488     si2drGroupsIdT   si2drPIGetGroups          SI2_ARGS(( si2drErrorT *err));
00489
00490
00491     si2drGroupsIdT   si2drGroupGetGroups       SI2_ARGS(( si2drGroupIdT group,
00492                                                    si2drErrorT *err));
00493
00494     si2drNamesIdT    si2drGroupGetNames        SI2_ARGS(( si2drGroupIdT group,
00495                                                    si2drErrorT *err));
00496
00497     si2drAttrsIdT    si2drGroupGetAttrs        SI2_ARGS(( si2drGroupIdT group,
00498                                                    si2drErrorT *err));
00499
00500     si2drDefinesIdT  si2drGroupGetDefines      SI2_ARGS(( si2drGroupIdT group,
00501                                                    si2drErrorT *err));
00502
00503     si2drGroupIdT    si2drIterNextGroup        SI2_ARGS(( si2drGroupsIdT iter,
00504                                                    si2drErrorT *err));
00505     si2drStringT     si2drIterNextName         SI2_ARGS(( si2drNamesIdT iter,
00506                                                    si2drErrorT *err));
00507     si2drAttrIdT     si2drIterNextAttr         SI2_ARGS(( si2drAttrsIdT iter,
00508                                                    si2drErrorT *err));
00509     si2drDefineIdT   si2drIterNextDefine       SI2_ARGS(( si2drDefinesIdT iter,
00510                                                    si2drErrorT *err));
00511     si2drVoidT       si2drIterQuit             SI2_ARGS(( si2drIterIdT iter,
00512                                                    si2drErrorT *err));
00513

```

```

00514
00515
00516     si2drVoidT      si2drObjectDelete    SI2_ARGS(( si2drObjectIdT object,
00517                                           si2drErrorT  *err));
00518
00519
00520
00521     si2drStringT    si2drPIGetErrorText    SI2_ARGS(( si2drErrorT errorCode,
00522                                           si2drErrorT  *err));
00523
00524     si2drObjectIdT  si2drPIGetNullId      SI2_ARGS(( si2drErrorT  *err));
00525
00526     si2drVoidT      si2drPIInit          SI2_ARGS(( si2drErrorT  *err));
00527
00528     si2drVoidT      si2drPIQuit          SI2_ARGS(( si2drErrorT  *err));
00529
00530     si2drObjectTypeT si2drObjectGetObjectType SI2_ARGS(( si2drObjectIdT object,
00531                                           si2drErrorT  *err));
00532
00533     si2drObjectIdT  si2drObjectGetOwner    SI2_ARGS(( si2drObjectIdT object,
00534                                           si2drErrorT  *err));
00535
00536     si2drBooleanT    si2drObjectIsNull     SI2_ARGS(( si2drObjectIdT object,
00537                                           si2drErrorT  *err));
00538
00539     si2drBooleanT    si2drObjectIsSame     SI2_ARGS(( si2drObjectIdT object1,
00540                                           si2drObjectIdT object2,
00541                                           si2drErrorT  *err));
00542
00543     si2drBooleanT    si2drObjectIsUsable   SI2_ARGS(( si2drObjectIdT object,
00544                                           si2drErrorT  *err));
00545
00546     si2drVoidT      si2drObjectSetFileName SI2_ARGS(( si2drObjectIdT object,
00547                                           si2drStringT  filename,
00548                                           si2drErrorT  *err));
00549
00550     si2drVoidT      si2drObjectSetLineNo   SI2_ARGS(( si2drObjectIdT object,
00551                                           si2drInt32T   lineno,
00552                                           si2drErrorT  *err));
00553
00554     si2drInt32T      si2drObjectGetLineNo  SI2_ARGS(( si2drObjectIdT object,
00555                                           si2drErrorT  *err));
00556
00557     si2drStringT      si2drObjectGetFileName SI2_ARGS(( si2drObjectIdT object,
00558                                           si2drErrorT  *err));
00559
00560     si2drVoidT      si2drReadLibertyFile   SI2_ARGS(( char *filename,
00561                                           si2drErrorT  *err));
00562
00563     si2drVoidT      si2drWriteLibertyFile   SI2_ARGS(( char *filename,
00564                                           si2drGroupIdT group,
00565                                           char* cellname,
00566                                           si2drErrorT  *err ));
00567
00568     si2drVoidT      si2drCheckLibertyLibrary SI2_ARGS(( si2drGroupIdT group,
00569                                           si2drErrorT  *err));
00570
00571
00572     si2drBooleanT    si2drPIGetTraceMode    SI2_ARGS((si2drErrorT  *err));
00573
00574     si2drVoidT      si2drPIUnSetTraceMode   SI2_ARGS((si2drErrorT  *err));
00575
00576     si2drVoidT      si2drPISetTraceMode     SI2_ARGS((si2drStringT fname,
00577                                           si2drErrorT  *err));
00578
00579
00580     si2drVoidT      si2drPISetDebugMode     SI2_ARGS((si2drErrorT  *err));
00581
00582     si2drVoidT      si2drPIUnSetDebugMode   SI2_ARGS((si2drErrorT  *err));

```

```

00583
00584     si2drBooleanT    si2drPIGetDebugMode    SI2_ARGS((si2drErrorT *err));
00585
00586
00587     si2drVoidT       si2drPISetNocheckMode    SI2_ARGS((si2drErrorT *err));
00588
00589     si2drVoidT       si2drPIUnSetNocheckMode    SI2_ARGS((si2drErrorT *err));
00590
00591     si2drBooleanT    si2drPIGetNocheckMode    SI2_ARGS((si2drErrorT *err));
00592
00593
00594     si2drVoidT       si2drGroupMoveAfter SI2_ARGS((si2drGroupIdT groupToMove,
00595                                                    si2drGroupIdT targetGroup,
00596                                                    si2drErrorT *err));
00597
00598     si2drVoidT       si2drGroupMoveBefore SI2_ARGS((si2drGroupIdT groupToMove,
00599                                                    si2drGroupIdT targetGroup,
00600                                                    si2drErrorT *err));
00601
00602
00603 /* HELPER FUNCTIONS/STRUCTURES */
00604
00605 #ifdef NO_LONG_DOUBLE
00606 #define LONG_DOUBLE double
00607 #define strtold strtod
00608 #else
00609 #define LONG_DOUBLE long double
00610 #endif
00611
00612 struct liberty_value_data
00613 {
00614     int dimensions;
00615     int *dim_sizes; /* a malloc'd array of <dimensions> numbers,
00616 each number is the size of the array in that dim.*/
00617     LONG_DOUBLE **index_info; /* a Ptr to a malloc'd array of size <dimensions>, ptrs to
00618 malloc'd arrays of long double index values.
00619 Each array of long double is of length set by
00620 the corresponding dim_sizes array values */
00621     LONG_DOUBLE *values; /* a ptr to a malloc'd array of long doubles,
00622 starting with [0,0,...,0,0], and ending with
00623 [z-1,y-1,...,b-1,a-1], where a-z are the max
00624 number of elements in each dimension. */
00625 };
00626
00627 LONG_DOUBLE liberty_get_element(struct liberty_value_data *vd, ...); /* returns NaN if bounds exceeded */
00628
00629 void liberty_destroy_value_data(struct liberty_value_data *vd);
00630
00631 struct liberty_value_data *liberty_get_values_data( si2drGroupIdT table_group);
00632
00633 // directly call the main function of liberty parser
00634 extern void print_version();
00635 extern void usage();
00636 extern int main_liberty_parser(int argc, char* argv[]);
00637 extern void get_function(char* filename);
00638
00639 #ifdef __cplusplus
00640 }
00641 #endif
00642
00643 #endif
00644

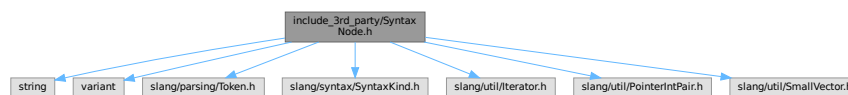
```

8.19 include_3rd_party/SyntaxNode.h File Reference

Base class and utilities for syntax nodes.

```
#include <string>
#include <variant>
#include "slang/parsing/Token.h"
#include "slang/syntax/SyntaxKind.h"
#include "slang/util/Iterator.h"
#include "slang/util/PointerIntPair.h"
#include "slang/util/SmallVector.h"
```

Include dependency graph for SyntaxNode.h:



Classes

- struct [slang::syntax::PtrTokenOrSyntax](#)
A token pointer or a syntax node.
- struct [slang::syntax::ConstTokenOrSyntax](#)
A token or a constant syntax node.
- struct [slang::syntax::TokenOrSyntax](#)
A token or a syntax node.
- class [slang::syntax::SyntaxNode](#)
Base class for all syntax nodes.
- class [slang::syntax::DeferredSourceRange](#)
- class [slang::syntax::SyntaxListBase](#)
A base class for syntax nodes that represent a list of items.
- class [slang::syntax::SyntaxList< T >](#)
A syntax node that represents a list of child syntax nodes.
- class [slang::syntax::TokenList](#)
A syntax node that represents a list of child tokens.
- class [slang::syntax::SeparatedSyntaxList< T >](#)
- class [slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >](#)

Namespaces

- namespace [slang](#)
- namespace [slang::syntax](#)

Functions

- SLANG_EXPORT SyntaxNode * [slang::syntax::clone](#) (const SyntaxNode &node, BumpAllocator &alloc)
Performs a shallow clone of the given syntax node.
- SLANG_EXPORT SyntaxNode * [slang::syntax::deepClone](#) (const SyntaxNode &node, BumpAllocator &alloc)
Performs a deep clone of the given syntax node.
- template<std::derived_from< SyntaxNode > T>
T * [slang::syntax::clone](#) (const T &node, BumpAllocator &alloc)
Performs a shallow clone of the given syntax node.
- template<std::derived_from< SyntaxNode > T>
T * [slang::syntax::deepClone](#) (const T &node, BumpAllocator &alloc)
Performs a deep clone of the given syntax node.
- template<typename T >
SyntaxList< T > * [slang::syntax::deepClone](#) (const SyntaxList< T > &node, BumpAllocator &alloc)
- template<typename T >
SeparatedSyntaxList< T > * [slang::syntax::deepClone](#) (const SeparatedSyntaxList< T > &node, BumpAllocator &alloc)
- TokenList * [slang::syntax::deepClone](#) (const TokenList &node, BumpAllocator &alloc)

8.19.1 Detailed Description

Base class and utilities for syntax nodes.

Definition in file [SyntaxNode.h](#).

8.20 SyntaxNode.h

[Go to the documentation of this file.](#)

```
00001 //-----
00004 //
00005 // SPDX-FileCopyrightText: Michael Popoloski
00006 // SPDX-License-Identifier: MIT
00007 //-----
00008 #pragma once
00009
00010 #include <string>
00011 #include <variant>
00012
00013 #include "slang/parsing/Token.h"
00014 #include "slang/syntax/SyntaxKind.h"
00015 #include "slang/util/Iterator.h"
00016 #include "slang/util/PointerIntPair.h"
00017 #include "slang/util/SmallVector.h"
```



```

00018
00019 namespace slang::syntax {
00020
00021 class SyntaxNode;
00022
00024 struct SLANG_EXPORT PtrTokenOrSyntax : public std::variant<parsing::Token*, SyntaxNode*> {
00025     using Base = std::variant<parsing::Token*, SyntaxNode*>;
00026     PtrTokenOrSyntax(parsing::Token* token) : Base(token) {}
00027     PtrTokenOrSyntax(SyntaxNode* node) : Base(node) {}
00028     PtrTokenOrSyntax(nullptr_t) : Base((parsing::Token*)nullptr) {}
00029
00031     bool isToken()const { return this->index() == 0; }
00032
00034     bool isNode()const { return this->index() == 1; }
00035
00038     parsing::Token* token()const { return std::get<0>(*this); }
00039
00042     SyntaxNode* node()const { return std::get<1>(*this); }
00043
00046     SourceRange range() const;
00047 };
00048
00050 struct SLANG_EXPORT ConstTokenOrSyntax : public std::variant<parsing::Token, const SyntaxNode*> {
00051     using Base = std::variant<parsing::Token, const SyntaxNode*>;
00052     ConstTokenOrSyntax(parsing::Token token) : Base(token) {}
00053     ConstTokenOrSyntax(const SyntaxNode* node) : Base(node) {}
00054     ConstTokenOrSyntax(nullptr_t) : Base(parsing::Token()) {}
00055
00057     bool isToken()const { return this->index() == 0; }
00058
00060     bool isNode()const { return this->index() == 1; }
00061
00064     parsing::Token token()const { return std::get<0>(*this); }
00065
00068     const SyntaxNode* node()const { return std::get<1>(*this); }
00069
00071     SourceRange range() const;
00072 };
00073
00075 struct SLANG_EXPORT TokenOrSyntax : public ConstTokenOrSyntax {
00076     TokenOrSyntax(parsing::Token token) : ConstTokenOrSyntax(token) {}
00077     TokenOrSyntax(SyntaxNode* node) : ConstTokenOrSyntax(node) {}
00078     TokenOrSyntax(nullptr_t) : ConstTokenOrSyntax(parsing::Token()) {}
00079
00082     SyntaxNode* node()const {
00083         // const_cast is safe because we only could have constructed this
00084         // object with a non-const pointer.
00085         return const_cast<SyntaxNode*>(std::get<1>(*this));
00086     }
00087 };
00088
00090 class SLANG_EXPORT SyntaxNode {
00091 public:
00092     using Token = parsing::Token;
00093
00095     SyntaxKind kind;
00096
00099     SyntaxNode* parent = nullptr;
00100
00108     const SyntaxNode* previewNode = nullptr;
00109
00111     std::string toString() const;
00112
00114     Token getFirstToken() const;
00115
00117     Token getLastToken() const;
00118
00120     Token* getFirstTokenPtr();
00121

```

```

00123     Token* getLastTokenPtr();
00124
00126     SourceRange sourceRange() const;
00127
00130     const SyntaxNode* childNode(size_t index) const;
00131
00134     SyntaxNode* childNode(size_t index);
00135
00139     Token childToken(size_t index) const;
00140
00144     Token* childTokenPtr(size_t index);
00145
00147     size_t getChildCount() const; // Note: implemented in AllSyntax.cpp
00148
00153     bool isEquivalentTo(const SyntaxNode& other) const;
00154
00157     template<typename T>
00158     T& as() {
00159         SLANG_ASSERT(T::isKind(kind));
00160         return *static_cast<T*>(this);
00161     }
00162
00165     template<typename T>
00166     const T& as()const {
00167         SLANG_ASSERT(T::isKind(kind));
00168         return *static_cast<const T*>(this);
00169     }
00170
00171     // Reinterprets this node as being of type T, if it is not belong type T, return nullptr
00172     template<typename T>
00173     T* as_if() {
00174         if (!T::isKind(kind))
00175             return nullptr;
00176         return static_cast<T*>(this);
00177     }
00178
00179     // Reinterprets this node as being of type T, if it is not belong type T, return nullptr
00180     template<typename T>
00181     const T* as_if()const {
00182         if (!T::isKind(kind))
00183             return nullptr;
00184         return static_cast<const T*>(this);
00185     }
00186
00189     template<typename TVisitor, typename... Args>
00190     decltype(auto) visit(TVisitor& visitor, Args&&... args);
00191
00194     template<typename TVisitor, typename... Args>
00195     decltype(auto) visit(TVisitor& visitor, Args&&... args) const;
00196
00200     static bool isKind(SyntaxKind) { return true; }
00201
00204     static bool isChildOptional(size_t) { return true; }
00205
00206 protected:
00207     explicit SyntaxNode(SyntaxKind kind) : kind(kind) {}
00208
00209 private:
00210     ConstTokenOrSyntax getChild(size_t index) const;
00211     TokenOrSyntax getChild(size_t index);
00212     PtrTokenOrSyntax getChildPtr(size_t index);
00213 };
00214
00219 SLANG_EXPORT SyntaxNode* clone(const SyntaxNode& node, BumpAllocator& alloc);
00220
00226 SLANG_EXPORT SyntaxNode* deepClone(const SyntaxNode& node, BumpAllocator& alloc);
00227
00232 template<std::derived_from<SyntaxNode> T>
00233 T* clone(const T& node, BumpAllocator& alloc) {

```

```

00234     return static_cast<T*>(clone(static_cast<const SyntaxNode*>(node), alloc));
00235 }
00236
00242 template<std::derived_from<SyntaxNode> T>
00243 T* deepClone(const T& node, BumpAllocator& alloc) {
00244     return static_cast<T*>(deepClone(static_cast<const SyntaxNode*>(node), alloc));
00245 }
00246
00250 class DeferredSourceRange {
00251 public:
00252     DeferredSourceRange() = default;
00253     DeferredSourceRange(SourceRange range) : range(range), node(nullptr, true) {}
00254     DeferredSourceRange(const SyntaxNode& node) : node(&node, false) {}
00255     DeferredSourceRange(const SyntaxNode& node, SourceRange range) :
00256         range(range), node(&node, true) {}
00257
00258     SourceRange get()const {
00259         if (!node.getInt()) {
00260             if (auto ptr = node.getPointer())
00261                 range = ptr->sourceRange();
00262             node.setInt(true);
00263         }
00264         return range;
00265     }
00266
00267     SourceRange operator*()const { return get(); }
00268
00269     const SyntaxNode* syntax()const { return node.getPointer(); }
00270
00271 private:
00272     mutable SourceRange range;
00273     mutable PointerIntPair<const SyntaxNode*, 1, 1, bool> node;
00274 };
00275
00277 class SLANG_EXPORT SyntaxListBase : public SyntaxNode {
00278 public:
00280     size_t getChildCount()const { return childCount; }
00281
00283     virtual TokenOrSyntax getChild(size_t index) = 0;
00284
00286     virtual ConstTokenOrSyntax getChild(size_t index) const = 0;
00287
00288     // Gets the child pointer (token or node) at given index.
00289     virtual PtrTokenOrSyntax getChildPtr(size_t index) = 0;
00290
00292     virtual void setChild(size_t index, TokenOrSyntax child) = 0;
00293
00295     virtual SyntaxListBase* clone(BumpAllocator& alloc) const = 0;
00296
00299     virtual void resetAll(BumpAllocator& alloc, std::span<const TokenOrSyntax> children) = 0;
00300
00301     static bool isKind(SyntaxKind kind);
00302
00303     static bool isChildOptional(size_t index);
00304
00305 protected:
00306     SyntaxListBase(SyntaxKind kind, size_t childCount) : SyntaxNode(kind), childCount(childCount) {}
00307
00308     size_t childCount;
00309 };
00310
00312 template<typename T>
00313 class SLANG_EXPORT SyntaxList : public SyntaxListBase, public std::span<T*> {
00314 public:
00315     SyntaxList(nullptr_t) : SyntaxList(std::span<T*>()) {}
00316     SyntaxList(std::span<T*> elements) :
00317         SyntaxListBase(SyntaxKind::SyntaxList, elements.size(), std::span<T*>(elements)) {}
00318
00319 private:

```

```

00320 TokenOrSyntax getChild(size_t index) final { return (*this)[index]; }
00321 ConstTokenOrSyntax getChild(size_t index) const final { return (*this)[index]; }
00322 PtrTokenOrSyntax getChildPtr(size_t index) final { return (*this)[index]; };
00323
00324 void setChild(size_t index, TokenOrSyntax child) final {
00325     (*this)[index] = &child.node()->as<T>();
00326 }
00327
00328 SyntaxListBase* clone(BumpAllocator& alloc) const final {
00329     return alloc.emplace<SyntaxList<T>>>(*this);
00330 }
00331
00332 void resetAll(BumpAllocator& alloc, std::span<const TokenOrSyntax> children) final {
00333     SmallVector<T*> buffer(children.size(), UninitializedTag());
00334     for (auto& t : children)
00335         buffer.push_back(&t.node()->as<T>());
00336
00337     *this = buffer.copy(alloc);
00338     childCount = buffer.size();
00339 }
00340 };
00341
00342 template<typename T>
00343 SyntaxList<T>* deepClone(const SyntaxList<T>& node, BumpAllocator& alloc) {
00344     SmallVector<T*> buffer(node.size(), UninitializedTag());
00345     for (auto& t : node)
00346         buffer.push_back(deepClone(*t, alloc));
00347
00348     return alloc.emplace<SyntaxList<T>>(buffer.copy(alloc));
00349 }
00350
00352 class SLANG_EXPORT TokenList : public SyntaxListBase, public std::span<parsing::Token> {
00353 public:
00354     TokenList(nullptr_t) : TokenList(std::span<Token>()) {}
00355     TokenList(std::span<Token> elements) :
00356         SyntaxListBase(SyntaxKind::TokenList, elements.size(), std::span<Token>(elements)) {}
00357 private:
00359     TokenOrSyntax getChild(size_t index) final { return (*this)[index]; }
00360     ConstTokenOrSyntax getChild(size_t index) const final { return (*this)[index]; }
00361     PtrTokenOrSyntax getChildPtr(size_t index) final { return &(*this)[index]; };
00362     void setChild(size_t index, TokenOrSyntax child) final { (*this)[index] = child.token(); }
00363
00364     SyntaxListBase* clone(BumpAllocator& alloc) const final {
00365         return alloc.emplace<TokenList>(*this);
00366     }
00367
00368     void resetAll(BumpAllocator& alloc, std::span<const TokenOrSyntax> children) final {
00369         SmallVector<Token> buffer(children.size(), UninitializedTag());
00370         for (auto& t : children)
00371             buffer.push_back(t.token());
00372
00373         *this = buffer.copy(alloc);
00374         childCount = buffer.size();
00375     }
00376 };
00377
00380 template<typename T>
00381 class SLANG_EXPORT SeparatedSyntaxList : public SyntaxListBase {
00382 public:
00383     template<typename U>
00384     class iterator_base : public iterator_facade<iterator_base<U>> {
00385     public:
00386         using ParentList = std::conditional_t<std::is_const_v<std::remove_pointer_t<U>>,
00387                                             const SeparatedSyntaxList, SeparatedSyntaxList>;
00388
00389         iterator_base() : list(nullptr), index(0) {}
00390         iterator_base(ParentList& list, size_t index) : list(&list), index(index) {}
00391     };

```

```

00394         U dereference()const { return (*list)[index]; }
00395
00396         bool equals(const iterator_base& other)const {
00397             return list == other.list && index == other.index;
00398         }
00399
00400         ptrdiff_t distance_to(const iterator_base& other)const {
00401             return ptrdiff_t(other.index) - ptrdiff_t(index);
00402         }
00403
00404         void advance(ptrdiff_t n) { index = size_t(ptrdiff_t(index) + n); }
00405
00406     private:
00407         ParentList* list;
00408         size_t index;
00409     };
00410
00411     using iterator = iterator_base<T*>;
00412     using const_iterator = iterator_base<const T*>;
00413
00414     SeparatedSyntaxList(nullptr_t) : SeparatedSyntaxList(std::span<TokenOrSyntax>()) {}
00415     SeparatedSyntaxList(std::span<TokenOrSyntax> elements) :
00416         SyntaxListBase(SyntaxKind::SeparatedList, elements.size(), elements(elements)) {}
00417
00419     [[nodiscard]] std::span<const ConstTokenOrSyntax> elems()const {
00420         return std::span<const ConstTokenOrSyntax>(
00421             static_cast<ConstTokenOrSyntax*>(elements.data()), elements.size());
00422     }
00423
00425     [[nodiscard]] std::span<TokenOrSyntax> elems() { return elements; }
00426
00428     [[nodiscard]] bool empty()const { return elements.empty(); }
00429
00431     [[nodiscard]] size_t size() const noexcept { return (elements.size() + 1) / 2; }
00432
00433     const T* operator[](size_t index)const {
00434         index <= 1;
00435         return &elements[index].node()->template as<T>();
00436     }
00437
00438     T* operator[](size_t index) {
00439         index <= 1;
00440         return &elements[index].node()->template as<T>();
00441     }
00442
00443     iterator begin() { return iterator(*this, 0); }
00444     iterator end() { return iterator(*this, size()); }
00445     const_iterator begin()const { return cbegin(); }
00446     const_iterator end()const { return cend(); }
00447     const_iterator cbegin()const { return const_iterator(*this, 0); }
00448     const_iterator cend()const { return const_iterator(*this, size()); }
00449
00450     private:
00451     TokenOrSyntax getChild(size_t index) final { return elements[index]; }
00452     ConstTokenOrSyntax getChild(size_t index) const final { return elements[index]; }
00453     PtrTokenOrSyntax getChildPtr(size_t index) final {
00454         if (elements[index].isNode())
00455             return elements[index].node();
00456         else
00457             return &(std::get<parsing::Token>(elements[index]));
00458     }
00459     void setChild(size_t index, TokenOrSyntax child) final { elements[index] = child; }
00460
00461     SyntaxListBase* clone(BumpAllocator& alloc) const final {
00462         return alloc.emplace<SeparatedSyntaxList<T*>>(*this);
00463     }
00464
00465     void resetAll(BumpAllocator& alloc, std::span<const TokenOrSyntax> children) final {
00466         SmallVector<TokenOrSyntax> buffer(children.size(), UninitializedTag());

```

```

00467         buffer.append_range(children);
00468
00469         elements = buffer.copy(alloc);
00470         childCount = buffer.size();
00471     }
00472
00473     std::span<TokenOrSyntax> elements;
00474 };
00475
00476 template<typename T>
00477 SeparatedSyntaxList<T>* deepClone(const SeparatedSyntaxList<T>& node, BumpAllocator& alloc) {
00478     SmallVector<TokenOrSyntax> buffer(node.size(), UninitializedTag());
00479     for (const auto& ele : node.elms()) {
00480         if (ele.isToken())
00481             buffer.push_back(ele.token().deepClone(alloc));
00482         else
00483             buffer.push_back(static_cast<T*>(deepClone(*ele.node(), alloc)));
00484     }
00485     return alloc.emplace<SeparatedSyntaxList<T>>(buffer.copy(alloc));
00486 }
00487
00488 inline TokenList* deepClone(const TokenList& node, BumpAllocator& alloc) {
00489     SmallVector<parsing::Token> buffer(node.size(), UninitializedTag());
00490     for (const auto& ele : node) {
00491         buffer.push_back(ele.deepClone(alloc));
00492     }
00493     return alloc.emplace<TokenList>(buffer.copy(alloc));
00494 }
00495
00496 } // namespace slang::syntax

```

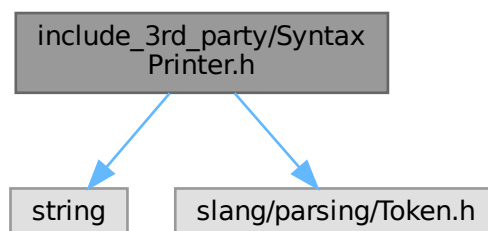
8.21 include_3rd_party/SyntaxPrinter.h File Reference

Support for printing syntax nodes and tokens.

```
#include <string>
```

```
#include "slang/parsing/Token.h"
```

Include dependency graph for SyntaxPrinter.h:



Classes

- class `slang::syntax::SyntaxPrinter`

Namespaces

- namespace `slang`
- namespace `slang::syntax`

8.21.1 Detailed Description

Support for printing syntax nodes and tokens.

Definition in file [SyntaxPrinter.h](#).

8.22 SyntaxPrinter.h

[Go to the documentation of this file.](#)

```
00001 -----
00004 //
00005 // SPDX-FileCopyrightText: Michael Popoloski
00006 // SPDX-License-Identifier: MIT
00007 -----
00008 #pragma once
00009
00010 #include <string>
00011
00012 #include "slang/parsing/Token.h"
00013
00014 namespace slang {
00015 class SourceManager;
00016 }
00017
00018 namespace slang::syntax {
00019
00020 class SyntaxNode;
00021 class SyntaxTree;
00022
00025 class SLANG_EXPORT SyntaxPrinter {
00026 public:
00027     SyntaxPrinter() = default;
00028     explicit SyntaxPrinter(const SourceManager& sourceManager);
00029
00032 SyntaxPrinter& append(std::string_view text);
00033
00036 SyntaxPrinter& print(parsing::Trivia trivia);
00037
00040 SyntaxPrinter& print(parsing::Token token);
00041
00044 SyntaxPrinter& print(const SyntaxNode& node);
00045
00048 SyntaxPrinter& print(const SyntaxTree& tree);
00049
00052 SyntaxPrinter& setIncludeTrivia(bool include) {
00053     includeTrivia = include;
00054     return *this;
00055 }
00056
00059 SyntaxPrinter& setIncludeMissing(bool include) {
00060     includeMissing = include;
00061     return *this;
00062 }
```

```

00063
00066     SyntaxPrinter& setIncludeSkipped(bool include) {
00067         includeSkipped = include;
00068         return *this;
00069     }
00070
00073     SyntaxPrinter& setIncludeDirectives(bool include) {
00074         includeDirectives = include;
00075         return *this;
00076     }
00077
00080     SyntaxPrinter& setIncludePreprocessed(bool include) {
00081         includePreprocessed = include;
00082         return *this;
00083     }
00084
00087     SyntaxPrinter& setIncludeComments(bool include) {
00088         includeComments = include;
00089         return *this;
00090     }
00091
00094     SyntaxPrinter& setSquashNewlines(bool include) {
00095         squashNewlines = include;
00096         return *this;
00097     }
00098
00100     std::string str()const { return buffer; }
00101
00105     static std::string printFile(const SyntaxTree& tree);
00106
00107 private:
00108     std::string buffer;
00109     const SourceManager* sourceManager = nullptr;
00110     bool includeTrivia = true;
00111     bool includeMissing = false;
00112     bool includeSkipped = false;
00113     bool includeDirectives = false;
00114     bool includePreprocessed = true;
00115     bool includeComments = true;
00116     bool squashNewlines = true;
00117 };
00118
00119 } // namespace slang::syntax

```

8.23 include_3rd_party/SyntaxTree.h File Reference

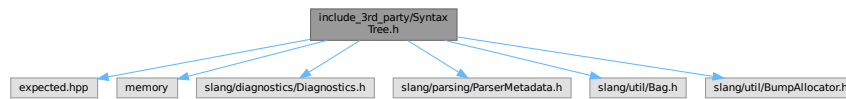
Top-level parser interface.

```

#include <expected.hpp>
#include <memory>
#include "slang/diagnostics/Diagnostics.h"
#include "slang/parsing/ParserMetadata.h"
#include "slang/util/Bag.h"
#include "slang/util/BumpAllocator.h"

```


Include dependency graph for SyntaxTree.h:



Classes

- class [slang::syntax::SyntaxTree](#)

Namespaces

- namespace [slang](#)
- namespace [slang::parsing](#)
- namespace [slang::syntax](#)

8.23.1 Detailed Description

Top-level parser interface.

Definition in file [SyntaxTree.h](#).

8.24 SyntaxTree.h

[Go to the documentation of this file.](#)

```

00001 //-----
00004 //
00005 // SPDX-FileCopyrightText: Michael Popoloski
00006 // SPDX-License-Identifier: MIT
00007 //-----
00008 #pragma once
00009
00010 #include <expected.hpp>
00011 #include <memory>
00012
00013 #include "slang/diagnostics/Diagnostics.h"
00014 #include "slang/parsing/ParserMetadata.h"
00015 #include "slang/util/Bag.h"
00016 #include "slang/util/BumpAllocator.h"
00017
00018 namespace slang {
00019
00020 class SourceManager;
00021 struct SourceBuffer;
00022
00023 } // namespace slang
00024

```

```

00025 namespace slang::parsing {
00026 struct ParserMetadata;
00027 }
00028
00029 namespace slang::syntax {
00030
00031 class SyntaxNode;
00032 struct DefineDirectiveSyntax;
00033
00039 class SLANG_EXPORT SyntaxTree {
00040 public:
00041     using TreeOrError =
00042         nonstd::expected<std::shared_ptr<SyntaxTree>, std::pair<std::error_code, std::string_view>>;
00043     using MacroList = std::span<const DefineDirectiveSyntax* const>;
00044
00047     bool isLibraryUnit = false;
00048
00049     SyntaxTree(SyntaxNode* root, SourceManager& sourceManager, BumpAllocator&& alloc,
00050         const SourceLibrary* library, const std::shared_ptr<SyntaxTree>& parent = nullptr);
00051
00052     SyntaxTree(SyntaxTree&& other) = default;
00053     ~SyntaxTree();
00054
00058     static TreeOrError fromFile(std::string_view path);
00059
00066     static TreeOrError fromFile(std::string_view path, SourceManager& sourceManager,
00067         const Bag& options = {});
00068
00073     static TreeOrError fromFiles(std::span<const std::string_view> paths);
00074
00081     static TreeOrError fromFiles(std::span<const std::string_view> paths,
00082         SourceManager& sourceManager, const Bag& options = {});
00083
00089     static std::shared_ptr<SyntaxTree> fromText(std::string_view text,
00090         std::string_view name = "source",
00091         std::string_view path = "");
00092
00099     static std::shared_ptr<SyntaxTree> fromText(std::string_view text, const Bag& options,
00100         std::string_view name = "source"sv,
00101         std::string_view path = "");
00102
00111     static std::shared_ptr<SyntaxTree> fromText(std::string_view text, SourceManager& sourceManager,
00112         std::string_view name = "source"sv,
00113         std::string_view path = "", const Bag& options = {},
00114         const SourceLibrary* library = nullptr);
00115
00123     static std::shared_ptr<SyntaxTree> fromFileInMemory(std::string_view text,
00124         SourceManager& sourceManager,
00125         std::string_view name = "source"sv,
00126         std::string_view path = "",
00127         const Bag& options = {});
00128
00135     static std::shared_ptr<SyntaxTree> fromBuffer(const SourceBuffer& buffer,
00136         SourceManager& sourceManager,
00137         const Bag& options = {},
00138         MacroList inheritedMacros = {});
00139
00147     static std::shared_ptr<SyntaxTree> fromBuffers(std::span<const SourceBuffer> buffers,
00148         SourceManager& sourceManager,
00149         const Bag& options = {},
00150         MacroList inheritedMacros = {});
00151
00157     static std::shared_ptr<SyntaxTree> fromLibraryMapFile(std::string_view path,
00158         SourceManager& sourceManager,
00159         const Bag& options = {});
00160
00168     static std::shared_ptr<SyntaxTree> fromLibraryMapText(std::string_view text,
00169         SourceManager& sourceManager,
00170         std::string_view name = "source"sv,

```

```

00171         std::string_view path = "",
00172         const Bag& options = {});
00173
00179     static std::shared_ptr<SyntaxTree> fromLibraryMapBuffer(const SourceBuffer& buffer,
00180         SourceManager& sourceManager,
00181         const Bag& options = {});
00182
00184     Diagnostics& diagnostics() { return diagnosticsBuffer; }
00185
00187     BumpAllocator& allocator() { return alloc; }
00188
00190     SourceManager& sourceManager() { return sourceMan; }
00191
00193     const SourceManager& sourceManager()const { return sourceMan; }
00194
00196     const SourceLibrary* getSourceLibrary()const { return library; }
00197
00199     SyntaxNode& root() { return *rootNode; }
00200
00202     const SyntaxNode& root()const { return *rootNode; }
00203
00205     const Bag& options()const { return options_; }
00206
00208     const parsing::ParserMetadata& getMetadata()const { return *metadata; }
00209
00211     MacroList getDefinedMacros()const { return macros; }
00212
00217     static SourceManager& getDefaultSourceManager();
00218
00219 private:
00220     SyntaxTree(SyntaxNode* root, const SourceLibrary* library, SourceManager& sourceManager,
00221         BumpAllocator&& alloc, Diagnostics&& diagnostics, parsing::ParserMetadata&& metadata,
00222         std::vector<const DefineDirectiveSyntax*>&& macros, Bag options);
00223
00224     static std::shared_ptr<SyntaxTree> create(SourceManager& sourceManager,
00225         std::span<const SourceBuffer> source,
00226         const Bag& options, MacroList inheritedMacros,
00227         bool guess);
00228
00229     SyntaxNode* rootNode;
00230     const SourceLibrary* library;
00231     SourceManager& sourceMan;
00232     BumpAllocator alloc;
00233     Diagnostics diagnosticsBuffer;
00234     Bag options_;
00235     std::unique_ptr<parsing::ParserMetadata> metadata;
00236     std::vector<const DefineDirectiveSyntax*> macros;
00237 };
00238
00239 } // namespace slang::syntax

```

8.25 include_3rd_party/SyntaxVisitor.h File Reference

Syntax tree visitor support.

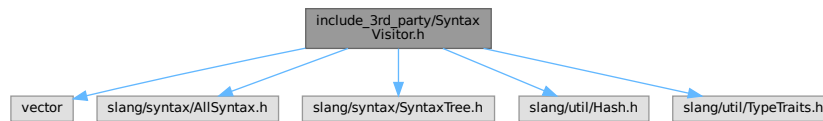
```

#include <vector>
#include "slang/syntax/AllSyntax.h"
#include "slang/syntax/SyntaxTree.h"
#include "slang/util/Hash.h"

```

```
#include "slang/util/TypeTraits.h"
```

Include dependency graph for SyntaxVisitor.h:



Classes

- class [slang::syntax::SyntaxVisitor< TDerived >](#)
- struct [slang::syntax::detail::SyntaxChange](#)
- struct [slang::syntax::detail::RemoveChange](#)
- struct [slang::syntax::detail::ReplaceChange](#)
- struct [slang::syntax::detail::ChangeCollection](#)
- class [slang::syntax::SyntaxRewriter< TDerived >](#)

Namespaces

- namespace [slang](#)
- namespace [slang::syntax](#)
- namespace [slang::syntax::detail](#)

Macros

- `#define` [DERIVED](#) `static_cast<TDerived*>(this)`

Typedefs

- using [slang::syntax::detail::InsertChangeMap](#) = `flat_hash_map< const SyntaxNode *, std::vector< SyntaxChange > >`
- using [slang::syntax::detail::ModifyChangeMap](#) = `flat_hash_map< const SyntaxNode *, std::variant< RemoveChange, ReplaceChange > >`
- using [slang::syntax::detail::ListChangeMap](#) = `flat_hash_map< const SyntaxNode *, std::vector< SyntaxChange > >`

Functions

- SLANG_EXPORT std::shared_ptr< SyntaxTree > [slang::syntax::detail::transformTree](#) (BumpAllocator &&alloc, const std::shared_ptr< SyntaxTree > &tree, const ChangeCollection &commits, const std::vector< std::shared_ptr< SyntaxTree > > &tempTrees, const SourceLibrary *library)

8.25.1 Detailed Description

Syntax tree visitor support.

Definition in file [SyntaxVisitor.h](#).

8.25.2 Macro Definition Documentation

8.25.2.1 DERIVED

```
#define DERIVED static_cast<TDerived*>(this)
```

Definition at line 19 of file [SyntaxVisitor.h](#).

8.26 SyntaxVisitor.h

[Go to the documentation of this file.](#)

```
00001 //-----
00004 //
00005 // SPDX-FileCopyrightText: Michael Popoloski
00006 // SPDX-License-Identifier: MIT
00007 //-----
00008 #pragma once
00009
00010 #include <vector>
00011
00012 #include "slang/syntax/AllSyntax.h"
00013 #include "slang/syntax/SyntaxTree.h"
00014 #include "slang/util/Hash.h"
00015 #include "slang/util/TypeTraits.h"
00016
00017 namespace slang::syntax {
00018
00019 #define DERIVED static_cast<TDerived*>(this)
00020
00024 template<typename TDerived>
00025 class SyntaxVisitor {
00026 public:
00028     template<typename T>
```

```

00029     void visit(T&& t) {
00030         if constexpr (requires { DERIVED->handle(t); })
00031             DERIVED->handle(t);
00032         else
00033             DERIVED->visitDefault(t);
00034     }
00035
00036     template<typename T>
00037     void visitDefault(T&& node) {
00038         for (uint32_t i = 0; i < node.getChildCount(); i++) {
00039             auto child = node.childNode(i);
00040             if (child)
00041                 child->visit(*DERIVED);
00042             else {
00043                 auto token = node.childToken(i);
00044                 if (token)
00045                     DERIVED->visitToken(token);
00046             }
00047         }
00048     }
00049 }
00050
00051 private:
00052     // This is to make things compile if the derived class doesn't provide an implementation.
00053     void visitToken(parsing::Token) {}
00054 };
00055
00056 namespace detail {
00057     struct SLANG_EXPORT SyntaxChange {
00058         const SyntaxNode* first = nullptr;
00059         SyntaxNode* second = nullptr;
00060         parsing::Token separator = {};
00061     };
00062
00063     struct SLANG_EXPORT RemoveChange : SyntaxChange {};
00064     struct SLANG_EXPORT ReplaceChange : SyntaxChange {};
00065
00066     using InsertChangeMap = flat_hash_map<const SyntaxNode*, std::vector<SyntaxChange>>;
00067     using ModifyChangeMap = flat_hash_map<const SyntaxNode*, std::variant<RemoveChange, ReplaceChange>>;
00068     using ListChangeMap = flat_hash_map<const SyntaxNode*, std::vector<SyntaxChange>>;
00069
00070     struct SLANG_EXPORT ChangeCollection {
00071         InsertChangeMap insertBefore;
00072         InsertChangeMap insertAfter;
00073         ModifyChangeMap removeOrReplace;
00074         ListChangeMap listInsertAtFront;
00075         ListChangeMap listInsertAtBack;
00076
00077         void clear() {
00078             insertBefore.clear();
00079             insertAfter.clear();
00080             removeOrReplace.clear();
00081             listInsertAtFront.clear();
00082             listInsertAtBack.clear();
00083         }
00084
00085         bool empty() const {
00086             return insertBefore.empty() && insertAfter.empty() && removeOrReplace.empty() &&
00087                 listInsertAtFront.empty() && listInsertAtBack.empty();
00088         }
00089     };
00090
00091     SLANG_EXPORT std::shared_ptr<SyntaxTree> transformTree(
00092         BumpAllocator&& alloc, const std::shared_ptr<SyntaxTree>& tree, const ChangeCollection& commits,
00093         const std::vector<std::shared_ptr<SyntaxTree>>& tempTrees, const SourceLibrary* library);
00094 } // namespace detail
00095
00096 template<typename TDerived>

```

```

00102 class SyntaxRewriter : public SyntaxVisitor<TDerived> {
00103 public:
00104     SyntaxRewriter() : factory(alloc) {}
00105
00116     std::shared_ptr<SyntaxTree> transform(const std::shared_ptr<SyntaxTree>& tree,
00117                                         const SourceLibrary* library = nullptr) {
00118         sourceManager = &tree->sourceManager();
00119         commits.clear();
00120         tempTrees.clear();
00121
00122         tree->root().visit(*DERIVED);
00123
00124         if (commits.empty())
00125             return tree;
00126
00127         return transformTree(std::move(alloc), tree, commits, tempTrees, library);
00128     }
00129
00130 protected:
00131     using Token = parsing::Token;
00132
00133     SyntaxNode& parse(std::string_view text) {
00134         tempTrees.emplace_back(SyntaxTree::fromText(text, *sourceManager));
00135         return tempTrees.back()->root();
00136     }
00137
00138     void remove(const SyntaxNode& oldNode) {
00141         if (auto [_, ok] = commits.removeOrReplace.emplace(&oldNode,
00142                                                         detail::RemoveChange{&oldNode, nullptr});
00143             !ok) {
00144             SLANG_THROW(std::logic_error("Node only permit one remove/replace operation"));
00145         }
00146     }
00147
00149     void replace(const SyntaxNode& oldNode, SyntaxNode& newNode) {
00150         if (auto [_, ok] = commits.removeOrReplace.emplace(
00151             &oldNode, detail::ReplaceChange{&oldNode, &newNode});
00152             !ok) {
00153             SLANG_THROW(std::logic_error("Node only permit one remove/replace operation"));
00154         }
00155     }
00156
00158     void insertBefore(const SyntaxNode& oldNode, SyntaxNode& newNode) {
00159         commits.insertBefore[&oldNode].push_back({&oldNode, &newNode});
00160     }
00161
00163     void insertAfter(const SyntaxNode& oldNode, SyntaxNode& newNode) {
00164         commits.insertAfter[&oldNode].push_back({&oldNode, &newNode});
00165     }
00166
00168     void insertAtFront(const SyntaxListBase& list, SyntaxNode& newNode, Token separator = {}) {
00169         commits.listInsertAtFront[&list].push_back({&list, &newNode, separator});
00170     }
00171
00173     void insertAtBack(const SyntaxListBase& list, SyntaxNode& newNode, Token separator = {}) {
00174         commits.listInsertAtBack[&list].push_back({&list, &newNode, separator});
00175     }
00176
00177     Token makeToken(parsing::TokenKind kind, std::string_view text,
00178                    std::span<const parsing::Trivia> trivia = {}) {
00179         return Token(alloc, kind, trivia, text, SourceLocation::NoLocation);
00180     }
00181
00182     Token makeId(std::string_view text, std::span<const parsing::Trivia> trivia = {}) {
00183         return makeToken(parsing::TokenKind::Identifier, text, trivia);
00184     }
00185
00186     Token makeId(std::string_view text, parsing::Trivia trivia) {
00187         auto triviaPtr = alloc.emplace<parsing::Trivia>(trivia);

```

```

00188         return makeId(text, {triviaPtr, 1});
00189     }
00190
00191     Token makeComma() { return makeToken(parsing::TokenKind::Comma, ", "sv); }
00192
00193     BumpAllocator alloc;
00194     SyntaxFactory factory;
00195
00196     static const parsing::Trivia SingleSpace;
00197
00198 private:
00199     SourceManager* sourceManager = nullptr;
00200     detail::ChangeCollection commits;
00201     std::vector<std::shared_ptr<SyntaxTree>> tempTrees;
00202 };
00203
00204 template<typename TDerived>
00205 const parsing::Trivia SyntaxRewriter<TDerived>::SingleSpace{parsing::TriviaKind::Whitespace, " sv};
00206
00207 #undef DERIVED
00208
00209 } // namespace slang::syntax

```

8.27 README.md File Reference

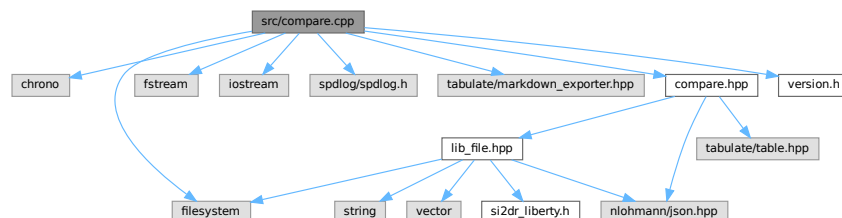
8.28 src/compare.cpp File Reference

```

#include <chrono>
#include <filesystem>
#include <fstream>
#include <iostream>
#include "spdlog/spdlog.h"
#include <tabulate/markdown_exporter.hpp>
#include "compare.hpp"
#include "version.h"

```

Include dependency graph for compare.cpp:



8.29 compare.cpp

[Go to the documentation of this file.](#)


```

00001 #include <chrono>
00002 #include <filesystem>
00003 #include <fstream>
00004 #include <iostream>
00005
00006 #include "spdlog/spdlog.h"
00007 #include <tabulate/markdown_exporter.hpp>
00008
00009 #include "compare.hpp"
00010 #include "version.h"
00011
00012
00031 LibraryComparator::LibraryComparator(LibFile &ref_libfile, LibFile &comp_libfile, double reltol,
00032                                     double abstol) {
00033     : reltol_(reltol), abstol_(abstol) {
00034     std::string ref_json_name = ref_libfile.basename_ + ".json";
00035     if (!std::filesystem::exists(ref_json_name)) {
00036         spdlog::info("Reference JSON file {} not found. Parsing first.", ref_json_name);
00037         ref_libfile.parse();
00038         ref_libfile.writeJsonToFile();
00039         ref_json_ = ref_libfile.lib_json_;
00040     } else {
00041         // Read the JSON file into json object
00042         std::ifstream ref_in(ref_json_name);
00043         if (!ref_in.is_open()) {
00044             spdlog::error("Could not open file '{}' for reading", ref_json_name);
00045             return;
00046         }
00047         try {
00048             ref_json_ = json::parse(ref_in);
00049         } catch (const json::parse_error &e) {
00050             spdlog::error("Error parsing reference JSON file '{}': {}", ref_json_name, e.what());
00051             return;
00052         }
00053     }
00054     std::string comp_json_name = comp_libfile.basename_ + ".json";
00055     if (!std::filesystem::exists(comp_json_name)) {
00056         spdlog::info("Comparison JSON file {} not found. Parsing first.", comp_json_name);
00057         comp_libfile.parse();
00058         comp_libfile.writeJsonToFile();
00059         comp_json_ = comp_libfile.lib_json_;
00060     } else {
00061         // Read the JSON file into json object
00062         std::ifstream comp_in(comp_json_name);
00063         if (!comp_in.is_open()) {
00064             spdlog::error("Could not open file '{}' for reading", comp_json_name);
00065             return;
00066         }
00067         try {
00068             comp_json_ = json::parse(comp_in);
00069         } catch (const json::parse_error &e) {
00070             spdlog::error("Error parsing comparison JSON file '{}': {}", comp_json_name, e.what());
00071             return;
00072         }
00073     }
00074
00075     ref_lib_path_ = ref_libfile.filepath_;
00076     comp_lib_path_ = comp_libfile.filepath_;
00077     spdlog::info("Successfully loaded JSON files for comparison");
00078 }
00079
00107 void LibraryComparator::compareLut(const std::string &cell_name, const std::string &pin_name,
00108                                   const std::string &timing_type, const std::string &related_pin,
00109                                   const std::string &arc_name, const json &ref_lut,
00110                                   const json &comp_lut, Table &table) {
00111     spdlog::info("Comparing LUT of '{}' ...", arc_name);
00112     std::vector<double> ref_index_1 = ref_lut["index_1"].get<std::vector<double>>();
00113     std::vector<double> ref_index_2 = ref_lut["index_2"].get<std::vector<double>>();
00114     std::vector<double> comp_index_1 = comp_lut["index_1"].get<std::vector<double>>();

```

```

00115     std::vector<double> comp_index_2 = comp_lut["index_2"].get<std::vector<double>>();
00116     if (ref_index_1 != comp_index_1 || ref_index_2 != comp_index_2) {
00117         spdlog::warn("Mismatch in LUT indices for '{}' in cell: '{}', pin: '{}', related_pin: '{}', "
00118             "timing_type: '{}'",
00119             arc_name, cell_name, pin_name, related_pin, timing_type);
00120         return;
00121     } else {
00122         spdlog::debug("LUT indices match for '{}'", arc_name);
00123
00124         std::vector<std::vector<double>> comp_value_matrix;
00125         // Generate a matrix of values from the JSON array for comparison library
00126         for (const auto &row_val : comp_lut["values"]) {
00127             std::vector<double> row_data;
00128             // Check if the row value is an array
00129             if (!row_val.is_array()) {
00130                 spdlog::error("Invalid LUT format: '{}' values should be an array of arrays in comp_lib '{}', "
00131                     "cell '{}', pin '{}', "
00132                     "related_pin '{}', timing_type '{}'",
00133                     arc_name, this->comp_lib_path_.string(), cell_name, pin_name, related_pin,
00134                     timing_type);
00135                 return;
00136             }
00137             // Check if non-numeric values
00138             for (const auto &val : row_val) {
00139                 if (val.is_number()) {
00140                     row_data.push_back(val.get<double>());
00141                 } else {
00142                     spdlog::error(
00143                         "Non-numeric value found in '{}'.values, skipping value: {} in comp_lib '{}', cell '{}', "
00144                         "pin '{}', related_pin '{}', timing_type '{}'",
00145                         arc_name, val.dump(), this->comp_lib_path_.string(), cell_name, pin_name, related_pin,
00146                         timing_type);
00147                     return;
00148                 }
00149             }
00150             comp_value_matrix.push_back(row_data);
00151         }
00152         // Generate a matrix of values from the JSON array for reference library
00153         std::vector<std::vector<double>> ref_value_matrix;
00154         for (const auto &row_val : ref_lut["values"]) {
00155             std::vector<double> row_data;
00156             // Check if the row value is an array
00157             if (!row_val.is_array()) {
00158                 spdlog::error("Invalid LUT format: '{}' values should be an array of arrays in ref_lib '{}', "
00159                     "cell '{}', pin '{}', "
00160                     "related_pin '{}', timing_type '{}'",
00161                     arc_name, this->ref_lib_path_.string(), cell_name, pin_name, related_pin,
00162                     timing_type);
00163                 return;
00164             }
00165             // Check if non-numeric values
00166             for (const auto &val : row_val) {
00167                 if (val.is_number()) {
00168                     row_data.push_back(val.get<double>());
00169                 } else {
00170                     spdlog::error(
00171                         "Non-numeric value found in '{}'.values, skipping value: {} in ref_lib '{}', cell '{}', "
00172                         "pin '{}', related_pin '{}', timing_type '{}'",
00173                         arc_name, val.dump(), this->ref_lib_path_.string(), cell_name, pin_name, related_pin,
00174                         timing_type);
00175                     return;
00176                 }
00177             }
00178             ref_value_matrix.push_back(row_data);
00179         }
00180         // Compare the matrix for relative tolerance
00181         if (!comp_value_matrix.empty() && !comp_value_matrix[0].empty()) {
00182             for (size_t i = 0; i < comp_value_matrix.size(); ++i) {
00183                 for (size_t j = 0; j < comp_value_matrix[i].size(); ++j) {

```

```

00184     double ref_val = ref_value_matrix[i][j];
00185     double comp_val = comp_value_matrix[i][j];
00186     if (std::abs(ref_val - comp_val) > reltol_ * std::abs(ref_val) &&
00187         std::abs(ref_val - comp_val) > abstol_) {
00188         spdlog::debug("LUT value mismatch for '{}', index_1: {}, index_2: {}", arc_name,
00189             ref_index_1[i], ref_index_2[j]);
00190         table.add_row({related_pin + "->" + pin_name, std::to_string(ref_val),
00191             std::to_string(comp_val), std::to_string(comp_val - ref_val),
00192             std::to_string((comp_val - ref_val) / ref_val * 100), timing_type, arc_name,
00193             std::to_string(i + 1), std::to_string(ref_index_1[i]), std::to_string(j + 1),
00194             std::to_string(ref_index_2[j]), "<"});
00195         spdlog::debug("table size: {}", table.size());
00196     } else {
00197         spdlog::debug("LUT value match for '{}', index_1: {}, index_2: {}", arc_name, ref_index_1[i],
00198             ref_index_2[j]);
00199     }
00200 }
00201 }
00202 } else {
00203     spdlog::error(
00204         "Empty or invalid 'values' array found for cell: '{}', pin: '{}', related_pin: '{}', "
00205         "timing_type: '{}', arc: '{}'",
00206         cell_name, pin_name, related_pin, timing_type, arc_name);
00207 }
00208 }
00209 }
00210
00229 void LibraryComparator::compareTimingArc(const std::string &cell_name, const std::string &pin_name,
00230     const std::string &timing_type, const json &ref_timing_arc,
00231     const json &comp_timing_arc, Table &table) {
00232     spdlog::info("Comparing timing type '{}' ...", timing_type);
00233
00234     std::string related_pin = ref_timing_arc["related_pin"].get<std::string>();
00235     spdlog::debug("Related pin: '{}'", related_pin);
00236
00237     std::vector<std::string> arc_names = {"cell_rise", "cell_fall", "rise_transition", "fall_transition"};
00238     for (auto arc_name : arc_names) {
00239         if (comp_timing_arc.contains(arc_name)) {
00240             spdlog::debug("Comparing timing arc: '{}'", arc_name);
00241             // Look for the arc in the reference JSON
00242             if (ref_timing_arc.contains(arc_name)) {
00243                 // Found this arc
00244                 spdlog::debug("Found timing arc: '{}'", arc_name);
00245                 compareLut(cell_name, pin_name, timing_type, related_pin, arc_name, ref_timing_arc[arc_name],
00246                     comp_timing_arc[arc_name], table);
00247             } else {
00248                 // arc not found in reference JSON
00249                 spdlog::warn("Timing arc: '{}' not found in reference JSON", arc_name);
00250             }
00251         }
00252     }
00253 }
00254
00269 void LibraryComparator::comparePin(const std::string &cell_name, const std::string &pin_name,
00270     const json &ref_pin, const json &comp_pin, Table &table) {
00271     spdlog::info("Comparing pin '{}' ...", pin_name);
00272
00273     if (comp_pin.contains("timing_arcs")) {
00274         for (const auto &comp_arc : comp_pin["timing_arcs"]) {
00275             std::string timing_type = comp_arc["timing_type"].get<std::string>();
00276             spdlog::debug("Comparing timing type: '{}'", timing_type);
00277             // Look for the timing type in the reference JSON
00278             auto ref_arc_it = std::find_if(
00279                 ref_pin["timing_arcs"].begin(), ref_pin["timing_arcs"].end(),
00280                 [&timing_type](const json &ref_arc) { return ref_arc["timing_type"] == timing_type; });
00281             if (ref_arc_it != ref_pin["timing_arcs"].end()) {
00282                 // Found this timing type
00283                 spdlog::debug("Found timing type: '{}'", timing_type);
00284                 compareTimingArc(cell_name, pin_name, timing_type, *ref_arc_it, comp_arc, table);

```

```

00285     } else {
00286         // timing arc not found in reference JSON
00287         spdlog::warn("Timing arc: '{}' not found in reference JSON", timing_type);
00288     }
00289 }
00290 } else {
00291     spdlog::info("No timing arcs found for pin: '{}'", comp_pin["pin_name"].get<std::string>());
00292 }
00293 }
00294
00308 void LibraryComparator::compareCell(const std::string &cell_name, const json &ref_cell,
00309                                     const json &comp_cell, Table &table) {
00310     spdlog::info("Comparing cell '{}' ...", cell_name);
00311
00312     if (comp_cell.contains("output_pins")) {
00313         for (const auto &comp_pin : comp_cell["output_pins"]) {
00314             std::string pin_name = comp_pin["pin_name"].get<std::string>();
00315             spdlog::debug("Comparing pin: '{}'", pin_name);
00316             // Look for the pin in the reference JSON
00317             auto ref_pin_it =
00318                 std::find_if(ref_cell["output_pins"].begin(), ref_cell["output_pins"].end(),
00319                             [&pin_name](const json &ref_pin) { return ref_pin["pin_name"] == pin_name; });
00320             if (ref_pin_it != ref_cell["output_pins"].end()) {
00321                 // Found this pin
00322                 spdlog::debug("Found pin: '{}'", pin_name);
00323                 comparePin(cell_name, pin_name, *ref_pin_it, comp_pin, table);
00324             } else {
00325                 // pin not found in reference JSON
00326                 spdlog::warn("Pin: '{}' not found in reference JSON", pin_name);
00327             }
00328         }
00329     } else {
00330         spdlog::info("No output pins found for cell: '{}'", cell_name);
00331     }
00332 }
00333
00350 void LibraryComparator::generateReport(const std::string &output_file) {
00351     std::ofstream outfile(output_file);
00352     outfile << "# LIBRARY comparison\n" << std::endl;
00353     outfile << "**Reference library: " << ref_lib_path_ << "**\n" << std::endl;
00354     outfile << "**Comparison library: " << comp_lib_path_ << "**\n" << std::endl;
00355     outfile << "**Absolute tolerance: " << abstol_ << "**\n" << std::endl;
00356     outfile << "**Relative tolerance: " << reltol_ << "**\n" << std::endl;
00357     outfile << "**Performed by " << APP_NAME << " v" << APP_VERSION << " from " << APP_AUTHOR;
00358     auto now = std::chrono::system_clock::now();
00359     std::time_t now_time_t = std::chrono::system_clock::to_time_t(now);
00360     std::tm *now_tm = std::localtime(&now_time_t);
00361     outfile << ". on: " << std::put_time(now_tm, "%c") << "**\n" << std::endl;
00362     outfile << "> Legend: < outlier, * scaled, ! indices switched, ^ slews extrapolated, ~ loads "
00363         "extrapolated,\n";
00364     outfile << "> \n";
00365     outfile
00366         << "> Legend: + padding added, /0 divide by zero, / slews interpolated, # loads interpolated\n";
00367     outfile << "> \n";
00368     outfile << "> Legend: << value is less but unknown, >> value is more but unknown\n\n";
00369
00370     spdlog::info("Starting comparison report generation ...");
00371     for (const auto &comp_cell : comp_json_["cells"]) {
00372         std::string cell_name = comp_cell["cell_name"].get<std::string>();
00373         spdlog::debug("Comparing cell: '{}'", cell_name);
00374         // Look for the cell in the reference JSON
00375         auto ref_cell_it =
00376             std::find_if(ref_json_["cells"].begin(), ref_json_["cells"].end(),
00377                         [&cell_name](const json &ref_cell) { return ref_cell["cell_name"] == cell_name; });
00378         if (ref_cell_it != ref_json_["cells"].end()) {
00379             // Found this cell
00380             spdlog::debug("Found cell: '{}'", cell_name);
00381             Table table;
00382             table.add_row({"Pin Name", "Reference", "Comparison", "Diff", "Diff %", "Type", "Arc Name",

```

```

00383         "Row #", "Index_1", "Col #", "Index_2", "Note"});
00384     // Header formatting
00385     for (size_t i = 0; i < table[0].size(); ++i) {
00386         table[0][i].format().font_color(Color::yellow).font_style({FontStyle::bold});
00387     }
00388     compareCell(cell_name, *ref_cell_it, comp_cell, table);
00389
00390     if (table.size() > 1) {
00391         // Data starts from row 1, row 0 is header
00392         size_t failed_count = table.size() - 1;
00393         double sum_diff = 0.0;
00394         double sum_diff_percent = 0.0;
00395         size_t outlier_count = 0;
00396         double max_diff = -1e9;
00397         double max_diff_percent = -1e9;
00398         std::vector<double> diff_values; // Store diff values for std deviation calculation
00399
00400         // Index of columns
00401         const int diff_index = 3;
00402         const int diff_percent_index = 4;
00403         const int note_index = 11;
00404
00405         for (size_t i = 1; i < table.size(); ++i) {
00406             try {
00407                 double diff = std::stod(table[i][diff_index].get_text());
00408                 double diff_percent = std::stod(table[i][diff_percent_index].get_text());
00409
00410                 sum_diff += diff;
00411                 sum_diff_percent += diff_percent;
00412                 diff_values.push_back(diff_percent);
00413
00414                 if (table[i][note_index].get_text() == "<") {
00415                     outlier_count++;
00416                 }
00417                 max_diff = std::max(max_diff, diff);
00418                 max_diff_percent = std::max(max_diff_percent, diff_percent);
00419
00420             } catch (const std::invalid_argument &e) {
00421                 spdlog::error("Could not convert value to double: {}", e.what());
00422                 continue; // Skip this row if there's an error
00423             }
00424         }
00425
00426         double avg_diff = sum_diff / failed_count;
00427         double avg_diff_percent = sum_diff_percent / failed_count;
00428
00429         outfile << "## " << cell_name << "\n\n";
00430         outfile << "Delay Comparison in ns\n\n";
00431         if (output_file.substr(output_file.size() - 3) == ".md") {
00432             MarkdownExporter exporter;
00433             auto markdown = exporter.dump(table);
00434             outfile << markdown << std::endl;
00435         } else {
00436             outfile << table << std::endl;
00437         }
00438         outfile << std::endl;
00439         std::cout << table << std::endl;
00440         outfile << cell_name << " delay SUMMARY (abstol: " << abstol_ << ", reltol: " << reltol_
00441             << ")\n";
00442
00443         // Create summary table
00444         Table summary_table;
00445         summary_table.add_row({"Cell Name", "Data Type", "Failed Count", "Avg Diff", "Avg Diff%",
00446             "Max Diff", "Max Diff%", "Outliers"});
00447         summary_table.add_row({cell_name, "delay(ns)", std::to_string(failed_count),
00448             std::to_string(avg_diff), std::to_string(avg_diff_percent) + "%",
00449             std::to_string(max_diff), std::to_string(max_diff_percent) + "%",
00450             std::to_string(outlier_count)});
00451

```

```

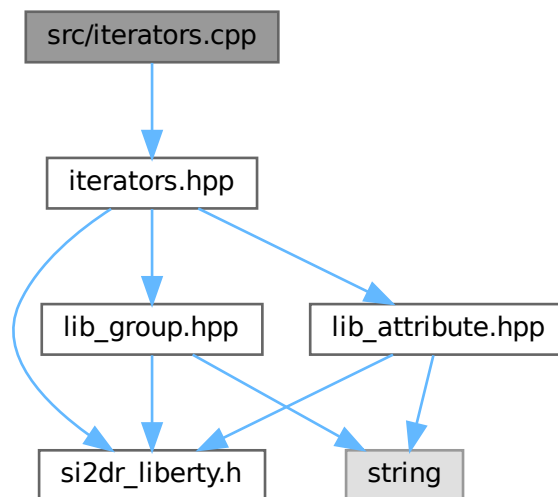
00452     // Output summary table
00453     if (output_file.substr(output_file.size() - 3) == ".md") {
00454         MarkdownExporter exporter;
00455         auto markdown = exporter.dump(summary_table);
00456         outfile << markdown << std::endl;
00457     } else {
00458         outfile << summary_table << std::endl;
00459     }
00460     outfile << "Worst delay outlier: Max Abs: " << max_diff << ", Max Rel: " << max_diff_percent
00461         << "%, Outliers: " << outlier_count << "\n\n";
00462 }
00463 } else {
00464     // cell not found in reference JSON
00465     spdlog::warn("Cell: '{}' not found in reference JSON", cell_name);
00466 }
00467 }
00468
00469 outfile.close();
00470 }

```

8.30 src/iterators.cpp File Reference

#include "iterators.hpp"

Include dependency graph for iterators.cpp:



8.31 iterators.cpp

[Go to the documentation of this file.](#)

```

00001 #include "iterators.hpp"
00002

```

```

00003 GroupsIterator::GroupsIterator(si2drGroupsIdT groups, si2drErrorT &err) : groups_(groups), err_(err) {
00004     group_ = si2drIterNextGroup(groups_, &err_);
00005 }
00006 GroupsIterator::~GroupsIterator() { si2drIterQuit(groups_, &err_); }
00007
00008 void GroupsIterator::next() { group_ = si2drIterNextGroup(groups_, &err_); }
00009 bool GroupsIterator::end() { return si2drObjectIsNull(group_, &err_); }
00010
00011 LibGroup GroupsIterator::get() { return LibGroup(group_, err_); }
00012
00013 AttributesIterator::AttributesIterator(si2drAttrsIdT attrs, si2drErrorT &err)
00014     : attrs_(attrs), err_(err) {
00015     attr_ = si2drIterNextAttr(attrs_, &err_);
00016 }
00017 AttributesIterator::~AttributesIterator() { si2drIterQuit(attrs_, &err_); }
00018
00019 void AttributesIterator::next() { attr_ = si2drIterNextAttr(attrs_, &err_); }
00020 bool AttributesIterator::end() { return si2drObjectIsNull(attr_, &err_); }
00021
00022 LibAttribute AttributesIterator::get() { return LibAttribute(attr_, err_); }
00023
00024 ValuesIterator::ValuesIterator(si2drValuesIdT values, si2drErrorT &err) : values_(values), err_(err) {
00025     si2drIterNextComplexValue(values_, &vtype_, &int_, &float_, &str_, &bool_, &exprp_, &err_);
00026 }
00027 ValuesIterator::~ValuesIterator() { si2drIterQuit(values_, &err_); }
00028
00029 void ValuesIterator::next() {
00030     si2drIterNextComplexValue(values_, &vtype_, &int_, &float_, &str_, &bool_, &exprp_, &err_);
00031 }
00032 bool ValuesIterator::end() { return vtype_ == SI2DR_UNDEFINED_VALUETYPE; }

```

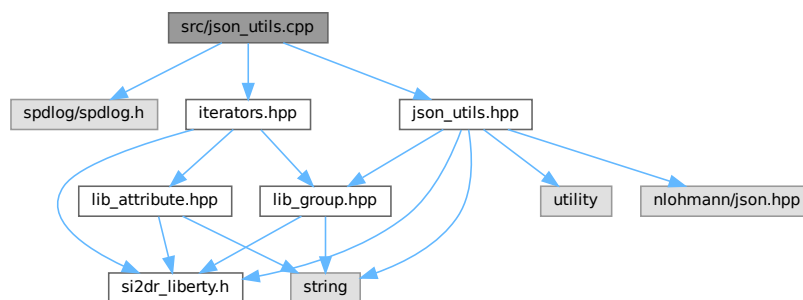
8.32 src/json_utils.cpp File Reference

```
#include "spdlog/spdlog.h"
```

```
#include "iterators.hpp"
```

```
#include "json_utils.hpp"
```

Include dependency graph for json_utils.cpp:



Functions

- `std::vector< double > parseStringToVector (const std::string &str)`

Parses a string representation of a vector of doubles into a vector of doubles.

- `json generateLutJson (LibGroup &lib_lut_group, si2drErrorT &err)`

Generates a JSON object representation of a look-up table (LUT) from a [LibGroup](#) object.

- `json generateTimingJson (LibGroup &lib_timing_group, si2drErrorT &err)`

Generates a JSON representation of timing information from a library timing group.

- `json generatePowerJson (LibGroup &lib_power_group, si2drErrorT &err)`

Converts a Liberty power group into a JSON representation.

- `std::pair< std::string, json > generatePinJson (LibGroup &lib_pin_group, si2drErrorT &err)`

Generates a JSON representation of a Liberty pin group.

- `json generateCellJson (LibGroup &lib_cell_group, si2drErrorT &err)`

Generates a JSON representation of a cell from a [LibGroup](#) object.

8.32.1 Function Documentation

8.32.1.1 generateCellJson()

```
json generateCellJson (
    LibGroup & lib_cell_group,
    si2drErrorT & err )
```

Generates a JSON representation of a cell from a [LibGroup](#) object.

This function processes a [LibGroup](#) representing a cell and converts it into a JSON object. It extracts the cell name and processes specific attributes such as area, cell_leakage_power, and cell_footprint. It also processes pins within the cell by categorizing them based on their direction (input, output, internal, inout).

Parameters

<i>lib_cell_group</i>	The LibGroup object representing the cell
<i>err</i>	Reference to a si2drErrorT object for error handling

Returns

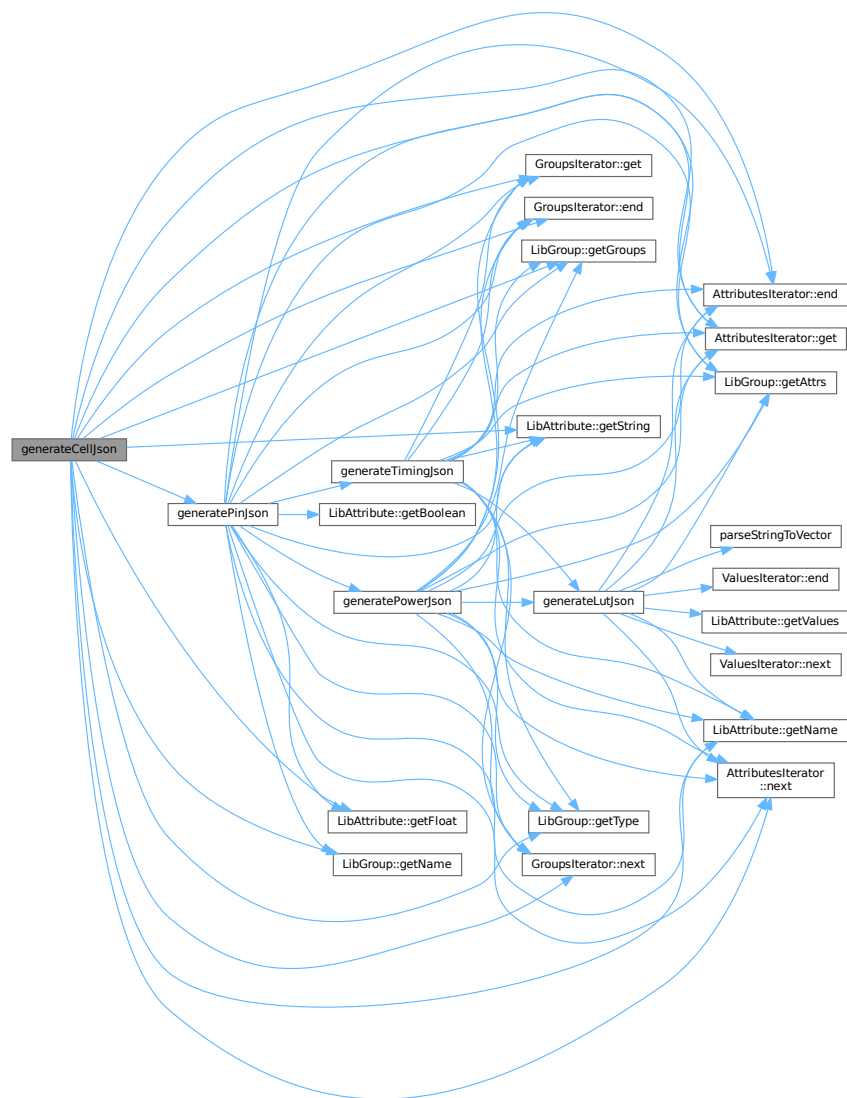
json A JSON object containing the cell's information with the following structure:

- "cell_name": string - name of the cell
- "area": float (optional) - cell area if present
- "cell_leakage_power": float (optional) - cell leakage power if present

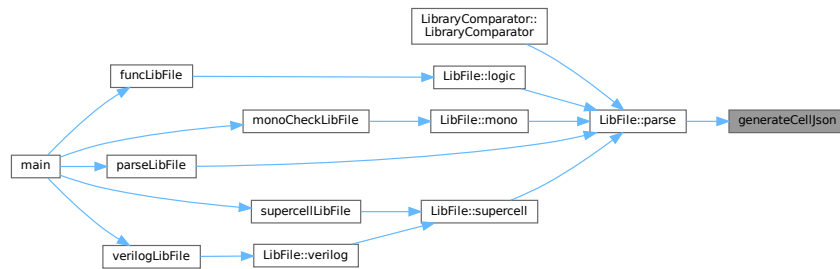
- "cell_footprint": string (optional) - cell footprint if present
- "input_pins": array - list of input pins
- "output_pins": array - list of output pins
- "internal_pins": array - list of internal pins
- "inout_pins": array - list of inout pins

Definition at line 272 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.32.1.2 generateLutJson()

```

json generateLutJson (
    LibGroup & lib_lut_group,
    si2drErrorT & err )

```

Generates a JSON object representation of a look-up table (LUT) from a [LibGroup](#) object.

This function iterates through the attributes of the provided [LibGroup](#) object representing a LUT and converts them into a JSON structure. It specifically handles the following attributes:

- "index_1": Converted to a vector and stored in the JSON
- "index_2": Converted to a vector and stored in the JSON
- "values": Each value is parsed into a vector and added to an array in the JSON

Any other attributes encountered will trigger a warning message.

Parameters

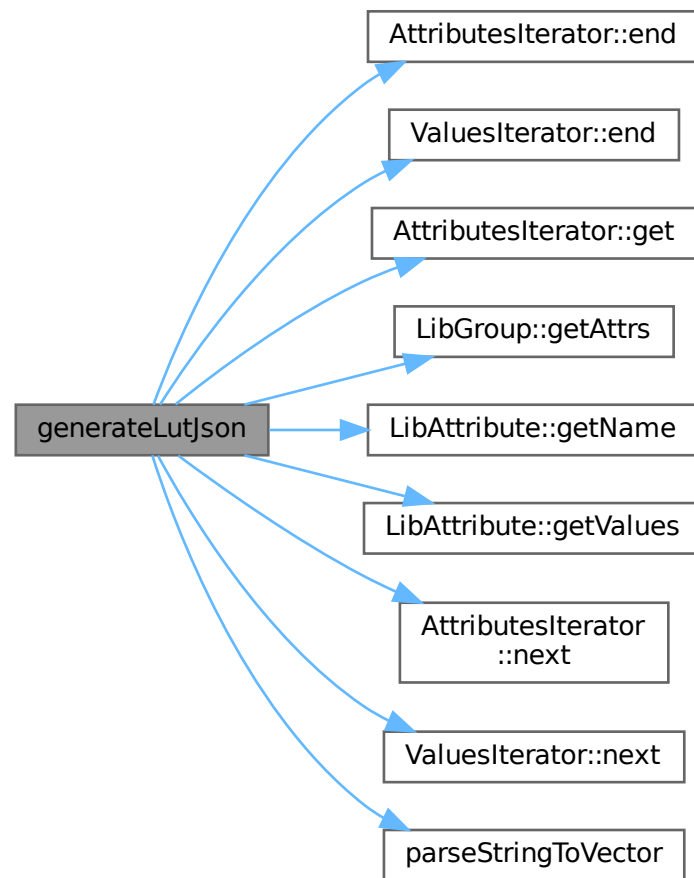
<i>lib_lut_group</i>	The LibGroup object containing the LUT data to be converted
<i>err</i>	Reference to an <code>si2drErrorT</code> object to track any errors during processing

Returns

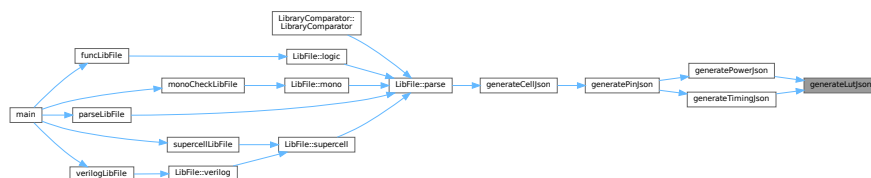
json A JSON object representing the LUT data with keys for "index_1", "index_2", and "values"

Definition at line 62 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.32.1.3 generatePinJson()

```
std::pair< std::string, json > generatePinJson (
```

```
LibGroup & lib_pin_group,  
si2drErrorT & err )
```

Generates a JSON representation of a Liberty pin group.

This function traverses a Liberty pin group, extracts relevant attributes and sub-groups, and converts them into a JSON object. It also determines the pin's direction.

Processes the following pin attributes:

- direction
- max_transition, capacitance, rise_capacitance, fall_capacitance, max_capacitance (float types)
- function, power_down_function, related_ground_pin, related_power_pin, three_state (string types)
- clock (boolean type)

Handles the following sub-groups:

- internal_power: Converted using [generatePowerJson\(\)](#)
- timing: Converted using [generateTimingJson\(\)](#)

Parameters

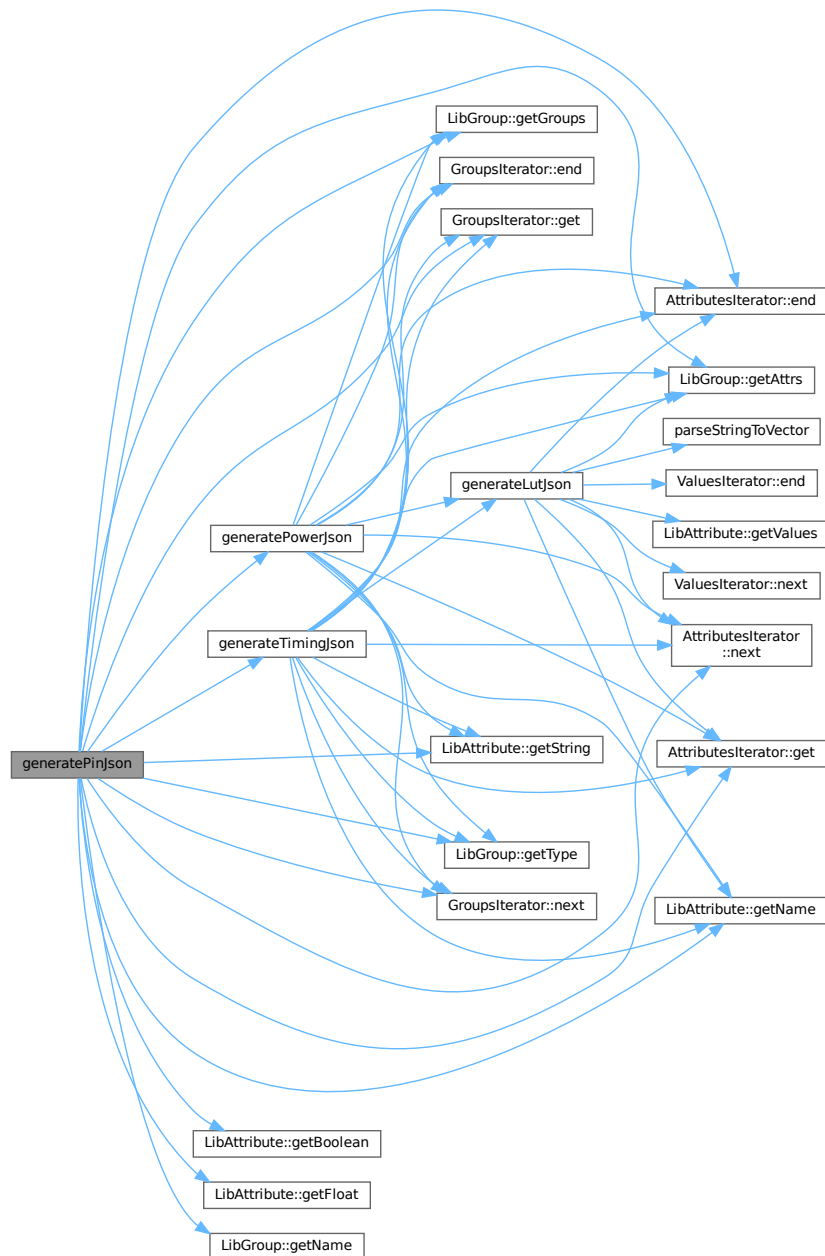
<i>lib_pin_group</i>	The Liberty pin group to process
<i>err</i>	Reference to an error object for tracking Liberty API errors

Returns

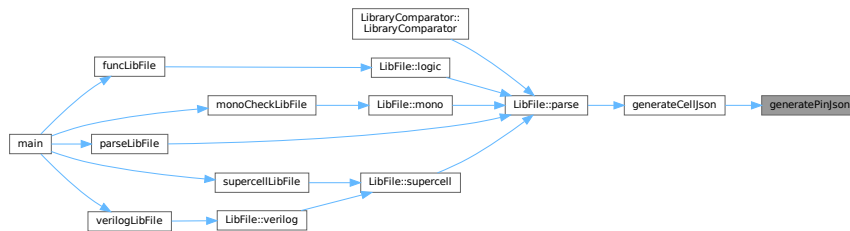
A pair containing the pin direction (string) and the JSON representation of the pin

Definition at line 206 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.32.1.4 generatePowerJson()

```

json generatePowerJson (
    LibGroup & lib_power_group,
    si2drErrorT & err )

```

Converts a Liberty power group into a JSON representation.

This function processes a Liberty power group and converts its attributes and subgroups into a JSON object. It handles attributes like "when", "related_pin", and "related_pg_pin", as well as "rise_power" and "fall_power" subgroups.

Parameters

<i>lib_power_group</i>	The Liberty power group to convert
<i>err</i>	Reference to an si2drErrorT object for error handling

Returns

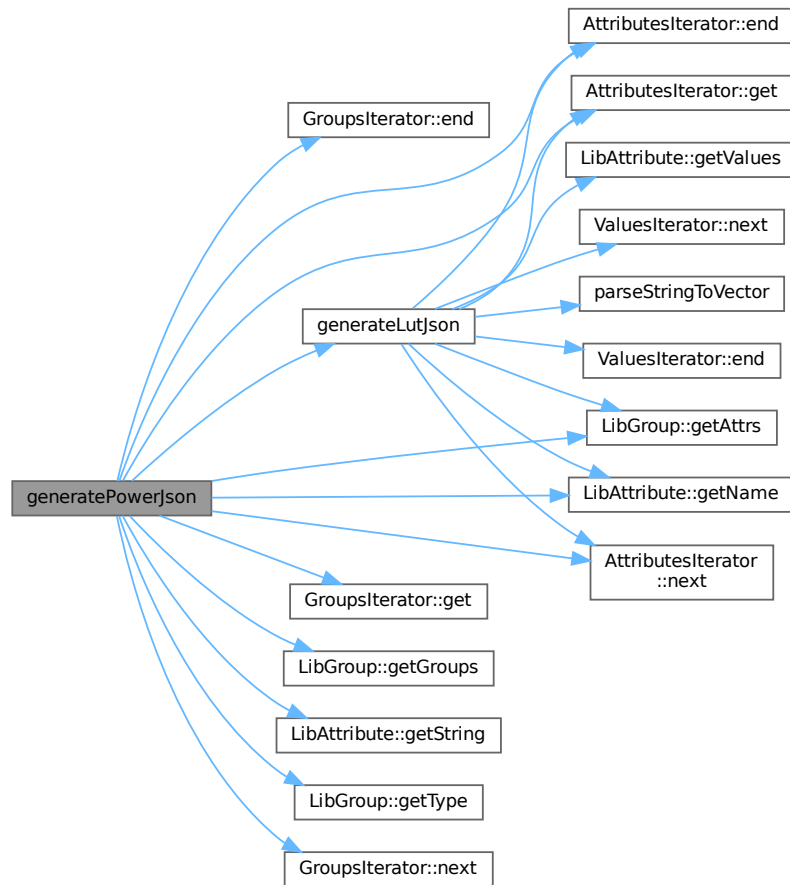
json A JSON object representing the power group data

The function:

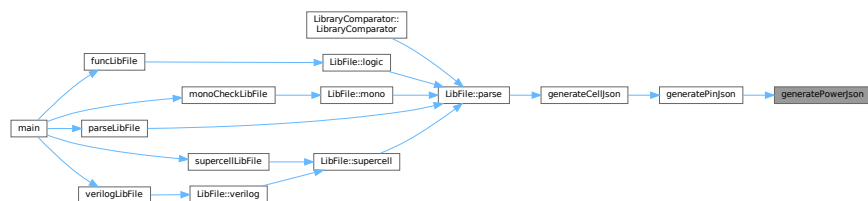
- Extracts string attributes (when, related_pin, related_pg_pin)
- Processes rise_power and fall_power subgroups by converting them to LUT JSON format
- Logs warnings for unknown power subgroup types

Definition at line 154 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.32.1.5 generateTimingJson()

```

json generateTimingJson (
    LibGroup & lib_timing_group,
    si2drErrorT & err )

```

Generates a JSON representation of timing information from a library timing group.

This function extracts timing attributes and sub-groups from a Liberty timing group and organizes them into a JSON object. It processes standard timing attributes like 'related_pin', 'timing_type', 'timing_sense', and 'when', as well as timing tables such as 'cell_fall', 'cell_rise', 'fall_transition', 'rise_transition', 'fall_constraint', and 'rise_constraint'.

Parameters

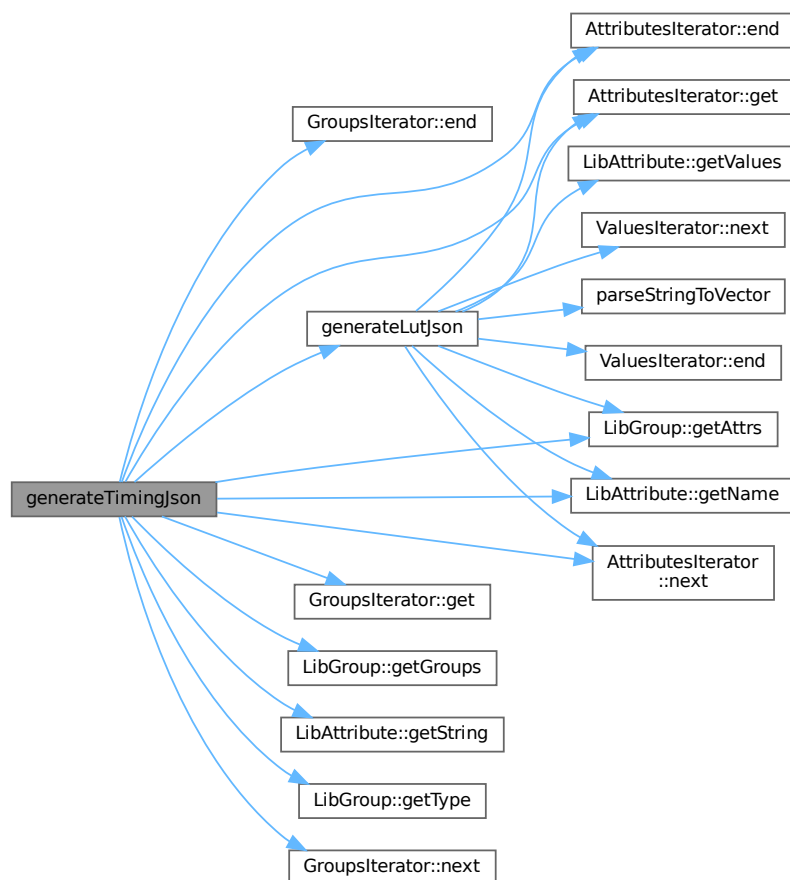
<i>lib_timing_group</i>	The Liberty timing group to process
<i>err</i>	Reference to an si2drErrorT object for error reporting

Returns

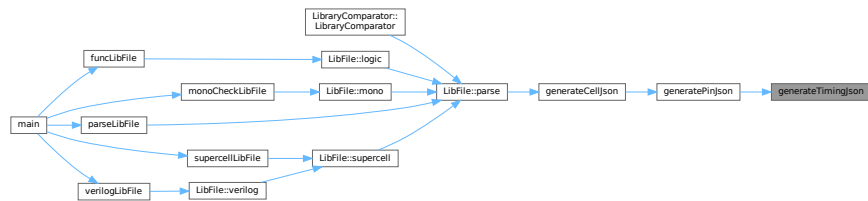
json A JSON object containing the extracted timing information

Definition at line 106 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.32.1.6 parseStringToVector()

```
std::vector< double > parseStringToVector (
    const std::string & str )
```

Parses a string representation of a vector of doubles into a vector of doubles.

This function takes a string containing comma-separated numbers, cleans it by removing backslashes and newline characters, and then converts each comma-separated value into a double that is added to the resulting vector.

Parameters

<i>str</i>	The input string to be parsed, containing comma-separated numbers
------------	---

Returns

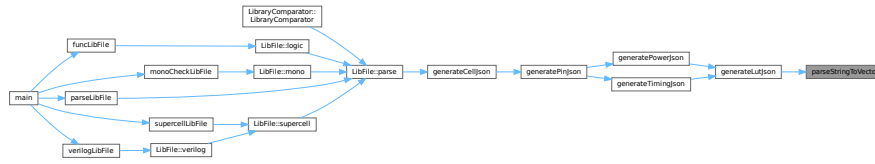
`std::vector<double>` A vector of doubles parsed from the input string

Exceptions

<code>std::invalid_argument</code>	If the string contains values that cannot be converted to double
<code>std::out_of_range</code>	If the string contains values that are out of range for double

Definition at line 28 of file [json_utils.cpp](#).

Here is the caller graph for this function:



8.33 json_utils.cpp

[Go to the documentation of this file.](#)

```

00001 #include "spdlog/spdlog.h"
00002
00003 #include "iterators.hpp"
00004 #include "json_utils.hpp"
00005
00006 /*
00007 json 类型的值可以是以下几种类型之一：
00008 方括号 []：用于包含一组有序的值（数组）。
00009 数组中的每个值可以是任意类型的 JSON 值，包括对象、数组、字符串、数字、布尔值或 null。
00010 花括号 {}：用于包含一组键值对（对象）。
00011 对象中的每个键都是一个字符串，值可以是任意类型的 JSON 值，包括对象、数组、字符串、数字、布尔值或
00012 null。
00013 */
00014
00028 std::vector<double> parseStringToVector(const std::string &str) {
00029     std::vector<double> result;
00030     std::string cleaned_str;
00031
00032     // Remove backslashes and newline characters
00033     for (char c : str) {
00034         if (c != '\\' && c != '\n') {
00035             cleaned_str += c;
00036         }
00037     }
00038
00039     std::stringstream ss(cleaned_str);
00040     std::string item;
00041     while (std::getline(ss, item, ',')) {
00042         result.push_back(std::stod(item));
00043     }
00044     return result;
00045 }
00046
00062 json generateLutJson(LibGroup &lib_lut_group, si2drErrorT &err) {
00063     json lut_json;
00064
00065     AttributesIterator attr_iter(lib_lut_group.getAttrs(), err);
00066     for (; !attr_iter.end(); attr_iter.next()) {
00067         LibAttribute lib_attr = attr_iter.get();
00068         std::string lut_attr_name = lib_attr.getName();
00069
00070         // spdlog::debug("LUT Attribute Name: {}", lib_attr.getName());
00071         // spdlog::debug("Is Complex? {}", lib_attr.isComplex());
00072
00073         if (lut_attr_name == "index_1" || lut_attr_name == "index_2") {
00074             ValuesIterator values_iter(lib_attr.getValues(), err);
00075             for (; !values_iter.end(); values_iter.next()) {
00076                 // spdlog::debug("Type: {}", int(values_iter.vtype_)); // 5 is string
00077                 // spdlog::debug("Str: {}", values_iter.str_);
00078             }
00079         }
00080     }
00081     return lut_json;
00082 }

```

```

00078         lut_json[lut_attr_name] = parseStringToVector(std::string(values_iter.str_));
00079     }
00080 } else if (lut_attr_name == "values") {
00081     ValuesIterator values_iter(lib_attr.getValues(), err);
00082     for (; !values_iter.end(); values_iter.next()) {
00083         // spdlog::debug("{}", values_iter.str_);
00084         lut_json["values"].push_back(parseStringToVector(std::string(values_iter.str_)));
00085     }
00086 } else {
00087     spdlog::warn("Unknown LUT attribute name: {}", lut_attr_name);
00088 }
00089 }
00090 return lut_json;
00091 }
00092
00106 json generateTimingJson(LibGroup &lib_timing_group, si2drErrorT &err) {
00107     json timing_json;
00108
00109     AttributesIterator attr_iter(lib_timing_group.getAttrs(), err);
00110     for (; !attr_iter.end(); attr_iter.next()) {
00111         LibAttribute lib_attr = attr_iter.get();
00112
00113         std::string attr_name = lib_attr.getName();
00114         if (attr_name == "related_pin" || attr_name == "timing_type" || attr_name == "timing_sense" ||
00115             attr_name == "when") {
00116             timing_json[attr_name] = lib_attr.getString();
00117         }
00118         // More timing attributes can be added here
00119     }
00120
00121     GroupsIterator timing_sub_group_iter(lib_timing_group.getGroups(), err);
00122     for (; !timing_sub_group_iter.end(); timing_sub_group_iter.next()) {
00123         LibGroup lib_timing_sub_group = timing_sub_group_iter.get();
00124
00125         std::string timing_sub_group_type = lib_timing_sub_group.getType();
00126         // std::string timing_sub_group_name = lib_timing_sub_group.getName();
00127         if (timing_sub_group_type == "cell_fall" || timing_sub_group_type == "cell_rise" ||
00128             timing_sub_group_type == "fall_transition" || timing_sub_group_type == "rise_transition" ||
00129             timing_sub_group_type == "fall_constraint" || timing_sub_group_type == "rise_constraint") {
00130             timing_json[timing_sub_group_type] = generateLutJson(lib_timing_sub_group, err);
00131         } else {
00132             spdlog::warn("Unknown timing sub group type: {}", timing_sub_group_type);
00133         }
00134     }
00135     return timing_json;
00136 }
00137
00154 json generatePowerJson(LibGroup &lib_power_group, si2drErrorT &err) {
00155     json power_json;
00156
00157     AttributesIterator attr_iter(lib_power_group.getAttrs(), err);
00158     for (; !attr_iter.end(); attr_iter.next()) {
00159         LibAttribute lib_attr = attr_iter.get();
00160
00161         std::string attr_name = lib_attr.getName();
00162         if (attr_name == "when" || attr_name == "related_pin" || attr_name == "related_pg_pin") {
00163             power_json[attr_name] = lib_attr.getString();
00164         }
00165         // More power attributes can be added here
00166     }
00167
00168     GroupsIterator power_sub_group_iter(lib_power_group.getGroups(), err);
00169     for (; !power_sub_group_iter.end(); power_sub_group_iter.next()) {
00170         LibGroup lib_power_sub_group = power_sub_group_iter.get();
00171
00172         std::string power_sub_group_type = lib_power_sub_group.getType();
00173         // std::string power_sub_group_name = lib_power_sub_group.getName();
00174         if (power_sub_group_type == "rise_power") {
00175             // spdlog::debug("Rise Power: {}", power_sub_group_name);

```

```

00176     power_json["rise_power"] = generateLutJson(lib_power_sub_group, err);
00177 } else if (power_sub_group_type == "fall_power") {
00178     power_json["fall_power"] = generateLutJson(lib_power_sub_group, err);
00179 } else {
00180     spdlog::warn("Unknown power sub group type: {}", power_sub_group_type);
00181 }
00182 }
00183 return power_json;
00184 }
00185
00206 std::pair<std::string, json> generatePinJson(LibGroup &lib_pin_group, si2drErrorT &err) {
00207     json pin_json;
00208     std::string direction;
00209     pin_json["pin_name"] = lib_pin_group.getName();
00210
00211     AttributesIterator attr_iter(lib_pin_group.getAttrs(), err);
00212     for (; !attr_iter.end(); attr_iter.next()) {
00213         LibAttribute lib_attr = attr_iter.get();
00214
00215         std::string attr_name = lib_attr.getName();
00216         if (attr_name == "direction") {
00217             direction = lib_attr.getString();
00218         } else if (attr_name == "max_transition" || attr_name == "capacitance" ||
00219                 attr_name == "rise_capacitance" || attr_name == "fall_capacitance" ||
00220                 attr_name == "max_capacitance") {
00221             pin_json[attr_name] = lib_attr.getFloat();
00222         } else if (attr_name == "function" || attr_name == "power_down_function" ||
00223                 attr_name == "related_ground_pin" || attr_name == "related_power_pin" ||
00224                 attr_name == "three_state") {
00225             pin_json[attr_name] = lib_attr.getString();
00226         } else if (attr_name == "clock") {
00227             pin_json[attr_name] = lib_attr.getBoolean();
00228         }
00229         // More pin attributes can be added here
00230     }
00231
00232     GroupsIterator pin_sub_group_iter(lib_pin_group.getGroups(), err);
00233     for (; !pin_sub_group_iter.end(); pin_sub_group_iter.next()) {
00234         LibGroup lib_pin_sub_group = pin_sub_group_iter.get();
00235
00236         std::string pin_sub_group_type = lib_pin_sub_group.getType();
00237         // std::string pin_sub_group_name = lib_pin_sub_group.getName();
00238         if (pin_sub_group_type == "internal_power") {
00239             json power_json = generatePowerJson(lib_pin_sub_group, err);
00240             pin_json["power_arcs"].push_back(power_json);
00241         } else if (pin_sub_group_type == "timing") {
00242             // spdlog::debug("Has Timing: {}", lib_pin_group.getName());
00243             json timing_json = generateTimingJson(lib_pin_sub_group, err);
00244             pin_json["timing_arcs"].push_back(timing_json);
00245         } else {
00246             spdlog::warn("Unknown pin sub group type: {}", pin_sub_group_type);
00247         }
00248     }
00249     return std::make_pair(direction, pin_json);
00250 }
00251
00272 json generateCellJson(LibGroup &lib_cell_group, si2drErrorT &err) {
00273     json cell_json;
00274     cell_json["cell_name"] = lib_cell_group.getName();
00275
00276     AttributesIterator attr_iter(lib_cell_group.getAttrs(), err);
00277     for (; !attr_iter.end(); attr_iter.next()) {
00278         LibAttribute lib_attr = attr_iter.get();
00279
00280         std::string attr_name = lib_attr.getName();
00281         if (attr_name == "area" || attr_name == "cell_leakage_power") {
00282             cell_json[attr_name] = lib_attr.getFloat();
00283         } else if (attr_name == "cell_footprint") {
00284             cell_json[attr_name] = lib_attr.getString();

```

```

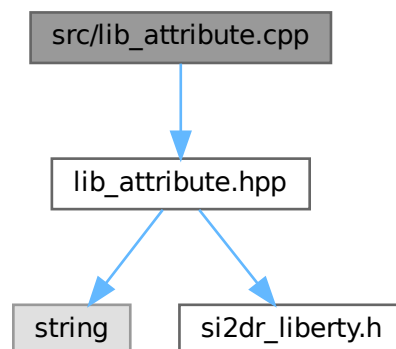
00285     }
00286     // More cell attributes can be added here
00287 }
00288
00289 GroupsIterator cell_sub_group_iter(lib_cell_group.getGroups(), err);
00290 for (; !cell_sub_group_iter.end(); cell_sub_group_iter.next()) {
00291     LibGroup lib_cell_sub_group = cell_sub_group_iter.get();
00292
00293     std::string cell_sub_group_type = lib_cell_sub_group.getType();
00294     // std::string cell_sub_group_name = lib_cell_sub_group.getName();
00295
00296     // spdlog::debug("Cell Sub Group Type: {}", cell_sub_group_type);
00297     // spdlog::debug("Cell Sub Group Name: {}", cell_sub_group_name);
00298
00299     if (cell_sub_group_type == "pin") {
00300         auto [direction, pin_json] = generatePinJson(lib_cell_sub_group, err);
00301         if (direction == "input") {
00302             cell_json["input_pins"].push_back(pin_json);
00303         } else if (direction == "output") {
00304             cell_json["output_pins"].push_back(pin_json);
00305         } else if (direction == "internal") {
00306             cell_json["internal_pins"].push_back(pin_json);
00307         } else if (direction == "inout") {
00308             cell_json["inout_pins"].push_back(pin_json);
00309         } else {
00310             spdlog::warn("Unknown direction: {}", direction);
00311         }
00312     }
00313 }
00314 return cell_json;
00315 }

```

8.34 src/lib_attribute.cpp File Reference

```
#include "lib_attribute.hpp"
```

Include dependency graph for lib_attribute.cpp:



8.37 lib_file.cpp

[Go to the documentation of this file.](#)

```

00001 #include <chrono>
00002 #include <filesystem>
00003 #include <fstream>
00004 #include <sstream>
00005 #include <unordered_set>
00006
00007 #include "spdlog/sinks/basic_file_sink.h"
00008 #include "spdlog/sinks/stdout_color_sinks.h"
00009 #include "spdlog/spdlog.h"
00010
00011 #include "iterators.hpp"
00012 #include "json_utils.hpp"
00013 #include "lib_file.hpp"
00014 #include "verilog_utils.hpp"
00015
00028 LibFile::LibFile(const std::string &filepath, const std::string &loggername)
00029     : filepath_(filepath), loggername_(loggername) {
00030     filename_ = filepath_.string();
00031     basename_ = filepath_.stem().string();
00032     jsonname_ = basename_ + ".json";
00033
00034     auto console_sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
00035     console_sink->set_level(spdlog::level::info);
00036
00037     auto file_sink = std::make_shared<spdlog::sinks::basic_file_sink_mt>(loggername_, true);
00038     file_sink->set_level(spdlog::level::debug);
00039
00040     std::vector<spdlog::sink_ptr> sinks{console_sink, file_sink};
00041     logger_ = std::make_shared<spdlog::logger>(loggername_, sinks.begin(), sinks.end());
00042     logger_->set_level(spdlog::level::debug);
00043
00044     logger_->info("Created LibFile object for '{}'", filename_);
00045     logger_->info("Debug log in: '{}', loggername_");
00046 }
00047
00048 LibFile::~LibFile() { logger_->info("Closing file: '{}', filename_"); }
00049
00059 void LibFile::writeJsonToFile() {
00060     std::ofstream out(jsonname_);
00061     if (!out.is_open()) {
00062         logger_->error("Could not open file '{}' for writing", jsonname_);
00063         return;
00064     }
00065     out << lib_json_.dump(2);
00066     out.close();
00067     logger_->info("JSON data written to '{}', jsonname_");
00068 }
00069
00090 void LibFile::read() {
00091     logger_->info("Reading '{} ...', filename_");
00092
00093     auto start = std::chrono::high_resolution_clock::now();
00094     si2drReadLibertyFile(const_cast<char *>(filepath_.c_str()), &err_);
00095     auto end = std::chrono::high_resolution_clock::now();
00096     std::chrono::duration<double> duration = end - start;
00097
00098     if (err_ == SI2DR_INVALID_NAME) {
00099         logger_->error("Could not open file '{}' for parsing, quitting...", filename_);
00100         exit(301);
00101     } else if (err_ == SI2DR_SYNTAX_ERROR) {
00102         logger_->error("Syntax Errors were detected in the input file!");
00103         exit(401);
00104     } else {
00105         logger_->info("Done. Read time: {:.2f} seconds", duration.count());

```

```

00106 }
00107 }
00108
00129 void LibFile::parse() {
00130     si2drPIInit(&err_); // Initialize private error handler
00131     this->read();        // Read the Liberty file
00132
00133     logger_>info("Parsing '{}' ...", filename_);
00134     // Create a scope for the top-level groups iteration
00135     {
00136         GroupsIterator group_iter(si2drPIGetGroups(&err_), err_);
00137         for (; !group_iter.end(); group_iter.next()) {
00138             LibGroup lib_group = group_iter.get();
00139
00140             if (lib_group.getName() != "") {
00141                 libname_ = lib_group.getName();
00142                 lib_json["library_name"] = this->libname_;
00143                 logger_>info("Library Name: {}", libname_);
00144             } else {
00145                 logger_>warn("Library Name: <NONAME>");
00146             }
00147             // Second level groups
00148             GroupsIterator sub_group_iter(lib_group.getGroups(), err_);
00149             for (; !sub_group_iter.end(); sub_group_iter.next()) {
00150                 LibGroup lib_sub_group = sub_group_iter.get();
00151
00152                 std::string sub_group_type = lib_sub_group.getType();
00153                 std::string sub_group_name = lib_sub_group.getName();
00154
00155                 if (sub_group_type == "cell") {
00156                     logger_>debug("Cell Name: {}", sub_group_name);
00157                     // if (sub_group_name != "AN2D0") {
00158                     //     continue;
00159                     // }
00160                     // Handle each cell
00161                     json cell_json = generateCellJson(lib_sub_group, err_);
00162                     lib_json["cells"].push_back(cell_json);
00163                 } else if (sub_group_type == "operating_conditions") {
00164                     logger_>info("Operating Conditions: {}", sub_group_name);
00165                     // Get PVT from Operating Conditions
00166                     AttributesIterator attr_iter(lib_sub_group.getAttrs(), err_);
00167                     for (; !attr_iter.end(); attr_iter.next()) {
00168                         LibAttribute lib_attr = attr_iter.get();
00169                         logger_>debug("Attribute Name: {}", lib_attr.getName());
00170                         logger_>debug("Int Value: {}", lib_attr.getInt());
00171                         logger_>debug("Float Value: {}", lib_attr.getFloat());
00172                         logger_>debug("String Value: {}", lib_attr.getString());
00173                         if (lib_attr.getName() == "process") {
00174                             process_ = lib_attr.getInt();
00175                             lib_json["process"] = process_;
00176                         } else if (lib_attr.getName() == "voltage") {
00177                             voltage_ = lib_attr.getFloat();
00178                             // Round to 2 decimal places to avoid floating-point precision issues
00179                             lib_json["voltage"] = std::round(voltage_ * 100) / 100.0;
00180                         } else if (lib_attr.getName() == "temperature") {
00181                             temperature_ = lib_attr.getInt();
00182                             lib_json["temperature"] = temperature_;
00183                         }
00184                     }
00185                     logger_>info("P: {}, V: {}, T: {}", process_, voltage_, temperature_);
00186                 }
00187                 // sub_group_iter's lifetime ends here, and the destructor is called
00188             }
00189         }
00190         // group_iter's lifetime ends here, and the destructor is called
00191     }
00192     si2drPIQuit(&err_);
00193 }
00194

```



```

00195 void LibFile::modify() { logger_>info("Modifying the file..."); }
00196
00218 bool LibFile::checkTimingArcMonotonicity(const json &cell, const json &pin, const json &arc,
00219                                           const std::string &timing_arc_name, const bool is_slew) {
00220     if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
00221         logger_>debug(
00222             "Checking cell: '{}', pin: '{}', related_pin: '{}', timing_arc: '{}', when: '{}'",
00223             cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
00224             arc["related_pin"].get<std::string>(), timing_arc_name, arc["when"].get<std::string>());
00225     } else {
00226         logger_>debug("Checking cell: '{}', pin: '{}', related_pin: '{}', timing_arc: '{}'",
00227             cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
00228             arc["related_pin"].get<std::string>(), timing_arc_name);
00229     }
00230     bool is_monotonic = true; // Assume the values are monotonic initially
00231     if (arc.contains(timing_arc_name) && arc[timing_arc_name].contains("values")) {
00232         std::vector<std::vector<double>> value_matrix;
00233         // Generate a matrix of values from the JSON array
00234         for (const auto &row_val : arc[timing_arc_name]["values"]) {
00235             std::vector<double> row_data;
00236             // Check if the value is an array
00237             if (!row_val.is_array()) {
00238                 logger_>error("Invalid format: '{}' values should be an array of arrays in cell '{}', pin "
00239                     "'{}', related_pin '{}'",
00240                     timing_arc_name, cell["cell_name"].get<std::string>(),
00241                     pin["pin_name"].get<std::string>(), arc["related_pin"].get<std::string>());
00242                 return false;
00243             }
00244             // Check for non-numeric values
00245             for (const auto &val : row_val) {
00246                 if (val.is_number()) {
00247                     row_data.push_back(val.get<double>());
00248                 } else {
00249                     logger_>warn("Non-numeric value found in '{}'.values, skipping value: {} in cell '{}', "
00250                         "pin '{}', related_pin '{}'",
00251                         timing_arc_name, val.dump(), cell["cell_name"].get<std::string>(),
00252                         pin["pin_name"].get<std::string>(), arc["related_pin"].get<std::string>());
00253                 }
00254             }
00255             value_matrix.push_back(row_data);
00256         }
00257         // Check the matrix for monotonicity
00258         if (!value_matrix.empty() && !value_matrix[0].empty()) {
00259             // Check the monotonic incrementality of rows (by output load capacitance, index 2)
00260             for (size_t i = 0; i < value_matrix.size(); ++i) {
00261                 for (size_t j = 1; j < value_matrix[i].size(); ++j) {
00262                     if ((value_matrix[i][j] < value_matrix[i][j - 1] &&
00263                         pin["pin_name"] != arc["related_pin"]) ||
00264                         (value_matrix[i][j] == 0 && value_matrix[i][j - 1] == 0)) {
00265                         // If contains "when" key, log the when value
00266                         if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
00267                             logger_>warn("Non-monotonic (by load) '{}' values: ({}), ({}), {} < ({}), ({}), {} "
00268                                 "for Cell: {} Pin: {}->{} when: '{}'",
00269                                 timing_arc_name, i, j, value_matrix[i][j], i, j - 1,
00270                                 value_matrix[i][j - 1], cell["cell_name"].get<std::string>(),
00271                                 arc["related_pin"].get<std::string>(),
00272                                 pin["pin_name"].get<std::string>(), arc["when"].get<std::string>());
00273                         } else {
00274                             logger_>warn("Non-monotonic (by load) '{}' values: ({}), ({}), {} < ({}), ({}), {} "
00275                                 "for Cell: {} Pin: {}->{}",
00276                                 timing_arc_name, i, j, value_matrix[i][j], i, j - 1,
00277                                 value_matrix[i][j - 1], cell["cell_name"].get<std::string>(),
00278                                 arc["related_pin"].get<std::string>(),
00279                                 pin["pin_name"].get<std::string>());
00280                         }
00281                         is_monotonic = false;
00282                     }
00283                 }
00284             }

```

```

00285     if (is_slew) {
00286         // Check the monotonic incrementality of columns (by input slew, index 1)
00287         for (size_t j = 0; j < value_matrix[0].size(); ++j) {
00288             for (size_t i = 1; i < value_matrix.size(); ++i) {
00289                 if ((value_matrix[i][j] < value_matrix[i - 1][j] &&
00290                     pin["pin_name"] != arc["related_pin"]) ||
00291                     (value_matrix[i][j] == 0 && value_matrix[i - 1][j] == 0)) {
00292                     // If contains "when" key, log the when value
00293                     if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
00294                         logger_>warn("Non-monotonic (by slew) '{}' values: ({}, {}) {} < ({}, {}) {} "
00295                                     "for Cell: {} Pin: {}->{} when: \"{}\"",
00296                                     timing_arc_name, i, j, value_matrix[i][j], i - 1, j,
00297                                     value_matrix[i - 1][j], cell["cell_name"].get<std::string>(),
00298                                     arc["related_pin"].get<std::string>(),
00299                                     pin["pin_name"].get<std::string>(), arc["when"].get<std::string>());
00300                     } else {
00301                         logger_>warn("Non-monotonic (by slew) '{}' values: ({}, {}) {} < ({}, {}) {} "
00302                                     "for Cell: {} Pin: {}->{}",
00303                                     timing_arc_name, i, j, value_matrix[i][j], i - 1, j,
00304                                     value_matrix[i - 1][j], cell["cell_name"].get<std::string>(),
00305                                     arc["related_pin"].get<std::string>(),
00306                                     pin["pin_name"].get<std::string>());
00307                     }
00308                     is_monotonic = false;
00309                 }
00310             }
00311         }
00312     }
00313 } else {
00314     logger_>warn(
00315         "Empty or invalid 'values' array found for cell: '{}', pin: '{}', related_pin: '{}', "
00316         "timing_arc: '{}'",
00317         cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
00318         arc["related_pin"].get<std::string>(), timing_arc_name);
00319 }
00320 }
00321 return is_monotonic;
00322 }
00323
00344 void LibFile::mono(const bool is_slew) {
00345     /*
00346     * Use this command to check the following data in the current library to ensure
00347     * the tables are monotonically increasing with respect to output load:
00348     * cell_rise retaining_rise
00349     * cell_fall retaining_fall
00350     * rise_transition retain_rise_slew
00351     * fall_transition retain_fall_slew
00352     * mpw
00353     */
00354     logger_>info("Monotonicity check of '{}' starting ...", filename_);
00355
00356     if (!std::filesystem::exists(jsonname_)) {
00357         logger_>info("JSON file not found. Parsing Liberty file first.");
00358         this->parse();
00359         this->writeJsonToFile();
00360     } else {
00361         // Read the JSON file into json object
00362         std::ifstream in(jsonname_);
00363         if (!in.is_open()) {
00364             logger_>error("Could not open file '{}' for reading", jsonname_);
00365             return;
00366         }
00367         try {
00368             lib_json_ = json::parse(in);
00369         } catch (const json::parse_error &e) {
00370             logger_>error("JSON parsing error in file '{}': {}", jsonname_, e.what());
00371             in.close();
00372             return;
00373         }

```

```

00374     in.close();
00375 }
00376
00377 std::map<std::string, bool> cell_monotonicity_status; // Track pass/fail status for each cell
00378 std::vector<std::string> failed_cells;                // List of failed cell names
00379 int total_cells = 0;
00380 int passed_cells = 0;
00381
00382 // Check the monotonicity of delay values
00383 // The index 1 (time values) must be monotonically increasing and >= 0
00384 for (const auto &cell : lib_json_["cells"]) {
00385     total_cells++;
00386     bool cell_is_monotonic = true; // Assume cell is monotonic initially
00387     std::string cell_name = cell["cell_name"].get<std::string>();
00388     cell_monotonicity_status[cell_name] =
00389         true; // Initialize to pass, will be set to false if any check fails
00390
00391     if (cell.contains("output_pins")) {
00392         for (const auto &pin : cell["output_pins"]) {
00393             if (pin.contains("timing_arcs")) {
00394                 for (const auto &arc : pin["timing_arcs"]) {
00395                     // Check 4 timing arc names and accumulate monotonicity status
00396                     for (const auto &name :
00397                         {"cell_rise", "cell_fall", "rise_transition", "fall_transition"}) {
00398                         if (!checkTimingArcMonotonicity(cell, pin, arc, name, is_slew)) {
00399                             cell_is_monotonic = false;
00400                         }
00401                     }
00402                 }
00403             }
00404         }
00405     }
00406     if (cell.contains("input_pins")) {
00407         for (const auto &pin : cell["input_pins"]) {
00408             if (!pin.contains("clock")) {
00409                 continue; // Skip non-clock pins
00410             }
00411             // logger->debug("Clock pin: {}", pin["pin_name"].get<std::string>());
00412             if (pin.contains("timing_arcs")) {
00413                 for (const auto &arc : pin["timing_arcs"]) {
00414                     if (arc.contains("timing_type")) {
00415                         // logger->debug("Timing type: {}", arc["timing_type"].get<std::string>());
00416                         if (arc["timing_type"].get<std::string>() == "min_pulse_width") {
00417                             for (const auto &name : {"rise_constraint", "fall_constraint"}) {
00418                                 if (!checkTimingArcMonotonicity(cell, pin, arc, name, is_slew)) {
00419                                     cell_is_monotonic = false;
00420                                 }
00421                             }
00422                         }
00423                     }
00424                 }
00425             }
00426         }
00427     }
00428     // Update cell status based on all checks
00429     cell_monotonicity_status[cell_name] = cell_is_monotonic;
00430     if (cell_is_monotonic) {
00431         passed_cells++;
00432     } else {
00433         failed_cells.push_back(cell_name);
00434     }
00435 }
00436 logger->info("Monotonicity check of '{}' completed.", filename_);
00437
00438 // Output summary statistics
00439 logger->info("validate_monotonicity : {} out of {} cells passed", passed_cells, total_cells);
00440 logger->info("validate_monotonicity : {} out of {} cells failed", total_cells - passed_cells,
00441             total_cells);
00442 if (!failed_cells.empty()) {

```

```

00443     std::stringstream failed_cells_ss;
00444     for (size_t i = 0; i < failed_cells.size(); ++i) {
00445         failed_cells_ss << failed_cells[i];
00446         if (i < failed_cells.size() - 1) {
00447             failed_cells_ss << ", "; // Add comma if not the last element
00448         }
00449     }
00450     logger_>info("Failed cell list : {}", failed_cells_ss.str());
00451 }
00452 }
00453
00473 void LibFile::supercell(const int chain_length, const std::vector<std::string> &cell_names) {
00474     /*
00475     * Supercells are named as follows:
00476     * <cellname>_X<chain_length>_<input_pin>_<output_pin>
00477     */
00478     logger_>info("Creating supercells for '{}', filename_);
00479
00480     if (!std::filesystem::exists(jsonname_)) {
00481         logger_>info("JSON file not found. Parsing Liberty file first.");
00482         this->parse();
00483         this->writeJsonToFile();
00484     } else {
00485         // Read the JSON file into json object
00486         std::ifstream in(jsonname_);
00487         if (!in.is_open()) {
00488             logger_>error("Could not open file '{}' for reading", jsonname_);
00489             return;
00490         }
00491         try {
00492             lib_json_ = json::parse(in);
00493         } catch (const json::parse_error &e) {
00494             logger_>error("JSON parsing error in file '{}': {}", jsonname_, e.what());
00495             in.close();
00496             return;
00497         }
00498         in.close();
00499     }
00500
00501     // write to .map file
00502     std::ofstream out(basename_ + ".map");
00503     if (!out.is_open()) {
00504         logger_>error("Could not open file '{}' for writing", basename_ + ".map");
00505         return;
00506     }
00507     // if cell_names is empty, create supercells for all cells
00508     // else create supercells for only the specified cells
00509
00510     // Check if we're processing specific cells or all cells
00511     bool process_all_cells = cell_names.empty();
00512
00513     if (process_all_cells) {
00514         logger_>info("Creating supercells for ALL cells in '{}', filename_);
00515     } else {
00516         logger_>info("Creating supercells for {} specified cells in '{}', cell_names.size(),
00517                     filename_);
00518     }
00519
00520     // Create a set for faster lookups if we have specific cell names
00521     std::unordered_set<std::string> cell_set;
00522     std::unordered_set<std::string> found_cells;
00523     if (!process_all_cells) {
00524         cell_set.insert(cell_names.begin(), cell_names.end());
00525     }
00526
00527     for (const auto &cell : lib_json_["cells"]) {
00528         bool is_sequential = false;
00529         std::string cell_name = cell["cell_name"].get<std::string>();
00530

```

```

00531     // Skip cells not in the specified list
00532     if (!process_all_cells && cell_set.find(cell_name) == cell_set.end()) {
00533         continue;
00534     }
00535
00536     // Mark this cell as found
00537     if (!process_all_cells) {
00538         found_cells.insert(cell_name);
00539     }
00540
00541     logger_>debug("Creating supercells for cell: '{}'", cell_name);
00542
00543     std::vector<std::string> output_pins;
00544     std::vector<std::string> input_pins;
00545     // Extract input and output pins
00546     if (cell.contains("output_pins")) {
00547         for (const auto &pin : cell["output_pins"]) {
00548             output_pins.push_back(pin["pin_name"].get<std::string>());
00549         }
00550     }
00551     if (cell.contains("input_pins")) {
00552         for (const auto &pin : cell["input_pins"]) {
00553             if (pin.contains("clock")) {
00554                 is_sequential = true;
00555                 // clock pin not use for supercell
00556                 continue;
00557             }
00558             input_pins.push_back(pin["pin_name"].get<std::string>());
00559         }
00560     }
00561
00562     // create supercells for all combinations of input and output pins
00563     for (const auto &output_pin : output_pins) {
00564         for (const auto &input_pin : input_pins) {
00565             if (is_sequential) {
00566                 const int chain_len = 1;
00567                 std::string supercell_name =
00568                     cell_name + "__X" + std::to_string(chain_len) + "__" + input_pin + "__" + output_pin;
00569                 out << cell_name << " " << supercell_name << std::endl;
00570             } else {
00571                 std::string supercell_name = cell_name + "__X" + std::to_string(chain_length) + "__" +
00572                     input_pin + "__" + output_pin;
00573                 out << cell_name << " " << supercell_name << std::endl;
00574             }
00575         }
00576     }
00577 }
00578
00579 // Check for cells that were specified but not found
00580 if (!process_all_cells) {
00581     std::vector<std::string> not_found_cells;
00582     for (const auto &requested_cell : cell_names) {
00583         if (found_cells.find(requested_cell) == found_cells.end()) {
00584             not_found_cells.push_back(requested_cell);
00585         }
00586     }
00587
00588     // Output warning for cells that weren't found
00589     if (!not_found_cells.empty()) {
00590         std::string missing_cells = not_found_cells[0];
00591         for (size_t i = 1; i < not_found_cells.size(); ++i) {
00592             missing_cells += ", " + not_found_cells[i];
00593         }
00594         logger_>warn("{} specified cell{} not found in the library: {}", not_found_cells.size(),
00595             not_found_cells.size() > 1 ? "s were" : " was", missing_cells);
00596     }
00597 }
00598
00599 out.close();

```

```

00600 logger_>info("Supercell creation complete in '{}'", basename_ + ".map");
00601 }
00602
00627 void LibFile::verilog(const int chain_length, const std::vector<std::string> &cell_names) {
00628 logger_>info("Creating Verilog for '{}'", filename_);
00629
00630 this->supercell(chain_length, cell_names);
00631
00632 // Create a temporary file for individual modules
00633 std::string temp_file = basename_ + "_temp.v";
00634 std::ofstream out(temp_file);
00635 if (!out.is_open()) {
00636 logger_>error("Could not open temp file '{}' for writing", temp_file);
00637 return;
00638 }
00639
00640 // Read the .map file into a vector to preserve all entries and their order
00641 std::vector<std::pair<std::string, std::string>> supercell_entries;
00642 std::ifstream in(basename_ + ".map");
00643 if (!in.is_open()) {
00644 logger_>error("Could not open file '{}' for reading", basename_ + ".map");
00645 return;
00646 }
00647
00648 std::string line;
00649 while (std::getline(in, line)) {
00650 std::istringstream iss(line);
00651 std::string cell_name, supercell_name;
00652 if (!(iss >> cell_name >> supercell_name)) {
00653 logger_>warn("Invalid line format in map file: '{}', line);
00654 continue;
00655 }
00656 supercell_entries.push_back({cell_name, supercell_name});
00657 }
00658 in.close();
00659 logger_>info("Read {} supercells from '{}'", supercell_entries.size(), basename_ + ".map");
00660
00661 // Store module information for top-level creation
00662 std::vector<std::string> module_names;
00663 std::map<std::string, std::vector<std::string>> module_inputs;
00664 std::map<std::string, std::vector<std::string>> module_outputs;
00665
00666 // Generate verilog module for each supercell
00667 for (const auto &supercell_entry : supercell_entries) {
00668 // Check if the cell is sequential
00669 bool is_sequential = false;
00670 // Count the number of instances will be created in verilog
00671 int instance_count = 0;
00672 // Get original cell name from pair
00673 const std::string &cell_name = supercell_entry.first;
00674 // Get supercell name(as well as module name) from pair
00675 const std::string &module_name = supercell_entry.second;
00676
00677 // Store module name for top-level creation
00678 module_names.push_back(module_name);
00679
00680 logger_>info("Creating Module: '{}' from Cell '{}'", module_name, cell_name);
00681
00682 // Get the cell JSON object
00683 json cell_json;
00684 for (const auto &cell : lib_json_["cells"]) {
00685 if (cell["cell_name"].get<std::string>() == cell_name) {
00686 cell_json = cell;
00687 break;
00688 }
00689 }
00690
00691 // Get input/output pins from the cell JSON object
00692 std::vector<std::string> input_pins;

```

```

00693     std::vector<std::string> output_pins;
00694     std::stringstream input_pins_ss;
00695     std::stringstream output_pins_ss;
00696     if (cell_json.contains("input_pins")) {
00697         for (const auto &pin : cell_json["input_pins"]) {
00698             if (pin.contains("clock")) {
00699                 is_sequential = true;
00700             }
00701             std::string pin_name = pin["pin_name"].get<std::string>();
00702             input_pins.push_back(pin_name);
00703             input_pins_ss << pin_name << ", ";
00704
00705             // Store input pin name for top-level creation
00706             module_inputs[module_name].push_back(pin_name);
00707         }
00708     }
00709     if (cell_json.contains("output_pins")) {
00710         for (const auto &pin : cell_json["output_pins"]) {
00711             std::string pin_name = pin["pin_name"].get<std::string>();
00712             output_pins.push_back(pin_name);
00713             output_pins_ss << pin_name << ", ";
00714
00715             // Store output pin name for top-level creation
00716             module_outputs[module_name].push_back(pin_name);
00717         }
00718     }
00719     logger_>debug("Input pins set: {}", input_pins_ss.str());
00720     logger_>debug("Output pins set: {}", output_pins_ss.str());
00721
00722     // Sequential cells will have only one instance
00723     // Combinational cells will have instances equal to chain length
00724     if (is_sequential) {
00725         instance_count = 1;
00726     } else {
00727         instance_count = chain_length;
00728     }
00729
00730     // Create ANSI port list
00731     std::string port_list_text = "(";
00732     // Add input ports
00733     bool isFirstPort = true;
00734     for (const auto &input_pin : input_pins) {
00735         if (!isFirstPort) {
00736             port_list_text += ", ";
00737         }
00738         port_list_text += "input " + input_pin;
00739         isFirstPort = false;
00740     }
00741     // Add output ports
00742     for (const auto &output_pin : output_pins) {
00743         if (!isFirstPort) {
00744             port_list_text += ", ";
00745         }
00746         port_list_text += "output " + output_pin;
00747         isFirstPort = false;
00748     }
00749     port_list_text += ")";
00750     logger_>debug("ANSI Port list: {}", port_list_text);
00751
00752     // Creat full module header text
00753     std::string fullModuleText = "module " + module_name + port_list_text + ";\nendmodule";
00754     logger_>debug("Full module text: {}", fullModuleText);
00755
00756     // Generate the syntax tree for the module using slang library
00757     auto tree = slang::syntax::SyntaxTree::fromText(fullModuleText);
00758     if (tree) {
00759         // Add ports to the syntax tree
00760         ModuleRewriter rewriter(input_pins, output_pins, supercell_entry, instance_count, logger_);
00761     }

```

```

00762     tree = rewriter.transform(tree);
00763     // Revisit the syntax tree to check architecture
00764     rewriter.visit(tree->root());
00765
00766     // Output the transformed syntax tree
00767     out << "`timescale 1ns/10ps" << std::endl;
00768     out << slang::syntax::SyntaxPrinter::printFile(*tree) << std::endl << std::endl;
00769 } else {
00770     logger->error("Failed to create syntax tree for module '{}'", module_name);
00771     continue;
00772 }
00773 }
00774
00775 out.close();
00776
00777 // Create the top-level module
00778 std::ofstream final_out(basename_ + ".v");
00779 if (!final_out.is_open()) {
00780     logger->error("Could not open file '{}' for writing", basename_ + ".v");
00781     return;
00782 }
00783
00784 // Collect all port names separately for inputs and outputs
00785 std::vector<std::string> all_input_ports;
00786 std::vector<std::string> all_output_ports;
00787 std::stringstream top_instances;
00788
00789 // First pass: collect all port names
00790 for (const auto &module_name : module_names) {
00791     // Collect input port names
00792     for (const auto &pin : module_inputs[module_name]) {
00793         all_input_ports.push_back(module_name + "__" + pin);
00794     }
00795
00796     // Collect output port names
00797     for (const auto &pin : module_outputs[module_name]) {
00798         all_output_ports.push_back(module_name + "__" + pin);
00799     }
00800
00801     // Create instance
00802     std::string instance_name = "I_" + module_name.substr(0, module_name.find("__X"));
00803     for (size_t i = module_name.find("__X") + 3; i < module_name.length(); i++) {
00804         if (module_name[i] == '_' && module_name[i + 1] == '_') {
00805             instance_name += "__" + module_name.substr(i + 2);
00806             break;
00807         }
00808     }
00809
00810     // Generate port connections for this instance
00811     std::stringstream instance_ports;
00812     for (const auto &pin : module_inputs[module_name]) {
00813         instance_ports << "." << pin << "(" << module_name << "__" << pin << ")", ";";
00814     }
00815     for (const auto &pin : module_outputs[module_name]) {
00816         instance_ports << "." << pin << "(" << module_name << "__" << pin << ")", ";";
00817     }
00818
00819     // Remove last comma and space
00820     std::string instance_ports_str = instance_ports.str();
00821     if (!instance_ports_str.empty()) {
00822         instance_ports_str = instance_ports_str.substr(0, instance_ports_str.length() - 2);
00823     }
00824
00825     top_instances << " " << module_name << " " << instance_name << " (" << instance_ports_str
00826         << ");\n";
00827 }
00828
00829 // Now generate the port list with all inputs first, then all outputs
00830 std::stringstream top_ports;

```



```

00831
00832 // Add all input ports first
00833 for (size_t i = 0; i < all_input_ports.size(); ++i) {
00834     top_ports << all_input_ports[i];
00835     if (i < all_input_ports.size() - 1 || !all_output_ports.empty()) {
00836         top_ports << ", ";
00837     }
00838 }
00839
00840 // Then add all output ports
00841 for (size_t i = 0; i < all_output_ports.size(); ++i) {
00842     top_ports << all_output_ports[i];
00843     if (i < all_output_ports.size() - 1) {
00844         top_ports << ", ";
00845     }
00846 }
00847
00848 // Generate input and output declarations
00849 std::stringstream top_inputs;
00850 std::stringstream top_outputs;
00851
00852 // Generate input declarations
00853 for (const auto &port : all_input_ports) {
00854     top_inputs << " input " << port << ";\n";
00855 }
00856
00857 // Generate output declarations
00858 for (const auto &port : all_output_ports) {
00859     top_outputs << " output " << port << ";\n";
00860 }
00861
00862 // Write the top module
00863 final_out << "timescale 1ns/10ps" << std::endl;
00864 final_out << "module validate_top ( " << top_ports.str() << " );" << std::endl;
00865 final_out << std::endl; // Add newline before port declarations
00866 final_out << top_inputs.str();
00867 final_out << top_outputs.str();
00868 final_out << std::endl;
00869 final_out << top_instances.str();
00870 final_out << "endmodule" << std::endl << std::endl;
00871
00872 // Append the module definitions from the temporary file
00873 std::ifstream temp_in(temp_file);
00874 if (temp_in.is_open()) {
00875     final_out << temp_in.rdbuf();
00876     temp_in.close();
00877 } else {
00878     logger_>error("Could not open temp file '{}'" for reading", temp_file);
00879 }
00880
00881 final_out.close();
00882
00883 // Remove temporary file
00884 // if (std::filesystem::exists(temp_file)) {
00885 //     std::filesystem::remove(temp_file);
00886 // }
00887
00888 logger_>info("Verilog creation complete in '{}'", basename_ + ".v");
00889 }
00890
00891 std::map<std::string, std::string> LibFile::logic(const std::string &cell_name) {
00892     std::map<std::string, std::string> logic_map = {};
00893
00894     logger_>info("Getting output pin logic function for '{}'", filename_);
00895
00896     if (!std::filesystem::exists(jsonname_)) {
00897         logger_>info("JSON file not found. Parsing Liberty file first.");
00898         this->parse();
00899         this->writeJsonToFile();

```

```

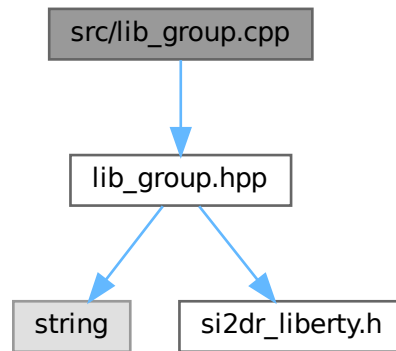
00900 } else {
00901     // Read the JSON file into json object
00902     std::ifstream in(jsonname_);
00903     if (!in.is_open()) {
00904         logger_>error("Could not open file '{}': for reading", jsonname_);
00905         return logic_map;
00906     }
00907     try {
00908         lib_json_ = json::parse(in);
00909     } catch (const json::parse_error &e) {
00910         logger_>error("JSON parsing error in file '{}': {}", jsonname_, e.what());
00911         in.close();
00912         return logic_map;
00913     }
00914     in.close();
00915 }
00916
00917 // Check if the cell name exists in the JSON object
00918 for (const auto &cell : lib_json_["cells"]) {
00919     if (cell["cell_name"].get<std::string>() == cell_name) {
00920         logger_>debug("Found cell: '{}', cell_name);
00921
00922         // Tranverse all output pins
00923         if (cell.contains("output_pins")) {
00924             for (const auto &pin : cell["output_pins"]) {
00925                 std::string pin_name = pin["pin_name"].get<std::string>();
00926                 // Check if the pin has a "timing_arcs" attribute
00927                 if (pin.contains("function")) {
00928                     std::string function = pin["function"].get<std::string>();
00929                     logic_map[pin_name] = function;
00930                     logger_>debug("Pin: '{}': Logic Function: '{}', pin_name, function);
00931                 } else {
00932                     logger_>warn("Pin: '{}': does not have a 'function' attribute", pin_name);
00933                 }
00934             }
00935         }
00936     }
00937 }
00938 if (logic_map.empty()) {
00939     logger_>warn("No logic functions found for cell: '{}', cell_name);
00940 } else {
00941     logger_>info("Found {} Logic functions for cell: '{}', logic_map.size(), cell_name);
00942     // Print the logic functions
00943     for (const auto &pair : logic_map) {
00944         logger_>info("Pin: '{}', Logic Function: '{}', pair.first, pair.second);
00945     }
00946 }
00947 return logic_map;
00948 }

```

8.38 src/lib_group.cpp File Reference

```
#include "lib_group.hpp"
```

Include dependency graph for lib_group.cpp:



8.39 lib_group.cpp

[Go to the documentation of this file.](#)

```

00001 #include "lib_group.hpp"
00002
00003 LibGroup::LibGroup(si2drGroupIdT group, si2drErrorT &err) : group_(group), err_(err) {}
00004
00005 LibGroup::~LibGroup() {}
00006
00007 std::string LibGroup::getName() {
00008     si2drNamesIdT names = si2drGroupGetNames(group_, &err_);
00009     si2drStringT name = si2drIterNextName(names, &err_);
00010     si2drIterQuit(names, &err_);
00011     return name ? std::string(name) : std::string();
00012 }
00013
00014 std::string LibGroup::getType() {
00015     si2drStringT type = si2drGroupGetGroupType(group_, &err_);
00016     return type ? std::string(type) : std::string();
00017 }
00018
00019 si2drAttrIdT LibGroup::getAttrs() { return si2drGroupGetAttrs(group_, &err_); }
00020
00021 si2drGroupsIdT LibGroup::getGroups() { return si2drGroupGetGroups(group_, &err_); }
  
```

8.40 src/main.cpp File Reference

```
#include <filesystem>
#include <iostream>
```

```

#include <string>
#include <thread>
#include <vector>
#include "CLI/CLI.hpp"
#include "si2dr_liberty.h"
#include "spdlog/sinks/basic_file_sink.h"
#include "spdlog/sinks/stdout_color_sinks.h"
#include "spdlog/spdlog.h"
#include "compare.hpp"
#include "lib_file.hpp"
#include "verilog_utils.hpp"
#include "version.h"

```

Include dependency graph for main.cpp:



Functions

- void [printInfo](#) ()
Sets up and configures the global logger and prints application information.
- void [parseLibFile](#) (const std::string &library_path, const std::string log_file_name)
Parses a library file and generates corresponding output.
- void [monoCheckLibFile](#) (const std::string &library_path, const std::string log_file_name, bool is↵_slew)
Performs monotonicity check on a library file.
- void [supercellLibFile](#) (const std::string &library_path, const std::string &log_file_name, int chain↵_length, const std::vector< std::string > &cell_names)
Creates supercell map structures from a Liberty library file.
- void [verilogLibFile](#) (const std::string &library_path, const std::string &log_file_name, int chain↵_length, const std::vector< std::string > &cell_names)
Generates Verilog files from a library file for specified cells.
- void [compareLibFiles](#) (const std::string &ref_lib, const std::string &comp_lib, const double reltol, const double abstol, std::string &report_file_name)
Compares two library files and generates a detailed comparison report.
- void [funcLibFile](#) (const std::string &ref_file, const std::string &comp_file, const std::vector< std↵::string > &cell_names, std::string &report_file_name)
- int [main](#) (int argc, char *argv[])

8.40.1 Function Documentation

8.40.1.1 compareLibFiles()

```
void compareLibFiles (
    const std::string & ref_lib,
    const std::string & comp_lib,
    const double reltol,
    const double abstol,
    std::string & report_file_name )
```

Compares two library files and generates a detailed comparison report.

This function compares a reference library file with another library file, performing validation checks based on specified tolerance parameters. The comparison results are written to a report file in markdown or text format.

Parameters

<i>ref_lib</i>	Path to the reference library file to use as the baseline
<i>comp_lib</i>	Path to the library file to compare against the reference
<i>reltol</i>	Relative tolerance for numerical comparisons (must be ≥ 0.0)
<i>abstol</i>	Absolute tolerance for numerical comparisons
<i>report_file_name</i>	[in,out] Name of the file to write the comparison report to. If empty, defaults to [comp_lib_basename].cmp.md. If provided but doesn't end with .txt or .md, .md will be appended.

Note

The function will log an error and return without comparing if reltol is invalid.

Log files will be created with the naming pattern [library_basename].cmp.log

The function uses the [LibraryComparator](#) class to perform the actual comparison.

Definition at line 219 of file [main.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:

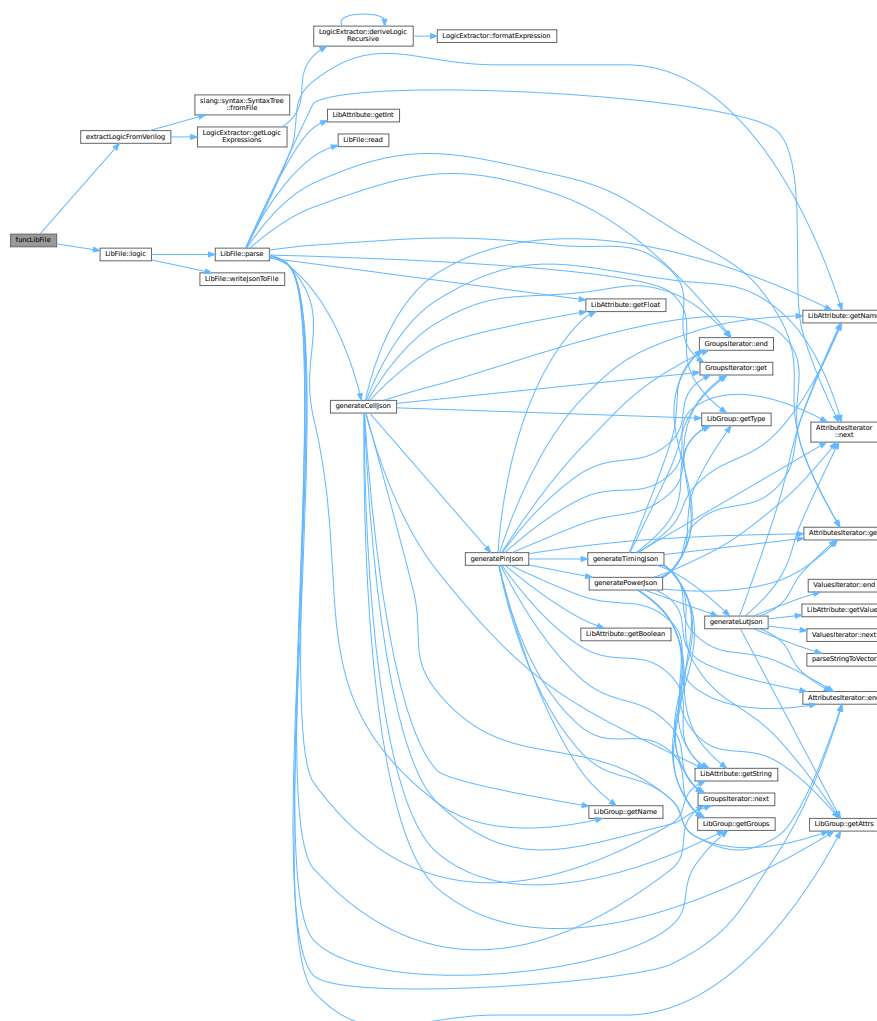


8.40.1.2 funcLibFile()

```
void funcLibFile (
    const std::string & ref_file,
    const std::string & comp_file,
    const std::vector< std::string > & cell_names,
    std::string & report_file_name )
```

Definition at line [247](#) of file [main.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.40.1.3 main()

```
int main (
```


Parameters

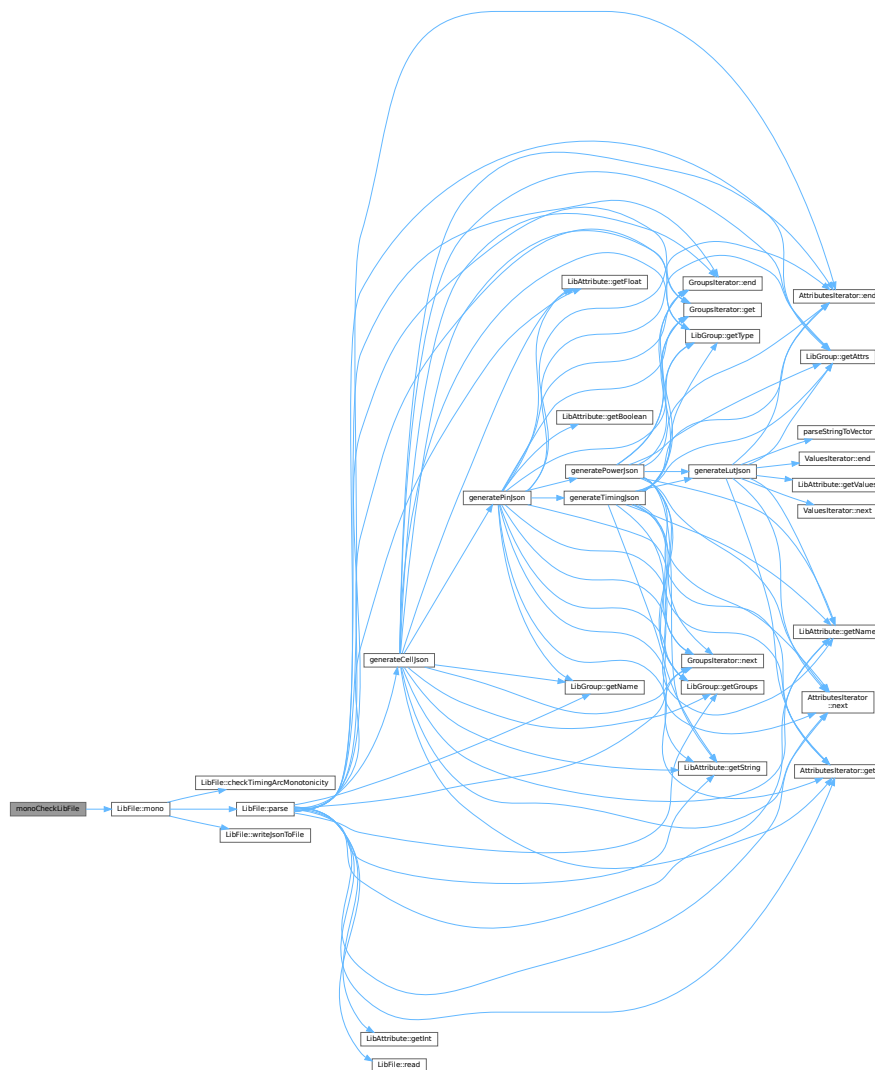
<i>library_path</i>	Path to the library file to check
<i>log_file_name</i>	Name of the log file to create (optional). If empty, a default name will be generated from the library file name
<i>is_slew</i>	Flag indicating whether to check input slew monotonicity (true) or output load monotonicity (false)

Exceptions

<i>The</i>	function catches and logs any exceptions but does not rethrow them
------------	--

Definition at line 97 of file `main.cpp`.

Here is the call graph for this function:



Here is the caller graph for this function:



8.40.1.5 parseLibFile()

```
void parseLibFile (  
    const std::string & library_path,  
    const std::string log_file_name )
```

Parses a library file and generates corresponding output.

This function processes the given library file, parsing its contents and generating a JSON output. It also logs the parsing process to a specified log file or creates a default log file if none is provided.

Parameters

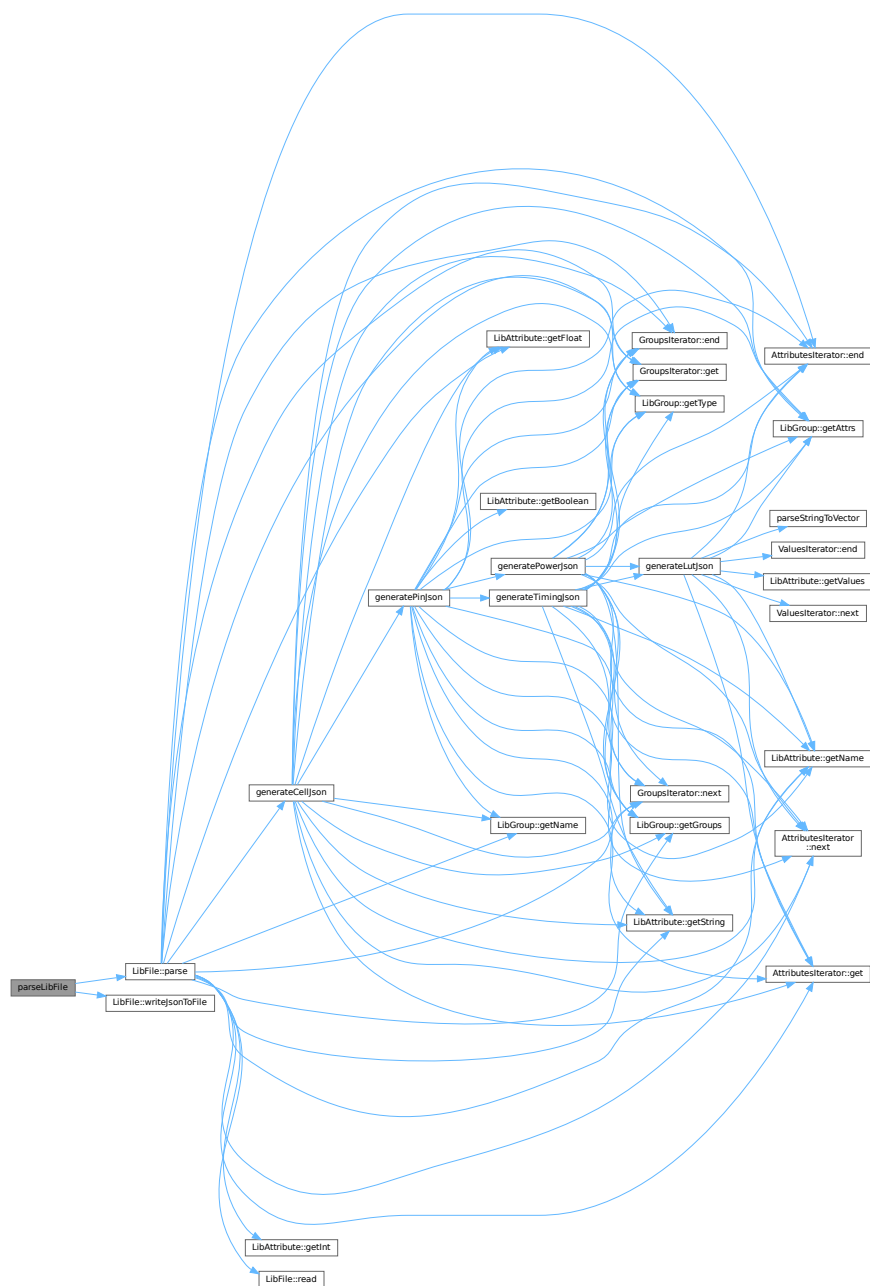
<i>library_path</i>	Path to the library file that needs to be parsed
<i>log_file_name</i>	Optional name for the log file. If empty, a default name is generated based on the library filename with ".parse.log" extension

Exceptions

<i>The</i>	function catches and logs any exceptions but doesn't propagate them
------------	---

Definition at line 65 of file [main.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.40.1.6 printInfo()

```
void printInfo ( )
```

Sets up and configures the global logger and prints application information.

This function performs the following operations:

1. Creates a console sink for logging with level set to INFO
2. Creates a file sink for logging with level set to DEBUG, saving to [APP_NAME].log
3. Configures a logger with both sinks and sets it as the default logger
4. Outputs basic application information:
 - Version and build timestamp
 - Author information
 - Log file location

Note

Uses spdlog library for logging functionality

Depends on APP_NAME, APP_VERSION, BUILD_TIMESTAMP, APP_AUTHOR, and APP↵_CONTACT macros

Definition at line 34 of file [main.cpp](#).

Here is the caller graph for this function:



8.40.1.7 supercellLibFile()

```
void supercellLibFile (
    const std::string & library_path,
    const std::string & log_file_name,
    int chain_length,
    const std::vector< std::string > & cell_names )
```

Creates supercell map structures from a Liberty library file.

This function reads a Liberty file, creates supercell structures based on the specified chain length and cell names, and logs the process to a file.

Parameters

<i>library_path</i>	Path to the Liberty file to process
<i>log_file_name</i>	Name of the log file (if empty, defaults to "[library_name].supercell.log")
<i>chain_length</i>	The length of chains to create (must be ≥ 1)
<i>cell_names</i>	Vector of cell names to process for supercell generation

Exceptions

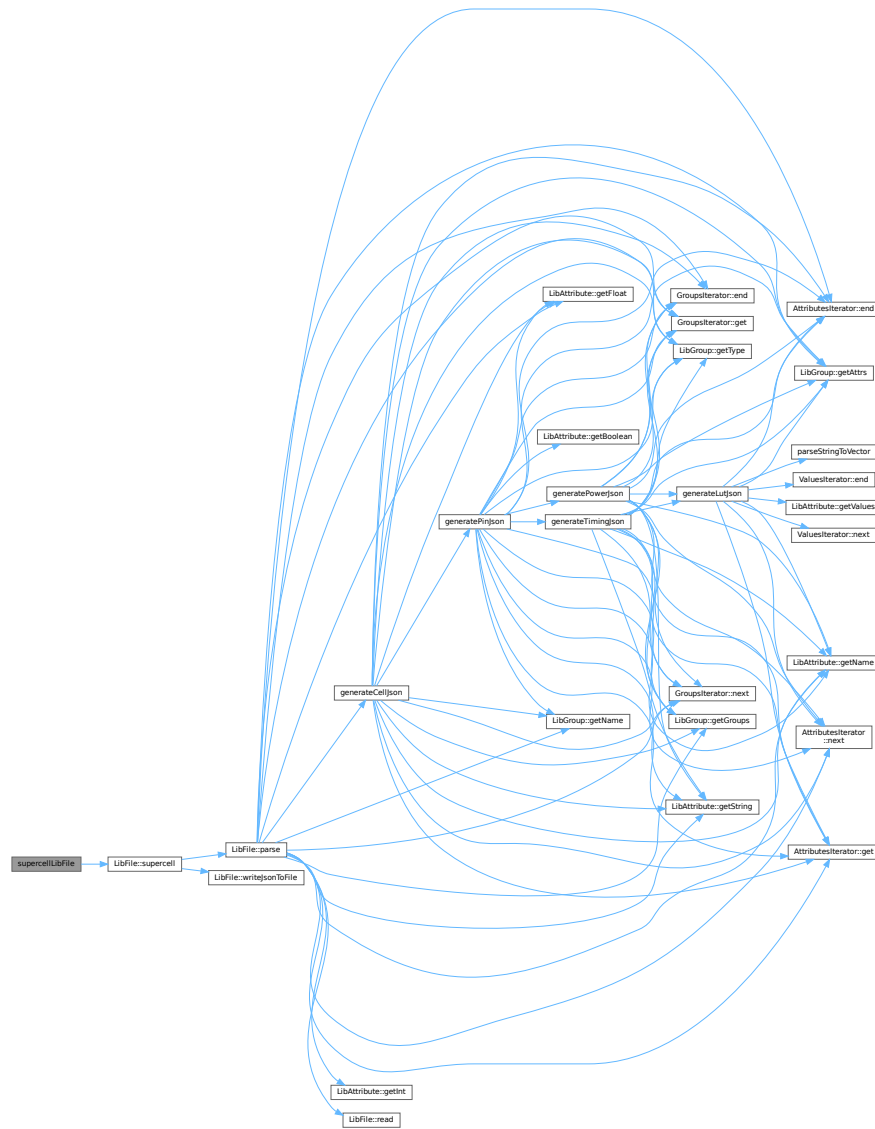
May	pass through exceptions from the LibFile::supercell method
-----	--

Note

The function validates the chain length and logs all activities including errors that might occur during processing

Definition at line [132](#) of file [main.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



8.40.1.8 verilogLibFile()

```
void verilogLibFile (
    const std::string & library_path,
    const std::string & log_file_name,
    int chain_length,
    const std::vector< std::string > & cell_names )
```

Generates Verilog files from a library file for specified cells.

This function processes a library file and generates Verilog representation for the specified cell names with a given chain length. The operation results are logged to a specified or default log file.

Parameters

<i>library_path</i>	Path to the library file to process
<i>log_file_name</i>	Name for the log file (if empty, a default name will be generated)
<i>chain_length</i>	Number of cells to chain together, must be ≥ 1
<i>cell_names</i>	Vector of cell names to generate Verilog for

Exceptions

<i>Catches</i>	any exceptions from the verilog generation process and logs them
----------------	--

Definition at line 173 of file [main.cpp](#).

[illegible]

```
graph LR; main --> verilogLibFile
```

[Go to the documentation of this file.](#)

```
00001 // ./src/main.cpp
00002 #include <filesystem>
00003 #include <iostream>
```



```

00004 #include <string>
00005 #include <thread>
00006 #include <vector>
00007
00008 #include "CLI/CLI.hpp"
00009 #include "si2dr_liberty.h"
00010 #include "spdlog/sinks/basic_file_sink.h"
00011 #include "spdlog/sinks/stdout_color_sinks.h"
00012 #include "spdlog/spdlog.h"
00013
00014 #include "compare.hpp"
00015 #include "lib_file.hpp"
00016 #include "verilog_utils.hpp"
00017 #include "version.h"
00018
00034 void printInfo() {
00035     // Set Global Logger
00036     auto console_sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
00037     console_sink->set_level(spdlog::level::info);
00038     std::string app_name = APP_NAME;
00039     auto file_sink = std::make_shared<spdlog::sinks::basic_file_sink_mt>(app_name + ".log", true);
00040     file_sink->set_level(spdlog::level::trace);
00041
00042     std::vector<spdlog::sink_ptr> sinks{console_sink, file_sink};
00043     auto logger = std::make_shared<spdlog::logger>(APP_NAME, sinks.begin(), sinks.end());
00044     logger->set_level(spdlog::level::debug);
00045     spdlog::set_default_logger(logger);
00046
00047     spdlog::info("Version {}, Built: {}", APP_VERSION, BUILD_TIMESTAMP);
00048     spdlog::info("Author: {}, Email: {}", APP_AUTHOR, APP_CONTACT);
00049     spdlog::info("Global log file in: '{}'", app_name + ".log");
00050 }
00051
00065 void parseLibFile(const std::string &library_path, const std::string log_file_name) {
00066     std::string logname = log_file_name.empty()
00067         ? std::filesystem::path(library_path).stem().string() + ".parse.log"
00068         : log_file_name;
00069     LibFile libfile(library_path, logname);
00070
00071     libfile.logger->info("Starting parse for file: '{} ...'", library_path);
00072
00073     try {
00074         libfile.parse();
00075         libfile.writeJsonToFile();
00076         libfile.logger->info("Successfully parsed file: '{}'", library_path);
00077     } catch (const std::exception &e) {
00078         libfile.logger->error("Error parsing file '{}': {}", library_path, e.what());
00079     }
00080 }
00081
00097 void monoCheckLibFile(const std::string &library_path, const std::string log_file_name,
00098     bool is_slew) {
00099     std::string logname = log_file_name.empty()
00100         ? std::filesystem::path(library_path).stem().string() + ".mono.log"
00101         : log_file_name;
00102     LibFile libfile(library_path, logname);
00103
00104     libfile.logger->info("Starting monotonicity check for file: '{}', input slew: {}", library_path,
00105         is_slew);
00106
00107     try {
00108         libfile.mono(is_slew);
00109         libfile.logger->info("Successfully completed monotonicity check for file: '{}'", library_path);
00110     } catch (const std::exception &e) {
00111         libfile.logger->error("Error during monotonicity check for file '{}': {}", library_path,
00112             e.what());
00113     }
00114 }
00115

```

```

00132 void supercellLibFile(const std::string &library_path, const std::string &log_file_name,
00133                       int chain_length, const std::vector<std::string> &cell_names) {
00134     std::string logname = log_file_name.empty()
00135         ? std::filesystem::path(library_path).stem().string() + ".supercell.log"
00136         : log_file_name;
00137     LibFile libfile(library_path, logname);
00138
00139     // Check chain length validity
00140     if (chain_length < 1) {
00141         libfile.logger_>error("Invalid chain length: {}. Chain length must be >= 1.", chain_length);
00142         return;
00143     }
00144     std::stringstream ss;
00145     for (const auto &cell : cell_names) {
00146         ss << cell << ", ";
00147     }
00148     libfile.logger_>info("Starting supercell generation for file: '{}', chain length: {}, cells: {}",
00149                          library_path, chain_length, ss.str());
00150     try {
00151         libfile.supercell(chain_length, cell_names);
00152         libfile.logger_>info("Successfully generated supercells for file: '{}', library_path);
00153     } catch (const std::exception &e) {
00154         libfile.logger_>error("Error during supercell generation for file '{}': {}", library_path,
00155                                e.what());
00156     }
00157 }
00158
00173 void verilogLibFile(const std::string &library_path, const std::string &log_file_name,
00174                    int chain_length, const std::vector<std::string> &cell_names) {
00175     std::string logname = log_file_name.empty()
00176         ? std::filesystem::path(library_path).stem().string() + ".verilog.log"
00177         : log_file_name;
00178     LibFile libfile(library_path, logname);
00179
00180     // Check chain length validity
00181     if (chain_length < 1) {
00182         libfile.logger_>error("Invalid chain length: {}. Chain length must be >= 1.", chain_length);
00183         return;
00184     }
00185     std::stringstream ss;
00186     for (const auto &cell : cell_names) {
00187         ss << cell << ", ";
00188     }
00189     libfile.logger_>info("Starting Verilog generation for file: '{}', chain length: {}, cells: {}",
00190                          library_path, chain_length, ss.str());
00191     try {
00192         libfile.verilog(chain_length, cell_names);
00193         libfile.logger_>info("Successfully generated Verilog for file: '{}', library_path);
00194     } catch (const std::exception &e) {
00195         libfile.logger_>error("Error during Verilog generation for file '{}': {}", library_path,
00196                                e.what());
00197     }
00198 }
00199
00219 void compareLibFiles(const std::string &ref_lib, const std::string &comp_lib, const double reltol,
00220                    const double abstol, std::string &report_file_name) {
00221     std::string ref_logname = std::filesystem::path(ref_lib).stem().string() + ".cmp.log";
00222     std::string comp_logname = std::filesystem::path(comp_lib).stem().string() + ".cmp.log";
00223     LibFile ref_libfile(ref_lib, ref_logname), comp_libfile(comp_lib, comp_logname);
00224
00225     // Check relative tolerance validity
00226     if (reltol < 0.0) {
00227         spdlog::error("Invalid relative tolerance: {}. Relative tolerance must be >= 0.0.", reltol);
00228         return;
00229     }
00230
00231     // Check the report file name
00232     if (report_file_name.empty()) {
00233         report_file_name = comp_libfile.basename_ + ".cmp.md";

```

```

00234 } else if (std::filesystem::path(report_file_name).extension() != ".txt" &&
00235           std::filesystem::path(report_file_name).extension() != ".md") {
00236     // report file name not end in .txt or .md, warn user and append .md
00237     report_file_name = report_file_name + ".md";
00238     spdlog::warn("Report file name does not end in .txt or .md. Report will be written to '{}'",
00239                report_file_name);
00240 }
00241
00242 LibraryComparator comparator(ref_libfile, comp_libfile, reltol, abstol);
00243 comparator.generateReport(report_file_name);
00244 spdlog::info("Comparison completed. Report written to: '{}'", report_file_name);
00245 }
00246
00247 void funcLibFile(const std::string &ref_file, const std::string &comp_file,
00248                 const std::vector<std::string> &cell_names, std::string &report_file_name) {
00249     // Check if the files are Liberty or Verilog
00250     std::string ref_ext = std::filesystem::path(ref_file).extension();
00251     std::string comp_ext = std::filesystem::path(comp_file).extension();
00252     std::string ref_logname = std::filesystem::path(ref_file).stem().string() + ".cmp.log";
00253     std::string comp_logname = std::filesystem::path(comp_file).stem().string() + ".cmp.log";
00254
00255     // Pre cell names check
00256     if (cell_names.empty()) {
00257         spdlog::error("No cell names provided for functional equivalence check.");
00258         return;
00259     }
00260
00261     // Check the report file name
00262     if (report_file_name.empty()) {
00263         report_file_name = std::filesystem::path(comp_file).stem().string() + ".cmp.md";
00264     } else if (std::filesystem::path(report_file_name).extension() != ".txt" &&
00265               std::filesystem::path(report_file_name).extension() != ".md") {
00266         // report file name not end in .txt or .md, warn user and append .md
00267         report_file_name = report_file_name + ".md";
00268         spdlog::warn("Report file name does not end in .txt or .md. Report will be written to '{}'",
00269                    report_file_name);
00270     }
00271
00272     // Declare LibFile objects as pointers, initialized to nullptr
00273     LibFile *ref_libfile = nullptr;
00274     LibFile *comp_libfile = nullptr;
00275
00276     // Check the reference file extension
00277     if (ref_ext == ".v") {
00278         spdlog::info("Reference file in Verilog format.");
00279     } else if (ref_ext == ".lib") {
00280         spdlog::info("Reference file in Liberty format.");
00281         ref_libfile = new LibFile(ref_file, ref_logname);
00282     } else {
00283         spdlog::error("Unsupported reference file format: '{}'", ref_ext);
00284         return;
00285     }
00286
00287     // Check the comparison file extension
00288     if (comp_ext == ".v") {
00289         spdlog::info("Comparison file in Verilog format.");
00290     } else if (comp_ext == ".lib") {
00291         spdlog::info("Comparison file in Liberty format.");
00292         comp_libfile = new LibFile(comp_file, comp_logname);
00293     } else {
00294         spdlog::error("Unsupported comparison file format: '{}'", comp_ext);
00295         return;
00296     }
00297
00298     // Tranverse the cell names and check functional equivalence
00299     for (const auto &cell : cell_names) {
00300         spdlog::info("Checking functional equivalence for cell: '{}'", cell);
00301         std::map<std::string, std::string> ref_outpin_map;
00302         std::map<std::string, std::string> comp_outpin_map;

```

```

00303     if (ref_ext == ".v") {
00304         ref_outpin_map = extractLogicFromVerilog(ref_file, cell);
00305     } else if (ref_ext == ".lib") {
00306         ref_outpin_map = ref_libfile->logic(cell);
00307     }
00308
00309     if (comp_ext == ".v") {
00310         // getAST(comp_file, cell);
00311         // extractAndPrintNetlistInfo(comp_file, cell);
00312         comp_outpin_map = extractLogicFromVerilog(comp_file, cell);
00313     } else if (comp_ext == ".lib") {
00314         comp_outpin_map = comp_libfile->logic(cell);
00315     }
00316
00317     spdlog::info("Logic function expressions collected for cell: '{}'", cell);
00318     spdlog::info("Starting logic comparison ...");
00319
00320     // TODO Perform functional equivalence check
00321     // LogicComparator comparator(ref_outpin_map, comp_outpin_map, cell);
00322     // comparator.generateReport(report_file_name);
00323     spdlog::info("Functional equivalence check completed for cell: '{}'", cell);
00324 }
00325 spdlog::info("Functional equivalence check completed for all cells,");
00326 spdlog::info("Report written to: '{}'", report_file_name);
00327 }
00328
00329 int main(int argc, char *argv[]) {
00330     // Parse command line arguments
00331     std::vector<std::string> library_paths; // Support multiple files
00332     std::string log_file_name = "";
00333
00334     // Command line parameter parsing using CLI11
00335     CLI::App app{APP_NAME};
00336     app.set_version_flag("-v,--version", APP_VERSION);
00337
00338     // Add subcommand for parse mode
00339     CLI::App *parse_cmd =
00340         app.add_subcommand("parse", "Parse the Liberty file and write JSON to a file");
00341     parse_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00342         ->check(CLI::ExistingFile)
00343         ->required();
00344     parse_cmd->add_option("-l,--log", log_file_name,
00345         "Specify the log file name. Default: <basename>.parse.log");
00346     parse_cmd->callback([&] {
00347         printInfo();
00348         // Check if multi files
00349         if (library_paths.size() > 1) {
00350             spdlog::info("Running sequential parsing for {} files.", library_paths.size());
00351             spdlog::info("Each library will write to its own log file.");
00352             // Sequential parsing
00353             for (const auto &library_path : library_paths) {
00354                 parseLibFile(library_path, log_file_name = "");
00355             }
00356         } else {
00357             parseLibFile(library_paths[0], log_file_name);
00358         }
00359     });
00360
00361     // Add subcommand for mono check mode
00362     bool is_slew = false;
00363     CLI::App *mono_cmd = app.add_subcommand("mono", "Check the monotonicity of timing arc values");
00364     mono_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00365         ->check(CLI::ExistingFile)
00366         ->required();
00367     mono_cmd->add_option("-l,--log", log_file_name,
00368         "Specify the log file name. Default: <basename>.mono.log");
00369     mono_cmd->add_flag("-s,--slew", is_slew,
00370         "Specify that monotonicity checks also include input slew.");
00371     mono_cmd->callback([&] {

```

```

00372     printInfo();
00373     // Check if multi files
00374     if (library_paths.size() > 1) {
00375         spdlog::info("Running multi-threaded monotonicity check for {} files.", library_paths.size());
00376         spdlog::info("Each thread will write to its own log file.");
00377
00378         // Sequential json file check
00379         for (const auto &library_path : library_paths) {
00380             if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() +
00381                                     ".json")) {
00382                 spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00383                 parseLibFile(library_path, log_file_name = "");
00384             }
00385         }
00386         spdlog::info("All JSON files prepared.");
00387         // Parallel monotonicity check
00388         std::vector<std::thread> threads;
00389         for (const auto &library_path : library_paths) {
00390             threads.emplace_back(monoCheckLibFile, library_path, log_file_name = "", is_slew);
00391         }
00392         for (auto &thread : threads) {
00393             thread.join();
00394         }
00395         spdlog::info("All threads completed.");
00396     } else {
00397         monoCheckLibFile(library_paths[0], log_file_name, is_slew);
00398     }
00399 });
00400
00401 // Add subcommand for compare mode
00402 std::string ref_lib, comp_lib, report_file_name;
00403 CLI::App *compare_cmd = app.add_subcommand(
00404     "compare",
00405     "Compare the comparison library against the reference library and report differences");
00406 compare_cmd->add_option("--ref", ref_lib, "Specify the reference library file")
00407     ->check(CLI::ExistingFile)
00408     ->required();
00409 compare_cmd->add_option("--comp", comp_lib, "Specify the comparison library file")
00410     ->check(CLI::ExistingFile)
00411     ->required();
00412 double abstol = 0.002;
00413 compare_cmd->add_option("--abstol", abstol,
00414     "Specify the absolute tolerance for comparison. Default: 0.002ns");
00415 double reltol = 0.02;
00416 compare_cmd->add_option("--reltol", reltol,
00417     "Specify the relative tolerance for comparison. Default: 0.02/2.0%");
00418 compare_cmd->add_option("--report", report_file_name,
00419     "Specify the report file name. Default: <comp_lib>.cmp.md");
00420 compare_cmd->callback([&] {
00421     printInfo();
00422     compareLibFiles(ref_lib, comp_lib, reltol, abstol, report_file_name);
00423 });
00424
00425 // Add subcommand for supercell generation
00426 int chain_length = 1;
00427 CLI::App *supercell_cmd =
00428     app.add_subcommand("supercell", "Generate supercells for the given Liberty file");
00429 supercell_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00430     ->check(CLI::ExistingFile)
00431     ->required();
00432 supercell_cmd->add_option("-l,--log", log_file_name,
00433     "Specify the log file name. Default: <basename>.supercell.log");
00434 supercell_cmd->add_option("-c,--chain", chain_length,
00435     "Specify the chain length for supercell generation. Default: 1");
00436 std::vector<std::string> cell_names = {}; // "CMPE42D1" "AN2D0", "DFQD1"
00437 supercell_cmd->add_option("--cells", cell_names,
00438     "Specify the cell names to generate supercells for");
00439 supercell_cmd->callback([&] {
00440     printInfo();

```

```

00441 // Check if multi files
00442 if (library_paths.size() > 1) {
00443     spdlog::info("Running multi-threaded supercell generation for {} files.",
00444         library_paths.size());
00445     spdlog::info("Each library will write to its own log file.");
00446
00447 // Sequential json file check
00448 for (const auto &library_path : library_paths) {
00449     if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() +
00450         ".json")) {
00451         spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00452         parseLibFile(library_path, log_file_name = "");
00453     }
00454 }
00455 spdlog::info("All JSON files prepared.");
00456 // Parallel supercell generation
00457 std::vector<std::thread> threads;
00458 for (const auto &library_path : library_paths) {
00459     threads.emplace_back(supercellLibFile, library_path, log_file_name = "", chain_length,
00460         cell_names);
00461 }
00462 for (auto &thread : threads) {
00463     thread.join();
00464 }
00465 spdlog::info("All threads completed.");
00466 } else {
00467     supercellLibFile(library_paths[0], log_file_name, chain_length, cell_names);
00468 }
00469 });
00470
00471 // Add subcommand for zlibboost
00472 CLI::App *zlibboost_cmd =
00473     app.add_subcommand("zlibboost", "ZlibBoost - Multi-threaded Library Processing Tool");
00474 std::string config_dir = "/home/songzx/Projects/zlibboost/config.tcl";
00475 std::string python_dir = "/home/guocj/anaconda3/envs/myenv/bin/python";
00476 std::string main_py_dir = "/home/songzx/Projects/zlibboost/zlibboost.py";
00477 zlibboost_cmd->add_option(
00478     "-c, --config", config_dir,
00479     "Specify the configuration TCL file. Default: /home/songzx/Projects/zlibboost/config.tcl");
00480 zlibboost_cmd->add_option(
00481     "--python", python_dir,
00482     "Specify the python directory. Default: /home/guocj/anaconda3/envs/myenv/bin/python");
00483 zlibboost_cmd->add_option("--main", main_py_dir,
00484     "Specify the main python script directory. Default: "
00485     "/home/songzx/Projects/zlibboost/zlibboost.py");
00486 zlibboost_cmd->callback([&] {
00487     printInfo();
00488 // Run the ZlibBoost tool
00489 std::string command = python_dir + " " + main_py_dir + " -c " + config_dir;
00490 spdlog::info("Running ZlibBoost with command: '{}'", command);
00491 int ret = std::system(command.c_str());
00492 if (ret == 0) {
00493     spdlog::info("ZlibBoost completed successfully.");
00494 } else {
00495     spdlog::error("ZlibBoost failed with return code: {}", ret);
00496 }
00497 });
00498
00499 // Add subcommand for clear
00500 CLI::App *clear_cmd =
00501     app.add_subcommand("clear", "Clear the log, JSON, map, markdown files in this directory");
00502 clear_cmd->callback([&] {
00503     printInfo();
00504     std::filesystem::path current_dir = std::filesystem::current_path();
00505     for (const auto &entry : std::filesystem::directory_iterator(current_dir)) {
00506         if (entry.path().extension() == ".log" || entry.path().extension() == ".json" ||
00507             entry.path().extension() == ".map" || entry.path().extension() == ".md") {
00508             spdlog::info("Removing file: '{}', entry.path().string());
00509             std::filesystem::remove(entry.path());

```

```

00510     }
00511 }
00512 spdlog::info("All log, JSON, map, markdown files cleared.");
00513 });
00514
00515 // Add subcommand for Verilog generation
00516 CLI::App *verilog_cmd =
00517     app.add_subcommand("verilog", "Generate Verilog file for given Liberty file");
00518 verilog_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00519     ->check(CLI::ExistingFile)
00520     ->required();
00521 verilog_cmd->add_option("-l,--log", log_file_name,
00522     "Specify the log file name. Default: <basename>.verilog.log");
00523 verilog_cmd->add_option("-c,--chain", chain_length,
00524     "Specify the chain length for verilog generation. Default: 1");
00525 verilog_cmd->add_option("--cells", cell_names, "Specify the cell names to generate Verilog for");
00526 verilog_cmd->callback([&] {
00527     printInfo();
00528     // Check if multi files
00529     if (library_paths.size() > 1) {
00530         spdlog::info("Running multi-threaded Verilog generation for {} files.", library_paths.size());
00531         spdlog::info("Each library will write to its own log file.");
00532
00533         // Sequential json file check
00534         for (const auto &library_path : library_paths) {
00535             if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() +
00536                 ".json")) {
00537                 spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00538                 parseLibFile(library_path, log_file_name = "");
00539             }
00540         }
00541         spdlog::info("All JSON files prepared.");
00542         // Parallel Verilog generation
00543         std::vector<std::thread> threads;
00544         for (const auto &library_path : library_paths) {
00545             threads.emplace_back(verilogLibFile, library_path, log_file_name = "", chain_length,
00546                 cell_names);
00547         }
00548         for (auto &thread : threads) {
00549             thread.join();
00550         }
00551         spdlog::info("All threads completed.");
00552     } else {
00553         verilogLibFile(library_paths[0], log_file_name, chain_length, cell_names);
00554     }
00555 });
00556
00557 // Add subcommand for SPICE generation
00558 CLI::App *spice_cmd = app.add_subcommand("spice", "Generate SPICE file for given Liberty file");
00559 spice_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00560     ->check(CLI::ExistingFile)
00561     ->required();
00562 spice_cmd->add_option("-l,--log", log_file_name,
00563     "Specify the log file name. Default: <basename>.spice.log");
00564 spice_cmd->add_option("--cells", cell_names, "Specify the cell names to generate SPICE for");
00565 spice_cmd->callback([&] {
00566     printInfo();
00567     // Check if multi files
00568     if (library_paths.size() > 1) {
00569         spdlog::info("Running multi-threaded SPICE generation for {} files.", library_paths.size());
00570         spdlog::info("Each library will write to its own log file.");
00571
00572         // Sequential json file check
00573         for (const auto &library_path : library_paths) {
00574             if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() +
00575                 ".json")) {
00576                 spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00577                 parseLibFile(library_path, log_file_name = "");
00578             }

```


Functions

- void [getAST](#) (const std::string &verilog_file, const std::string &cell)
- void [extractAndPrintNetlistInfo](#) (const std::string &verilog_file, const std::string &cell)
- std::map< std::string, std::string > [extractLogicFromVerilog](#) (const std::string &verilog_file, const std::string &cell)

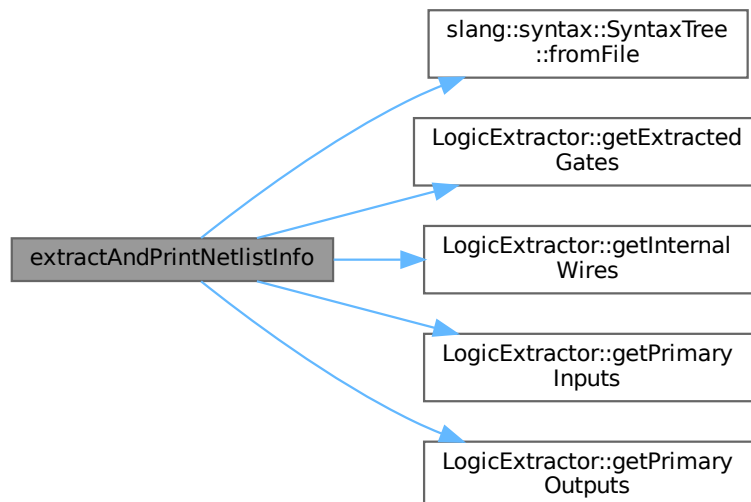
8.42.1 Function Documentation

8.42.1.1 [extractAndPrintNetlistInfo\(\)](#)

```
void extractAndPrintNetlistInfo (  
    const std::string & verilog_file,  
    const std::string & cell )
```

Definition at line [1123](#) of file [verilog_utils.cpp](#).

Here is the call graph for this function:

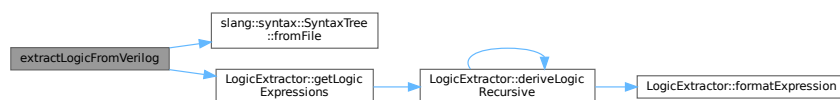


8.42.1.2 extractLogicFromVerilog()

```
std::map< std::string, std::string > extractLogicFromVerilog (
    const std::string & verilog_file,
    const std::string & cell )
```

Definition at line 1184 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:

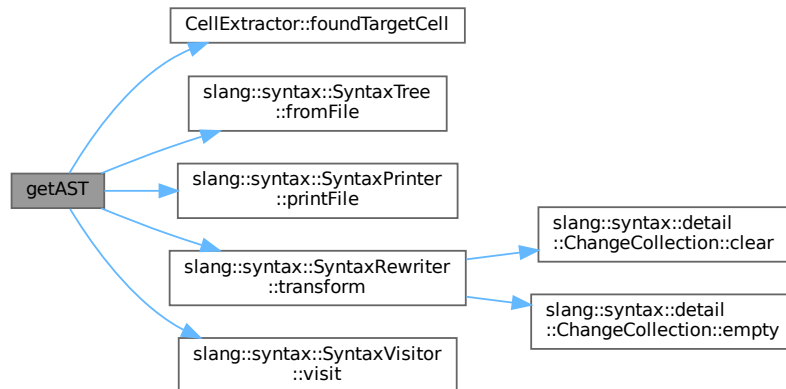


8.42.1.3 getAST()

```
void getAST (
    const std::string & verilog_file,
    const std::string & cell )
```

Definition at line 546 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



8.43 verilog_utils.cpp

[Go to the documentation of this file.](#)

```

00001 // verilog_utils.cpp
00002
00003 #include <filesystem> // For path operations if needed later
00004
00005 #include "verilog_utils.hpp"
00006
00007 // Adding a generic handler method to record all visited nodes and increment the depth counter
00008 void VerilogVisitor::handle(const slang::syntax::SyntaxNode &node) {
00009     std::string indent(depth_ * 2, ' ');
00010     spdlog::debug("{}Node: {}", indent, slang::syntax::toString(node.kind));
00011
00012     // Increase depth, visit child nodes, then decrease depth
00013     depth_++;
00014     visitDefault(node);
00015     depth_--;
00016 }
00017
00018 // Handle module declarations
00019 void VerilogVisitor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00020     std::string indent(depth_ * 2, ' ');
00021
00022     if (module.header && module.header->name.valid()) {
00023         std::string_view moduleName = module.header->name.valueText();
00024         spdlog::debug("{}Module Name: {}", indent, moduleName);
00025
00026         inTargetModule_ = !targetCell_.empty() && moduleName == targetCell_;
00027         if (!inTargetModule_) {
00028             return;
00029         }
00030
00031         // If a specific module name is specified, check if it matches
00032         if (inTargetModule_) {
00033             spdlog::info("{}Found target module: {}", indent, targetCell_);
00034             // Print the module ports
00035             if (module.header->ports) {
00036                 spdlog::info("{}Ports: {}", indent, module.header->ports->toString());
00037             }
00038         }
00039     }
00040 }

```

```

00038     }
00039 }
00040
00041 // Continue processing child nodes
00042 depth++;
00043 visitDefault(module);
00044 depth--;
00045 }
00046
00047 // Handle port declarations
00048 void VerilogVisitor::handle(const slang::syntax::PortDeclarationSyntax &portDecl) {
00049     if (!inTargetModule_) {
00050         return;
00051     }
00052
00053     std::string indent = std::string(depth_ * 2, ' ');
00054
00055     // Get port direction
00056     std::string direction = "unknown";
00057     if (portDecl.header) {
00058         // Try to convert the header to different types of port headers
00059         if (auto *varPort = portDecl.header->as_if<slang::syntax::VariablePortHeaderSyntax>()) {
00060             if (varPort->direction) {
00061                 direction = varPort->direction.valueText();
00062             }
00063         } else if (auto *netPort = portDecl.header->as_if<slang::syntax::NetPortHeaderSyntax>()) {
00064             if (netPort->direction) {
00065                 direction = netPort->direction.valueText();
00066             }
00067         }
00068
00069         // Get port name
00070         for (const auto &declarator : portDecl.declarators) {
00071             if (declarator->name.valid()) {
00072                 std::string_view portName = declarator->name.valueText();
00073                 spdlog::info("{}Port Name: {} ({})", indent, portName, direction);
00074             }
00075         }
00076     }
00077 }
00078
00079 // Handle hierarchical instantiation (module instance)
00080 void VerilogVisitor::handle(const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst) {
00081     if (!inTargetModule_) {
00082         return;
00083     }
00084
00085     std::string indent(depth_ * 2, ' ');
00086
00087     if (hierarchyInst.type.valid()) {
00088         std::string_view instType = hierarchyInst.type.valueText();
00089         spdlog::info("{}Hierarchy Instance Type: {}", indent, instType);
00090
00091         // Safely print parameter values (if any)
00092         try {
00093             if (hierarchyInst.parameters) {
00094                 spdlog::info("{}Parameters:", indent);
00095                 for (const auto &param : hierarchyInst.parameters->parameters) {
00096                     if (!param)
00097                         continue; // Null pointer check
00098
00099                     if (auto *orderedParam = param->as_if<slang::syntax::OrderedParamAssignmentSyntax>()) {
00100                         if (orderedParam->expr) {
00101                             spdlog::info("{} Parameter: {}", indent, orderedParam->expr->toString());
00102                         }
00103                     } else if (auto *namedParam = param->as_if<slang::syntax::NamedParamAssignmentSyntax>()) {
00104                         if (namedParam->name.valid() && namedParam->expr) {
00105                             spdlog::info("{} Parameter {}: {}", indent, namedParam->name.valueText(),
00106                                     namedParam->expr->toString());
00107                         }
00108                     }
00109                 }
00110             }
00111         } catch (...) {
00112             // Ignore exceptions
00113         }
00114     }
00115 }

```

```

00107         }
00108     }
00109 }
00110 }
00111 } catch (const std::exception &e) {
00112     spdlog::warn("{}Exception while processing parameters: {}", indent, e.what());
00113 }
00114
00115 // Safely handle instances
00116 try {
00117     for (const auto &instance : hierarchyInst.instances) {
00118         if (!instance) {
00119             spdlog::warn("{}Invalid instance", indent);
00120             continue;
00121         } else if (!instance->decl) {
00122             spdlog::warn("{}Invalid instance declaration", indent);
00123             // continue;
00124         } else {
00125             try {
00126                 if (instance->decl->name.valid()) {
00127                     std::string_view instName = instance->decl->name.valueText();
00128                     spdlog::info("{}Instance Name: {}", indent, instName);
00129                 }
00130             } catch (...) {
00131                 spdlog::debug("{}Error printing name", indent);
00132             }
00133
00134             // Safely print dimensions
00135             try {
00136                 if (!instance->decl->dimensions.empty()) {
00137                     spdlog::info("{} Dimensions: {}", indent, instance->decl->dimensions.toString());
00138                 }
00139             } catch (...) {
00140                 spdlog::debug("{}Error printing dimensions", indent);
00141             }
00142         }
00143
00144         // Safely print port connections
00145         spdlog::info("{} Port connections:", indent);
00146         for (const auto &conn : instance->connections) {
00147             if (!conn) {
00148                 spdlog::warn("{} Null connection", indent);
00149                 continue; // Null pointer check
00150             }
00151
00152             try {
00153                 // Use as_if method for safe type conversion
00154                 if (auto *ordered = conn->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
00155                     if (ordered->expr) {
00156                         spdlog::info("{} Ordered connection: {}", indent, ordered->expr->toString());
00157                     } else {
00158                         spdlog::info("{} Ordered connection: <empty>", indent);
00159                     }
00160                 } else if (auto *named = conn->as_if<slang::syntax::NamedPortConnectionSyntax>()) {
00161                     if (named->name.valid()) {
00162                         std::string exprStr = named->expr ? named->expr->toString() : "<empty>";
00163                         spdlog::info("{} .{}({})", indent, named->name.valueText(), exprStr);
00164                     }
00165                 } else if (conn->as_if<slang::syntax::EmptyPortConnectionSyntax>()) {
00166                     spdlog::info("{} <empty connection>", indent);
00167                 } else if (conn->as_if<slang::syntax::WildcardPortConnectionSyntax>()) {
00168                     spdlog::info("{} .* (wildcard connection)", indent);
00169                 } else {
00170                     spdlog::info("{} Unknown connection type: {}", indent,
00171                                     slang::syntax::toString(conn->kind));
00172                 }
00173             } catch (const std::exception &e) {
00174                 spdlog::warn("{}Exception processing connection: {}", indent, e.what());
00175             } catch (...) {

```

```

00176         spdlog::warn("{}Unknown exception processing connection", indent);
00177     }
00178 }
00179 }
00180 } catch (const std::exception &e) {
00181     spdlog::warn("{}Exception while processing instances: {}", indent, e.what());
00182 }
00183
00184 // Continue processing child nodes
00185 depth_++;
00186 visitDefault(hierarchyInst);
00187 depth--;
00188 }
00189 }
00190 /*
00191 // Handle primitive gate instantiation
00192 void VerilogVisitor::handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst) {
00193     if (!inTargetModule_) {
00194         return;
00195     }
00196
00197     std::string indent(depth_ * 2, ' ');
00198
00199     // Get primitive gate type
00200     if (primitiveInst.type.valid()) {
00201         std::string_view gateType = primitiveInst.type.valueText();
00202         spdlog::info("{}Primitive Gate: {}", indent, gateType);
00203
00204         // Print delay information (if any)
00205         if (primitiveInst.delay) {
00206             spdlog::info("{}Delay: {}", indent, primitiveInst.delay->toString());
00207         }
00208
00209         // Print strength information (if any)
00210         if (primitiveInst.strength) {
00211             spdlog::info("{}Strength: {}", indent, primitiveInst.strength->toString());
00212         }
00213
00214         // Print each instance
00215         for (const auto &instance : primitiveInst.instances) {
00216             // Primitive gates usually don't have names, but if they do, print them
00217             if (instance->decl && instance->decl->name.valid()) {
00218                 std::string_view instName = instance->decl->name.valueText();
00219                 spdlog::info("{} Gate instance: {}", indent, instName);
00220             }
00221
00222             // Print connections
00223             spdlog::info("{} Gate connections:", indent);
00224             for (size_t i = 0; i < instance->connections.size(); ++i) {
00225                 const auto &conn = instance->connections[i];
00226                 // For gate-level instantiation, connections are usually ordered
00227                 if (auto *ordered = conn->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
00228                     if (ordered->expr) {
00229                         // The first is usually the output, the rest are inputs
00230                         std::string portType = (i == 0) ? "output" : "input";
00231                         spdlog::info("{} {} {}: {}", indent, portType, i, ordered->expr->toString());
00232                     }
00233                 }
00234             }
00235
00236             // Continue to traverse deeper when specific standard gate type
00237             // Create a set of safe standard gate types
00238             static const std::unordered_set<std::string_view> safeGateTypes = {
00239                 "and", "or", "nand", "nor", "xor", "xnor", "not",
00240                 "buf", "bufif0", "bufif1", "notif0", "notif1", "pullup", "pulldown",
00241                 "cmos", "rcmos", "nmos", "pmos", "rnmos", "rpmos", "tran",
00242                 "rtran", "tranif0", "tranif1", "rtranif0", "rtranif1";
00243             }
00244         }

```

```

00245 if (safeGateTypes.find(gateType) != safeGateTypes.end()) {
00246 // Continue processing child nodes
00247 depth++;
00248 visitDefault(primitiveInst);
00249 depth--;
00250 } else {
00251 spdlog::warn("{}Skipping deeper traversal of primitive: {}", indent, gateType);
00252 }
00253
00254 } else {
00255 spdlog::warn("{}Primitive instantiation missing gate type", indent);
00256 }
00257 }
00258 */
00259 // Handle specify block
00260 void VerilogVisitor::handle(const slang::syntax::SpecifyBlockSyntax &specifyBlock) {
00261 if (!inTargetModule_) {
00262 return;
00263 }
00264
00265 std::string indent(depth_ * 2, ' ');
00266 spdlog::info("{}Specify Block:", indent);
00267
00268 // Iterate through all path declarations
00269 for (const auto &item : specifyBlock.items) {
00270 if (auto *pathDecl = item->as_if<slang::syntax::PathDeclarationSyntax>()) {
00271 if (pathDecl->desc) {
00272 std::string pathSrc;
00273 std::string pathDst;
00274
00275 // Get path source
00276 if (!pathDecl->desc->inputs.empty()) {
00277 // Get the first input
00278 if (auto *identifier =
00279 pathDecl->desc->inputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
00280 if (identifier->identifier.valid()) {
00281 pathSrc = identifier->identifier.valueText();
00282 }
00283 }
00284 }
00285
00286 // Get path destination
00287 if (pathDecl->desc->suffix) {
00288 if (auto *simpleSuffix =
00289 pathDecl->desc->suffix->as_if<slang::syntax::SimplePathSuffixSyntax>()) {
00290 if (!simpleSuffix->outputs.empty()) {
00291 if (auto *identifier =
00292 simpleSuffix->outputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
00293 if (identifier->identifier.valid()) {
00294 pathDst = identifier->identifier.valueText();
00295 }
00296 }
00297 } else if (auto *edgeSuffix =
00298 pathDecl->desc->suffix
00299 ->as_if<slang::syntax::EdgeSensitivePathSuffixSyntax>()) {
00300 if (!edgeSuffix->outputs.empty()) {
00301 if (auto *identifier =
00302 edgeSuffix->outputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
00303 if (identifier->identifier.valid()) {
00304 pathDst = identifier->identifier.valueText();
00305 }
00306 }
00307 }
00308 }
00309 }
00310
00311 // Construct path string
00312 std::string pathStr = pathSrc + " => " + pathDst;
00313

```

```

00314         // Get path delay
00315         std::string delayStr = "(";
00316         for (size_t i = 0; i < pathDecl->delays.size(); ++i) {
00317             if (i > 0)
00318                 delayStr += ", ";
00319             delayStr += pathDecl->delays[i]->toString();
00320         }
00321         delayStr += ")";
00322
00323         spdlog::info("{} Path: {} = {}", indent, pathStr, delayStr);
00324     }
00325 }
00326 // Can add handling for ConditionalPathDeclarationSyntax and IfNonePathDeclarationSyntax
00327 else if (auto *condPath = item->as_if<slang::syntax::ConditionalPathDeclarationSyntax>()) {
00328     if (condPath->path && condPath->predicate) {
00329         spdlog::info("{} Conditional Path (if {})", indent, condPath->predicate->toString());
00330         // Recursively handle internal path
00331         handle(*condPath->path);
00332     }
00333 } else if (auto *ifNonePath = item->as_if<slang::syntax::IfNonePathDeclarationSyntax>()) {
00334     if (ifNonePath->path) {
00335         spdlog::info("{} If-None Path", indent);
00336         // Recursively handle internal path
00337         handle(*ifNonePath->path);
00338     }
00339 }
00340 }
00341
00342 // Continue processing child nodes
00343 depth_++;
00344 visitDefault(specifyBlock);
00345 depth--;
00346 }
00347 }
00348
00349 // Handle module declarations, only keep the target module
00350 void CellExtractor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00351     if (module.header && module.header->name.valid()) {
00352         std::string_view moduleName = module.header->name.valueText();
00353
00354         // If it is the target module, mark as found, otherwise remove it
00355         if (!targetCell_.empty() && moduleName == targetCell_) {
00356             foundTarget_ = true;
00357             // Do not modify, keep this module
00358         } else {
00359             // Remove non-target modules
00360             remove(module);
00361         }
00362     }
00363 }
00364
00365 // Get result
00366 bool CellExtractor::foundTargetCell()const { return foundTarget_; }
00367
00368 // Handle module declarations
00369 void CellPrinter::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00370     if (module.header && module.header->name.valid()) {
00371         std::string_view moduleName = module.header->name.valueText();
00372
00373         // Check if it is the target module
00374         if (!targetCell_.empty() && moduleName == targetCell_) {
00375             foundTarget_ = true;
00376
00377             // Print module definition
00378             out_ << "`timescale 1ns/10ps\n";
00379             out_ << module.toString();
00380
00381             return; // Do not traverse further
00382         }

```



```

00383 }
00384
00385 // Continue traversing other modules
00386 visitDefault(module);
00387 }
00388
00398 void ModuleRewriter::handle(const slang::syntax::SyntaxNode &node) {
00399     std::string indent(depth_ * 2, ' ');
00400     logger_->debug("{}Node: {}", indent, slang::syntax::toString(node.kind));
00401
00402     // Continue processing child nodes
00403     depth_++;
00404     visitDefault(node);
00405     depth_--;
00406 }
00407
00408 void ModuleRewriter::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00409     logger_->debug("Processing module: {}", module.header->name.valueText());
00410
00411     // Create intermediate wires for connections between instances
00412     for (int i = 0; i < instance_count_ - 1; i++) {
00413         auto &newNetNode = parse("\n wire OP_" + std::to_string(i) + ";");
00414         insertAtBack(module.members, newNetNode);
00415         logger_->debug("Added intermediate wire: {}", newNetNode.toString());
00416     }
00417
00418     // Get critical input port & output port from
00419     // the module name pattern: CELLNAME_X#_CRITICALPORT__OUTPUTPORT
00420     std::string criticalInputPort = "";
00421     std::string criticalOutputPort = "";
00422     if (!this->moduleName_.empty()) {
00423         size_t firstSep = this->moduleName_.find("__");
00424         if (firstSep != std::string::npos) {
00425             size_t secondSep = this->moduleName_.find("__", firstSep + 2);
00426             if (secondSep != std::string::npos) {
00427                 size_t thirdSep = this->moduleName_.find("__", secondSep + 2);
00428                 if (thirdSep != std::string::npos) {
00429                     criticalInputPort = this->moduleName_.substr(secondSep + 2, thirdSep - (secondSep + 2));
00430                     criticalOutputPort = this->moduleName_.substr(thirdSep + 2);
00431                     logger_->debug("Extracted critical input port: {}", criticalInputPort);
00432                     logger_->debug("Extracted output port: {}", criticalOutputPort);
00433                 } else {
00434                     logger_->warn("Can't find thirdSep. Invalid module name pattern: {}", this->moduleName_);
00435                     return;
00436                 }
00437             } else {
00438                 logger_->warn("Can't find secondSep. Invalid module name pattern: {}", this->moduleName_);
00439                 return;
00440             }
00441         } else {
00442             logger_->warn("Can't find firstSep. Invalid module name pattern: {}", this->moduleName_);
00443             return;
00444         }
00445     } else {
00446         logger_->warn("Module name is empty. Can't extract critical input port.");
00447         return;
00448     }
00449
00450     // Add additional wires for intermediate outputs that aren't part of the chain
00451     if (this->outputPins_.size() > 1) { // Only add if there are multiple output pins
00452         for (int i = 0; i < instance_count_ - 1; i++) { // Skip the first and last instances
00453             for (const auto &outputPin : this->outputPins_) {
00454                 if (outputPin != criticalOutputPort) { // Found an intermediate output pin
00455                     auto &newNetNode = parse("\n wire P_" + std::to_string(i) + "__" + outputPin + ";");
00456                     insertAtBack(module.members, newNetNode);
00457                     logger_->debug("Added intermediate wire: {}", newNetNode.toString());
00458                 }
00459             }
00460         }
00461     }

```

```

00461 }
00462
00463 std::string portList_str = module.header->ports->toString();
00464 logger_>debug("Port list: {}", portList_str);
00465
00466 std::vector<std::string> allPorts;
00467 auto ansiPortList = module.header->ports->as_if<slang::syntax::AnsiPortListSyntax>();
00468 if (!ansiPortList) {
00469     logger_>warn("Port list is not ANSI style.");
00470     return;
00471 }
00472 portInfoMap_.clear(); // Clear the port info map
00473 for (const auto portMember : ansiPortList->ports) {
00474     if (portMember->kind == slang::syntax::SyntaxKind::ImplicitAnsiPort) {
00475         const auto implicitPort = portMember->as_if<slang::syntax::ImplicitAnsiPortSyntax>();
00476         const auto &directionToken =
00477             implicitPort->header->as_if<slang::syntax::VariablePortHeaderSyntax>()->direction;
00478         const auto &nameToken = implicitPort->declarator->name;
00479
00480         std::string_view portName = nameToken.valueText();
00481         std::string_view direction = directionToken.valueText();
00482         portInfoMap_[std::string(portName)] = std::string(direction); // Store port info in the map
00483         logger_>debug("Port Name: {}, Direction: {}", portName, direction);
00484     } else {
00485         logger_>warn("Port member is not ImplicitAnsiPort.");
00486     }
00487 }
00488
00489 for (int i = 0; i < instance_count_; i++) {
00490     std::string instanceName = "I_" + this->cellName_ + "__X" + std::to_string(i) + "__" +
00491         criticalInputPort + "__" + criticalOutputPort;
00492     std::string instanceCode = "\n " + this->cellName_ + " " + instanceName + " (";
00493     std::vector<std::string> portConnections;
00494
00495     for (const auto &pair : portInfoMap_) {
00496         std::string portName = pair.first;
00497         std::string direction = pair.second;
00498         std::string connectionName;
00499
00500         if (portName == criticalInputPort) {
00501             if (i == 0) {
00502                 connectionName = portName; // First instance: connect to module input port
00503             } else {
00504                 connectionName =
00505                     "OP_" + std::to_string(i - 1); // Intermediate instances: connect to previous OP wire
00506             }
00507         } else if (portName == criticalOutputPort) {
00508             if (i == instance_count_ - 1) {
00509                 connectionName = portName; // Last instance: connect to module output port
00510             } else {
00511                 connectionName = "OP_" + std::to_string(i); // Intermediate instances: connect to OP wire
00512             }
00513         } else if (direction == "output") { // Handle other output ports
00514             if (i < instance_count_ - 1) {
00515                 connectionName = "P_" + std::to_string(i) + "__" +
00516                     portName; // Connect to intermediate P_i__portName wire
00517             } else {
00518                 connectionName = portName; // Last instance: connect to module output port
00519             }
00520         } else { // For input ports (and inout?), connect directly to module port
00521             connectionName = portName;
00522         }
00523         portConnections.push_back("." + portName + "(" + connectionName + ")");
00524     }
00525
00526     // Connect all ports with comma and space
00527     if (!portConnections.empty()) {
00528         instanceCode += portConnections[0];
00529         for (size_t i = 1; i < portConnections.size(); ++i) {

```

```

00530         instanceCode += " , " + portConnections[i];
00531     }
00532 }
00533 instanceCode += ");";
00534
00535 // Blank line before the first instance
00536 if (i == 0) {
00537     instanceCode = "\n" + instanceCode;
00538 }
00539
00540 // Insert the instance code
00541 auto &instanceNode = parse(instanceCode);
00542 insertAtBack(module.members, instanceNode);
00543 }
00544 }
00545
00546 void getAST(const std::string &verilog_file, const std::string &cell) {
00547     try {
00548         spdlog::info("Starting get AST from Verilog file: '{}'", verilog_file);
00549         auto result = slang::syntax::SyntaxTree::fromFile(verilog_file);
00550         if (result) {
00551             spdlog::info("Successfully parsed Verilog file.");
00552
00553             try {
00554                 // First, use the visitor to print basic information
00555                 VerilogVisitor visitor(cell);
00556                 visitor.visit(result.value()->root());
00557
00558                 // If a target cell is specified, use the rewriter to extract the relevant code
00559                 if (!cell.empty()) {
00560                     // Create a rewriter to only keep the target cell related code
00561                     CellExtractor extractor(cell);
00562                     auto extractedTree = extractor.transform(result.value());
00563
00564                     if (extractor.foundTargetCell()) {
00565                         // Use SyntaxPrinter to print the extracted code
00566                         std::string extractedCode = slang::syntax::SyntaxPrinter::printFile(*extractedTree);
00567
00568                         // Save to a file named after the cell name
00569                         std::string outputFile = cell + ".v";
00570                         std::ofstream cellOut(outputFile);
00571                         if (cellOut) {
00572                             cellOut << extractedCode;
00573                             cellOut.close();
00574                             spdlog::info("Extracted '{}' cell code to '{}'", cell, outputFile);
00575                         } else {
00576                             spdlog::error("Failed to write extracted cell code to '{}'", outputFile);
00577                         }
00578                     } else {
00579                         spdlog::warn("Target cell '{}' not found in the Verilog file", cell);
00580                     }
00581                 }
00582
00583                 // spdlog::info("Print full source code to 'full_source_code.v'");
00584                 // // Optionally save the entire syntax tree
00585                 // std::string fullOutput = slang::syntax::SyntaxPrinter::printFile(*result.value());
00586                 // std::ofstream out("full_source_code.v");
00587                 // out << fullOutput;
00588                 // out.close();
00589
00590                 spdlog::info("Print target cell code to 'cell_code.v using toString()'");
00591                 // Use CellPrinter to print the target cell's code
00592                 std::ofstream cellOut("cell_code.v");
00593                 CellPrinter cellPrinter(cell, cellOut);
00594                 cellPrinter.visit(result.value()->root());
00595                 cellOut.close();
00596
00597             } catch (const std::exception &e) {
00598                 spdlog::error("Exception during AST traversal: {}", e.what());
00599             }
00600         }
00601     }
00602 }

```

```

00599     } catch (...) {
00600         spdlog::error("Unknown exception during AST traversal");
00601     }
00602 } else {
00603     spdlog::error("Error parsing Verilog file.");
00604 }
00605 } catch (const std::exception &e) {
00606     spdlog::error("Exception during Verilog parsing: {}", e.what());
00607 } catch (...) {
00608     spdlog::error("Unknown exception during Verilog parsing");
00609 }
00610 }
00611
00612 // --- Implementation for LogicExtractor ---
00613
00614 void LogicExtractor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00615     if (parsingComplete_)
00616         return; // Don't re-process if called again
00617
00618     if (module.header && module.header->name.valid()) {
00619         std::string_view moduleName = module.header->name.valueText();
00620         if (!targetCell_.empty() && moduleName == targetCell_) {
00621             spdlog::info("LogicExtractor: Found target module: {}", targetCell_);
00622             inTargetModule_ = true;
00623
00624             // Reset state for the target module
00625             primaryInputs_.clear();
00626             primaryOutputs_.clear();
00627             internalWires_.clear();
00628             portDirections_.clear(); // Clear temporary direction map
00629             gateOutputDrivers_.clear();
00630             // logicCache_.clear(); // For Step 2
00631
00632             // Visit children (ports, declarations, instances) IN ORDER
00633             // It's often better to visit declarations first, then the port list
00634             // Slang's default visitDefault might handle this, but be aware.
00635             visitDefault(module);
00636
00637             // --- Finalize Ports after visiting declarations and port list ---
00638             // This part might be better placed *after* visiting NonAnsiPortList,
00639             // assuming the list visit confirms the names from the header.
00640             // Let's move the finalization logic to handle(NonAnsiPortListSyntax)
00641
00642             // Mark that we finished processing the target module
00643             inTargetModule_ = false; // Important to prevent processing other modules
00644             parsingComplete_ = true; // Stop processing after finding the target
00645             spdlog::info("LogicExtractor: Finished visiting target module '{}'.", targetCell_);
00646
00647         } else if (!parsingComplete_) {
00648             // If not the target module yet, continue searching
00649             visitDefault(module);
00650         }
00651     } else if (!parsingComplete_) {
00652         // Handle modules without valid headers if necessary, or just traverse
00653         visitDefault(module);
00654     }
00655 }
00656
00657 // Handle Port Declarations (defines direction and type, usually inside module body)
00658 void LogicExtractor::handle(const slang::syntax::PortDeclarationSyntax &portDecl) {
00659     if (!inTargetModule_)
00660         return;
00661     spdlog::debug("LogicExtractor: Handling Port Declaration");
00662
00663     std::string direction = "unknown";
00664     if (portDecl.header) {
00665         // Determine direction (input/output/inout)
00666         // Important: Use valueText() as Slang typically represents keywords as text tokens
00667         if (auto varHeader = portDecl.header->as_if<slang::syntax::VariablePortHeaderSyntax>()) {

```

```

00668     if (varHeader->direction.valid()) {
00669         direction = std::string(varHeader->direction.valueText());
00670     }
00671 } else if (auto netHeader = portDecl.header->as_if<slang::syntax::NetPortHeaderSyntax>()) {
00672     if (netHeader->direction.valid()) {
00673         direction = std::string(netHeader->direction.valueText());
00674     }
00675 }
00676 // Add InterfacePortHeaderSyntax if needed
00677 } else {
00678     spdlog::warn("LogicExtractor: Port declaration without header found.");
00679 }
00680
00681 for (const auto &declarator : portDecl.declarators) {
00682     if (declarator->name.valid()) {
00683         std::string portName = std::string(declarator->name.valueText());
00684         if (portDirections_.count(portName) && portDirections_[portName] != "unknown") {
00685             spdlog::warn("LogicExtractor: Multiple direction declarations for port '{}'. Keeping first "
00686                 "one '{}'. New direction: '{}'",
00687                 portName, portDirections_[portName], direction);
00688         } else {
00689             spdlog::info("LogicExtractor: Storing direction '{}' for port '{}'", direction, portName);
00690             portDirections_[portName] = direction;
00691             if (direction == "input") {
00692                 primaryInputs_.insert(portName);
00693                 internalWires_.insert(portName); // Inputs are also technically 'wires' usable internally
00694             } else if (direction == "output") {
00695                 primaryOutputs_.insert(portName);
00696                 internalWires_.insert(portName); // Outputs are also wires driven by something
00697             } else if (direction == "inout") {
00698                 spdlog::info("LogicExtractor: Inout port '{}' found. Adding to both inputs and outputs.",
00699                     portName);
00700                 primaryInputs_.insert(portName);
00701                 primaryOutputs_.insert(portName);
00702                 internalWires_.insert(portName);
00703             } else {
00704                 spdlog::warn("LogicExtractor: Port '{}' found in declaration but has unknown or missing "
00705                     "direction '{}'. Treating as internal wire.",
00706                     portName, direction);
00707                 internalWires_.insert(portName);
00708             }
00709             // logicCache[portName] = portName; // For Step 2
00710         }
00711     } else {
00712         spdlog::warn(
00713             "LogicExtractor: Port declarator without a name found in PortDeclarationSyntax.");
00714     }
00715 }
00716 // Don't call visitDefault here, as it might revisit things unexpectedly.
00717 // Let the main module visitor handle traversing into children if necessary.
00718 }
00719
00720 // Handle Non-ANSI Port List (defines names, usually in module header)
00721 void LogicExtractor::handle(const slang::syntax::NonAnsiPortListSyntax &portList) {
00722     if (!inTargetModule_)
00723         return;
00724     spdlog::debug("LogicExtractor: Handling NonAnsi Port List");
00725
00726     for (const auto portSyntax : portList.ports) {
00727         if (!portSyntax)
00728             continue;
00729
00730         std::string portName = "";
00731         // Most common case: ImplicitNonAnsiPortSyntax contains the expression (usually just the name)
00732         if (auto implicitPort = portSyntax->as_if<slang::syntax::ImplicitNonAnsiPortSyntax>()) {
00733             if (implicitPort->expr) {
00734                 // The expression itself might be complex, try to get the simple name
00735                 if (auto portRef = implicitPort->expr->as_if<slang::syntax::PortReferenceSyntax>()) {
00736                     if (portRef->name.valid()) {

```

```

00737         portName = std::string(portRef->name.valueText());
00738     }
00739 } else if (auto portConcat =
00740     implicitPort->expr->as_if<slang::syntax::PortConcatenationSyntax>()) {
00741     // Handle concatenations if necessary - complex
00742     spdlog::warn("LogicExtractor: Port concatenation found in NonAnsi port list - currently "
00743         "not fully handled for logic extraction. Port: {}",
00744         implicitPort->toString());
00745     continue; // Skip complex ports for now
00746 } else {
00747     // Fallback: try getting name from the expression directly (might be just identifier)
00748     portName = implicitPort->toString();
00749 }
00750 }
00751 }
00752 // Handle ExplicitNonAnsiPortSyntax (e.g., .A(A)) if needed
00753 else if (auto explicitPort = portSyntax->as_if<slang::syntax::ExplicitNonAnsiPortSyntax>()) {
00754     if (explicitPort->name.valid()) {
00755         portName = std::string(explicitPort->name.valueText());
00756         // Note: explicitPort->expr is the internal signal it connects to.
00757         // We are primarily interested in the port's own name (explicitPort->name) here.
00758     }
00759 }
00760 // Handle EmptyNonAnsiPortSyntax (commas for placeholders) if necessary
00761 else if (portSyntax->kind == slang::syntax::SyntaxKind::EmptyNonAnsiPort) {
00762     spdlog::debug("LogicExtractor: Skipping empty non-ANSI port placeholder.");
00763     continue;
00764 } else {
00765     spdlog::warn("LogicExtractor: Unhandled NonAnsi port syntax kind: {}",
00766         slang::syntax::toString(portSyntax->kind));
00767     continue;
00768 }
00769
00770 // Now we (hopefully) have the portName.
00771 if (!portName.empty()) {
00772     if (portDirections_.count(portName)) {
00773         const std::string &direction = portDirections_[portName];
00774         spdlog::debug("LogicExtractor: Finalizing port '{}' with direction '{}'", portName,
00775             direction);
00776
00777         if (direction == "input") {
00778             primaryInputs_.insert(portName);
00779             internalWires_.insert(portName); // Inputs are also technically 'wires' usable internally
00780             // logicCache_[portName] = portName; // For Step 2
00781         } else if (direction == "output") {
00782             primaryOutputs_.insert(portName);
00783             internalWires_.insert(portName); // Outputs are also wires driven by something
00784         } else if (direction == "inout") {
00785             spdlog::info("LogicExtractor: Inout port '{}' found. Adding to both inputs and outputs.",
00786                 portName);
00787             primaryInputs_.insert(portName);
00788             primaryOutputs_.insert(portName);
00789             internalWires_.insert(portName);
00790             // logicCache_[portName] = portName; // For Step 2
00791         } else {
00792             spdlog::warn("LogicExtractor: Port '{}' found in list but has unknown or missing "
00793                 "direction '{}'. Treating as internal wire.",
00794                 portName, direction);
00795             internalWires_.insert(portName);
00796         }
00797     } else {
00798         spdlog::debug("LogicExtractor: Port '{}' found in NonAnsi list, no direction declaration ",
00799             portName);
00800     }
00801 } else {
00802     spdlog::warn("LogicExtractor: Could not determine port name from NonAnsi port list item: {}",
00803         portSyntax->toString());
00804 }
00805 }

```

```

00806 // Don't call visitDefault here
00807 }
00808
00809 // Handle explicit wire declarations
00810 void LogicExtractor::handle(const slang::syntax::NetDeclarationSyntax &netDecl) {
00811     if (!inTargetModule_)
00812         return;
00813
00814     for (const auto &declarator : netDecl.declarators) {
00815         if (declarator->name.valid()) {
00816             std::string wireName = std::string(declarator->name.valueText());
00817             spdlog::debug("LogicExtractor: Found wire declaration: {}", wireName);
00818             // Avoid adding ports again if they were also declared as nets (common)
00819             if (!primaryInputs_.count(wireName) && !primaryOutputs_.count(wireName)) {
00820                 internalWires_.insert(wireName);
00821             } else {
00822                 spdlog::trace("LogicExtractor: Wire '{}' is already known as a port.", wireName);
00823             }
00824         }
00825     }
00826     // Don't call visitDefault here
00827 }
00828
00829 // Handle primitive gate instantiations (MOST IMPORTANT PART)
00830 void LogicExtractor::handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst) {
00831     if (!inTargetModule_)
00832         return;
00833
00834     if (!primitiveInst.type.valid()) {
00835         spdlog::warn("LogicExtractor: Primitive instance without a type token found.");
00836         return;
00837     }
00838
00839     // Use token kind for reliable checking, text for name storage
00840     slang::parsing::TokenKind gateKind = primitiveInst.type.kind;
00841     std::string gateTypeName = std::string(primitiveInst.type.valueText());
00842
00843     spdlog::debug("LogicExtractor: Found Primitive Instance of Type: {} (Kind: {})", gateTypeName,
00844                  slang::parsing::toString(gateKind));
00845
00846     for (const auto &instance : primitiveInst.instances) {
00847         // if (!instance || !instance->connections.isInitialized()) { // Check if connections are valid
00848         //     spdlog::warn("LogicExtractor: Skipping primitive instance of type {} due to null pointer or
00849         //         "
00850         //             "uninitialized connections.",
00851         //             gateTypeName);
00852         //     continue;
00853         // }
00854
00855         GateInfo currentGateInfo;
00856         currentGateInfo.gateTypeName = gateTypeName;
00857         currentGateInfo.kind = gateKind;
00858
00859         // Primitives usually have ordered connections.
00860         // The FIRST connection is typically the OUTPUT.
00861         // The REST are INPUTS.
00862         if (instance->connections.empty()) {
00863             spdlog::warn("LogicExtractor: Gate instance of type {} has no connections.", gateTypeName);
00864             continue;
00865         }
00866
00867         // Extract Output Signal
00868         // Ensure the connection itself is not null
00869         if (!instance->connections[0]) {
00870             spdlog::error("LogicExtractor: First connection (output) is null for primitive {}",
00871                          gateTypeName);
00872             continue;
00873         }
00874         if (auto firstConn =

```

```

00875         instance->connections[0]->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
00876         currentGateInfo.outputSignal =
00877             firstConn->expr->toString(); // Get the output signal name from the connection
00878             // remove extra spaces in the signal name
00879         currentGateInfo.outputSignal.erase(
00880             std::remove_if(currentGateInfo.outputSignal.begin(), currentGateInfo.outputSignal.end(),
00881                 [](unsigned char x) { return std::isspace(x); }),
00882             currentGateInfo.outputSignal.end());
00883         if (!currentGateInfo.outputSignal.empty()) {
00884             spdlog::debug(" Output Signal: {}", currentGateInfo.outputSignal);
00885             // Gate outputs are internal signals (unless they are module outputs)
00886             // Add to internal wires if not already a primary output.
00887             if (!primaryOutputs_.count(currentGateInfo.outputSignal)) {
00888                 internalWires_.insert(currentGateInfo.outputSignal);
00889             }
00890         } else {
00891             spdlog::error(
00892                 "LogicExtractor: Could not determine output signal name for gate instance of type {}",
00893                 gateTypeName);
00894             continue; // Skip this instance if output is unknown
00895         }
00896     } else {
00897         spdlog::error("LogicExtractor: Expected OrderedPortConnectionSyntax for output of primitive "
00898             "{}", but got kind: {}. Conn: {}",
00899             gateTypeName, slang::syntax::toString(instance->connections[0]->kind),
00900             instance->connections[0]->toString());
00901         continue; // Skip this instance
00902     }
00903
00904     // Extract Input Signals
00905     for (size_t i = 1; i < instance->connections.size(); ++i) {
00906         // Ensure the connection itself is not null
00907         if (!instance->connections[i]) {
00908             spdlog::warn("LogicExtractor: Input connection {} is null for primitive {}", i,
00909                 gateTypeName);
00910             continue;
00911         }
00912         if (auto conn =
00913             instance->connections[i]->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
00914             std::string inputSig =
00915                 conn->expr->toString(); // Get the input signal name from the connection
00916             // remove extra spaces in the signal name
00917             inputSig.erase(std::remove_if(inputSig.begin(), inputSig.end(),
00918                 [](unsigned char x) { return std::isspace(x); }),
00919                 inputSig.end());
00920             if (!inputSig.empty()) {
00921                 currentGateInfo.inputSignals.push_back(inputSig);
00922                 spdlog::debug(" Input Signal {}: {}", i, inputSig);
00923             } else {
00924                 spdlog::warn("LogicExtractor: Could not determine input signal name for input {} of gate "
00925                     "instance type {}. Conn: {}",
00926                     i, gateTypeName, conn->toString());
00927                 // Decide whether to skip or continue with partial inputs
00928             }
00929         } else {
00930             spdlog::warn("LogicExtractor: Expected OrderedPortConnectionSyntax for input {} of "
00931                 "primitive {}, but got kind: {}. Conn: {}",
00932                 i, gateTypeName, slang::syntax::toString(instance->connections[i]->kind),
00933                 instance->connections[i]->toString());
00934         }
00935     }
00936
00937     // Store the gate information, mapping the output signal to its driving gate
00938     if (!currentGateInfo.outputSignal.empty()) {
00939         if (gateOutputDrivers_.count(currentGateInfo.outputSignal)) {
00940             // This is a critical warning - indicates multiple drivers for the same net!
00941             spdlog::error("LogicExtractor: Multiple drivers found for signal '{}'. Previous driver: "
00942                 "{}", New driver: {}. Netlist is likely invalid.",
00943                 currentGateInfo.outputSignal,

```



```

00944         gateOutputDrivers_[currentGateInfo.outputSignal].gateTypeName, gateTypeName);
00945         // Keep the first one found for now, or decide on error handling
00946     } else {
00947         spdlog::info(" Storing driver for '{}': Gate Type '{}'", currentGateInfo.outputSignal,
00948             gateTypeName);
00949         gateOutputDrivers_[currentGateInfo.outputSignal] = currentGateInfo;
00950     }
00951 }
00952 }
00953 // Don't call visitDefault here
00954 }
00955
00956 // --- Logic Derivation Implementation ---
00957
00958 // Public method called after visiting the tree
00959 std::map<std::string, std::string> LogicExtractor::getLogicExpressions() {
00960     std::map<std::string, std::string> result_map;
00961     if (!parsingComplete_) {
00962         spdlog::error("LogicExtractor: AST parsing did not complete or target module '{}' not found. "
00963             "Cannot extract logic.",
00964             targetCell_);
00965         return result_map; // Return empty map
00966     }
00967
00968     spdlog::info("LogicExtractor: Deriving logic expressions for {} output ports...",
00969         primaryOutputs_.size());
00970
00971     for (const std::string &outputPort : primaryOutputs_) {
00972         spdlog::debug("LogicExtractor: Deriving logic for output: {}", outputPort);
00973         try {
00974             result_map[outputPort] = deriveLogicRecursive(outputPort);
00975             spdlog::info(" Output: {} => {}", outputPort, result_map[outputPort]);
00976         } catch (const std::runtime_error &e) {
00977             spdlog::error("LogicExtractor: Error deriving logic for output '{}': {}", outputPort,
00978                 e.what());
00979             result_map[outputPort] = "/* Error deriving logic */";
00980         } catch (...) {
00981             spdlog::error("LogicExtractor: Unknown error deriving logic for output '{}'", outputPort);
00982             result_map[outputPort] = "/* Unknown error deriving logic */";
00983         }
00984     }
00985     return result_map;
00986 }
00987
00988 // Recursive function with memoization
00989 std::string LogicExtractor::deriveLogicRecursive(const std::string &signalName) {
00990     // 1. Check Cache (Memoization)
00991     if (logicCache_.count(signalName)) {
00992         return logicCache_.at(signalName);
00993     }
00994
00995     // 2. Base Case: Is it a primary input?
00996     if (primaryInputs_.count(signalName)) {
00997         // Already cached during port handling, but double-check
00998         if (!logicCache_.count(signalName)) {
00999             logicCache_[signalName] = signalName;
01000         }
01001         return signalName;
01002     }
01003
01004     // 3. Recursive Step: Is it driven by a gate?
01005     if (gateOutputDrivers_.count(signalName)) {
01006         const GateInfo &driverGate = gateOutputDrivers_.at(signalName);
01007         std::vector<std::string> inputExpressions;
01008         spdlog::debug(" Tracing signal '{}', driven by {} gate", signalName,
01009             driverGate.gateTypeName);
01010
01011         // Recursively find expressions for all inputs of this gate
01012         for (const std::string &inputSig : driverGate.inputSignals) {

```

```

01013     if (inputSig.empty()) {
01014         throw std::runtime_error("Empty input signal name encountered for gate driving " +
01015                                 signalName);
01016     }
01017     spdlog::debug("      Recursing for input: {}", inputSig);
01018     inputExpressions.push_back(deriveLogicRecursive(inputSig));
01019 }
01020
01021 // Format the expression based on gate type and input expressions
01022 std::string currentExpr = formatExpression(driverGate, inputExpressions);
01023
01024 // Cache the result
01025 logicCache_[signalName] = currentExpr;
01026 return currentExpr;
01027 }
01028
01029 // 4. Handle Assign statements (if implemented)
01030 // if (assignDrivers_.count(signalName)) { ... }
01031
01032 // 5. Error Case: Signal not found or not driven by known element
01033 // Check if it's just an internal wire that wasn't driven?
01034 if (internalWires_.count(signalName)) {
01035     throw std::runtime_error(
01036         "Signal '" + signalName +
01037         "' is an internal wire but has no identified driver (gate or assign).");
01038 } else {
01039     throw std::runtime_error(
01040         "Signal '" + signalName +
01041         "' is not a primary input, known wire, or driven by a recognized gate/assignment.");
01042 }
01043 }
01044
01045 // Helper to format the expression string based on gate type
01046 std::string LogicExtractor::formatExpression(const GateInfo &gateInfo,
01047                                             const std::vector<std::string> &inputExprs) {
01048     if (inputExprs.empty() && gateInfo.kind != slang::parsing::TokenKind::NotKeyword &&
01049         gateInfo.kind != slang::parsing::TokenKind::BufKeyword) {
01050         // Gates like AND/OR/XOR need inputs
01051         spdlog::warn("Gate type {} requires inputs, but none were provided/derived for output {}",
01052                     gateInfo.gateTypeName, gateInfo.outputSignal);
01053         return "/*<Error: Missing Inputs for " + gateInfo.gateTypeName + ">*/";
01054     }
01055
01056     std::string result = "";
01057
01058     // Use gateInfo.type (enum) for reliable checking
01059     switch (gateInfo.kind) {
01060     case slang::parsing::TokenKind::AndKeyword:
01061         result = "(" + inputExprs[0];
01062         for (size_t i = 1; i < inputExprs.size(); ++i)
01063             result += " * " + inputExprs[i]; // Use * for AND
01064         result += ")";
01065         break;
01066     case slang::parsing::TokenKind::NandKeyword:
01067         result = "!(" + inputExprs[0];
01068         for (size_t i = 1; i < inputExprs.size(); ++i)
01069             result += " * " + inputExprs[i];
01070         result += ")";
01071         break;
01072     case slang::parsing::TokenKind::OrKeyword:
01073         result = "(" + inputExprs[0];
01074         for (size_t i = 1; i < inputExprs.size(); ++i)
01075             result += " + " + inputExprs[i]; // Use + for OR
01076         result += ")";
01077         break;
01078     case slang::parsing::TokenKind::NorKeyword:
01079         result = "!(" + inputExprs[0];
01080         for (size_t i = 1; i < inputExprs.size(); ++i)
01081             result += " + " + inputExprs[i];

```

```

01082     result += ")";
01083     break;
01084 case slang::parsing::TokenKind::XorKeyword:
01085     result = "(" + inputExprs[0];
01086     for (size_t i = 1; i < inputExprs.size(); ++i)
01087         result += " ^ " + inputExprs[i]; // Use ^ for XOR
01088     result += ")";
01089     break;
01090 case slang::parsing::TokenKind::XnorKeyword:
01091     result = "!(" + inputExprs[0];
01092     for (size_t i = 1; i < inputExprs.size(); ++i)
01093         result += " ^ " + inputExprs[i];
01094     result += ")";
01095     break;
01096 case slang::parsing::TokenKind::NotKeyword: // NOT gate
01097     if (inputExprs.size() != 1) {
01098         spdlog::warn("NOT gate expects 1 input, got {}", inputExprs.size());
01099         return "/*<Error: Incorrect Inputs for NOT>*/";
01100     }
01101     result = "!" + inputExprs[0]; // Use ! for NOT
01102     break;
01103 case slang::parsing::TokenKind::BufKeyword: // BUF gate
01104     if (inputExprs.size() != 1) {
01105         spdlog::warn("BUF gate expects 1 input, got {}", inputExprs.size());
01106         return "/*<Error: Incorrect Inputs for BUF>*/";
01107     }
01108     result = inputExprs[0]; // Output is the same as input
01109     break;
01110 // Add cases for bufif0, bufif1, notif0, notif1, pullup, pulldown, cmos, nmos, pmos, tran etc. if
01111 // needed
01112 default:
01113     spdlog::warn("Unsupported primitive gate type for logic expression generation: {}",
01114                 gateInfo.gateTypeName);
01115     result = "/*<Unsupported Gate: " + gateInfo.gateTypeName + ">*/";
01116     break;
01117 }
01118 return result;
01119 }
01120
01121 // --- New Function Implementation (Step 1: Visit and Print) ---
01122
01123 void extractAndPrintNetlistInfo(const std::string &verilog_file, const std::string &cell) {
01124     spdlog::info("--- Step 1: Starting Netlist Info Extraction ---");
01125     spdlog::info("Verilog file: '{}', Target cell: '{}'", verilog_file, cell);
01126
01127     try {
01128         auto result = slang::syntax::SyntaxTree::fromFile(verilog_file);
01129         if (!result) { // Check if result is valid
01130             spdlog::error("Error parsing Verilog file '{}'. Cannot extract info.", verilog_file);
01131             return;
01132         }
01133
01134         spdlog::info("Successfully parsed Verilog file.");
01135         std::shared_ptr<slang::syntax::SyntaxTree> tree = result.value();
01136
01137         LogicExtractor extractor(cell);
01138         tree->root().visit(extractor); // Populate extractor's internal state
01139
01140         // --- Print Summary (Debug for Step 1) ---
01141         spdlog::info("--- Extraction Summary for Cell '{}': ---", cell);
01142
01143         const auto &inputs = extractor.getPrimaryInputs();
01144         spdlog::info("Found {} Primary Inputs:", inputs.size());
01145         for (const auto &name : inputs) {
01146             spdlog::info(" - {}", name);
01147         }
01148
01149         const auto &outputs = extractor.getPrimaryOutputs();
01150         spdlog::info("Found {} Primary Outputs:", outputs.size());

```

```

01151     for (const auto &name : outputs) {
01152         spdlog::info(" - {}", name);
01153     }
01154
01155     const auto &wires = extractor.getInternalWires();
01156     spdlog::info("Found {} Internal Wires:", wires.size());
01157     for (const auto &name : wires) {
01158         spdlog::info(" - {}", name);
01159     }
01160
01161     const auto &gates = extractor.getExtractedGates();
01162     spdlog::info("Found {} Gate Drivers:", gates.size());
01163     for (const auto &pair : gates) {
01164         const std::string &outputNet = pair.first;
01165         const GateInfo &info = pair.second;
01166         std::string inputsStr = "";
01167         for (size_t i = 0; i < info.inputSignals.size(); ++i) {
01168             inputsStr += info.inputSignals[i] + (i == info.inputSignals.size() - 1 ? " " : ", ");
01169         }
01170         spdlog::info(" - Output: {} <= Driven by: {} ({} Inputs: [{}]", outputNet,
01171             info.gateTypeName, slang::parsing::toString(info.kind), inputsStr);
01172     }
01173     spdlog::info("--- End Netlist Info Extraction ---");
01174
01175 } catch (const std::exception &e) {
01176     spdlog::error("Exception during Verilog parsing or info extraction: {}", e.what());
01177 } catch (...) {
01178     spdlog::error("Unknown exception during Verilog parsing or info extraction.");
01179 }
01180 }
01181
01182 // --- (Step 2: Extract Logic Expressions) ---
01183
01184 std::map<std::string, std::string> extractLogicFromVerilog(const std::string &verilog_file,
01185     const std::string &cell) {
01186     spdlog::info("--- Starting Logic Expression Extraction for cell: '{}' ---", cell);
01187     std::map<std::string, std::string> logicMap; // Default empty map
01188
01189     try {
01190         auto result = slang::syntax::SyntaxTree::fromFile(verilog_file);
01191         if (!result) {
01192             spdlog::error("Error parsing Verilog file '{}'. Cannot extract logic.", verilog_file);
01193             return logicMap;
01194         }
01195
01196         spdlog::info("Successfully parsed Verilog file.");
01197         std::shared_ptr<slang::syntax::SyntaxTree> tree = result.value();
01198
01199         LogicExtractor extractor(cell);
01200         tree->root().visit(extractor); // Populate extractor's internal state
01201
01202         // Now, call the method to derive and get the expressions
01203         logicMap = extractor.getLogicExpressions();
01204
01205     } catch (const std::exception &e) {
01206         spdlog::error("Exception during Verilog parsing or logic extraction: {}", e.what());
01207     } catch (...) {
01208         spdlog::error("Unknown exception during Verilog parsing or logic extraction.");
01209     }
01210
01211     if (logicMap.empty()) {
01212         spdlog::warn("Logic extraction finished, but no expressions were derived for cell '{}'. Check "
01213             "if cell exists and is correctly defined.",
01214             cell);
01215     } else {
01216         spdlog::info("Logic extraction completed for cell '{}'. Found {} output expression strings", cell,
01217             logicMap.size());
01218     }
01219

```

```
01220     spdlog::info("--- End Logic Expression Extraction ---");
01221     return logicMap;
01222 }
```


索引

- ~AttributesIterator
 - AttributesIterator, [32](#)
- ~GroupsIterator
 - GroupsIterator, [52](#)
- ~LibAttribute
 - LibAttribute, [59](#)
- ~LibFile
 - LibFile, [66](#)
- ~LibGroup
 - LibGroup, [83](#)
- ~SyntaxTree
 - slang::syntax::SyntaxTree, [168](#)
- ~ValuesIterator
 - ValuesIterator, [189](#)
- abstol_
 - LibraryComparator, [94](#)
- advance
 - slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >, [56](#)
- alloc
 - slang::syntax::SyntaxRewriter< TDerived >, [163](#)
 - slang::syntax::SyntaxTree, [177](#)
- allocator
 - slang::syntax::SyntaxTree, [169](#)
- APP_AUTHOR
 - version.h, [221](#)
- APP_CONTACT
 - version.h, [221](#)
- APP_NAME
 - version.h, [222](#)
- APP_VERSION
 - version.h, [222](#)
- APP_VERSION_MAJOR
 - version.h, [222](#)
- APP_VERSION_MINOR
 - version.h, [222](#)
- APP_VERSION_PATCH
 - version.h, [222](#)
- append
 - slang::syntax::SyntaxPrinter, [148](#)
- as
 - slang::syntax::SyntaxNode, [141](#)
- as_if
 - slang::syntax::SyntaxNode, [141](#), [142](#)
- attr_
 - AttributesIterator, [34](#)
 - LibAttribute, [63](#)
- AttributesIterator, [31](#)
 - ~AttributesIterator, [32](#)
 - attr_, [34](#)
 - AttributesIterator, [32](#)
 - attrs_, [34](#)
 - end, [32](#)
 - err_, [34](#)
 - get, [33](#)
 - next, [33](#)
- attrs_
 - AttributesIterator, [34](#)
- b
 - si2drExprT, [125](#)
- Base
 - slang::syntax::ConstTokenOrSyntax, [44](#)
 - slang::syntax::PtrTokenOrSyntax, [110](#)
- basename_
 - LibFile, [79](#)
- begin
 - slang::syntax::SeparatedSyntaxList< T >, [118](#)
- bool_
 - ValuesIterator, [190](#)

- buffer
 - slang::syntax::SyntaxPrinter, 153
- BUILD_TIMESTAMP
 - version.h, 223
- cbegin
 - slang::syntax::SeparatedSyntaxList< T >, 119
- CellExtractor, 35
 - CellExtractor, 36
 - foundTarget_, 37
 - foundTargetCell, 36
 - handle, 36
 - targetCell_, 37
- cellName_
 - ModuleRewriter, 107
- CellPrinter, 38
 - CellPrinter, 39
 - foundTarget_, 39
 - handle, 39
 - out_, 40
 - targetCell_, 40
- cend
 - slang::syntax::SeparatedSyntaxList< T >, 119
- checkTimingArcMonotonicity
 - LibFile, 67
- childCount
 - slang::syntax::SyntaxListBase, 138
- childNodes
 - slang::syntax::SyntaxNode, 142
- childToken
 - slang::syntax::SyntaxNode, 142
- childTokenPtr
 - slang::syntax::SyntaxNode, 142
- clear
 - slang::syntax::detail::ChangeCollection, 41
- clone
 - slang::syntax, 24, 25
 - slang::syntax::SeparatedSyntaxList< T >, 119
 - slang::syntax::SyntaxList< T >, 132
 - slang::syntax::SyntaxListBase, 136
 - slang::syntax::TokenList, 184
- commits
 - slang::syntax::SyntaxRewriter< TDerived >, 163
- comp_json_
 - LibraryComparator, 94
- comp_lib_path_
 - LibraryComparator, 94
- compare.hpp
 - json, 198
- compareCell
 - LibraryComparator, 88
- compareLibFiles
 - main.cpp, 312
- compareLut
 - LibraryComparator, 89
- comparePin
 - LibraryComparator, 90
- compareTimingArc
 - LibraryComparator, 91
- const_iterator
 - slang::syntax::SeparatedSyntaxList< T >, 117
- ConstTokenOrSyntax
 - slang::syntax::ConstTokenOrSyntax, 44, 45
- create
 - slang::syntax::SyntaxTree, 169
- d
 - si2drExprT, 125
- deepClone
 - slang::syntax, 25–27
- DeferredSourceRange
 - slang::syntax::DeferredSourceRange, 47
- depth_
 - ModuleRewriter, 107
 - VerilogVisitor, 196
- dereference
 - slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >, 57
- DERIVED
 - SyntaxVisitor.h, 273
- deriveLogicRecursive
 - LogicExtractor, 97
- diagnostics
 - slang::syntax::SyntaxTree, 169
- diagnosticsBuffer

- slang::syntax::SyntaxTree, 177
- dim_sizes
 - liberty_value_data, 64
- dimensions
 - liberty_value_data, 64
- distance_to
 - slang::syntax::SeparatedSyntaxList< T, iterator_base< U >, 57
- elements
 - slang::syntax::SeparatedSyntaxList< T >, 124
- elems
 - slang::syntax::SeparatedSyntaxList< T >, 119, 120
- empty
 - slang::syntax::detail::ChangeCollection, 41
 - slang::syntax::SeparatedSyntaxList< T >, 120
- end
 - AttributesIterator, 32
 - GroupsIterator, 52
 - slang::syntax::SeparatedSyntaxList< T >, 120, 121
 - ValuesIterator, 189
- equals
 - slang::syntax::SeparatedSyntaxList< T, iterator_base< U >, 57
- err_
 - AttributesIterator, 34
 - GroupsIterator, 53
 - LibAttribute, 63
 - LibFile, 79
 - LibGroup, 85
 - ValuesIterator, 190
- exprp_
 - ValuesIterator, 191
- extractAndPrintNetlistInfo
 - verilog_utils.cpp, 333
 - verilog_utils.hpp, 216
- extractLogicFromVerilog
 - verilog_utils.cpp, 333
 - verilog_utils.hpp, 217
- factory
 - slang::syntax::SyntaxRewriter< TDerived >, 163
 - filename_
 - LibFile, 79
 - filepath_
 - LibFile, 80
 - first
 - slang::syntax::detail::SyntaxChange, 129
 - float_
 - ValuesIterator, 191
 - formatExpression
 - LogicExtractor, 97
 - foundTarget_
 - CellExtractor, 37
 - CellPrinter, 39
 - foundTargetCell
 - CellExtractor, 36
 - fromBuffer
 - slang::syntax::SyntaxTree, 169
 - fromBuffers
 - slang::syntax::SyntaxTree, 170
 - fromFile
 - slang::syntax::SyntaxTree, 170, 171
 - fromFileInMemory
 - slang::syntax::SyntaxTree, 171
 - fromFiles
 - slang::syntax::SyntaxTree, 172
 - fromLibraryMapBuffer
 - slang::syntax::SyntaxTree, 172
 - fromLibraryMapFile
 - slang::syntax::SyntaxTree, 173
 - fromLibraryMapText
 - slang::syntax::SyntaxTree, 173
 - fromText
 - slang::syntax::SyntaxTree, 174
 - funcLibFile
 - main.cpp, 314
 - GateInfo, 49
 - gateTypeName, 50
 - inputSignals, 50
 - kind, 50
 - outputSignal, 50

- gateOutputDrivers_
 - LogicExtractor, 101
- gateTypeName
 - GateInfo, 50
- generateCellJson
 - json_utils.cpp, 284
 - json_utils.hpp, 202
- generateLutJson
 - json_utils.cpp, 286
 - json_utils.hpp, 204
- generatePinJson
 - json_utils.cpp, 287
 - json_utils.hpp, 205
- generatePowerJson
 - json_utils.cpp, 290
 - json_utils.hpp, 208
- generateReport
 - LibraryComparator, 93
- generateTimingJson
 - json_utils.cpp, 291
- get
 - AttributesIterator, 33
 - GroupsIterator, 52
 - slang::syntax::DeferredSourceRange, 48
- get_function
 - si2dr_liberty.h, 240
- getAST
 - verilog_utils.cpp, 334
 - verilog_utils.hpp, 217
- getAttrs
 - LibGroup, 83
- getBoolean
 - LibAttribute, 60
- getChild
 - slang::syntax::SeparatedSyntaxList< T >, 121
 - slang::syntax::SyntaxList< T >, 132
 - slang::syntax::SyntaxListBase, 136
 - slang::syntax::SyntaxNode, 143
 - slang::syntax::TokenList, 184
- getChildCount
 - slang::syntax::SyntaxListBase, 136
 - slang::syntax::SyntaxNode, 143
- getChildPtr
 - slang::syntax::SeparatedSyntaxList< T >, 121
 - slang::syntax::SyntaxList< T >, 133
 - slang::syntax::SyntaxListBase, 137
 - slang::syntax::SyntaxNode, 143
 - slang::syntax::TokenList, 184
- getDefaultSourceManager
 - slang::syntax::SyntaxTree, 175
- getDefinedMacros
 - slang::syntax::SyntaxTree, 175
- getExtractedGates
 - LogicExtractor, 98
- getFirstToken
 - slang::syntax::SyntaxNode, 143
- getFirstTokenPtr
 - slang::syntax::SyntaxNode, 143
- getFloat
 - LibAttribute, 60
- getGroups
 - LibGroup, 83
- getInt
 - LibAttribute, 60
- getInternalWires
 - LogicExtractor, 98
- getLastToken
 - slang::syntax::SyntaxNode, 144
- getLastTokenPtr
 - slang::syntax::SyntaxNode, 144
- getLogicExpressions
 - LogicExtractor, 99
- getMetadata
 - slang::syntax::SyntaxTree, 175
- getName
 - LibAttribute, 61
 - LibGroup, 84
- getPrimaryInputs
 - LogicExtractor, 99
- getPrimaryOutputs
 - LogicExtractor, 100
- getSourceLibrary
 - slang::syntax::SyntaxTree, 176
- getString
 - LibAttribute, 61
- getType

- LibGroup, [84](#)
- getValues
 - LibAttribute, [62](#)
- group__
 - GroupsIteator, [53](#)
 - LibGroup, [85](#)
- groups__
 - GroupsIteator, [54](#)
- GroupsIteator, [51](#)
 - ~GroupsIteator, [52](#)
 - end, [52](#)
 - err__, [53](#)
 - get, [52](#)
 - group__, [53](#)
 - groups__, [54](#)
 - GroupsIteator, [51](#)
 - next, [53](#)
- handle
 - CellExtractor, [36](#)
 - CellPrinter, [39](#)
 - LogicExtractor, [100](#), [101](#)
 - ModuleRewriter, [106](#)
 - VerilogVisitor, [194](#), [195](#)
- i
 - si2drExprT, [125](#)
- include/compare.hpp, [197](#), [198](#)
- include/iterators.hpp, [199](#), [200](#)
- include/json_utils.hpp, [200](#), [209](#)
- include/lib_attribute.hpp, [210](#), [211](#)
- include/lib_file.hpp, [211](#), [213](#)
- include/lib_group.hpp, [213](#), [214](#)
- include/verilog_utils.hpp, [215](#), [218](#)
- include/version.h, [221](#), [223](#)
- include_3rd_party/si2dr_liberty.h, [223](#), [249](#)
- include_3rd_party/SyntaxNode.h, [259](#), [260](#)
- include_3rd_party/SyntaxPrinter.h, [266](#), [267](#)
- include_3rd_party/SyntaxTree.h, [268](#), [269](#)
- include_3rd_party/SyntaxVisitor.h, [271](#), [273](#)
- includeComments
 - slang::syntax::SyntaxPrinter, [153](#)
- includeDirectives
 - slang::syntax::SyntaxPrinter, [153](#)
- includeMissing
 - slang::syntax::SyntaxPrinter, [153](#)
- includePreprocessed
 - slang::syntax::SyntaxPrinter, [154](#)
- includeSkipped
 - slang::syntax::SyntaxPrinter, [154](#)
- includeTrivia
 - slang::syntax::SyntaxPrinter, [154](#)
- index
 - slang::syntax::SeparatedSyntaxList< T
>::iterator_base< U >, [58](#)
- index_info
 - liberty_value_data, [64](#)
- inputPins__
 - ModuleRewriter, [107](#)
- inputSignals
 - GateInfo, [50](#)
- insertAfter
 - slang::syntax::detail::ChangeCollection, [41](#)
 - slang::syntax::SyntaxRewriter< TDerived >, [157](#)
- insertAtBack
 - slang::syntax::SyntaxRewriter< TDerived >, [158](#)
- insertAtFront
 - slang::syntax::SyntaxRewriter< TDerived >, [158](#)
- insertBefore
 - slang::syntax::detail::ChangeCollection, [42](#)
 - slang::syntax::SyntaxRewriter< TDerived >, [158](#)
- InsertChangeMap
 - slang::syntax::detail, [28](#)
- instance_count__
 - ModuleRewriter, [107](#)
- int__
 - ValuesIteator, [191](#)
- inTargetModule__
 - LogicExtractor, [102](#)
 - VerilogVisitor, [196](#)
- internalWires__
 - LogicExtractor, [102](#)
- isChildOptional

- slang::syntax::SyntaxListBase, [137](#)
- slang::syntax::SyntaxNode, [144](#)
- isComplex
 - LibAttribute, [62](#)
- isEquivalentTo
 - slang::syntax::SyntaxNode, [144](#)
- isKind
 - slang::syntax::SyntaxListBase, [137](#)
 - slang::syntax::SyntaxNode, [144](#)
- isLibraryUnit
 - slang::syntax::SyntaxTree, [178](#)
- isNode
 - slang::syntax::ConstTokenOrSyntax, [45](#)
 - slang::syntax::PtrTokenOrSyntax, [111](#)
- isToken
 - slang::syntax::ConstTokenOrSyntax, [45](#)
 - slang::syntax::PtrTokenOrSyntax, [111](#)
- iterator
 - slang::syntax::SeparatedSyntaxList< T >, [117](#)
- iterator_base
 - slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >, [56](#)
- json
 - compare.hpp, [198](#)
 - json_utils.hpp, [202](#)
 - lib_file.hpp, [212](#)
- json_utils.cpp
 - generateCellJson, [284](#)
 - generateLutJson, [286](#)
 - generatePinJson, [287](#)
 - generatePowerJson, [290](#)
 - generateTimingJson, [291](#)
 - parseStringToVector, [293](#)
- json_utils.hpp
 - generateCellJson, [202](#)
 - generateLutJson, [204](#)
 - generatePinJson, [205](#)
 - generatePowerJson, [208](#)
 - json, [202](#)
- jsonname_
 - LibFile, [80](#)
- kind
 - GateInfo, [50](#)
 - slang::syntax::SyntaxNode, [146](#)
- left
 - si2drExprT, [125](#)
- lib_file.hpp
 - json, [212](#)
- lib_json_
 - LibFile, [80](#)
- LibAttribute, [58](#)
 - ~LibAttribute, [59](#)
 - attr_, [63](#)
 - err_, [63](#)
 - getBoolean, [60](#)
 - getFloat, [60](#)
 - getInt, [60](#)
 - getName, [61](#)
 - getString, [61](#)
 - getValues, [62](#)
 - isComplex, [62](#)
 - LibAttribute, [59](#)
- liberty_destroy_value_data
 - si2dr_liberty.h, [240](#)
- liberty_get_element
 - si2dr_liberty.h, [241](#)
- liberty_get_values_data
 - si2dr_liberty.h, [241](#)
- liberty_value_data, [63](#)
 - dim_sizes, [64](#)
 - dimensions, [64](#)
 - index_info, [64](#)
 - values, [64](#)
- LibFile, [64](#)
 - ~LibFile, [66](#)
 - basename_, [79](#)
 - checkTimingArcMonotonicity, [67](#)
 - err_, [79](#)
 - filename_, [79](#)
 - filepath_, [80](#)
 - jsonname_, [80](#)
 - lib_json_, [80](#)
 - LibFile, [66](#)
 - libname_, [80](#)

- logger_, 80
- loggername_, 81
- logic, 68
- modify, 70
- mono, 70
- parse, 71
- process_, 81
- read, 74
- supercell, 75
- temperature_, 81
- verilog, 77
- voltage_, 81
- writeJsonToFile, 78
- LibGroup, 82
 - ~LibGroup, 83
 - err_, 85
 - getAttrs, 83
 - getGroups, 83
 - getName, 84
 - getType, 84
 - group_, 85
 - LibGroup, 82
- libname_
 - LibFile, 80
- library
 - slang::syntax::SyntaxTree, 178
- LibraryComparator, 85
 - abstol_, 94
 - comp_json_, 94
 - comp_lib_path_, 94
 - compareCell, 88
 - compareLut, 89
 - comparePin, 90
 - compareTimingArc, 91
 - generateReport, 93
 - LibraryComparator, 87
 - ref_json_, 94
 - ref_lib_path_, 94
 - reitol_, 95
- list
 - slang::syntax::SeparatedSyntaxList<
 - >::iterator_base< U >, 58
- ListChangeMap
 - slang::syntax::detail, 28
- listInsertAtBack
 - slang::syntax::detail::ChangeCollection, 42
- listInsertAtFront
 - slang::syntax::detail::ChangeCollection, 42
- logger_
 - LibFile, 80
 - ModuleRewriter, 107
- loggername_
 - LibFile, 81
- logic
 - LibFile, 68
- logicCache_
 - LogicExtractor, 102
- LogicExtractor, 95
 - deriveLogicRecursive, 97
 - formatExpression, 97
 - gateOutputDrivers_, 101
 - getExtractedGates, 98
 - getInternalWires, 98
 - getLogicExpressions, 99
 - getPrimaryInputs, 99
 - getPrimaryOutputs, 100
 - handle, 100, 101
 - inTargetModule_, 102
 - internalWires_, 102
 - logicCache_, 102
 - LogicExtractor, 97
 - parsingComplete_, 102
 - portDirections_, 102
 - primaryInputs_, 103
 - primaryOutputs_, 103
 - targetCell_, 103
- LONG_DOUBLE
 - si2dr_liberty.h, 228
- MacroList
 - slang::syntax::SyntaxTree, 167
- macros
 - slang::syntax::SyntaxTree, 178
- main
 - main.cpp, 315
- main.cpp

- compareLibFiles, [312](#)
- funcLibFile, [314](#)
- main, [315](#)
- monoCheckLibFile, [316](#)
- parseLibFile, [318](#)
- printInfo, [320](#)
- supercellLibFile, [320](#)
- verilogLibFile, [322](#)
- main_liberty_parser
 - si2dr_liberty.h, [241](#)
- makeComma
 - slang::syntax::SyntaxRewriter< TDerived >, [159](#)
- makeld
 - slang::syntax::SyntaxRewriter< TDerived >, [159](#), [160](#)
- makeToken
 - slang::syntax::SyntaxRewriter< TDerived >, [160](#)
- metadata
 - slang::syntax::SyntaxTree, [178](#)
- modify
 - LibFile, [70](#)
- ModifyChangeMap
 - slang::syntax::detail, [29](#)
- moduleName_
 - ModuleRewriter, [108](#)
- ModuleRewriter, [104](#)
 - cellName_, [107](#)
 - depth_, [107](#)
 - handle, [106](#)
 - inputPins_, [107](#)
 - instance_count_, [107](#)
 - logger_, [107](#)
 - moduleName_, [108](#)
 - ModuleRewriter, [105](#)
 - outputPins_, [108](#)
 - portInfoMap_, [108](#)
- mono
 - LibFile, [70](#)
- monoCheckLibFile
 - main.cpp, [316](#)
- next
 - AttributesIterator, [33](#)
 - GroupsIterator, [53](#)
 - ValuesIterator, [190](#)
- node
 - slang::syntax::ConstTokenOrSyntax, [45](#)
 - slang::syntax::DeferredSourceRange, [49](#)
 - slang::syntax::PtrTokenOrSyntax, [111](#)
 - slang::syntax::TokenOrSyntax, [188](#)
- operator*
 - slang::syntax::DeferredSourceRange, [48](#)
- operator[]
 - slang::syntax::SeparatedSyntaxList< T >, [122](#)
- options
 - slang::syntax::SyntaxTree, [176](#)
- options_
 - slang::syntax::SyntaxTree, [178](#)
- out_
 - CellPrinter, [40](#)
- outputPins_
 - ModuleRewriter, [108](#)
- outputSignal
 - GateInfo, [50](#)
- parent
 - slang::syntax::SyntaxNode, [146](#)
- ParentList
 - slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >, [56](#)
- parse
 - LibFile, [71](#)
 - slang::syntax::SyntaxRewriter< TDerived >, [161](#)
- parseLibFile
 - main.cpp, [318](#)
- parseStringToVector
 - json_utils.cpp, [293](#)
- parsingComplete_
 - LogicExtractor, [102](#)
- portDirections_
 - LogicExtractor, [102](#)
- portInfoMap_
 - ModuleRewriter, [108](#)

- previewNode
 - slang::syntax::SyntaxNode, [146](#)
- primaryInputs_
 - LogicExtractor, [103](#)
- primaryOutputs_
 - LogicExtractor, [103](#)
- print
 - slang::syntax::SyntaxPrinter, [148](#), [149](#)
- print_version
 - si2dr_liberty.h, [241](#)
- printFile
 - slang::syntax::SyntaxPrinter, [150](#)
- printlnInfo
 - main.cpp, [320](#)
- process_
 - LibFile, [81](#)
- PtrTokenOrSyntax
 - slang::syntax::PtrTokenOrSyntax, [110](#), [111](#)
- range
 - slang::syntax::ConstTokenOrSyntax, [46](#)
 - slang::syntax::DeferredSourceRange, [49](#)
 - slang::syntax::PtrTokenOrSyntax, [112](#)
- read
 - LibFile, [74](#)
- README.md, [276](#)
- ref_json_
 - LibraryComparator, [94](#)
- ref_lib_path_
 - LibraryComparator, [94](#)
- reitol_
 - LibraryComparator, [95](#)
- remove
 - slang::syntax::SyntaxRewriter< TDerived >, [161](#)
- removeOrReplace
 - slang::syntax::detail::ChangeCollection, [42](#)
- replace
 - slang::syntax::SyntaxRewriter< TDerived >, [161](#)
- resetAll
 - slang::syntax::SeparatedSyntaxList< T >, [122](#)
 - slang::syntax::SyntaxList< T >, [133](#)
- slang::syntax::SyntaxListBase, [137](#)
- slang::syntax::TokenList, [185](#)
- right
 - si2drExprT, [125](#)
- root
 - slang::syntax::SyntaxTree, [176](#)
- rootNode
 - slang::syntax::SyntaxTree, [179](#)
- s
 - si2drExprT, [126](#)
- second
 - slang::syntax::detail::SyntaxChange, [129](#)
- SeparatedSyntaxList
 - slang::syntax::SeparatedSyntaxList< T >, [118](#)
- separator
 - slang::syntax::detail::SyntaxChange, [129](#)
- setChild
 - slang::syntax::SeparatedSyntaxList< T >, [123](#)
 - slang::syntax::SyntaxList< T >, [133](#)
 - slang::syntax::SyntaxListBase, [137](#)
 - slang::syntax::TokenList, [185](#)
- setIncludeComments
 - slang::syntax::SyntaxPrinter, [150](#)
- setIncludeDirectives
 - slang::syntax::SyntaxPrinter, [150](#)
- setIncludeMissing
 - slang::syntax::SyntaxPrinter, [151](#)
- setIncludePreprocessed
 - slang::syntax::SyntaxPrinter, [151](#)
- setIncludeSkipped
 - slang::syntax::SyntaxPrinter, [151](#)
- setIncludeTrivia
 - slang::syntax::SyntaxPrinter, [152](#)
- setSquashNewlines
 - slang::syntax::SyntaxPrinter, [152](#)
- SI2_ARGS
 - si2dr_liberty.h, [228](#), [241–248](#)
- SI2_BOOLEAN
 - si2dr_liberty.h, [240](#)
- SI2_DOUBLE
 - si2dr_liberty.h, [240](#)
- SI2_EXPR

- si2dr_liberty.h, [240](#)
- SI2_FALSE
 - si2dr_liberty.h, [237](#)
- SI2_FLOAT
 - si2dr_liberty.h, [240](#)
- SI2_ITER
 - si2dr_liberty.h, [240](#)
- SI2_LONG
 - si2dr_liberty.h, [240](#)
- SI2_LONG_MAX
 - si2dr_liberty.h, [228](#)
- SI2_LONG_MIN
 - si2dr_liberty.h, [228](#)
- SI2_MAX_VALUETYPE
 - si2dr_liberty.h, [240](#)
- SI2_OID
 - si2dr_liberty.h, [240](#)
- SI2_STRING
 - si2dr_liberty.h, [240](#)
- SI2_TRUE
 - si2dr_liberty.h, [237](#)
- SI2_ULONG_MAX
 - si2dr_liberty.h, [229](#)
- SI2_UNDEFINED_VALUETYPE
 - si2dr_liberty.h, [240](#)
- SI2_VOID
 - si2dr_liberty.h, [240](#)
- si2BooleanEnum
 - si2dr_liberty.h, [237](#)
- si2BooleanT
 - si2dr_liberty.h, [231](#)
- si2DoubleT
 - si2dr_liberty.h, [231](#)
- SI2DR_ATTR
 - si2dr_liberty.h, [239](#)
- SI2DR_BOOLEAN
 - si2dr_liberty.h, [240](#)
- SI2DR_COMPLEX
 - si2dr_liberty.h, [237](#)
- SI2DR_COMPLEXVAL
 - si2dr_liberty.h, [239](#)
- SI2DR_DEFINE
 - si2dr_liberty.h, [239](#)
- SI2DR_EXPR
 - si2dr_liberty.h, [240](#)
- SI2DR_EXPR_OP_ADD
 - si2dr_liberty.h, [238](#)
- SI2DR_EXPR_OP_DIV
 - si2dr_liberty.h, [238](#)
- SI2DR_EXPR_OP_EXP
 - si2dr_liberty.h, [238](#)
- SI2DR_EXPR_OP_LOG10
 - si2dr_liberty.h, [238](#)
- SI2DR_EXPR_OP_LOG2
 - si2dr_liberty.h, [238](#)
- SI2DR_EXPR_OP_MUL
 - si2dr_liberty.h, [238](#)
- SI2DR_EXPR_OP_PAREN
 - si2dr_liberty.h, [238](#)
- SI2DR_EXPR_OP_SUB
 - si2dr_liberty.h, [238](#)
- SI2DR_EXPR_VAL
 - si2dr_liberty.h, [238](#)
- SI2DR_FALSE
 - si2dr_liberty.h, [229](#)
- SI2DR_FLOAT64
 - si2dr_liberty.h, [240](#)
- SI2DR_GROUP
 - si2dr_liberty.h, [239](#)
- SI2DR_INT32
 - si2dr_liberty.h, [240](#)
- SI2DR_INTERNAL_SYSTEM_ERROR
 - si2dr_liberty.h, [238](#)
- SI2DR_INVALID_ATTRTYPE
 - si2dr_liberty.h, [238](#)
- SI2DR_INVALID_NAME
 - si2dr_liberty.h, [238](#)
- SI2DR_INVALID_OBJECTTYPE
 - si2dr_liberty.h, [238](#)
- SI2DR_INVALID_VALUE
 - si2dr_liberty.h, [238](#)
- si2dr_liberty.h
 - get_function, [240](#)
 - liberty_destroy_value_data, [240](#)
 - liberty_get_element, [241](#)
 - liberty_get_values_data, [241](#)

- LONG_DOUBLE, 228
- main_liberty_parser, 241
- print_version, 241
- SI2_ARGS, 228, 241–248
- SI2_BOOLEAN, 240
- SI2_DOUBLE, 240
- SI2_EXPR, 240
- SI2_FALSE, 237
- SI2_FLOAT, 240
- SI2_ITER, 240
- SI2_LONG, 240
- SI2_LONG_MAX, 228
- SI2_LONG_MIN, 228
- SI2_MAX_VALUETYPE, 240
- SI2_OID, 240
- SI2_STRING, 240
- SI2_TRUE, 237
- SI2_ULONG_MAX, 229
- SI2_UNDEFINED_VALUETYPE, 240
- SI2_VOID, 240
- si2BooleanEnum, 237
- si2BooleanT, 231
- si2DoubleT, 231
- SI2DR_ATTR, 239
- SI2DR_BOOLEAN, 240
- SI2DR_COMPLEX, 237
- SI2DR_COMPLEXVAL, 239
- SI2DR_DEFINE, 239
- SI2DR_EXPR, 240
- SI2DR_EXPR_OP_ADD, 238
- SI2DR_EXPR_OP_DIV, 238
- SI2DR_EXPR_OP_EXP, 238
- SI2DR_EXPR_OP_LOG10, 238
- SI2DR_EXPR_OP_LOG2, 238
- SI2DR_EXPR_OP_MUL, 238
- SI2DR_EXPR_OP_PAREN, 238
- SI2DR_EXPR_OP_SUB, 238
- SI2DR_EXPR_VAL, 238
- SI2DR_FALSE, 229
- SI2DR_FLOAT64, 240
- SI2DR_GROUP, 239
- SI2DR_INT32, 240
- SI2DR_INTERNAL_SYSTEM_ERROR, 238
- SI2DR_INVALID_ATTRTYPE, 238
- SI2DR_INVALID_NAME, 238
- SI2DR_INVALID_OBJECTTYPE, 238
- SI2DR_INVALID_VALUE, 238
- SI2DR_MAX_ERROR, 238
- SI2DR_MAX_FLOAT32, 229
- SI2DR_MAX_FLOAT64, 229
- SI2DR_MAX_INT32, 229
- SI2DR_MAX_OBJECTTYPE, 239
- SI2DR_MAX_STRING_LEN, 230
- SI2DR_MAX_VALUETYPE, 240
- SI2DR_MIN_FLOAT32, 230
- SI2DR_MIN_FLOAT64, 230
- SI2DR_MIN_INT32, 230
- SI2DR_NO_ERROR, 238
- SI2DR_OBJECT_ALREADY_EXISTS, 238
- SI2DR_OBJECT_NOT_FOUND, 238
- SI2DR_PIINIT_NOT_CALLED, 238
- SI2DR_REFERENCE_ERROR, 238
- SI2DR_SEMANTIC_ERROR, 238
- SI2DR_SEVERITY_ERR, 239
- SI2DR_SEVERITY_NOTE, 239
- SI2DR_SEVERITY_WARN, 239
- SI2DR_SIMPLE, 237
- SI2DR_STRING, 240
- SI2DR_SYNTAX_ERROR, 238
- SI2DR_TRACE_FILES_CANNOT_BE_OPENED, 238
- SI2DR_TRUE, 230
- SI2DR_UNDEFINED_OBJECTTYPE, 239
- SI2DR_UNDEFINED_VALUETYPE, 239
- SI2DR_UNUSABLE_OID, 238
- si2drAttrComplexValIdT, 231
- si2drAttrIdT, 231
- si2drAttrIdT, 231
- si2drAttrTypeT, 232, 237
- si2drBooleanT, 232
- si2drDefaultMessageHandler, 248
- si2drDefinIdT, 232
- si2drDefinesIdT, 232
- si2drErrorT, 232, 237
- si2drExprT, 232
- si2drExprTypeT, 233, 238

- si2drFloat32T, [233](#)
- si2drFloat64T, [233](#)
- si2drGroupIdT, [233](#)
- si2drGroupsIdT, [233](#)
- si2drInt32T, [233](#)
- si2drIterIdT, [234](#)
- si2drMessageHandlerT, [234](#)
- si2drNamesIdT, [234](#)
- si2drObjectIdT, [234](#)
- si2drObjectsIdT, [234](#)
- si2drObjectTypeT, [235](#), [239](#)
- si2drPIGetMessageHandler, [248](#)
- si2drPISetMessageHandler, [249](#)
- si2drSeverityT, [235](#), [239](#)
- si2drStringT, [235](#)
- si2drValuesIdT, [235](#)
- si2drValueTypeT, [235](#), [239](#)
- si2drVoidT, [235](#)
- si2FloatT, [236](#)
- si2IteratorIdT, [236](#)
- si2LongT, [236](#)
- si2ObjectIdT, [236](#)
- si2StringT, [236](#)
- si2TypeEnum, [240](#)
- si2TypeT, [236](#)
- si2VoidT, [237](#)
- usage, [249](#)
- SI2DR_MAX_ERROR
 - si2dr_liberty.h, [238](#)
- SI2DR_MAX_FLOAT32
 - si2dr_liberty.h, [229](#)
- SI2DR_MAX_FLOAT64
 - si2dr_liberty.h, [229](#)
- SI2DR_MAX_INT32
 - si2dr_liberty.h, [229](#)
- SI2DR_MAX_OBJECTTYPE
 - si2dr_liberty.h, [239](#)
- SI2DR_MAX_STRING_LEN
 - si2dr_liberty.h, [230](#)
- SI2DR_MAX_VALUETYPE
 - si2dr_liberty.h, [240](#)
- SI2DR_MIN_FLOAT32
 - si2dr_liberty.h, [230](#)
- SI2DR_MIN_FLOAT64
 - si2dr_liberty.h, [230](#)
- SI2DR_MIN_INT32
 - si2dr_liberty.h, [230](#)
- SI2DR_NO_ERROR
 - si2dr_liberty.h, [238](#)
- SI2DR_OBJECT_ALREADY_EXISTS
 - si2dr_liberty.h, [238](#)
- SI2DR_OBJECT_NOT_FOUND
 - si2dr_liberty.h, [238](#)
- SI2DR_PIINIT_NOT_CALLED
 - si2dr_liberty.h, [238](#)
- SI2DR_REFERENCE_ERROR
 - si2dr_liberty.h, [238](#)
- SI2DR_SEMANTIC_ERROR
 - si2dr_liberty.h, [238](#)
- SI2DR_SEVERITY_ERR
 - si2dr_liberty.h, [239](#)
- SI2DR_SEVERITY_NOTE
 - si2dr_liberty.h, [239](#)
- SI2DR_SEVERITY_WARN
 - si2dr_liberty.h, [239](#)
- SI2DR_SIMPLE
 - si2dr_liberty.h, [237](#)
- SI2DR_STRING
 - si2dr_liberty.h, [240](#)
- SI2DR_SYNTAX_ERROR
 - si2dr_liberty.h, [238](#)
- SI2DR_TRACE_FILES_CANNOT_BE_OPENED
 - si2dr_liberty.h, [238](#)
- SI2DR_TRUE
 - si2dr_liberty.h, [230](#)
- SI2DR_UNDEFINED_OBJECTTYPE
 - si2dr_liberty.h, [239](#)
- SI2DR_UNDEFINED_VALUETYPE
 - si2dr_liberty.h, [239](#)
- SI2DR_UNUSABLE_OID
 - si2dr_liberty.h, [238](#)
- si2drAttrComplexValIdT
 - si2dr_liberty.h, [231](#)
- si2drAttrIdT
 - si2dr_liberty.h, [231](#)
- si2drAttrsIdT

- si2dr_liberty.h, [231](#)
- si2drAttrTypeT
 - si2dr_liberty.h, [232](#), [237](#)
- si2drBooleanT
 - si2dr_liberty.h, [232](#)
- si2drDefaultMessageHandler
 - si2dr_liberty.h, [248](#)
- si2drDefinIdT
 - si2dr_liberty.h, [232](#)
- si2drDefinesIdT
 - si2dr_liberty.h, [232](#)
- si2drErrorT
 - si2dr_liberty.h, [232](#), [237](#)
- si2drExprT, [124](#)
 - b, [125](#)
 - d, [125](#)
 - i, [125](#)
 - left, [125](#)
 - right, [125](#)
 - s, [126](#)
 - si2dr_liberty.h, [232](#)
 - type, [126](#)
 - u, [126](#)
 - valuetype, [126](#)
- si2drExprTypeT
 - si2dr_liberty.h, [233](#), [238](#)
- si2drFloat32T
 - si2dr_liberty.h, [233](#)
- si2drFloat64T
 - si2dr_liberty.h, [233](#)
- si2drGroupIdT
 - si2dr_liberty.h, [233](#)
- si2drGroupsIdT
 - si2dr_liberty.h, [233](#)
- si2drInt32T
 - si2dr_liberty.h, [233](#)
- si2drIterIdT
 - si2dr_liberty.h, [234](#)
- si2drMessageHandlerT
 - si2dr_liberty.h, [234](#)
- si2drNamesIdT
 - si2dr_liberty.h, [234](#)
- si2drObjectIdT
 - si2dr_liberty.h, [234](#)
- si2drObjectsIdT
 - si2dr_liberty.h, [234](#)
- si2drObjectTypeT
 - si2dr_liberty.h, [235](#), [239](#)
- si2drPIGetMessageHandler
 - si2dr_liberty.h, [248](#)
- si2drPISetMessageHandler
 - si2dr_liberty.h, [249](#)
- si2drSeverityT
 - si2dr_liberty.h, [235](#), [239](#)
- si2drStringT
 - si2dr_liberty.h, [235](#)
- si2drValuesIdT
 - si2dr_liberty.h, [235](#)
- si2drValueTypeT
 - si2dr_liberty.h, [235](#), [239](#)
- si2drVoidT
 - si2dr_liberty.h, [235](#)
- si2FloatT
 - si2dr_liberty.h, [236](#)
- si2IteratorIdT
 - si2dr_liberty.h, [236](#)
- si2LongT
 - si2dr_liberty.h, [236](#)
- si2ObjectIdT
 - si2dr_liberty.h, [236](#)
- si2OID, [127](#)
 - v1, [127](#)
 - v2, [127](#)
- si2StringT
 - si2dr_liberty.h, [236](#)
- si2TypeEnum
 - si2dr_liberty.h, [240](#)
- si2TypeT
 - si2dr_liberty.h, [236](#)
- si2VoidT
 - si2dr_liberty.h, [237](#)
- SingleSpace
 - slang::syntax::SyntaxRewriter< TDerived >, [163](#)
- size
 - slang::syntax::SeparatedSyntaxList< T >, [123](#)

- slang, 23
- slang::parsing, 23
- slang::syntax, 23
 - clone, 24, 25
 - deepClone, 25–27
- slang::syntax::ConstTokenOrSyntax, 43
 - Base, 44
 - ConstTokenOrSyntax, 44, 45
 - isNode, 45
 - isToken, 45
 - node, 45
 - range, 46
 - token, 46
- slang::syntax::DeferredSourceRange, 46
 - DeferredSourceRange, 47
 - get, 48
 - node, 49
 - operator*, 48
 - range, 49
 - syntax, 48
- slang::syntax::detail, 28
 - InsertChangeMap, 28
 - ListChangeMap, 28
 - ModifyChangeMap, 29
 - transformTree, 29
- slang::syntax::detail::ChangeCollection, 40
 - clear, 41
 - empty, 41
 - insertAfter, 41
 - insertBefore, 42
 - listInsertAtBack, 42
 - listInsertAtFront, 42
 - removeOrReplace, 42
- slang::syntax::detail::RemoveChange, 113
- slang::syntax::detail::ReplaceChange, 114
- slang::syntax::detail::SyntaxChange, 128
 - first, 129
 - second, 129
 - separator, 129
- slang::syntax::PtrTokenOrSyntax, 109
 - Base, 110
 - isNode, 111
 - isToken, 111
 - node, 111
 - PtrTokenOrSyntax, 110, 111
 - range, 112
 - token, 112
- slang::syntax::SeparatedSyntaxList< T >, 115
 - begin, 118
 - cbegin, 119
 - cend, 119
 - clone, 119
 - const_iterator, 117
 - elements, 124
 - elems, 119, 120
 - empty, 120
 - end, 120, 121
 - getChild, 121
 - getChildPtr, 121
 - iterator, 117
 - operator[], 122
 - resetAll, 122
 - SeparatedSyntaxList, 118
 - setChild, 123
 - size, 123
- slang::syntax::SeparatedSyntaxList< T >::iterator_base< U >, 54
 - advance, 56
 - dereference, 57
 - distance_to, 57
 - equals, 57
 - index, 58
 - iterator_base, 56
 - list, 58
 - ParentList, 56
- slang::syntax::SyntaxList< T >, 130
 - clone, 132
 - getChild, 132
 - getChildPtr, 133
 - resetAll, 133
 - setChild, 133
 - SyntaxList, 131
- slang::syntax::SyntaxListBase, 134
 - childCount, 138
 - clone, 136
 - getChild, 136

- getChildCount, [136](#)
- getChildPtr, [137](#)
- isChildOptional, [137](#)
- isKind, [137](#)
- resetAll, [137](#)
- setChild, [137](#)
- SyntaxListBase, [135](#)
- slang::syntax::SyntaxNode, [138](#)
 - as, [141](#)
 - as_if, [141](#), [142](#)
 - childNode, [142](#)
 - childToken, [142](#)
 - childTokenPtr, [142](#)
 - getChild, [143](#)
 - getChildCount, [143](#)
 - getChildPtr, [143](#)
 - getFirstToken, [143](#)
 - getFirstTokenPtr, [143](#)
 - getLastToken, [144](#)
 - getLastTokenPtr, [144](#)
 - isChildOptional, [144](#)
 - isEquivalentTo, [144](#)
 - isKind, [144](#)
 - kind, [146](#)
 - parent, [146](#)
 - previewNode, [146](#)
 - sourceRange, [145](#)
 - SyntaxNode, [141](#)
 - Token, [140](#)
 - toString, [145](#)
 - visit, [145](#)
- slang::syntax::SyntaxPrinter, [147](#)
 - append, [148](#)
 - buffer, [153](#)
 - includeComments, [153](#)
 - includeDirectives, [153](#)
 - includeMissing, [153](#)
 - includePreprocessed, [154](#)
 - includeSkipped, [154](#)
 - includeTrivia, [154](#)
 - print, [148](#), [149](#)
 - printFile, [150](#)
 - setIncludeComments, [150](#)
 - setIncludeDirectives, [150](#)
 - setIncludeMissing, [151](#)
 - setIncludePreprocessed, [151](#)
 - setIncludeSkipped, [151](#)
 - setIncludeTrivia, [152](#)
 - setSquashNewlines, [152](#)
 - sourceManager, [154](#)
 - squashNewlines, [154](#)
 - str, [152](#)
 - SyntaxPrinter, [148](#)
- slang::syntax::SyntaxRewriter< TDerived >, [155](#)
 - alloc, [163](#)
 - commits, [163](#)
 - factory, [163](#)
 - insertAfter, [157](#)
 - insertAtBack, [158](#)
 - insertAtFront, [158](#)
 - insertBefore, [158](#)
 - makeComma, [159](#)
 - makeld, [159](#), [160](#)
 - makeToken, [160](#)
 - parse, [161](#)
 - remove, [161](#)
 - replace, [161](#)
 - SingleSpace, [163](#)
 - sourceManager, [164](#)
 - SyntaxRewriter, [157](#)
 - tempTrees, [164](#)
 - Token, [157](#)
 - transform, [162](#)
- slang::syntax::SyntaxTree, [164](#)
 - ~SyntaxTree, [168](#)
 - alloc, [177](#)
 - allocator, [169](#)
 - create, [169](#)
 - diagnostics, [169](#)
 - diagnosticsBuffer, [177](#)
 - fromBuffer, [169](#)
 - fromBuffers, [170](#)
 - fromFile, [170](#), [171](#)
 - fromFileInMemory, [171](#)
 - fromFiles, [172](#)
 - fromLibraryMapBuffer, [172](#)

- fromLibraryMapFile, [173](#)
- fromLibraryMapText, [173](#)
- fromText, [174](#)
- getDefaultSourceManager, [175](#)
- getDefinedMacros, [175](#)
- getMetadata, [175](#)
- getSourceLibrary, [176](#)
- isLibraryUnit, [178](#)
- library, [178](#)
- MacroList, [167](#)
- macros, [178](#)
- metadata, [178](#)
- options, [176](#)
- options_, [178](#)
- root, [176](#)
- rootNode, [179](#)
- sourceMan, [179](#)
- sourceManager, [177](#)
- SyntaxTree, [168](#)
- TreeOrError, [167](#)
- slang::syntax::SyntaxVisitor< TDerived >, [179](#)
 - visit, [180](#)
 - visitDefault, [180](#)
 - visitToken, [181](#)
- slang::syntax::TokenList, [182](#)
 - clone, [184](#)
 - getChild, [184](#)
 - getChildPtr, [184](#)
 - resetAll, [185](#)
 - setChild, [185](#)
 - TokenList, [183](#)
- slang::syntax::TokenOrSyntax, [186](#)
 - node, [188](#)
 - TokenOrSyntax, [187](#)
- sourceMan
 - slang::syntax::SyntaxTree, [179](#)
- sourceManager
 - slang::syntax::SyntaxPrinter, [154](#)
 - slang::syntax::SyntaxRewriter< TDerived >, [164](#)
 - slang::syntax::SyntaxTree, [177](#)
- sourceRange
 - slang::syntax::SyntaxNode, [145](#)
- squashNewlines
 - slang::syntax::SyntaxPrinter, [154](#)
- src/compare.cpp, [276](#)
- src/iterators.cpp, [282](#)
- src/json_utils.cpp, [283](#), [294](#)
- src/lib_attribute.cpp, [297](#), [298](#)
- src/lib_file.cpp, [298](#), [299](#)
- src/lib_group.cpp, [311](#)
- src/main.cpp, [311](#), [324](#)
- src/verilog_utils.cpp, [332](#), [335](#)
- str
 - slang::syntax::SyntaxPrinter, [152](#)
- str_
 - ValuesIterator, [191](#)
- supercell
 - LibFile, [75](#)
- supercellLibFile
 - main.cpp, [320](#)
- syntax
 - slang::syntax::DeferredSourceRange, [48](#)
- SyntaxList
 - slang::syntax::SyntaxList< T >, [131](#)
- SyntaxListBase
 - slang::syntax::SyntaxListBase, [135](#)
- SyntaxNode
 - slang::syntax::SyntaxNode, [141](#)
- SyntaxPrinter
 - slang::syntax::SyntaxPrinter, [148](#)
- SyntaxRewriter
 - slang::syntax::SyntaxRewriter< TDerived >, [157](#)
- SyntaxTree
 - slang::syntax::SyntaxTree, [168](#)
- SyntaxVisitor.h
 - DERIVED, [273](#)
- targetCell_
 - CellExtractor, [37](#)
 - CellPrinter, [40](#)
 - LogicExtractor, [103](#)
 - VerilogVisitor, [196](#)
- temperature_
 - LibFile, [81](#)

- tempTrees
 - slang::syntax::SyntaxRewriter< TDerived >, [164](#)
- Token
 - slang::syntax::SyntaxNode, [140](#)
 - slang::syntax::SyntaxRewriter< TDerived >, [157](#)
- token
 - slang::syntax::ConstTokenOrSyntax, [46](#)
 - slang::syntax::PtrTokenOrSyntax, [112](#)
- TokenList
 - slang::syntax::TokenList, [183](#)
- TokenOrSyntax
 - slang::syntax::TokenOrSyntax, [187](#)
- toString
 - slang::syntax::SyntaxNode, [145](#)
- transform
 - slang::syntax::SyntaxRewriter< TDerived >, [162](#)
- transformTree
 - slang::syntax::detail, [29](#)
- TreeOrError
 - slang::syntax::SyntaxTree, [167](#)
- type
 - si2drExprT, [126](#)
- u
 - si2drExprT, [126](#)
- usage
 - si2dr_liberty.h, [249](#)
- v1
 - si2OID, [127](#)
- v2
 - si2OID, [127](#)
- values
 - liberty_value_data, [64](#)
- values_
 - ValuesIterator, [191](#)
- ValuesIterator, [188](#)
 - ~ValuesIterator, [189](#)
 - bool_, [190](#)
 - end, [189](#)
 - err_, [190](#)
 - exprp_, [191](#)
 - float_, [191](#)
 - int_, [191](#)
 - next, [190](#)
 - str_, [191](#)
 - values_, [191](#)
 - ValuesIterator, [189](#)
 - vtype_, [192](#)
- valuetype
 - si2drExprT, [126](#)
- verilog
 - LibFile, [77](#)
- verilog_utils.cpp
 - extractAndPrintNetlistInfo, [333](#)
 - extractLogicFromVerilog, [333](#)
 - getAST, [334](#)
- verilog_utils.hpp
 - extractAndPrintNetlistInfo, [216](#)
 - extractLogicFromVerilog, [217](#)
 - getAST, [217](#)
- verilogLibFile
 - main.cpp, [322](#)
- VerilogVisitor, [192](#)
 - depth_, [196](#)
 - handle, [194](#), [195](#)
 - inTargetModule_, [196](#)
 - targetCell_, [196](#)
 - VerilogVisitor, [193](#)
- version.h
 - APP_AUTHOR, [221](#)
 - APP_CONTACT, [221](#)
 - APP_NAME, [222](#)
 - APP_VERSION, [222](#)
 - APP_VERSION_MAJOR, [222](#)
 - APP_VERSION_MINOR, [222](#)
 - APP_VERSION_PATCH, [222](#)
 - BUILD_TIMESTAMP, [223](#)
- visit
 - slang::syntax::SyntaxNode, [145](#)
 - slang::syntax::SyntaxVisitor< TDerived >, [180](#)
- visitDefault
 - slang::syntax::SyntaxVisitor< TDerived >, [180](#)

[180](#)

visitToken

slang::syntax::SyntaxVisitor< TDerived >,

[181](#)

voltage_

LibFile, [81](#)

vtype_

ValuesIterator, [192](#)

writeJsonToFile

LibFile, [78](#)